Chapter 1: Why Rust?

Rust for RTL Verification is a book for verification engineers who want to write testbenches in Rust. It teaches the language from zero — not one line of prior Rust is assumed — and then uses rustdv, a UVM verification framework written in Rust, to build a complete, running testbench for a small ALU. If you have written UVM testbenches in any language, this book was written for you.

Verification engineers come to Rust from two directions. Most write SystemVerilog UVM testbenches and have spent years with `uvm_config_db`, `type_id::create()`, and the sequencer handshake. A growing number write Python testbenches with cocotb and pyuvm, and know the same methodology by its Python names. This book addresses both of you, usually at the same time, because you share the thing that matters: you know what a driver is for, why a scoreboard subscribes to a monitor, and what the factory buys you. That shared knowledge — the UVM, not any particular language — is the ground this book builds on.

What the book assumes, then, is verification: the UVM's concepts and vocabulary. If you have that from work, you are ready. If you want to build it first, two earlier books in this series teach it from first principles: *The UVM Primer* in SystemVerilog and *Python for RTL Verification* in Python. Either one prepares you for this book. Neither is required.

A note on what you do *not* need: any Rust. Not one line. If you have heard alarming rumors about a thing called the borrow checker, you have heard correctly, and we will make friends with it in Chapters 5 and 6.

Two revolutions

The first revolution is the one you lived through, or inherited. In the 1990s, verification engineers realized that testbenches are not test fixtures but *software* — software that happens to talk to a simulated design. That realization gave us the verification languages (e, Vera, SUPERLOG), then SystemVerilog, then a decade of methodology wars that ended with every EDA vendor blessing

a single winner: the Universal Verification Methodology. Later the same realization pushed further — if testbenches are software, why not write them in a general-purpose software language? — and produced cocotb and pyuvm, which moved testbenches off the simulator and into Python. Every step followed the same moral: testbenches are software, and software deserves a software language.

Rust's story rhymes with it. In 2006, a Mozilla engineer named Graydon Hoare started a personal project to answer an uncomfortable question: why, decades into the software era, were our foundational programs — browsers, kernels, the code we bet everything on — still written in languages that let one stray pointer corrupt everything? C and C++ were fast because they trusted the programmer completely, and every security bulletin showed what that trust cost.[1]

The conventional answer was garbage collection: let a runtime babysit memory, and accept the slowdown. That is Python's answer, and SystemVerilog's too — class objects in SV live until the last handle drops, collected automatically, exactly like Python objects. Rust proposed a third answer: what if the *compiler* proved memory safety, at compile time, and the finished program paid nothing at all? No garbage collector, no interpreter, no runtime babysitter. The rules that make this possible — ownership and borrowing — are the subject of Chapters 5 and 6, and they will bend your brain exactly once, after which you will wonder how you ever tracked object lifetimes in your head.

Rust 1.0 shipped in 2015. Since then the language has spent year after year at the top of developer-survey "most admired" lists, and it has done something no other language managed in half a century: it convinced the Linux kernel, Windows, and Android teams to admit a second systems language into their codebases. That is not fashion. That is an industry deciding that memory safety without a garbage collector is worth learning something hard.

[1] The security community eventually put numbers on it: both Microsoft and Google's Chrome team reported that roughly 70% of their serious security bugs were memory-safety bugs — the exact category Rust eliminates at compile time.

Why Rust for verification?

It is fair to ask why a verification engineer with a working flow should care. Rust's compiler earns its keep on data and ownership. Transactions are plain structs whose fields the compiler knows by name: rename a field, and it produces the complete list of every place that must change, and the testbench does not build until you have addressed all of them. Every object has exactly one owner: hand a transaction to the driver, and the compiler knows you no longer have it, which settles at compile time a question — who may still touch this object? — that verification methodologies otherwise answer with convention and code review. These checks pay off in proportion to the size of the testbench and the number of hands on it. A fifty-component environment refactored by four people gets something from them that a five-hundred-line testbench does not need.

Configuration, the factory, and TLM connection, on the other hand, resolve at run time in rustdv — deliberately, because that is what late binding *is*. The whole point of those three layers is to defer decisions so that one environment serves many tests, and a compiler can only check what is known before the program runs. A wrong configuration key, a missing factory override, an unconnected port: these fail at run time in rustdv as they do in every UVM, and the chapters that teach each layer show exactly what the failure looks like.

The other reason to care is throughput. Rust compiles to the same kind of native code as the simulator itself, with no interpreter and no garbage collector. If you come from Python, you know that every signal read, every transaction compare, every scoreboard update runs through the interpreter; for a small ALU it does not matter, but for a regression farm running thousands of seeds against a large SoC, testbench overhead is real money and real schedule. And a testbench with no collector pauses and no interpreter in the loop is the kind of testbench that can keep up with an emulator. This argument is about speed, not correctness — but speed is a verification resource like any other.

The rest of the ledger is smaller but real:

Unit testing without a simulator. Rust testbench components are ordinary structs, so the pure-software parts of your testbench — predictors, transaction operations, coverage logic — can be tested with `cargo test` in milliseconds, on your laptop, with no simulator license anywhere in sight. If you have ever

queued for a license to test a scoreboard change that never touches a signal, this feature alone may justify the book.

One binary, no environment. A Rust testbench compiles to a single library that the simulator loads. There is no interpreter version to match, no virtual environment to activate, no `pip install` on the farm machines. If it built, it runs.

An open toolchain. The Rust compiler, the cargo build system, the package ecosystem, and rustdv itself are free and open source. The simulator is the only licensed tool left in the loop — and this book's examples run on Icarus Verilog, which is also free.

What it costs

I owe you the other side of the ledger, because there is one.

Rust is hard to learn. Not a little hard — the ownership system is a new idea, and for your first weeks the compiler will reject code you are certain is fine. (It is almost never fine. This is the maddening part. Later it becomes the endearing part.) If you come from Python, you are trading a language that works hard to be unsurprising for one that holds opinions and holds them at compile time. If you come from SystemVerilog, take heart: you have already mastered one of the largest languages in engineering, and Rust is smaller, more consistent, and better documented than what you already know. Different, though. Chapter 2 maps what carries over and what must be unlearned, for both of you.

The edit-run loop includes a compile. For testbench work the compile is usually seconds, not minutes, but the rhythm is different from an interpreted flow and you will feel it.

The verification ecosystem is younger. SystemVerilog has two decades of UVM infrastructure; Python has cocotb, pyuvm, and years of conference papers; Rust verification is early. That is part of why this book exists — someone gets to write the early chapters of that story, and it may as well be us.

If your testbenches are small, your regressions short, and your team fluent in its current language, that language remains a fine answer, and I will not

pretend otherwise. This book is for when one of those stops being true.

Code examples

Every example has a figure number; code is followed by `--` and then its output, and every transcript in this book is genuine tool output. You can get a copy of the examples from the book's repository, organized in directories named after their chapters, each with a `README.md` explaining how to run it. Early chapters use small standalone cargo projects as playgrounds; from Chapter 15 on, examples are simulation directories.

Our running example is the TinyALU: a two-input ALU with a `start` / `done` handshake, small enough that the testbench, not the design, stays the subject. Readers of the earlier books will recognize it; Chapter 18 specifies it fully, so newcomers lose nothing. By the final chapter you will have built its testbench eight times, versions 1.0 through 8.0, each version adding one architectural idea — the same climb the earlier books made.

We begin where every programming book begins. In figure 1, we create a program with `cargo new`, Rust's project generator — meet `cargo` now, because it is the package manager, build system, test runner, and project generator fused into one tool, and it will be everywhere.

```
# Figure 1: Creating our first program

% cargo new hello
    Creating binary (application) `hello` package
```

`cargo new` writes a tiny project containing `src/main.rs`, which is where figure 2 lives. Rust asks even the smallest program for a function — `fn main()` is where every Rust program begins. The exclamation point on `println!` marks it as a *macro* rather than a function, a distinction that will matter a great deal in Chapter 21 and not at all before then.

```
// Figure 2: The classic first program

fn main() {
    println!("Hello, world.");
}
```

```
% cargo run
   Compiling hello v0.1.0
    Finished `dev` profile
     Running `target/debug/hello`
Hello, world.
--
Hello, world.
```

Notice the compile between `cargo run` and the greeting. That step is the new resident in your edit-run loop, and whether it earns its rent is the question the rest of this book answers with running testbenches.

The plan

Part I (Chapters 1–14) teaches the Rust you need, always against the languages you know: ownership where they had automatic collection, traits where they had inheritance, `Result` where they had exceptions or error signals, `match` where they had `case` and `if` chains.

Between the two parts sits an Interlude: the complete TinyALU testbench, presented whole and unexplained. It is your first sight of the destination — read it the way you would walk through a finished house before studying the blueprints.

Part II (Chapters 15–40) climbs to it. First the machinery: async/await — the same coroutine idea that powers both SystemVerilog's tasks and cocotb's scheduler, except that Rust hands you the engine — then tasks and queues, then the simulator, then testbenches 1.0 and 2.0, then one load-bearing chapter on macros. From Chapter 22 the UVM is rebuilt piece by piece: tests, components and the phase lifecycle, environments, logging, configuration, the factory, component communication, transactions, and sequences, through testbench versions 3.0 to 8.0. Chapter 40 returns to the Interlude's testbench

and walks it line by line, with everything explained. The appendices map this book's chapters onto the two earlier books and collect the idiom translations from Python and SystemVerilog.

The TinyALU is waiting. First, the language.

Chapter 2: Rust Concepts

Programming began with engineers pushing bits around, and types were invented so that a 16-bit `int` couldn't silently trample an 8-bit `char`. Ever since, languages have arranged themselves along a spectrum of strictness. VHDL and Pascal demanded permission for everything. C and SystemVerilog were permissive to a fault — SystemVerilog will happily chop the top eight bits off a 16-bit value to cram it into a byte. Python opted out of the argument entirely: since variables hold handles to objects rather than bits, there is nothing to chop, and the question of type compatibility gets answered at runtime, one operation at a time.

Rust takes a position on that spectrum you have seen before — it is statically typed, like VHDL — but it occupies the position so differently that the comparison will mislead you if you stop there. This chapter is about the difference. Not the syntax (that starts in Chapter 3), but the worldview: where the types live, when the checking happens, and why the strictest compiler you have ever met is going to become the most useful colleague you have.

Where you're coming from

Verification engineers arrive at this chapter from two directions, and each brings luggage worth inspecting at the door.

From SystemVerilog: declarations, static types, and compile errors are old friends, and much of your instinct transfers directly — you have always known how wide your buses are, and Rust agrees that widths matter. What needs unlearning is the *escape hatches*. SystemVerilog's type system is honeycombed with them: silent truncation on assignment, implicit conversions between anything bit-shaped, `$cast` to launder a class handle at runtime, and a class world where objects float free of the type checker until a cast succeeds or fails mid-simulation. Rust has no silent escapes. Every conversion is written where a reviewer can see it, and the checks you are used to postponing until elaboration or runtime happen before anything runs at all. Expect the compiler

to reject code your simulator would have accepted — and expect, a few chapters from now, to regard that as the feature it is.

From Python: you bring fluency in objects, iterators, and the ask-forgiveness style — try the operation, catch the exception. What needs unlearning is *try-and-see itself*. There is no `type()` to interrogate a value at runtime, because by runtime the types are gone; there is no exception to catch for a wrong-type operation, because the program containing it never gets built. And Python's most invisible habit — assignment copies a handle, and any number of names can share one object — is precisely the habit Chapter 5 exists to replace.

Both of you already believe the important thing: testbenches are software, and software correctness is worth machinery. You disagree only about when the machinery should run. Rust's answer is: as early as possible, all at once, every time.

Where the objects went

In Python, the number `5` is an object of class `int`, and it knows it. The type information travels with the object at runtime; that is what makes `type(5)` possible, and it is what makes dynamic typing work — every operation on every object begins with the interpreter looking up, right then, whether the object can do the thing. SystemVerilog splits the difference: nets and variables are compiled bits, but class objects — every transaction and component you have ever created — carry their type at runtime. That runtime tag is exactly what `$cast` consults when you down-cast a `uvm_object` handle and find out, mid-simulation, whether you guessed right.

Rust's `5` has a type too — but the type exists only in the compiler's head. Every value in a Rust program has exactly one type, known completely before the program runs. The compiler uses that knowledge to check every operation in the program, and then it throws the types away. The compiled binary contains no type tags, no class objects, no lookup tables — just the machine code that the types proved correct. There is no `type()` function in Rust for the same reason there is no ghost in a finished building: by the time the program runs, the types have done their work and left.[1]

This means the checking that a dynamic language spreads across the whole runtime, Rust performs all at once, up front. The cleanest way to see it is to make a classic dynamic-typing mistake and watch *when* it fails. In figure 1, `ends_with` is a real method on Rust strings — Python's `endswith()`, to the letter — and calling a method looks just as you'd expect: `mystring.ends_with("orld")`. Then we try it on an integer.

```
// Figure 1: Calling an undefined method

fn main() {
    let mystring = "Hello, World";
    println!("{}", mystring.ends_with("orld"));
    let myint: u8 = 42;
    println!("{}", myint.ends_with("a whimper"));
}
```

```
--
% cargo run
   Compiling concepts v0.1.0
error[E0599]: no method named `ends_with` found for type `u8` in
the current scope
 --> src/main.rs:5:26
  |
5 |     println!("{}", myint.ends_with("a whimper"));
  |                          ^^^^^^^^^ method not found in `u8`

error: could not compile `concepts` (bin "concepts") due to 1
previous error
```

Compare this to what Python does with the same mistake. There, the program *runs*: it prints `True` for the string, creates the integer, and only then dies with an `AttributeError` at line 4. The first three lines execute; the trouble is discovered in the act of transgressing.

Here, nothing ran. Not the broken line, and — look carefully — not the correct lines either. Rust refused to produce a program at all. That is the trade in its purest form: a dynamic language checks each operation the moment it happens, so a mistake costs you a run; Rust checks every operation before any of them happen, so a mistake costs you a compile. For verification work this trade is nearly a gift, because in our world "a run" is not a millisecond of interpreter time — it is a simulator license, an elaboration, and a lunch break. Delete the offending line, as in figure 2, and the program compiles and runs.

```
// Figure 2: The corrected program

fn main() {
    let mystring = "Hello, World";
    println!("{}", mystring.ends_with("orld"));
}
```

```
--
true
```

One reassurance before we move on: statically typed does not mean verbosely typed. If you are bracing for pages of SystemVerilog-style declarations, relax — Rust's compiler performs *type inference*. In figure 1 we never told it that `mystring` was a string; it worked that out from the value. You will write far fewer type annotations than SystemVerilog demands, and the ones you do write (like the `u8` above) tend to be the ones a verification engineer *wants* to write, because in our business the sizes are the specification. The TinyALU's A port really is eight bits wide, and Chapter 3 will make that `u8` official.

[1] Rust does keep a sliver of type information around for one feature we'll meet much later (trait objects, Chapter 10), but the rule to internalize is: no runtime type lookup, because no runtime types.

The compiler as collaborator

Everything in this book now depends on a mental adjustment, so let's make it explicitly.

When a runtime error arrives — a Python exception, a SystemVerilog `$fatal`, a failed `$cast` — it is reporting an accident that has already happened: the program was running, it hit something it couldn't do, and it stopped. When the Rust compiler rejects your program, nothing has happened yet. A rejection is not a failure report — it is a *prediction*, delivered while the mistake is still free. The compiler is not an adversary blocking the door to the simulator. It is a

reviewer who reads every line of your testbench, every time, in seconds, and never gets bored or skims.

The catch is that this reviewer communicates in a format you have to learn to read, and reading it well is a genuine skill — the single most valuable skill in this book, which is why our figures will so often show error messages rather than output. Let's dissect one. Figure 3 makes a mistake with TinyALU flavor: an ADD of two 8-bit operands can carry into nine bits, so the result belongs in a `u16` — but we try to store it back into a `u8` register. This is precisely the assignment from this chapter's opening history — the one SystemVerilog performs by silently chopping off the top eight bits.

```
// Figure 3: The mistake SystemVerilog would have allowed

fn main() {
    let a: u8 = 0xFF;
    let b: u8 = 0x01;
    let result: u16 = a as u16 + b as u16;
    let reg: u8 = result;
    println!("{}", reg);
}
```

```
--
error[E0308]: mismatched types
 --> src/main.rs:5:19
  |
5 |     let reg: u8 = result;
  |              --   ^^^^^^ expected `u8`, found `u16`
  |              |
  |              expected due to this
  |
help: you can convert a `u16` to a `u8` and panic if the converted
value
      doesn't fit
  |
5 |     let reg: u8 = result.try_into().unwrap();
  |                           ++++++++++++++++++++
For more information about this error, try `rustc --explain E0308`.
```

Read it from the top, because the compiler writes for readers who do:

- **The headline**: `error[E0308]: mismatched types`. Every error has a code, and the last line of the output tells you that `rustc --explain E0308` will

print a small essay about this class of error, with examples. That command is the book behind the book.

- **The location and the arrows**: file, line, column, then your own code quoted back with carets under the guilty expression. The compiler doesn't just point at the line — it points at the exact expression, and the `expected due to this` arrow points at the *other* place involved, the annotation that created the expectation. Two pointers, because a type mismatch always has two ends.
- **The `help` block**: a suggested fix, often as a ready-to-paste diff (those `+` signs mark what to insert). And notice what this particular suggestion says out loud: converting a `u16` to a `u8` *can lose data*, so the suggested conversion is one that checks at runtime and panics if the value doesn't fit. Rust will let you chop bits — with `result as u8`, exactly the truncation SystemVerilog performs silently — but you must write the chop yourself, in ink, where a reviewer can see it.

Figure 4 takes the honest fix: our register was simply too small for an ADD result, so we widen it. The point of the figure is how little drama the fix involves once the message has told you both ends of the mismatch.

```
// Figure 4: The corrected program

fn main() {
    let a: u8 = 0xFF;
    let b: u8 = 0x01;
    let result: u16 = a as u16 + b as u16;
    let reg: u16 = result;
    println!("{}", reg);
}
```

```
--
256
```

The compiler's helpfulness extends past types into plain proofreading. A typo'd name in Python surfaces as a runtime `AttributeError`, possibly weeks later, possibly on the one branch of the testbench that only executes when the DUT misbehaves. (SystemVerilog, to its credit, catches unknown names at compile time — though rarely this politely.) In Rust:

```
// Figure 5: The compiler as proofreader

fn main() {
    let result = 42;
    println!("{}", resutl);
}
```

```
--
error[E0425]: cannot find value `resutl` in this scope
 --> src/main.rs:3:20
  |
3 |     println!("{}", resutl);
  |                    ^^^^^^ help: a local variable with a similar
name
  |                           exists: `result`
```

It found the typo, and then it found the variable you meant. This is the texture of working in Rust: the error messages are not walls, they are directions. When this book's later chapters claim that a misconnected TLM port or a mistyped configuration "becomes a compile error," figures like these are what that will look like — and by then you will read them at a glance.

A word of honest preparation, though. The errors in this chapter are the friendly kind, because the mistakes were simple. Starting in Chapter 5, the compiler will begin rejecting programs for *ownership* reasons — code that looks obviously fine to any eye trained on a garbage-collected language, which is to say Python and SystemVerilog eyes alike — and those messages take real practice to read. The habit to build now, on easy errors, is the one that will carry you through the hard ones: read the first error first (later errors are often echoes of it), read the *whole* message including the help block, and trust that the compiler is describing a real problem even when you don't yet see it. In several years of Rust's existence, the compiler has been wrong about this far less often than its users have.[2]

[2] The Rust project treats confusing error messages as bugs and accepts bug reports about them. It is the only compiler I know with a customer-service department.

No interpreter, no garbage collector, no GIL

Three pieces of runtime machinery are simply absent from a running Rust program, and each absence will matter to us.

No interpreter. Python source is executed by a program that reads it — every signal comparison, every scoreboard update pays the interpreter's overhead, and every deployment needs the right interpreter version installed. Rust source is *compiled away*: `cargo run` in Chapter 1 produced a native binary, the same species of artifact as the simulator itself, and the source code is not consulted again. This is where the speed comes from, and it is also where the "no environment" benefit from Chapter 1 comes from — there is nothing to install on the farm machines because the program is already finished.

No garbage collector. Garbage collection is why neither Python nor SystemVerilog ever asks who is responsible for destroying an object. A Python object lives while any name points at it; an SV class object lives until the last handle drops; in both, memory management is somebody else's job, done invisibly, at times of the runtime's choosing. Rust has no such somebody. Memory is reclaimed at points the compiler determines *while compiling*, according to rules about which variable owns which value. Those rules are the famous part of Rust — ownership — and they are the subject of Chapter 5, where we will need a full chapter to replace the instinct that "assignment copies a handle and both names stay alive." For now, one sentence of preview: in Rust, assignment usually *moves* a value to its new owner, and that single idea eliminates the garbage collector, the pauses, and — more interesting to us — a whole family of "two tasks touched one transaction" testbench bugs.

No GIL. Python's Global Interpreter Lock serializes threads, which is why cocotb never pretended to give you parallelism — everything ran cooperatively on one thread, much as SystemVerilog processes take turns inside the simulator's event loop. Rust has no GIL; its compiler can prove threaded code free of data races, and "fearless concurrency" is a genuine Rust selling point. I mention it mostly to lower your expectations: our testbenches will still run cooperatively on the simulator's single thread, because the simulator interface demands it, and the design of our libraries enforces that rule at compile time rather than in documentation. The GIL's absence is real; our use for it, in this book, is modest.

The toolchain

Rust's tooling deserves a proper introduction, because it is unusually good and because you will touch all of it in the next few chapters. The pleasant surprise is that the whole kit arrives as a set with well-defined jobs — a verification engineer who has juggled `pip`, `venv`, `pyenv`, `black`, and `pylint`, or wrangled `.f` files, `+define` soup, and a Makefile only one person understands, will recognize every silhouette here, minus the juggling.

`rustup` installs and manages Rust itself — the job `pyenv` did for Python versions. One command installs the whole toolchain; `rustup update` moves you to the latest release. Rust ships a new stable version every six weeks, and — this matters — the project holds a strong backward-compatibility promise, so updating is routine rather than an event.

Which raises the obvious worry for anyone who lived through Python 2-to-3, or who maintains testbenches against three simulators' disagreements about the LRM: what happens when the language needs to change incompatibly? Rust's answer is **editions**. Every few years (2015, 2018, 2021, 2024) the project bundles its rare breaking changes into a named edition, and each crate — Rust's word for a package, official introductions in Chapter 14 — declares in its manifest which edition it speaks. The compiler supports all of them, forever, and crates from different editions link together in one program. It is Python 3 without the decade of schism: old code keeps compiling, new code gets the improvements, and nobody writes a farewell blog post. The projects in this book use the 2024 edition, which is what `cargo new` selects for you.

`cargo` you met in Chapter 1 — `pip`, `venv`, `make`, and `pytest` fused into one tool. It creates projects (`cargo new`), builds them (`cargo build`), builds-and-runs them (`cargo run`), fetches dependencies declared in `Cargo.toml`, and runs tests (`cargo test` — the star of Chapter 14, where we will unit-test testbench components with no simulator in sight). Because every Rust project is a cargo project with the same layout, there is no per-project incantation to learn; every example in this book builds the same way.

`rustfmt` reformats your source into the community-standard style, the job `black` did in Python: `cargo fmt`, and the formatting debate is over. This

book's figures are all `rustfmt`-clean, so what you read is what your editor will produce.

`clippy` is the linter — `pylint`'s job — but with a compiler's leverage, since it analyzes the same typed, checked view of your program the compiler sees. It catches correctness hazards, but its everyday gift to a Rust learner is idiom: clippy knows what fluent Rust looks like and will tell you, kindly and specifically, when you have written your old language in Rust syntax. Run it as `cargo clippy`; a representative complaint (output abridged) looks like this:

```
// Figure 6: Clippy teaching idiom

fn main() {
    let done = true;
    if done == true {
        println!("finished");
    }
}
```

```
--
% cargo clippy
warning: equality checks against true are unnecessary
 --> src/main.rs:3:8
  |
3 |     if done == true {
  |        ^^^^^^^^^^^^ help: try simplifying it as shown: `done`
```

Notice the shape: the same location-caret-help anatomy as a compiler error, because it comes from the same machinery. Reading one teaches you to read the other. Make `cargo clippy` a habit early — while you are learning, it is the closest thing to a Rust mentor watching over your shoulder, and unlike a mentor it never sighs.

Summary

Python's core idea is that everything is an object carrying its type at runtime, checked operation by operation; SystemVerilog checks its bits at compile time but lets its class objects carry runtime types that `$cast` and the config database interrogate mid-simulation. Rust's core idea is that every value has a

type known *before* the program runs, checked exhaustively by a compiler that then erases the types entirely — leaving a native binary with no interpreter, no garbage collector, and no GIL. The cost is that the compiler rejects programs your current language would have happily started; the payoff is that it rejects them in seconds, with error messages that name the problem, point at both ends of it, and usually propose the fix. Learning to read those messages is the skill this book will exercise constantly, and the toolchain — `rustup` for the compiler, `cargo` for everything, `rustfmt` for style, `clippy` for idiom — is the same set of jobs your old toolbelt did, consolidated and sharpened.

Concepts in hand, it is time to write some actual Rust. Chapter 3 starts where every language course starts — variables, numbers, and printing — and the TinyALU's 8-bit operands are about to get types that say so.

Chapter 3: Rust Basics

When you write `byte xx = 5;` in SystemVerilog, the program sets aside an 8-bit memory location and stores 5 in it. When you write `xx = 5` in Python, you get a handle to an `int` object on the heap — no bit width, no maximum value, no declared type; the *object* has a type, and the variable is just a name you can point at anything else a line later. Rust is about to hand the SystemVerilog reader something familiar and the Python reader something forgotten: the bits are back.

Chapter 2 introduced the compiler as a collaborator. This chapter puts it to work on the smallest possible material: variables, numbers, strings of output, and functions. None of it is hard, but nearly all of it is *different* from what you write today in ways that will matter every day, so we will move through it with small figures you can run and diff against your instincts.

If you want to follow along (you should), make a playground the way we did in Chapter 1:

```
# Figure 1: A playground for this chapter

% cargo new basics
    Creating binary (application) `basics` package
```

Everything in this chapter goes inside `fn main()` in `src/basics/src/main.rs` unless a figure shows its own function.

`let`: bindings, not name tags

In Python, a variable is a name tag you can peel off one object and stick on another. In Rust, `let` creates a **binding**: it associates a name with a value, and — here is the first surprise — that binding is **immutable by default**. Assigning to it a second time is not merely discouraged. It does not compile.

```
// Figure 2: Assigning twice to an immutable binding

fn main() {
    let xx = 5;
    println!("xx: {xx}");
    xx = 6;
    println!("xx: {xx}");
}
```

–

```
% cargo run
error[E0384]: cannot assign twice to immutable variable `xx`
 --> src/main.rs:4:5
  |
2 |     let xx = 5;
  |         -- first assignment to `xx`
3 |     println!("xx: {xx}");
4 |     xx = 6;
  |     ^^^^^^ cannot assign twice to immutable variable
  |
help: consider making this binding mutable
  |
2 |     let mut xx = 5;
  |         +++
```

Read that error the way Chapter 2 taught you: it names the rule, points at both the first assignment and the offending one, and then *tells you the fix*. If you want a variable that varies, you ask for one with `mut`:

```
// Figure 3: A mutable binding

fn main() {
    let mut xx = 5;
    println!("xx: {xx}");
    xx = 6;
    println!("xx: {xx}");
}
```

–

```
xx: 5
xx: 6
```

Whichever language you come from, immutability-by-default feels backwards for about a week. Then you start reading other people's testbench code and discover what it buys you: every `mut` in a Rust program is a signpost saying *this value changes — watch it*. In Python and SystemVerilog alike, every variable carried that warning implicitly, which is the same as no variable carrying it at all. When you read a Rust monitor and see that only one binding in it is `mut`, you know where the state lives. The compiler is not restricting you; it is making your intentions legible.[1]

[1] It will also nag you in the other direction: declare something `mut` and never mutate it, and the compiler warns you to take the `mut` off. It wants the signposts accurate in both directions.

Scalar types: the bits are back

Python gets by with exactly three number types — `bool`, `int`, and `float` — and its `int` has no maximum value. Rust returns us to the world hardware people never really left, and SystemVerilog readers never left at all: integers have widths, and the widths are in the names. Read `u8` as `bit [7:0]` — with the width actually enforced on every assignment.

The scalar types you will actually use:

- **Integers, unsigned:** `u8`, `u16`, `u32`, `u64` — 8 to 64 bits, no sign bit. There is also `usize`, the pointer-width integer that Rust uses for indexing and lengths.
- **Integers, signed:** `i8`, `i16`, `i32`, `i64` — two's complement, exactly as your DUT stores them. Bare integer literals like `5` default to `i32`.
- **Floats:** `f32` and `f64`. Bare float literals like `2.0` default to `f64`, which is Python's `float`.
- **`bool`:** `true` and `false` — lowercase now, and the *only* things a condition accepts. Python's habit of treating `None`, `0`, and empty containers as falsy does not exist here; an `if` takes a `bool`, full stop.
- **`char`:** a single Unicode character, in single quotes: `'A'`. Not a one-character string — a distinct type, four bytes wide.

Why should a verification engineer care? Because *your DUT already thinks this way*, and now your testbench can fully agree with it. The TinyALU's A and B legs are eight bits wide. A Python testbench drives them from an `int` and relies on discipline (and the BFM) to keep values in range — nothing in the language stops a careless test from generating `a = 300`. A SystemVerilog testbench declares the width but not the enforcement — assign 300 to a `byte` and the tool quietly keeps the bottom eight bits. In Rust, a TinyALU operand is a `u8`, and 300 *is not a value that type can hold*. The type system now knows something true about the hardware, and it never forgets it:

```
// Figure 4: The TinyALU's A leg really is a u8

fn main() {
    let aa: u8 = 0xFF;
    let bb: u8 = 300;
    println!("aa: {aa}, bb: {bb}");
}
```

–

```
% cargo run
error: literal out of range for `u8`
 --> src/main.rs:3:18
  |
3 |     let bb: u8 = 300;
  |                  ^^^
  |
  = note: the literal `300` does not fit into the type `u8`
    whose range is `0..=255`
```

Notice the `let aa: u8 = 0xFF;` syntax: a colon and a type after the name is a **type annotation**. Most of the time you can leave it off and Rust infers the type from context — this is why Figures 2 and 3 didn't need one — but when you mean *eight bits*, say so.

One honest wrinkle while we are here: what happens when arithmetic *overflows* a `u8` at runtime — say, `200 + 100` where both values arrived from a random generator? In a debug build, the program panics (halts with an error) at the overflowing operation; in a release build, the value wraps around modulo 256, the way the hardware would. If wrapping is what you *mean* — and in ALU-prediction code it often is — Rust provides methods like `wrapping_add` that say

so explicitly. We will use exactly that when we write the TinyALU predictor, because “the model overflows exactly like the DUT, on purpose, in writing” is the kind of sentence verification sign-off meetings love.

No implicit conversions

Both of your languages convert numeric types behind your back. In Python, a `float` in an operation means a `float` result — `ii + ff` quietly promotes the `int`, and `ii/ii` produces a `float` even from two `int`s. SystemVerilog goes further and implicitly converts nearly anything bit-shaped to anything else, sign and width be damned. Let’s write the mixed-type program and watch Rust refuse to play:

```
// Figure 5: A float in operations means... a compile error

fn main() {
    let ii: i32 = 1;
    let ff: f64 = 2.0;
    let ss = ii + ff;
    println!("ss: {ss}");
}
```

–

```
% cargo run
error[E0277]: cannot add a `f64` to `i32`
 --> src/main.rs:4:17
  |
4 |     let ss = ii + ff;
  |                 ^ no implementation for `i32 + f64`
```

Rust performs **no implicit numeric conversions**. Not int-to-float, not `u8`-to-`u16`, nothing. If you want a conversion, you write one, using the `as` keyword — and once you do, the rest of the ported figure behaves recognizably, with one telling difference in the last line:

```rust
// Figure 6: The same figure, with the conversions made explicit

fn main() {
    let ii: i32 = 1;
    let ff: f64 = 2.0;
    let ss = ii as f64 + ff;
    println!("ss: {ss}");
    let dd = ii / ii;
    println!("dd: {dd}");
}
```

–

```
ss: 3
dd: 1
```

In Python, `ii/ii` prints `1.0` — division *always* returns a `float`. In Rust, dividing two integers is integer division: `dd` is `1`, an `i32`, and `7 / 2` would be `3` — SystemVerilog agrees with Rust on this one. If you want the fractional answer, convert to floats first. Neither behavior is right or wrong, but they are different, and scoreboard math is exactly where that difference bites — so it is worth one figure now instead of one confused afternoon later.

The same strictness governs augmented assignment. A Python variable that starts as an `int` holding 1 has, after `xx /= 4`, silently become a `float` holding 1.5 — the variable changed *type* mid-flight. Watch the Rust version:

```rust
// Figure 7: Augmented assignments — the type never changes

fn main() {
    let mut xx = 1;
    println!("xx: {xx}");
    xx += 1;
    println!("xx += 1: {xx}");
    xx *= 3;
    println!("xx *= 3: {xx}");
    xx /= 4;
    println!("xx /= 4: {xx}");
}
```

–

```
xx: 1
xx += 1: 2
xx *= 3: 6
xx /= 4: 1
```

Same operators, same rhythm — Rust has `+=`, `*=`, `/=`, and friends, though like Python it has no `++`. But `xx` was born an `i32` and will die an `i32`; `6 /= 4` gives 1, not 1.5. A binding's type is fixed for the binding's whole life. Which raises an obvious question: what do we do when we want the Python pattern — same *idea*, new *type*?

Shadowing: same name, new binding

Python turns the string `"3.14159"` into a number by constructing a new object — `pi = float("3.14159")` — and pointing the old name at it. Rust's version of that pattern is **shadowing**: declaring a *new* binding, with `let`, that reuses an old name.

```rust
// Figure 8: Creating a number from a string, by shadowing

fn main() {
    let pi = "3.14159";
    let pi: f64 = pi.parse().expect("not a number");
    println!("pi: {pi}");
}
```

–

```
pi: 3.14159
```

The first `pi` is a string; the second `pi` is a brand-new binding, a `f64`, whose value came from parsing the first. From that line on, the name `pi` means the number; the string version is shadowed — inaccessible, retired with honors. This is not mutation (nothing was `mut`) and it is not a type change (each binding kept its type); it is the "same idea, new type" idiom, done with two immutable bindings instead of one shape-shifting variable. You will see it

constantly in testbench code: parse a string into a number, convert raw bits into a transaction, and keep the natural name at every step.

Two small notes on figure 8. First, `parse` can fail — `"pi".parse()` has nowhere good to go — so it returns a `Result`, Rust's replacement for exceptions; `.expect("...")` says "give me the value, and halt with this message if it failed." That is a blunt instrument we will trade for proper tools in Chapter 9; Python's version has the same rough edge, raising `ValueError` on `int("3.14159")`. Second, the annotation `: f64` is doing real work: it is how `parse` knows *what* to parse the string into.

`println!` and format strings

You have been reading `println!` output all chapter; now let's look at the format strings themselves. `{}` is the placeholder and arguments fill placeholders in order — the `$display` and `str.format()` model — and, the part that makes Rust feel almost Pythonic, a variable name can go directly inside the braces, like an f-string:

```
// Figure 9: Format strings, next to the f-strings you know

fn main() {
    let aa: u8 = 0x2A;
    let bb: u8 = 7;
    println!("aa is {} and bb is {}", aa, bb);  // like
str.format()
    println!("aa is {aa} and bb is {bb}");      // like an f-string
    println!("aa in hex: {aa:#04x}");
    println!("aa in binary: {aa:#010b}");
    println!("sum: {}", aa + bb);
}
```

–

```
aa is 42 and bb is 7
aa is 42 and bb is 7
aa in hex: 0x2a
aa in binary: 0b00101010
sum: 49
```

The format specifiers after the colon will feel familiar from both Python and `$display`: `{aa:#04x}` means hexadecimal, `#` for the `0x` prefix, padded to width 4. The hex and binary forms in figure 9 are the ones you will reach for when a scoreboard mismatch needs to be read against a waveform.

One genuine difference from f-strings: the braces capture *names only*, not arbitrary expressions. Python lets you write `f"{aa + bb}"`; Rust makes you write the expression as an argument, as in the last line of figure 9. And the exclamation point still means what Chapter 1 said it means: `println!` is a macro, which is precisely *why* it can type-check your format string against your arguments at compile time — pass one argument too few, or hand `%d` the wrong-shaped value in spirit, and the program does not build, where Python's `"{} {}".format(x)` and a mismatched `$display` wait until runtime to complain.

Expressions vs. statements

Here is the concept in this chapter most likely to be new, rather than a stricter spelling of something you had. Python and SystemVerilog both divide the world into statements (`if`, `for`, assignments) and expressions (things with values), and mostly keep them apart. In Rust, nearly everything is an **expression** — nearly everything *has a value* — and the language leans on this constantly.

Two demonstrations. First, `if` is an expression, which means it can sit on the right-hand side of a `let`:

```
// Figure 10: if is an expression

fn main() {
    let count: u8 = 42;
    let parity = if count % 2 == 0 { "even" } else { "odd" };
    println!("count is {parity}");
}
```

–

```
count is even
```

SystemVerilog has `? :` and Python has `"even" if count % 2 == 0 else "odd"` for exactly this job; Rust simply has no separate ternary, because ordinary `if` already returns a value. The compiler checks that both arms produce the same type — an `if` that gives you a string on Mondays and an integer on Tuesdays does not compile — and an `else` is required when you use the value, because the value must exist either way.

Second, a block — any `{ ... }` — is an expression whose value is its **last expression, written without a semicolon**:

```
// Figure 11: A block is an expression; the semicolon is the switch

fn main() {
    let nn = {
        let doubled = 2 * 3;
        doubled + 1
    };
    println!("nn: {nn}");
}
```

–

```
nn: 7
```

Look hard at `doubled + 1` — no semicolon. That is not sloppy punctuation; it is load-bearing syntax. A trailing expression without a semicolon is the block's value; add a semicolon and you have turned it into a statement, the block's value becomes the empty "unit" value `()`, and the compiler will greet you with an error message that — helpfully — points straight at the semicolon and suggests removing it. Every Rust programmer alive has been rescued by that message.[2] Once this clicks, you will start to see Rust code as a tree of nested expressions, each yielding a value to the one above it, and the language's whole shape gets simpler.

[2] Usually within the first hour.

Functions

Which brings us, with suspicious convenience, to functions — because a function body is just another block, and returning a value is just the block-expression rule again.

```
// Figure 12: A TinyALU prediction function

fn predict_add(aa: u8, bb: u8) -> u16 {
    aa as u16 + bb as u16
}

fn main() {
    let sum = predict_add(0xFF, 0xFF);
    println!("predicted sum: {sum:#06x}");
}
```

–

```
predicted sum: 0x01fe
```

Everything Python left optional is now required, and everything required is now checked. Each parameter declares its type; the `-> u16` arrow declares the return type; and the last expression of the body — `aa as u16 + bb as u16`, no semicolon — is the return value. (An explicit `return` keyword exists for bailing out early, but idiomatic Rust lets the final expression speak for itself.) A function with no `->` returns `()`, Rust's cousin of `void` and of Python's implicit `None`.

Figure 12 also carries the chapter's TinyALU payload. A Python prediction function takes unbounded `int`s and returns one, so the fact that the TinyALU's result port is *sixteen* bits while its operands are *eight* lives only in prose and in the DUT; a SystemVerilog function declares those widths but lets a careless assignment truncate through them. Here the fact lives in the signature, enforced: `fn predict_add(aa: u8, bb: u8) -> u16` *is* the TinyALU's ADD operation, as a type. `0xFF + 0xFF` overflows a `u8` — which is exactly why the hardware has a wide result port, and exactly why the function converts each operand to `u16` before adding. Try deleting the two `as u16` conversions and read the error you get; the compiler will explain the TinyALU datasheet to you.

And that signature pays one more dividend: the compiler checks every *call*. Pass `predict_add` a 16-bit value, or three arguments, or use its result where a `u8` is expected, and the testbench does not build. In Python, a mis-called prediction function is a runtime discovery; here it never gets as far as the simulator.

Comments, briefly

Line comments are `//` to end of line, exactly as in SystemVerilog, doing the job of Python's `#`. Block comments `/* ... */` exist but are rare in practice. What Rust has that Python approximated with docstrings and SystemVerilog never had is **doc comments**: lines beginning `///` above a function or type are documentation the toolchain actually compiles into browsable HTML (`cargo doc`). We will start writing them when we write code worth documenting, which is soon.

Summary

This chapter covered Rust's nuts and bolts, each one a deliberate diff against the languages you know:

- `let` **bindings** — immutable by default; `mut` is an explicit, visible request for mutability
- **scalar types** — `u8` through `i64`, `f32` / `f64`, `bool`, `char`; bit widths are back, and the TinyALU's operands are honest `u8`s at last
- **no implicit conversions** — mixed-type arithmetic is a compile error; `as` makes conversions visible; integer division stays integer
- **shadowing** — the "same idea, new type" pattern, done with a fresh binding instead of a shape-shifting variable
- `println!` **and format strings** — f-string-like `{name}` captures, plus compile-time checking of the format string itself
- **expressions vs. statements** — `if` and blocks have values; the trailing-expression-without-semicolon rule

- **functions** — typed parameters, declared return types, and signatures the compiler enforces at every call site

Every figure here was straight-line code — no branches worth mentioning, no loops at all. A testbench that never loops is not much of a testbench, and besides, Rust is holding back its best conditional construct: `match`, which does what `case` and `if` chains only gesture at, and which the rest of this book will use on nearly every page. Both are waiting in Chapter 4.

Chapter 4: Conditions, Loops, and Match

Conditions and loops are where your old languages already agree: Python tests with `if` / `elif` / `else` and loops with `while` and `for`; SystemVerilog tests with `if` / `else` and `case` and loops with everything from `for` to `forever`. Two-thirds of this chapter is warm-up, because these constructs transfer to Rust almost without friction. The last third introduces the construct the rest of this book leans on constantly: `match` — the statement Python never had and the one SystemVerilog's `case` always wanted to be. When we meet `Result` in Chapter 9, `Option` alongside it, and the `Ops` enum in Chapter 7, `match` is how we will take them apart. Learn it well here, in miniature, and every later chapter gets easier.

if and else

Rust's `if` looks like Python's with the punctuation swapped: braces instead of indentation, and no colon. Like Python — and unlike C and SystemVerilog — Rust does not require parentheses around the condition. Unlike everybody, Rust *insists* on the braces, even for one-line bodies.

```
// Figure 1: A Rust if statement

fn main() {
    let name = "Roy";
    if name != "Danny" {
        println!("Hey, you're not Danny.");
    }
}
```

```
--
Hey, you're not Danny.
```

One difference matters more than it looks. Python happily tested any value for truth: `if nn:` meant "if `nn` is nonzero," `if my_list:` meant "if the list is nonempty." Rust conditions must be `bool` — actually, literally `bool`. Write `if nn {` where `nn` is an integer and the compiler stops you:

```
// Figure 2: Rust has no truthiness

fn main() {
    let nn = 5;
    if nn {
        println!("nonzero");
    }
}
```

```
--
error[E0308]: mismatched types
 --> src/main.rs:3:8
  |
3 |     if nn {
  |        ^^ expected `bool`, found integer
```

You will grumble about this for a week and then remember every testbench where `if dut.done:` silently tested the wrong thing — a handle instead of a value, a list instead of its contents. Rust makes you write `if nn != 0`, and the intent goes on the record.[1]

There is no `elif` keyword. Rust spells it `else if`, and a chain of them stands in for a switch — for now:

```
// Figure 3: else if as a switch (for now)

fn main() {
    let (a, b) = (5, 5);
    let operation = "divide";
    if operation == "add" {
        println!("A + B = {}", a + b);
    } else if operation == "subtract" {
        println!("A - B = {}", a - b);
    } else if operation == "multiply" {
        println!("A * B = {}", a * b);
    } else {
        println!("Illegal Operation: {operation}");
    }
}
```

```
--
Illegal Operation: divide
```

Python called this trade "more flexibility at the cost of more code," and SystemVerilog readers reaching for `case` should hold that reflex a few pages. The cost is about to be refunded, with interest, in the `match` section.

[1] SystemVerilog veterans have the opposite scar: `if (sig)` on a 4-state signal, where an `X` quietly takes the false branch. Rust's answer to both languages is the same: say what you mean, and the compiler will hold you to it.

if is an expression

Here is the first new idea. In Chapter 3 we met Rust's distinction between statements and expressions: an expression produces a value. In Rust, `if` / `else` *is an expression* — the whole construct evaluates to the value of whichever branch ran. That means you can bind it with `let`:

```rust
// Figure 4: Conditional assignment — no ternary needed

fn main() {
    let aa = 5;
    let message = if aa == 5 { "five_val" } else { "other_val" };
    println!("{message}");
}
```

```
--
five_val
```

Python needed special ternary syntax — `x if cond else y` — because its `if` statement produces nothing. Rust needs no ternary because its ordinary `if` already produces a value.[2] Two rules come along with this power. First, both branches must produce the *same type* — a branch handing back a string while the other hands back an integer is a compile error, because `message` must be exactly one type. Second, if you use `if` as an expression, the `else` is mandatory; the compiler will not let you bind a value that might not exist. Both rules are the compiler asking the question Python deferred to runtime: *what, exactly, is this variable?*

[2] C and SystemVerilog programmers may now retire the `?` operator with full honors.

Three loops, one of them new

Python has two loops, `while` and `for`. SystemVerilog, characteristically, has five. Rust has three: `while`, `for`, and `loop`. The first two are your old friends in new clothes; the third will look familiar to exactly half of you.

`while` works exactly as you expect, condition first, braces around the body. Here is a counting loop:

```rust
// Figure 5: A while loop in action

fn main() {
    let mut nn = 0;
    while nn <= 13 {
        print!("{nn} ");
        nn += 1;
    }
    println!();
}
```

```
--
0 1 2 3 4 5 6 7 8 9 10 11 12 13
```

Two Chapter 3 details show up here: `nn` needs `mut` because we reassign it, and `print!` (without the `ln`) is how Rust says `end=" "` — it prints without the newline. `continue` and `break` exist, spelled the same and meaning the same as in Python; I will not re-teach them.

Now the third loop. Python emulates the run-forever pattern with `while True:` and a well-placed `break`; SystemVerilog gave it a keyword, `forever`. Rust agrees with SystemVerilog — intentionally infinite loops are common enough to deserve their own construct — and then improves the idea:

```rust
// Figure 6: loop — the intentional infinite loop

fn main() {
    let mut nn = 0;
    let first_big_square = loop {
        nn += 1;
        if nn * nn > 200 {
            break nn * nn;
        }
    };
    println!("{first_big_square}");
}
```

```
--
225
```

Look closely at that `break`: it carries a value, and the whole `loop` evaluates to it — which is why we could write `let first_big_square = loop { ... }`. Like `if`, `loop` is an expression. A "run until something happens, then hand back what you found" pattern that took a flag variable and a post-loop read in Python is one construct in Rust. Only `loop` gets this privilege; `while` and `for` loops cannot `break` with a value, because their conditions mean they might never produce one.

If `loop` sounds like a novelty, consider what you already know is coming: every driver loop and monitor loop you have ever written was a `forever` or a `while True:` at heart. In Part II, those become `loop` — the language admitting what the code always meant.

Ranges

Python's `range()` was a constructor with three calling conventions. Rust builds ranges from operators instead:

- `0..8` — start at 0, stop *before* 8. The same half-open interval as `range(0, 8)`.
- `0..=8` — start at 0, stop *at* 8, inclusive. Python had no direct equivalent; you wrote `range(0, 9)` and remembered why.

The `for` loop iterates over a range just as Python's did:

```
// Figure 7: Looping through numbers using a range

fn main() {
    for ii in 0..4 {
        print!("{ii} ");
    }
    println!();
}
```

```
--
0 1 2 3
```

Where is `step`? Ranges are iterators (Chapter 12 makes that concept rigorous), and iterators have adapter methods. Python's `range(1, 14, 2)` becomes:

```
// Figure 8: Stepping through a range

fn main() {
    for ii in (1..14).step_by(2) {
        print!("{ii} ");
    }
    println!();
}
```

```
--
1 3 5 7 9 11 13
```

There is also `.rev()` for counting down, and a whole catalog of adapters waiting in Chapter 12. For now: `..` exclusive, `..=` inclusive, adapters for everything else. And notice `0..=8` reads naturally for hardware — an inclusive range is how you think about the values a bus can carry. A TinyALU operand is a `u8`; the values it can take are `0..=255`, and you can write exactly that.

match: the construct Python never had

Now the centerpiece. Python replaced `case` / `switch` with `elif` chains and called the trade "more flexibility at the cost of more code." Rust's `match` refuses the trade: it delivers more flexibility than `case` *and* less code than `elif` — and then adds a property neither language offered, one that this book will spend many chapters collecting dividends on.

Start with the direct port. Figure 3's `else if` chain becomes:

```
// Figure 9: match as a switch

fn main() {
    let (a, b) = (5, 5);
    let operation = "multiply";
    let answer = match operation {
        "add" => a + b,
        "subtract" => a - b,
        "multiply" => a * b,
        _ => panic!("Illegal Operation: {operation}"),
    };
    println!("answer = {answer}");
}
```

```
--
answer = 25
```

Read it as: compare `operation` against each *pattern* on the left of a `=>`; run the code on the right of the first pattern that fits. The underscore `_` is the wildcard — it matches anything, playing the role of `else` or `default`. Three things to notice, each an upgrade over both `elif` and `case`:

1. **`match` is an expression.** Like `if` and `loop`, the whole construct produces a value — here it computes `answer` directly, where figure 3 could only print from inside each branch. Every arm must produce the same type, same rule as `if`.
2. **There is no fallthrough.** C and SystemVerilog programmers carry decades of missing-`break` scar tissue; `match` arms are separate, always, no `break` required or even possible.
3. **The compiler checks that the patterns cover every case.** This one gets its own section.

Exhaustiveness: the compiler counts your cases

Delete the `_` arm from a `match` and something remarkable happens. Here is a `match` on a raw op code — the TinyALU's four operations, numbered 1 through 4 as the spec has always numbered them — with no wildcard:

```
// Figure 10: The compiler catches missing cases

fn main() {
    let op_code: u8 = 2;
    let name = match op_code {
        1 => "ADD",
        2 => "AND",
        3 => "XOR",
        4 => "MUL",
    };
    println!("{name}");
}
```

```
--
error[E0004]: non-exhaustive patterns: `0_u8` and `5_u8..=u8::MAX`
not covered
 --> src/main.rs:3:22
  |
3 |     let name = match op_code {
  |                      ^^^^^^^ not covered
  |
  = note: the matched value is of type `u8`
```

Sit with that error message for a moment, because it is doing something neither of your testbench languages ever did for free. The compiler enumerated every value a `u8` can hold, subtracted the four we handled, and reported precisely what we missed: zero, and everything from 5 up. An `elif` chain that forgets a case is a runtime surprise — figure 3 only caught its illegal `"divide"` because we remembered to write the `else`. A SystemVerilog `case` that forgets one falls through in silence unless you wrote the `default`, and even `unique case` merely upgrades the silence to a runtime warning that waits for the right stimulus to arrive before it speaks. A `match` that forgets a case *does not compile*. The fix is either a `_` arm (an explicit decision to lump the leftovers together) or arms for the missing values (an explicit decision about each) — but it is always a decision, never an oversight.

For a `u8`, exhaustiveness is a nice safety net. The reason this book teaches `match` in Chapter 4 rather than Chapter 14 is what happens when the thing being matched has a *small, meaningful* set of cases. In Chapter 7, `Ops` returns as a true Rust enum with exactly four values, and a `match` on it needs exactly four arms — no wildcard, no dead cases, and if the TinyALU ever grows a fifth operation, **every `match` in the testbench that fails to handle it becomes a**

compile error. Your scoreboard, your coverage collector, your predictor: the compiler hands you the complete list of code that must learn about the new op.

Patterns: ranges, tuples, and taking things apart

The left side of a `match` arm is not limited to constants. Patterns can be ranges — and here the TinyALU gives us a real example: ADD, AND, and XOR complete in one cycle while MUL takes three. With op codes 1 through 4:

```
// Figure 11: Matching on ranges

fn main() {
    let op_code: u8 = 4;
    let cycles = match op_code {
        1..=3 => 1,
        4 => 3,
        _ => panic!("Illegal op code: {op_code}"),
    };
    println!("This operation takes {cycles} cycle(s)");
}
```

```
--
This operation takes 3 cycle(s)
```

Range patterns use the inclusive `..=` form, and you can also combine alternatives with `|`, as in `1 | 3 => ...`. The compiler still does its exhaustiveness arithmetic across all of it.

Patterns can also take structured data apart. Match on a tuple, and each position of the pattern matches the corresponding element — with `_` skipping positions you don't care about and plain names *binding* the values so the arm can use them:

```
// Figure 12: Matching and destructuring a tuple

fn main() {
    let operands: (u8, u8) = (0, 200);
    let comment = match operands {
        (0, 0) => String::from("both operands zero"),
        (0, _) | (_, 0) => String::from("one operand zero"),
        (a, b) if a == b => format!("equal operands: {a}"),
        (a, b) => format!("ordinary operands: {a}, {b}"),
    };
    println!("{comment}");
}
```

```
--
one operand zero
```

Three new tricks in one figure. The `|` combines two patterns into one arm. The names `a` and `b` in the later arms are bindings — the arm receives the matched values under those names, which is how `match` goes beyond comparing and starts *extracting*. And `if a == b` is a *match guard*, an extra boolean test bolted onto a pattern for the cases patterns alone can't express. You have just watched a `match` classify stimulus into coverage-bin-shaped categories in four lines — remember this figure when we build the coverage collector.

That extraction ability is the real reason `match` anchors this book. Here is a preview of what is coming — do not worry about the details yet. In Chapter 9 you will learn that a fallible operation like looking up a DUT signal returns a `Result`, which is either `Ok(handle)` carrying the goods or `Err(e)` carrying the explanation. The way you get the goods out is a `match`:

```
match dut.child("clk") {
    Ok(clk) => { /* use clk */ }
    Err(e) => { /* the signal wasn't there; e says why */ }
}
```

One construct checks which case you got *and* hands you its contents *and* forces you — at compile time — to say what happens in the failure case you would rather not think about. Python's `try` / `except` let you skip the `except` and hope; SystemVerilog mostly declined to have an error story at all; `match` on a

`Result` has no such loophole. Exhaustiveness, it turns out, is not a switch-statement garnish. It is how Rust makes error handling mandatory, and Chapters 7 and 9 are where that bill comes due — in our favor.

Summary

Rust's conditions and loops hold no terrors: `if` / `else if` / `else` with mandatory braces and honest `bool` conditions, `while` and `for` behaving as you expect, `continue` and `break` unchanged. The differences all push the same direction. No truthiness, and no silently-tested `X`. `if` is an expression, retiring the ternary. `loop` is `forever` with a diploma — it names the intentional infinite loop and can `break` with a value. Ranges are syntax (`0..8` exclusive, `0..=8` inclusive) rather than a constructor, with iterator adapters like `step_by` covering the rest.

And `match` is the construct Python never had and `case` wanted to be: patterns instead of comparisons, expression instead of statement, no fallthrough, destructuring with bindings and guards — and exhaustiveness checking, the compiler's guarantee that every case is a decision and no case is an oversight. We will `match` on integers and tuples this week, on `Ops` in Chapter 7, and on `Option` and `Result` for the rest of our verification careers.

So far, every value we have used has lived and died inside `fn main` without our attention — garbage-collected habits, still serving us fine. In the next chapter, we hand a value from one variable to another and discover that Rust has been keeping track of who owns what all along. Chapter 5 is ownership: the idea with no mirror in either of your languages, and the hinge of the whole one you're learning.

Chapter 5: Ownership

Every chapter so far has had a twin in the languages you know. `let` had assignment, `match` had `case` and `elif` chains, `u8` had `byte`. This chapter has no twin, because it answers a question neither of your languages ever let you hear: *when a value's time is up, who is responsible for destroying it?*

You have created millions of objects — transactions, components, queues full of commands — and you have never once destroyed one. You didn't have to. Python's garbage collector followed you around the testbench like a diligent stagehand, watching which objects still had names pointing at them and quietly disposing of the ones that didn't;[1] SystemVerilog's runtime does the same for class objects, keeping each one alive until its last handle drops. Both stagehands do the job so well, and so silently, that most programmers never learn the job exists.

Rust has no garbage collector. There is no stagehand. And yet Rust programs do not leak memory, do not free things twice, and do not touch things after they're freed — the classic sins of C. Rust pulls this off with one idea, enforced by the compiler, and that idea is the hinge of the entire language: **ownership**. Chapter 1 promised that the rules behind Rust's no-runtime-cost safety would bend your brain exactly once. This is the chapter where the bending happens.

[1] CPython's stagehand is mostly a reference counter — every object carries a count of the names and containers pointing at it, and hits the trash at zero — with a cycle-detecting garbage collector mopping up the cases where two objects point at each other and the counts never fall. The details don't matter here; the silence does.

Who frees this? The old answer

Let's watch the stagehand work. In Python, a variable is not a box holding a value — it is a name tag stuck onto an object that lives somewhere on the heap. Assignment copies the name tag, never the object — exactly as assigning one

SystemVerilog class handle to another copies the handle, never the object. In figure 1, `a` and `b` are two tags on one transaction-ish list, which we can prove by mutating through one name and looking through the other.

```
# Figure 1: Python assignment: two names, one object

a = ["ADD", 5, 3]
b = a
b[0] = "MUL"
print(a)
print(a is b)
```

```
--
['MUL', 5, 3]
True
```

You knew this, in whichever spelling: every UVM component holding a handle to the BFM, every scoreboard holding the same transaction object the monitor held. The part you may never have said out loud is the lifetime question: that list stays alive as long as *any* tag points at it, and it dies whenever the last tag disappears, at a moment of the runtime's choosing. Nobody owns the list. Ownership is smeared across every name that ever touched it, and the runtime keeps the books.

For testbenches this policy is comfortable right up until it isn't: the monitor that holds a stale handle to a component the test rebuilt, the two subscribers that received the "same" transaction and one of them mutated it, the destructor that runs at a time no document will commit to. The garbage-collected answer to "who frees this?" is *nobody in particular, eventually*. Rust's answer is one word long.

Rust's answer: one owner

Here is the rule the rest of this book stands on:

> Every value in Rust has exactly one **owner** — the variable (or, later, the struct field) responsible for it. When the owner goes out of scope, the

value is destroyed, immediately. Assignment doesn't copy a name tag; it **moves** ownership to the new variable, and the old one is dead.

That last clause is the shock. Let's take the shock now, on purpose, with the compiler watching. Figure 2 is the two-names experiment from figure 1, translated into Rust.

```
// Figure 2: Assignment moves — and the old name is gone

fn main() {
    let a = String::from("ADD 5 3");
    let b = a;
    println!("{a}");
    println!("{b}");
}
```

```
--
error[E0382]: borrow of moved value: `a`
 --> src/main.rs:4:15
  |
2 |     let a = String::from("ADD 5 3");
  |         - move occurs because `a` has type `String`, which does
not
  |           implement the `Copy` trait
3 |     let b = a;
  |             - value moved here
4 |     println!("{a}");
  |               ^^^ value borrowed here after move
  |
help: consider cloning the value if the performance cost is
acceptable
  |
3 |     let b = a.clone();
  |              ++++++++
```

Read that error the way Chapter 2 taught you, because it is one of the best-written error messages in any compiler and it narrates the whole story. Line 2: the string was created and `a` owned it. Line 3: `value moved here` — the assignment handed ownership to `b`, and `a` stopped being a valid name for anything. Line 4: we tried to use `a` after the move, and the compiler refused to build the program.

Sit with what did *not* happen. The program did not run and print something surprising. It did not crash at 2 a.m. in seed 8,441 of a regression. It never existed as a program at all. In your old languages, `b = a` gives you two live names and a shrug about lifetimes; in Rust, `let b = a;` is a baton pass — after it, exactly one variable is responsible for that string, and the compiler will name the exact line where responsibility changed hands. One value, one owner, at every moment, provable at compile time. That invariant is the entire trick, and everything Rust does that no garbage-collected language can — no GC, no data races, deterministic cleanup — falls out of it.

Notice, too, the compiler's parting suggestion: `a.clone()`. It has read your mind — or at least your options. We'll take it up on that shortly.

Scope is the destructor

If every value has exactly one owner, then "who frees this?" has a mechanical answer: the owner does, at the moment it goes out of scope. Rust calls this **dropping** the value, and it is as deterministic as the closing brace it happens at.

```
// Figure 3: Values die at the closing brace — every time, on time

fn main() {
    {
        let cmd = String::from("MUL 7 6");
        println!("inside the scope: {cmd}");
    }   // <- cmd's owner goes out of scope RIGHT HERE.
        //    The String is dropped, its memory freed, before
        //    the next line runs. No collector. No "eventually."
    println!("after the scope");
}
```

```
--
inside the scope: MUL 7 6
after the scope
```

The output is unremarkable; the guarantee is not. That string's memory was returned *at the brace* — not at the next garbage-collection pause, not when a reference count happened to hit zero, not "at interpreter shutdown, probably."

If you have ever tried to close a file, flush a log, or release a queue in a Python `__del__` method, you know what that "probably" costs: the language reference makes carefully hedged promises about when `__del__` runs, and none at all in some shutdown cases.[2] Rust replaces the hedging with a rule you can point to in the source code. From Chapter 24 on, this rule will be doing serious work — an objection guard that ends a run phase by dropping at the closing brace, tasks whose cleanup runs the instant they're cancelled — but the whole mechanism is already in front of you in figure 3. Deterministic destruction isn't a feature bolted onto ownership. It *is* ownership, viewed from the value's last moment.

[2] The Python language reference notes that it is "not guaranteed" that `__del__` will be called for objects still alive when the interpreter exits — one of the great load-bearing "not guaranteed"s of our time.

Wait — my integers have been fine

A fair objection: you've been assigning integers back and forth since Chapter 3 without the compiler saying a word about moves. Figure 4 confirms it.

```
// Figure 4: Copy types don't move — small values are simply copied

fn main() {
    let a: u8 = 5;
    let b = a;              // copies the byte; a is still alive
    println!("a = {a}, b = {b}");
}
```

```
--
a = 5, b = 5
```

The error message in figure 2 quietly explained why: the string moved *"because `a` has type `String`, which does not implement the `Copy` trait."* Types whose values are small, fixed-size, and self-contained — `u8`, `u16`, `bool`, the TinyALU's operands, all the scalars from Chapter 3 — are **`Copy`** types:

assignment duplicates the bits, both variables live, nothing to negotiate. There is no shared object for two names to fight over, so ownership has nothing to protect. (Traits, including how a type comes to be `Copy`, are Chapter 10's business.)

Move semantics governs everything else: `String`, the collections coming in Chapter 8, and — most importantly for us — the structs we build ourselves. Which brings us to the reason this chapter exists.

The monitor and the scoreboard

Every mechanism in this chapter has been abstract enough to shrug at. So let's make it a testbench problem — *the* testbench problem, the one you've written a dozen times: a monitor observes a transaction on the TinyALU's command bus and hands it to a scoreboard.

We need a transaction type. Structs get their own chapter (Chapter 7); for today, read the first four lines of figure 5 as "a class with only data and no methods" — fields, types, no ceremony. The `op` field really wants to be a proper enum, and in Chapter 7 it becomes one; a `u8` stands in for now. And we need the two parties: a `scoreboard` function that takes a `Transaction` *by value*, and a `main` that plays the monitor. In the UVM these would be components connected by an analysis port; here in our cargo playground, two functions are enough to expose the question that matters.

```
// Figure 5: The monitor hands off a transaction — and learns what
"hands off" means

struct Transaction {
    a: u8,
    b: u8,
    op: u8,
}

fn scoreboard(t: Transaction) {
    println!("scoreboard checking: {} op {} (code {})", t.a, t.b,
t.op);
}   // <- t dropped here: the scoreboard owned it, the scoreboard's
    //    scope ends, the transaction is destroyed. Question
answered.

fn main() {
    // main is playing the monitor today.
    let t = Transaction { a: 5, b: 3, op: 1 };
    scoreboard(t);
    println!("monitor logging: a was {}", t.a);
}
```

```
--
error[E0382]: borrow of moved value: `t`
  --> src/main.rs:15:44
   |
13 |     let t = Transaction { a: 5, b: 3, op: 1 };
   |         - move occurs because `t` has type `Transaction`,
which
   |           does not implement the `Copy` trait
14 |     scoreboard(t);
   |                - value moved here
15 |     println!("monitor logging: a was {}", t.a);
   |                                           ^^^ value borrowed
here after move
   |
note: consider changing this parameter type in function
`scoreboard` to
      borrow instead if owning the value isn't necessary
help: consider cloning the value if the performance cost is
acceptable
```

Passing a value to a function moves it, exactly as assignment did — `scoreboard(t)` is a baton pass, and line 15 is the monitor trying to run the next leg without the baton.

Here is what I want you to see: **the compiler is not reporting a syntax mistake. It is asking you a design question.** When the monitor hands this transaction to the scoreboard, what *should* happen? In the UVM you never had to decide. The analysis port handed every subscriber the same handle to the same object, ownership belonged to nobody, and the design question got answered by accident — which worked fine until one subscriber mutated the transaction another was still reading, a bug both earlier books could only warn you about. Rust makes you answer on purpose, and there are exactly two honest answers.

Answer one: it's a true handoff. The monitor's job was to observe the transaction and pass it on; it has no business touching it afterward. Then the code is wrong and the compiler is right — delete line 15, and the program compiles. Ownership flows monitor → scoreboard, the scoreboard checks it, and when the scoreboard's scope ends the transaction is dropped, on time, by its one owner. The comment on `scoreboard`'s closing brace in figure 5 is the answer to this chapter's title question, sitting in plain sight.

Answer two: the monitor still needs it — to log it, to hand a second copy to a coverage collector. Then the monitor must keep *a real copy*, and Rust has an explicit word for that.

`.clone()`: copies you can see

The compiler suggested it twice, so let's take the hint. Adding `#[derive(Clone)]` above the struct asks the compiler to write a field-by-field copy routine for us (that one magic line gets a full explanation in Chapters 10 and 21; for now, "please generate the copying code" is the whole story). Then `.clone()` makes an independent duplicate wherever we ask.

```rust
// Figure 6: The monitor keeps a copy — explicitly

#[derive(Clone)]
struct Transaction {
    a: u8,
    b: u8,
    op: u8,
}

fn scoreboard(t: Transaction) {
    println!("scoreboard checking: {} op {} (code {})", t.a, t.b,
t.op);
}

fn main() {
    let t = Transaction { a: 5, b: 3, op: 1 };
    scoreboard(t.clone());   // the scoreboard owns the copy...
    println!("monitor logging: a was {}", t.a);   // ...the monitor
owns the original
}
```

```
--
scoreboard checking: 5 op 3 (code 1)
monitor logging: a was 5
```

Two transactions now exist, each with exactly one owner, each dropped at its own owner's closing brace. No sharing, no aliasing, no way for the scoreboard's copy and the monitor's original to interfere. And crucially: mutation of one can never surprise the other — the "two subscribers, one mutated object" bug is not merely discouraged, it is unrepresentable in this code.

It's worth pausing on why Rust makes you *spell out* the copy. In Python, whether an assignment copied or aliased depended on the type — and `copy.deepcopy` existed for the cases you usually met through a bug. SystemVerilog engineers are ahead here — SV never copied a class object on assignment either, and the UVM made you call `clone()` in ink. Rust agrees with that instinct and adds the enforcement: trivial bit-copies (`Copy` types) are silent because they cannot matter; every copy that allocates or duplicates real state is written `.clone()`, visible in the diff, greppable in the review. When a testbench is cloning a million transactions a second, you can *find every clone* and decide whether each one earns its cost.

One more note while the handoff is fresh. When components exchange transactions later in the book, the exchange takes the transaction by value: a `put` into a TLM FIFO (Chapter 31) is a move, and so is a sequence's `finish_item` (Chapter 36) — the monitor-to-scoreboard baton pass, made into infrastructure. The decision you just made line by line (hand it off, or clone and keep one?) is the same decision you'll make at every port, and the compiler will hold you to your answer every time.

The mental model

There is one thread left hanging, and the compiler dangled it in figure 5's output on purpose: *"consider changing this parameter type in function `scoreboard` to borrow instead if owning the value isn't necessary."* Because ask yourself: does a scoreboard need to *own* the transaction? It needs to read the fields, compare against a prediction, and be done. Taking ownership just to look at something is like requiring the title to a car in order to check its odometer. Rust has a mechanism for looking without taking, written `&`, and it is called borrowing; it is the whole subject of Chapter 6, and I will not steal that chapter's material beyond telling you it exists.

What I will do is hand you the sentence this book will use, from here to testbench 8.0, whenever an API makes you choose between taking a value and referencing it. Say it with me, and keep it:

> **Ownership is about responsibility. Borrowing is about access.**
>
> Own a value when you are responsible for it — for its storage, its lifetime, its eventual destruction, its answer to "who frees this?" Borrow a value when you merely need access to it for a while — to read it, or briefly change it — while responsibility stays exactly where it was.

Every design decision ahead of us is an application of that sentence. The component hierarchy of Chapter 24 works because parents *own* their children — responsibility flows down the tree. FIFOs move transactions because a handoff transfers *responsibility*. Monitors will hand scoreboards references or

clones depending on whether the scoreboard needs *access* or needs to *keep* something. When you're stuck on a compiler error anywhere in this book, ask the sentence's question first: does this code need responsibility for the value, or just access to it? The answer usually types itself.

You now hold half the model — the responsibility half. Chapter 6 supplies the access half: `&`, the borrow checker Chapter 1 warned you about, and the rule that lets Rust catch at compile time a race condition earlier testbenches could only fear.

Chapter 6: Borrowing and References

> **In the UVM…** we got a warning instead of a rule. Every dialect has this bug: two processes — a monitor and a consumer — sharing a `transaction_data` variable, both waking on the same rising edge, with no guarantee about which runs first. SystemVerilog's LRM declares the ordering indeterminate and grew `program` blocks and clocking blocks trying to fence testbenches away from the consequences; cocotb's documentation, examining the same temptation around `NullTrigger`, says flatly **Do not do this** — the scheduling order "is not deterministic and should generally not be relied upon" — and tells you to synchronize with an event instead. Both toolchains could warn you. Neither could stop you. This chapter is about the language feature that stops you.

Chapter 5 ended with a mental model we will now spend a whole chapter earning: *ownership is about responsibility, borrowing is about access.* Ownership answered the question the garbage collectors answered silently — who frees this transaction? — by declaring exactly one owner, and by making assignment a *move* of that responsibility. When the monitor handed a transaction to the scoreboard, the monitor was done with it, and the compiler enforced its being done with it.

But that model, taken alone, is unlivable. A scoreboard that must *own* every transaction it merely wants to *look at* would be a scoreboard fed entirely by `clone()` calls. A coverage collector that steals the transaction from the scoreboard is not a testbench, it is a relay race. Most of the time a component does not need responsibility for a value. It needs access. Rust's mechanism for access-without-responsibility is the **reference**, and the act of granting one is called **borrowing** — a word chosen carefully, because a borrow, unlike a Python reference or a SystemVerilog class handle, comes with an obligation to give the value back and rules about what you may do while you hold it.

Shared references: many readers

A shared reference is written `&T` — "a reference to a T" — and you create one with `&`:

```
// Figure 1: Shared references — everyone may look, nobody may
touch

struct Transaction {
    data: u8,
}

fn report(t: &Transaction) {
    println!("Saw transaction with data {}", t.data);
}

fn main() {
    let t = Transaction { data: 42 };

    let monitor_view = &t;    // a borrow
    let coverage_view = &t;   // another borrow -- readers may
alias freely

    report(monitor_view);
    report(coverage_view);

    println!("Still the owner: {}", t.data);   // t was never moved
}
```

```
--
Saw transaction with data 42
Saw transaction with data 42
Still the owner: 42
```

(I am reusing the `Transaction` struct from Chapter 5. Structs get their full treatment in Chapter 7; for now it is a labeled box holding a `data` field.)

Three things to notice, each a quiet contrast with Chapter 5. First, `report` takes `&Transaction`, not `Transaction` — so calling it does not move `t`. The function borrows the transaction, reads it, and the borrow ends when the function returns. Second, we made *two* references to `t` at the same time and the compiler did not object: shared references may alias freely, any number of them at once. Third, after all that lending, `t` is still ours to use in the final `println!`. Responsibility never changed hands; only access did.

The price of this generosity is stamped into the type: through a `&T` you may *read* and nothing else. Try `monitor_view.data = 7` and the compiler will refuse — not because `t` is immutable (though it is; we never said `let mut`), but because a shared reference never grants write access, period. Everyone may look. Nobody may touch.

If you want an analogy, `&T` is what every handle in a well-disciplined codebase *pretended* to be: something you pass around for reading, trusting that nobody mutates through it. Rust removes the trust and keeps the handle.

Exclusive references: one writer

Sometimes touching is the point. A driver that receives a transaction may legitimately need to fill in a timestamp; a BFM may need to update a field before sending. For write access you need the second kind of reference, `&mut T` — an **exclusive reference**:

```
// Figure 2: An exclusive reference grants write access

struct Transaction {
    data: u8,
}

fn scramble(t: &mut Transaction) {
    t.data = 99;
}

fn main() {
    let mut t = Transaction { data: 42 };   // mut: the owner
permits mutation

    scramble(&mut t);   // lend write access, briefly

    println!("After scramble: {}", t.data);
}
```

```
--
After scramble: 99
```

Two spellings of `mut` appear here and they are doing different jobs. `let mut t` is the owner declaring that this value may ever be mutated at all — Chapter 3's immutable-by-default rule. `&mut t` is the owner lending *exclusive, writable* access to somebody else. You cannot create a `&mut` borrow of a variable that was not declared `mut`; permission flows downhill from the owner.

"Exclusive" is the load-bearing word. While a `&mut T` exists, it is the *only* live reference to that value — no other `&mut`, no `&`, and the owner itself may not so much as read the value until the exclusive borrow ends. The writer works alone, with the door locked.

The rule: aliasing XOR mutability

We can now state the rule the whole chapter — arguably the whole language — hangs on. Memorize it in this form:

> **At any moment, a value may have many readers or one writer — never both.**

Rust programmers call this **aliasing XOR mutability**: a value may be aliased (many `&T`), or it may be mutable (one `&mut T`), but never both at once.[1] Everything the borrow checker does is enforcement of this one sentence, plus bookkeeping about *when* each borrow ends.

Why is this the rule worth building a language around? Because every data race you have ever debugged — and every race those warnings at the top of this chapter were about — has the same anatomy: one piece of state, at least one writer, at least one other reader or writer, and no enforced ordering between them. Aliasing XOR mutability makes that anatomy unrepresentable. If there is a writer, there is nobody else; if there is anybody else, there is no writer. The race has no room to exist.

That is a large claim, so let us go back to the scene of the crime.

[1] The rule has the same shape as a conference-room whiteboard policy: any number of people may stand and read it, or exactly one person may hold the marker — but a reader standing at a whiteboard while someone else is erasing it learns nothing trustworthy, and everybody knows it.

The NullTrigger race, rejected at compile time

Recall the pattern every dialect tells you never to write: a shared `transaction_data` variable, a monitor process that writes it, a consumer process that reads it, both triggered by the same clock edge, with nothing but hope deciding who runs first. In SystemVerilog and Python alike, that code runs. It even works, usually, on your simulator, until a scheduler's ordering shifts and it silently doesn't.

We cannot yet write real concurrent tasks in Rust — the executor and `spawn` arrive in Part II — but we do not need them to expose the bug, because the bug was never really about tasks. It was about *two live accessors of one value, one of them a writer, with no enforced order*. We can write exactly that in six lines, giving the monitor its write access and the scoreboard its read access:

```
// Figure 3: Two tasks' worth of access to one value -- the borrow
checker objects

fn main() {
    let mut transaction_data: Option<u8> = None;

    let monitor = &mut transaction_data;   // the monitor's claim:
write access
    let scoreboard = &transaction_data;    // the scoreboard's
claim: read access

    *monitor = Some(42);                   // the monitor sees a
transaction...

    match scoreboard {                     // ...and the scoreboard
checks it
        Some(data) => println!("Scoreboard checking {data}"),
        None => println!("Nothing to check yet"),
    }
}
```

```
--
error[E0502]: cannot borrow `transaction_data` as immutable because
it is also borrowed as mutable
 --> src/main.rs:5:22
  |
4 |     let monitor = &mut transaction_data;
  |                   ---------------------- mutable borrow occurs
here
5 |     let scoreboard = &transaction_data;
  |                      ^^^^^^^^^^^^^^^^^^ immutable borrow occurs
here
...
7 |     *monitor = Some(42);
  |     ------------------- mutable borrow later used here

For more information about this error, try `rustc --explain E0502`.
error: could not compile `borrow_race` (bin "borrow_race") due to 1
previous error
```

(`Option<u8>` is Rust's typed replacement for the value-or-nothing idiom — Python's `None`, SystemVerilog's null or sentinel value. Read `Some(42)` as "has a value" and `None` as "doesn't." Chapter 9 gives `Option` its full due; here it is set dressing.)

Read the error the way Chapter 2 taught you to — as a collaborator's comment, not a rejection slip. The compiler identifies all three actors in the crime: where the mutable borrow was created (line 4), where the immutable borrow was created while the writer still existed (line 5 — that is the actual error, marked with carets), and — this is the part no Python tool could ever tell you — where the writer is *used again afterward* (line 7), proving the two claims overlap in time. Writer alive, reader alive, same value, same moment: aliasing AND mutability. The program does not compile.

Sit with what just happened. This bug has been documented for decades. cocotb's docstring says **Do not do this** in bold; the SystemVerilog LRM calls the ordering indeterminate and leaves you to your fate; a generation of verification leads has taught the discipline that avoids it. And still, nothing in either toolchain would stop a tired engineer from writing it on a Friday afternoon, and nothing would flag it until a scheduler reordering made a regression flicker, weeks later, in someone else's test. Rust converts the whole affair into three seconds of compile time and an error message with line numbers.

The fix: make the ordering real

The classic fix was an event — SystemVerilog's `->done` and `@(done)`, cocotb's `Event`: the monitor writes, *then* sets the event; the consumer awaits the event, *then* reads. Notice what the event actually contributed — it forced the write and the read into a definite order, so that the writer was finished before the reader began. The borrow checker demands precisely the same thing, and in straight-line code you provide it the same way: sequence the accesses so they do not overlap.

```
// Figure 4: The same actors, with the ordering made real

fn main() {
    let mut transaction_data: Option<u8> = None;

    let monitor = &mut transaction_data;
    *monitor = Some(42);                  // the monitor writes...
                                          // ...and its borrow ends
at its last use

    let scoreboard = &transaction_data;   // now the reader may
claim access
    match scoreboard {
        Some(data) => println!("Scoreboard checking {data}"),
        None => println!("Nothing to check yet"),
    }
}
```

```
--
Scoreboard checking 42
```

The only change from figure 3 is the order of the lines — the monitor finishes its writing *before* the scoreboard's borrow begins — and that is the entire point. The fix for an unordered write/read conflict is an ordering, in Rust as in Python; the difference is that Rust would not let you skip it.

One subtlety in figure 4 deserves a sentence, because it will save you fights with the compiler later: `monitor`'s borrow ended at its *last use* (the write on the line above), not at the closing brace of its scope. The borrow checker tracks how long each borrow is actually needed, not merely where the variable was declared. If it worked scope-to-brace, figure 4 would not compile either;

because it works use-to-use, tidy sequential code like this passes without ceremony.

And one note of caution, so you do not over-generalize: when the monitor and scoreboard become concurrent tasks in Chapter 16, they cannot simply take turns borrowing a local variable — each task needs its own durable handle to shared state, which is exactly the situation `&` / `&mut` alone cannot express. Rust's answers there are the queue (the two tasks never share the value at all — Chapter 16) and, when sharing truly is the design, the `Rc<RefCell<T>>` escape hatch, which moves this chapter's rule from compile time to runtime checking (Chapter 13). Both of those tools are built on top of the rule you just learned, not exemptions from it. The rule is the constant; only the enforcement point moves.

Lifetimes: recognize the syntax, decline the rabbit hole

There is one more piece of borrowing syntax you will meet in the wild, and I want you to be able to greet it without alarm. Sometimes a function signature carries a small tick-marked annotation:

```
fn newer<'a>(x: &'a Transaction, y: &'a Transaction) -> &'a
Transaction
```

That `'a` (pronounced "tick-a") is a **lifetime** — a name for *how long a borrow lasts*. Every reference in every program has one; the compiler has been inferring them silently in every figure of this chapter. They surface in the syntax only when the compiler needs your help connecting inputs to outputs: `newer` returns a reference, and the compiler must know whether that returned borrow is tied to `x`, to `y`, or to something else, because whoever *receives* the returned reference is now borrowing from one of them, and the checker must know which value has to stay alive. Writing `'a` on all three says: the returned reference lives no longer than the shorter-lived of the two arguments. That is the whole idea — lifetimes are the paperwork that lets borrow checking work *across* function boundaries.

Here is what you need at this point in the book, in full: recognize `'a` as "a named lifetime," know that it connects the lifetime of an output reference to the lifetimes of input references, and know that when the compiler asks you for one, it is asking "if I hand this reference back to your caller, which argument is it borrowing from?" You will see lifetimes in error messages and in library documentation; you will write them rarely, because the compiler's inference rules cover the overwhelming majority of testbench-shaped code, and rustdv's public API is deliberately designed to keep user-facing lifetimes rare besides.

And here I am going to make an editorial call with precedent: *Python for RTL Verification* taught every Python feature a testbench needed and then, at the door of the metaclass, stopped — noting that pyuvm used them internally but that a testbench author never needed to write one. Lifetimes get the same treatment here, for the same reason. There is real depth behind that tick mark — variance, higher-ranked bounds, a small literature of compiler lore — and none of it, not one line, appears in the testbenches this book builds. When a lifetime annotation is forced on us later, I will explain that occurrence on the spot. Until then: recognize it, read past it, keep going.[2]

[2] Rust folklore holds that you do not truly understand lifetimes until you have argued with the borrow checker about them for a weekend. This book's position is that your weekends belong to your regressions.

What borrowing buys the testbench

Step back and look at the shape of what you now know. Ownership (Chapter 5) decides who is responsible for every value — who frees the transaction, in a world with no garbage collector. Borrowing (this chapter) decides who may access it meanwhile, under a rule — many readers or one writer, never both — that turns the shared-state race every dialect warns about into a compile error with line numbers. Together they are the machinery that turns a whole class of 2 a.m. testbench bugs into a red squiggle at your desk before lunch.

So far, though, our transactions have been a struct with one sad little `u8` in it, and our operations have been strings of field pokes. A real TinyALU transaction

has two operands and an *operation* — and "an operation" is a value that is exactly one of ADD, AND, XOR, or MUL, a kind of value Python approximated with `IntEnum`, SystemVerilog approximates with `typedef enum`, and Rust does beautifully. Structs, methods, and enums with payloads are Chapter 7, and they are where Rust starts being fun.

Chapter 7: Structs, Enums, and Methods

Both earlier books in this series built on the same claim: object-oriented programming is the foundation of the expandability and reusability of UVM-based verification. That claim survives the trip to Rust intact. What does not survive is the machinery. Rust has no `class` statement, no `self` or `this` sliding invisibly into every method, no `__init__`, and no ability to bolt a data attribute onto an object whenever the mood strikes. In their place it offers two building blocks — structs and enums — and a place to hang behavior on them, the `impl` block. By the end of this chapter you will have both, plus the chapter's real payoff: Rust enums, which are so much more capable than Python's or SystemVerilog's that they will quietly restructure how you think about testbench data.

> **In the UVM...** every transaction and component was a class. Python let us build objects on the fly — `walrus = Animal()` then `walrus.kg = 1000`, no declaration anywhere — a freedom that cut both ways, surfacing a forgotten attribute as a runtime `AttributeError`. SystemVerilog made us declare every field at definition, as this chapter will too. And both dialects named the TinyALU's operations with an enum — `typedef enum` in SV, `Ops(enum.IntEnum)` in Python: `ADD = 1, AND = 2, XOR = 3, MUL = 4`, names mapped to opcode integers. Both patterns return in this chapter, and both come back changed.

Structs: the data, declared

A Rust struct is the part of a class that holds data — and only that part. You declare every field, with its type, up front:

```rust
// Figure 1: Defining and instantiating a struct

struct Animal {
    kg: f64,
}

fn main() {
    let walrus = Animal { kg: 1000.0 };
    println!("Walrus mass: {}", walrus.kg);
}
```

```
--
Walrus mass: 1000
```

Two things deserve a look. First, the declaration reads like SystemVerilog, not Python: the fields are part of the definition, not something we improvise later. Second, instantiation uses a *struct literal* — `Animal { kg: 1000.0 }` — which names every field and supplies every value. There is no separate "create empty, then populate" step, because an empty `Animal` is not a thing Rust permits to exist.

That last point is not a style preference; it is enforcement. In Python you could create a `yorkie`, forget to set `kg`, and get an `AttributeError` — at runtime, at the moment of use, which in a testbench means mid-simulation. Here is the same mistake in Rust:

```rust
// Figure 2: Forgetting a field is now a compile error

struct Animal {
    kg: f64,
}

fn main() {
    let yorkie = Animal {};
    println!("Yorkie mass: {}", yorkie.kg);
}
```

```
--
error[E0063]: missing field `kg` in initializer of `Animal`
 --> src/main.rs:6:18
  |
6 |     let yorkie = Animal {};
  |                  ^^^^^^ missing `kg`
```

The program never ran. There is no such thing as an `Animal` with an undefined mass, so the class of bug Python could only warn about is not a bug you can write.[1] SystemVerilog engineers will feel at home — and should note the upgrade: an SV field you never assign holds a default value and simulates anyway; a Rust field you never assign stops the build. This is the pattern of the whole chapter — of the whole book, really: where your old language offered a convention and a warning, Rust offers a rule and a compile error.

One freedom is gone: you cannot add a field to an object after the fact. `walrus.age = 12` on a struct with no `age` field does not compile. Every field a transaction will ever carry is visible in one place, in its definition, forever. For quick scripts this feels confining. For a transaction type that five components and three engineers share, it is exactly what you want.

[1] The walrus, having survived one book already, takes this in stride.

`impl` blocks: where behavior lives

Python and SystemVerilog classes bundle data and behavior in one block. Rust separates them: the struct declares the data, and an `impl` block — short for *implementation* — attaches the behavior. Here is a `get_pounds()` method:

```rust
// Figure 3: A method in an impl block

struct Animal {
    kg: f64,
}

impl Animal {
    fn get_pounds(&self) -> f64 {
        self.kg / 2.2
    }
}

fn main() {
    let walrus = Animal { kg: 1000.0 };
    println!("Walrus weight in pounds {:.2}", walrus.get_pounds());
}
```

```
--
Walrus weight in pounds 454.55
```

The call site reads like every language you know: `walrus.get_pounds()`. The definition is where the languages diverge, and the divergence is instructive.

Python's `self` is an ordinary argument that the interpreter fills in for you — `Animal.get_pounds(walrus)` works, passing the object by hand. (SystemVerilog's `this` is closer to true magic; it simply appears.) Rust's `&self` is doing the same job, but it carries more information: that ampersand says this method *borrows* the animal, read-only, exactly as Chapter 6 taught. The method can look at `self.kg`; it cannot modify it, and it does not take ownership of the walrus. The signature tells you the method's relationship to the object before you read a line of the body.

And yes, the explicit form still works: `Animal::get_pounds(&walrus)` is legal Rust and does exactly what Python's hand-passed version does. What Python offered as a party trick, Rust treats as the ordinary meaning that the dot syntax abbreviates.

When a method needs to *change* the object, it says so:

```rust
// Figure 4: A method that mutates takes &mut self

struct Animal {
    kg: f64,
}

impl Animal {
    fn feed(&mut self, meal_kg: f64) {
        self.kg += meal_kg;
    }
}

fn main() {
    let mut walrus = Animal { kg: 1000.0 };
    walrus.feed(3.5);
    println!("Walrus mass after lunch: {}", walrus.kg);
}
```

```
--
Walrus mass after lunch: 1003.5
```

Notice `let mut walrus`. A method that takes `&mut self` can only be called on a mutable binding — immutability by default, as in Chapter 3, extends all the way into method calls. Skim any `impl` block and the `&self` / `&mut self` split hands you a map of which methods observe and which methods mutate. In your old languages, discovering that a method quietly modified your transaction was an afternoon with the debugger; here it is a fact printed in the signature. When we build monitors and scoreboards, this distinction stops being philosophy: a scoreboard's check wants `&self`, its update wants `&mut self`, and the compiler holds every caller to it.

Associated functions: `new()` and the end of `__init__`

Python forces initialization with `__init__()`, a magic method called implicitly when you invoke the class name; SystemVerilog answers to the `new` keyword and a `function new()` you write. Rust has no magic methods and no implicit calls. What it has is the *associated function*: a function in an `impl` block that takes no `self` at all, called through the type name with `::`.

By near-universal convention, the constructor is an associated function named `new` — SystemVerilog engineers may enjoy that Rust agrees with them about the name:

```
// Figure 5: The new() associated function

struct Animal {
    kg: f64,
}

impl Animal {
    fn new(kg: f64) -> Self {
        Self { kg }
    }

    fn get_pounds(&self) -> f64 {
        self.kg / 2.2
    }
}

fn main() {
    let yorkie = Animal::new(20.0);
    println!("The Yorkie weighs {:.1} pounds",
yorkie.get_pounds());
}
```

```
--
The Yorkie weighs 9.1 pounds
```

A few notes on the pattern:

- `Self` (capital S) is shorthand for "the type this `impl` block belongs to" — here, `Animal`. Writing `-> Self` and `Self { kg }` means the code survives a rename.
- `Self { kg }` uses *field init shorthand*: when a variable and a field share a name, you may write the name once instead of `kg: kg`.
- There is nothing special about `new`. It is not a keyword, the language does not call it for you, and you could name it `hatch()` if you enjoyed confusing people. The compiler's only contribution is the guarantee from figure 2: however an `Animal` gets built, every field gets a value.

Python sorts methods into three kinds — instance, class, and static, a decorator apiece — and SystemVerilog splits its own hairs with `static` methods and

variables. Rust flattens the taxonomy to two: if it takes `self` in some form, it is a *method*, called with a dot; if it does not, it is an *associated function*, called with `::`. The static and class-method patterns both collapse into the second kind, and the class-variable pattern comes along as an *associated constant*:

```
// Figure 6: Associated constants and functions replace class
variables and static methods

struct Triangle;

impl Triangle {
    const SIDE_COUNT: u32 = 3;

    fn print_side_count() {
        println!("I have {} sides.", Self::SIDE_COUNT);
    }
}

fn main() {
    Triangle::print_side_count();
}
```

```
--
I have 3 sides.
```

(`struct Triangle;` with no braces is a *unit struct* — a type with no data, which is all our triangle ever needed.) The two classic reasons to keep a static method inside a class — a logical home, and a consistent calling style — apply verbatim here; the `impl` block *is* the logical home, and `Triangle::print_side_count()` is the consistent style.

Enums: the star of the chapter

Now for the construct that earns this chapter its place in the book.

Python names the TinyALU's operations with an `enum.IntEnum` — named constants, each secretly an integer wearing a name tag. SystemVerilog's `typedef enum` is the same idea with the same secret: underneath, it is an integer, and the language happily casts it back to one at the first opportunity. Rust enums start at that point and keep going.

Here is `Ops`, together with the prediction function it exists to serve:

```
// Figure 7: The Ops enumeration and an exhaustive match

#[derive(Clone, Copy, Debug, PartialEq)]
enum Ops {
    Add = 1,
    And = 2,
    Xor = 3,
    Mul = 4,
}

fn alu_prediction(a: u8, b: u8, op: Ops) -> u16 {
    match op {
        Ops::Add => a as u16 + b as u16,
        Ops::And => (a & b) as u16,
        Ops::Xor => (a ^ b) as u16,
        Ops::Mul => a as u16 * b as u16,
    }
}

fn main() {
    let op = Ops::Add;
    println!("{:?} of 0x0A and 0x05 -> 0x{:04x}", op,
alu_prediction(0x0A, 0x05, op));
}
```

```
--
Add of 0x0A and 0x05 -> 0x000f
```

Working through the new pieces:

The `derive` line. `#[derive(Clone, Copy, Debug, PartialEq)]` asks the compiler to generate standard capabilities: copying, comparison with `==`, and the `{:?}` debug printing you see in `main`. These are *traits*, and Chapter 10 gives them their due; for now, read the line as "make this type behave like a sensible value." IntEnum gave Python's `Ops` comparison and printing for free by inheritance; the derive line is where Rust's version of "for free" lives.

The discriminants. `Add = 1` assigns the variant an integer value, just as the IntEnum and the `typedef enum` did, and `Ops::Add as u8` recovers it when the opcode has to go onto a bus. But note which way the equivalence runs. An IntEnum member *is* an int — you can add three to `Ops.ADD` and Python will let you — and an SV enum converts to an integer whenever the expression around

it shrugs. `Ops::Add` is not a number that happens to have a name; it is a value of type `Ops`, and arithmetic on it is a compile error. The integer is available on request, one direction only. (Going the other way — from a raw integer read off a bus back to an `Ops` — takes explicit code that must confront the possibility of an illegal opcode. Chapter 9's `Result` is built for exactly that confrontation.)

The `match`. Chapter 4 introduced `match` as the load-bearing construct; here is the load. A `match` on an enum must be *exhaustive*: every variant handled, or the code does not compile. The prediction function cannot silently do nothing for `Mul` the way a Python `if/elif` chain with no `else` can.

That guarantee sounds abstract until the day it saves you. Suppose the TinyALU grows a subtract instruction. Add `SUB = 5` to the IntEnum or the `typedef enum`, and every `if/elif` chain and every `case` over ops in the testbench — the prediction function, the coverage model, the driver — keeps running, silently wrong, until a failing test (or worse, a passing one) sends you hunting. Watch what happens in Rust the moment we add the variant and change nothing else:

```
// Figure 8: The compiler finds every match the new variant breaks

#[derive(Clone, Copy, Debug, PartialEq)]
enum Ops {
    Add = 1,
    And = 2,
    Xor = 3,
    Mul = 4,
    Sub = 5,   // the new operation — and the only edit we made
}
```

```
--
error[E0004]: non-exhaustive patterns: `Ops::Sub` not covered
  --> src/main.rs:11:11
   |
11 |     match op {
   |           ^^ pattern `Ops::Sub` not covered
   |
note: `Ops` defined here
   = note: the matched value is of type `Ops`
help: ensure that all possible cases are being handled by
      adding a match arm with an explicit pattern
```

We changed the specification — one line — and the compiler produced a complete list of every place in the testbench that has not yet heard the news, before any simulator license was checked out, before any simulation ran, before any test could pass for the wrong reason. In a runtime-checked flow, this bug costs a simulation run at minimum and a shipped escape at maximum. Here it costs one compile, a few seconds, and it *cannot* be skipped.[2]

[2] There is an escape hatch — a `_ => ...` wildcard arm matches everything not yet named, and using one forfeits this protection. The idiom this book follows: never use a wildcard when matching an enum you own. Spend the extra lines; they are the tripwire.

Four states, no integers: `Logic`

SystemVerilog carries `x` and `z` natively — that is what `logic` is for. Python does not: every value in a cocotb testbench was ultimately an integer, and `get_int()` existed precisely to coerce simulator values that might contain `x` or `z` into something Python could compute with. Rust sides with SystemVerilog — hardware signals are not integers — and lets us say so directly:

```
// Figure 9: A four-state Logic enum

#[derive(Clone, Copy, Debug, PartialEq)]
enum Logic {
    Zero,
    One,
    X,
    Z,
}

fn to_char(v: Logic) -> char {
    match v {
        Logic::Zero => '0',
        Logic::One => '1',
        Logic::X => 'x',
        Logic::Z => 'z',
    }
}

fn main() {
    let bit = Logic::X;
    println!("The signal reads: {}", to_char(bit));
}
```

```
--
The signal reads: x
```

`Logic` is not a teaching toy: when we reach rustdv-sim in Chapter 17, reading a signal hands you a `LogicArray` — a vector of exactly this type — ported from the same four-state value types cocotb defines. Notice what the enum buys us that an IntEnum encoding (say, `X = 2`) never could — and that even SystemVerilog's native `logic` does not: there is no integer pretense to leak. In SV, an `x` rides along in arithmetic as x-propagation and surfaces downstream as a mystery; here, nothing can accidentally add `X` to a running sum, because `X` is not a number — it is one of four states a wire can be in, and any code that consumes a `Logic` must, thanks to exhaustive `match`, say what it does about `x` and `z`. The "forgot to handle the unknown state" bug is unrepresentable.

Variants that carry data

Everything so far, an IntEnum could at least gesture at. This last capability it could not. Rust enum variants can *carry data* — different data per variant — which makes an enum a type that says "this value is exactly one of the following shapes." Computer scientists call this a *sum type*; testbench authors will call it the right way to model outcomes:

```
// Figure 10: An enum whose variants carry payloads

#[derive(Debug)]
enum CheckResult {
    Pass,
    Fail { expected: u16, actual: u16 },
}

fn check(expected: u16, actual: u16) -> CheckResult {
    if expected == actual {
        CheckResult::Pass
    } else {
        CheckResult::Fail { expected, actual }
    }
}

fn main() {
    let result = check(0x000f, 0x0005);
    match result {
        CheckResult::Pass => println!("PASS"),
        CheckResult::Fail { expected, actual } => {
            println!("FAIL: expected 0x{expected:04x}, got
0x{actual:04x}")
        }
    }
}
```

```
--
FAIL: expected 0x000f, got 0x0005
```

Read the `match` arms closely, because a new thing is happening: the second arm doesn't just select the `Fail` variant, it *unpacks* it, binding `expected` and `actual` right there in the pattern. A passing check carries nothing; a failing check carries the evidence. In your old languages this was a boolean plus a couple of variables that are only meaningful when the boolean is false — a convention, enforced by hope. The enum makes the convention structural: the

payload *exists* only in the variant it belongs to, and the only way to touch it is to admit, in a pattern, which case you are in.

Once you see this shape, you will see it everywhere in Rust, because the standard library's two most important types are exactly this pattern: `Option<T>` is an enum whose variants are `Some(T)` and `None`, and `Result<T, E>` is an enum whose variants are `Ok(T)` and `Err(E)`. Absence and failure, modeled as payload-carrying enums, matched exhaustively — that is the entire story of how Rust replaced both `None`-checking and exceptions, and it gets Chapter 9 to itself.

Summary

Rust splits the class into parts and makes each part explicit. A **struct** declares its data completely — every field, every type, no after-the-fact attributes — and the compiler guarantees no instance ever exists with a field unset, retiring Python's `AttributeError` surprise and SystemVerilog's silently-defaulted field alike. An **`impl` block** attaches behavior: **methods** take `&self` to observe or `&mut self` to mutate, telling every caller which is which, while **associated functions** take no `self` and answer to the type name — `Animal::new(20.0)` — absorbing constructors, class methods, and static methods with one mechanism and zero magic.

Enums are the chapter's prize. `Ops` returns not as named integers but as a true sum type: exhaustively matched, so that adding a variant produces a compiler-generated to-do list of every site that must change — a testbench bug class eliminated before simulation. `Logic` models a wire's four states without pretending they are numbers. And payload-carrying variants let one type hold differently-shaped alternatives — the pattern behind `Option` and `Result`, and behind more testbench types to come.

Our transactions can now hold data and our enumerations can now hold their own. What we cannot yet do is hold *many* of anything — a queue of commands, a list of results, a scoreboard's worth of expectations. Chapter 8 takes up Rust's collections, where the Python sequences you know are waiting, with ownership along for the ride.

Chapter 8: Collections: Vec, String, and HashMap

In the UVM... we kept testbench data in whatever the language gave us. SystemVerilog gave us queues — `cmd_q[$]` with `push_back` and `pop_front` — plus dynamic arrays and associative arrays keyed by nearly anything. Python gave us lists ("the workhorse"), tuples, sets, and dictionaries that raise `KeyError` when a key is missing. Scoreboards were queues of expected transactions; coverage tallies were associative arrays or dicts of counts.

This chapter covers all of that ground at speed, and I want to be honest about why: most of what you know transfers directly. Rust has a growable list (`Vec<T>` — your list, your queue), a string type (`String`), and a hash-keyed store (`HashMap<K, V>` — your dict, your associative array). You index with square brackets, you slice with ranges, you loop with `for`, you check membership, you sort. If I walked through every operation, you would be bored and I would be padding.

So instead this chapter spends its words on the three places where your old instincts will actively mislead you:

1. A `Vec` *owns* its elements — pushing a value into it is a move, and Chapter 5 comes due.
2. Rust has *two* string types, `String` and `&str`, and until you know which is which, every function signature involving text will look like a typo.
3. Iterating over a collection can either *borrow* it or *consume* it, and the difference is one ampersand.

Everything else — the operations that work the way you expect — gets a fast tour at the end.

Vec: the list that owns its contents

A `Vec<T>` is Rust's growable array: the Python list and the SystemVerilog queue, one type doing both jobs, and every bit the workhorse they were. The first difference is visible in the type: a Python list holds *anything* (`['a', 3, LL]` was a perfectly good list), while a `Vec<T>` holds values of exactly one type `T` — the bargain SystemVerilog's typed queues always drove, and usually what a testbench wanted anyway. A history log of ALU transactions should hold transactions, all of them, and nothing else.

Let's build exactly that. Figure 1 creates a log of the `AluCommand` transactions we defined in Chapter 7 and pushes commands into it, the way a monitor might record everything it sees.[1]

```
// Figure 1: A Vec<AluCommand> as a transaction history log

#[derive(Clone, Copy, Debug, PartialEq)]
enum Ops { Add = 1, And = 2, Xor = 3, Mul = 4 }

#[derive(Clone, Debug, PartialEq)]
struct AluCommand { a: u8, b: u8, op: Ops }

fn main() {
    let mut log: Vec<AluCommand> = Vec::new();
    log.push(AluCommand { a: 5, b: 3, op: Ops::Add });
    log.push(AluCommand { a: 2, b: 2, op: Ops::Mul });

    println!("{} commands logged", log.len());
    println!("first: {:?}", log[0]);
}
```

```
--
2 commands logged
first: AluCommand { a: 5, b: 3, op: Add }
```

Familiar territory: `push` is `append` or `push_back`, `len()` is `len()` or `size()`, `log[0]` is `log[0]`. Note that `log` must be `let mut` — Chapter 3's immutability-by-default applies to collections with no exceptions, which means a testbench data structure cannot be quietly modified by code you didn't expect to modify it. Also note `Vec::new()` gave us an empty vector; the `vec!`

macro is the literal syntax, so `vec![1, 2, 3]` is Rust's `[1, 2, 3]` or `'{1, 2, 3}`.

Now the part that is new. A Python list — like every SystemVerilog queue of class handles — holds *references*. When you append a transaction, the container gets one more name for an object that still has all its other names; the monitor keeps its handle, the scoreboard keeps its handle, and the garbage collector sorts out the afterlife. A `Vec` holds *values*. When you push a transaction into a `Vec`, the transaction **moves** into the `Vec` — the vector becomes the owner, exactly as if you had assigned it to a new variable in Chapter 5. Figure 2 shows what happens when we forget.

```
// Figure 2: Pushing is a move

fn main() {
    let mut log: Vec<AluCommand> = Vec::new();
    let cmd = AluCommand { a: 5, b: 3, op: Ops::Add };
    log.push(cmd);
    println!("sent: {:?}", cmd);   // cmd moved into the Vec
}
```

```
--
error[E0382]: borrow of moved value: `cmd`
  --> src/main.rs:12:28
   |
10 |     let cmd = AluCommand { a: 5, b: 3, op: Ops::Add };
   |         --- move occurs because `cmd` has type `AluCommand`,
   |             which does not implement the `Copy` trait
11 |     log.push(cmd);
   |              --- value moved here
12 |     println!("sent: {:?}", cmd);
   |                            ^^^ value borrowed here after move
   |
help: consider cloning the value if the performance cost is
acceptable
   |
11 |     log.push(cmd.clone());
   |                 ++++++++
```

Read that error the way Chapter 2 taught you: the compiler names the move, points at the line where it happened, and offers the fix. If you want to log the command *and* keep using it, `push(cmd.clone())` makes the copy explicit — the choice your old containers made for you silently (share the reference) is now a decision you make on the line where it matters. If the log is the transaction's

final destination, push the value and let the `Vec` own it. This is the monitor-hands-a-transaction-to-the-scoreboard question from Chapter 5, answered by a container: *whoever holds the `Vec` owns everything in it*, and when the `Vec` is dropped, every transaction inside is dropped with it. No cycles, no GC, no doubt about who frees what.

One more `Vec` note before we move on: indexing past the end panics, where Python raised `IndexError` and SystemVerilog — brace yourself — returns a default value and keeps simulating. There is also `log.get(7)`, which neither panics nor invents data, but instead returns a value that is either something or nothing — a type called `Option` that keeps appearing in this chapter's corners and gets the full treatment in Chapter 9.

Iteration: borrow or consume, your choice

The `for` loop over a `Vec` looks exactly like Python — and hides the chapter's second lesson in a single character. Figure 3 loops over the log the obvious way and then tries to use it afterward.

```
// Figure 3: A for loop can consume the collection

fn main() {
    let log = vec![
        AluCommand { a: 5, b: 3, op: Ops::Add },
        AluCommand { a: 2, b: 2, op: Ops::Mul },
    ];
    for cmd in log {
        println!("{:?}", cmd);
    }
    println!("{} commands", log.len());   // log is gone
}
```

```
--
error[E0382]: borrow of moved value: `log`
  --> src/main.rs:14:29
   |
9  |     for cmd in log {
   |                --- `log` moved due to this implicit call
   |                    to `.into_iter()`
...
14 |     println!("{} commands", log.len());
   |                             ^^^ value borrowed here after move
   |
help: consider borrowing to avoid moving into the for loop
   |
9  |     for cmd in &log {
   |                +
```

`for cmd in log` *consumes* the vector: ownership of the whole `Vec` moves into the loop, each element moves into `cmd` in turn, and when the loop ends there is nothing left. That is occasionally exactly what you want — a scoreboard draining its queue at end of test, say. But most of the time you want Python's behavior, looking at the elements while leaving the collection intact, and the compiler's `help` line hands you the idiom: borrow it.

```
// Figure 4: Borrowing iteration leaves the Vec intact

fn main() {
    let log = vec![
        AluCommand { a: 5, b: 3, op: Ops::Add },
        AluCommand { a: 2, b: 2, op: Ops::Mul },
    ];
    for cmd in &log {
        println!("{:?}", cmd);
    }
    println!("{} commands still logged", log.len());
}
```

```
--
AluCommand { a: 5, b: 3, op: Add }
AluCommand { a: 2, b: 2, op: Mul }
2 commands still logged
```

With `for cmd in &log`, each `cmd` is a `&AluCommand` — a shared borrow, read-only, governed by Chapter 6's rules. The third form, `for cmd in &mut log`,

borrows each element mutably so you can edit in place. The whole story is three spellings:

- `for x in &v` — borrow each element; the loop reads. *This is your Python `for` and your SystemVerilog `foreach`.*
- `for x in &mut v` — mutably borrow each element; the loop edits.
- `for x in v` — take ownership of each element; the loop consumes, and `v` is gone.

Chapter 6's aliasing rule follows you into the loop body, and it quietly retires a bug both your languages know well: with `for cmd in &log` active, you cannot also `log.push(...)` inside the loop, because that would mutate a collection you are currently borrowing. Your old languages could only warn you never to modify a container while iterating over it; the borrow checker rejects the program.

String and &str: the two-string problem

Here is the single most confusing thing this chapter has to teach, so let's take it slowly. Python has one string type; SystemVerilog has one (plus an attic of packed-array conversions nobody enjoys). Rust, from where you sit, appears to have two, and every Rust learner spends a bewildered week discovering which functions want which.

The two types are:

- **`String`** — an owned, growable string, allocated on the heap. This is the closest thing to a mutable Python `str`-builder: you can push characters onto it, and whoever owns it is responsible for it, per Chapter 5.
- **`&str`** (pronounced "string slice") — a *borrowed view* of string data that lives somewhere else. It doesn't own anything; it points at characters and knows how many of them it covers. It is Chapter 6's `&T`, specialized for text.

The reason beginners meet the confusing one first: **every string literal is a `&str`.** When you write `"TinyALU"`, those seven bytes are baked into your compiled program itself, and the literal is a borrowed slice pointing into that

program memory. It costs nothing, it lives forever, and it is read-only. Figure 5 shows both types and the traffic between them.

```
// Figure 5: String literals are borrowed; String is owned

fn main() {
    let dut: &str = "TinyALU";            // borrowed slice into
program memory
    let mut name: String = String::from("Tiny");
    name.push_str("ALU");                 // owned and growable
    let banner = format!("*** Testing the {} ***", name);
    println!("{}", dut);
    println!("{}", banner);
}
```

```
--
TinyALU
*** Testing the TinyALU ***
```

`String::from("Tiny")` copies the literal's characters into a fresh, owned, heap-allocated `String` that we can grow with `push_str`. And `format!` is your f-string and your `$sformatf`: `format!("Testing the {}", name)` builds a new `String` the way `f"Testing the {name}"` and `$sformatf("Testing the %s", name)` did — same job, and like Python's strings, Rust's are immutable-by-default; growing one requires `mut`, and replacing text builds a new string rather than editing in place.[2]

Now the question that actually bites: when you write a function that takes text, which type goes in the signature? The rule of thumb is worth memorizing, because it appears throughout rustdv's own API:

> **Store `String`, pass `&str`.** Struct fields that keep text own it as `String`; function parameters that read text borrow it as `&str`.

The reasoning is pure Chapter 6. A function that only *reads* a name has no business demanding ownership of it — that would force every caller to give up (or clone) their string just so you could look at it. A parameter of type `&str` says "lend me a view," and it is maximally accepting: a `&String` coerces to a

`&str` automatically, so callers can pass a literal, a slice, or a borrowed `String`, all with no copying. Figure 6 shows the shape.

```
// Figure 6: Take &str; accept everything

fn report_pass(test_name: &str) {
    println!("PASSED: {}", test_name);
}

fn main() {
    let owned = String::from("random_ops");
    report_pass("smoke_test");    // a literal is already a &str
    report_pass(&owned);          // a &String coerces to &str
    println!("still have {}", owned);
}
```

```
--
PASSED: smoke_test
PASSED: random_ops
still have random_ops
```

If instead the function is going to *keep* the text — storing a component's name in a struct field, say — it should take a `String` and own it outright, and the move at the call site documents the handoff. When rustdv asks for `&str` in a signature, it is promising to look and not keep; when it asks for `String`, it is telling you the name is moving in permanently.

One habit does not survive the crossing at all: indexing into a string. `line[45]` was everyday Python (and legal SV); in Rust, `s[0]` on a `String` does not compile. Rust strings are UTF-8 encoded, so a character can occupy anywhere from one to four bytes, and Rust refuses to guess whether you want the byte or the character.[3] When you need the characters, say so — `for ch in s.chars()` iterates over them — and methods like `split`, `trim` (Python's `strip`), `replace`, and `contains` cover the daily string chores you already know by name.

HashMap: the dictionary, minus the ordering promise

`HashMap<K, V>` is the Python dict and the SystemVerilog associative array: store a value under a key, get it back later. It lives in the standard library's collections module rather than the language itself, so it needs an import — your first `use` statement doing real work.

Python's classic dict demonstration counts the letters in `"Mississippi"`, handling the first appearance of each letter with `KeyError` and then `setdefault`. Ours counts something a verification engineer actually tallies: how many times the testbench has exercised each ALU operation — the raw material of functional coverage. Rust's replacement for the whole first-time-key dance is the *entry API*, and it is one line.

```
// Figure 7: A HashMap<Ops, u32> op-frequency counter

use std::collections::HashMap;

#[derive(Clone, Copy, Debug, PartialEq, Eq, Hash)]
enum Ops { Add = 1, And = 2, Xor = 3, Mul = 4 }

fn main() {
    let op_stream = vec![Ops::Add, Ops::Mul, Ops::Add,
                         Ops::Xor, Ops::Add, Ops::Mul];
    let mut freq: HashMap<Ops, u32> = HashMap::new();
    for op in &op_stream {
        *freq.entry(*op).or_insert(0) += 1;
    }
    for (op, count) in &freq {
        println!("{:?}: {}", op, count);
    }
}
```

```
--
Mul: 2
Xor: 1
Add: 3
```

Three things to unpack. First, the derive line grew: a type used as a `HashMap` key must support hashing and full equality, so `Ops` now derives `Eq` and `Hash` alongside the derives from Chapter 7. In Python, hashability was a runtime

property you discovered when a `dict` rejected your key; in Rust it is a capability you declare on the type, and using a non-hashable key is a compile error. (What deriving really does, and the family of traits behind it, is Chapter 10's story.)

Second, the entry line: `freq.entry(*op).or_insert(0)` means "find this key's slot, filling it with 0 if it's empty, and hand me access to the value" — Python's `setdefault`, near-verbatim, with the leading `*` saying "increment the value the entry points at," Chapter 6's dereference doing honest work. For simple lookup, `freq.get(&Ops::Add)` plays the role of Python's `dict.get()`: it returns that same something-or-nothing `Option` type that `Vec::get` gave us — Chapter 9 is circling closer.

Third — and this one is a genuine behavioral difference, worth a flag in your notes — **look at the output order.** We inserted Add first; the printout led with Mul. Python dicts (since 3.7) return keys in insertion order; SystemVerilog associative arrays iterate in sorted key order; if you have written either, you have leaned on the habit. `HashMap` makes *no* ordering promise: iteration order is arbitrary, and it is deliberately randomized from run to run, so the same program can print `Add, Mul, Xor` today and `Xor, Add, Mul` tomorrow.[4] Any testbench logic that quietly depends on iteration order — golden log files compared line-by-line are the classic offender — must either sort the keys before printing or use `BTreeMap`, `HashMap`'s sibling that keeps keys sorted (the associative-array behavior) at a small cost. Iterating `for (op, count) in &freq` borrows, exactly per this chapter's rules, and destructures each key-value pair into two variables the way Python's `.items()` did.

The rest of the toolbox, briefly

Here is the promised fast tour of what transfers without drama, each item in a sentence or two.

Tuples exist and you have already used them: `let pair = (5, Ops::Add);` groups values of different types, `pair.0` indexes them, and `let (a, op) = pair;` destructures — so functions return multiple values exactly as they did in Python. **Ranges** you met in Chapter 4: `0..5` is `range(5)`, `0..=4` includes the

end. **Sets** are `HashSet<T>`, with `insert`, `contains`, and the union/intersection/difference operations — and the same no-ordering caveat as `HashMap`, which shares its machinery.

Of the common sequence operations: `v.len()` is `len(v)`; `v.contains(&x)` is `x in v` or an SV `inside`; `v.sort()` sorts in place like `list.sort()` and `q.sort()`, with `sort_by_key` playing the `key=` / `with` role; `v.iter().max()` and `.min()` return — you can guess by now — an `Option`, because the vector might be empty, a case Python handled by raising `ValueError`. Slicing carries over almost keystroke-for-keystroke: `&log[1..3]` is Python's `log[1:3]`, a borrowed view of elements one and two, and the same range syntax slices strings and arrays. The borrowed-ness is the Rust twist: a slice is a reference into the original, so Chapter 6's rules apply while you hold it. What you will *not* find are `+` and `*` on vectors; Rust spells concatenation and repetition through methods (`extend`, `concat`, `"–".repeat(15)` for a horizontal bar). And list comprehensions have no literal syntax — their job is done by iterator chains like `map` and `filter`, which are Chapter 12's whole subject and worth the wait.

Summary

Rust's collections are your old collections with ownership made visible. A `Vec<T>` is the list and the queue, but it owns its elements: pushing a transaction moves it in, dropping the `Vec` drops everything inside, and the who-frees-this question of Chapter 5 has a container-sized answer. Iteration comes in three spellings — `&v` borrows, `&mut v` borrows mutably, bare `v` consumes — and the borrow checker enforces the never-mutate-while-iterating rule your old languages could only put in a warning box. Text comes in two types: owned, growable `String` and borrowed `&str`, with string literals being `&str` slices into your program's own memory, and the rule of thumb *store* `String`, *pass* `&str`. `HashMap<K, V>` is the dict and the associative array, with the entry API replacing the first-time-key dance and no iteration-order promise — sort your keys before comparing log files, or reach for `BTreeMap`. Tuples, ranges, sets, slices, sorting, and membership tests all transfer nearly unchanged.

Along the way, one type kept appearing in the corners of figures and refusing to explain itself: `Vec::get`, `HashMap::get`, `max`, and `min` all returned an `Option`, Rust's way of saying "there might be nothing here" — in the type, where the compiler can see it. Python answered the same question with `None` checks and `KeyError`; SystemVerilog answered with a default value and a note in the log you never read. Rust's answer is better than both, and it comes with a partner named `Result` that replaces exceptions entirely. That is Chapter 9.

---

[1] Standalone playground project, per our Part I convention: `cargo new collections` and everything in this chapter runs in `src/main.rs`. And yes, `vec!` has the exclamation point of a macro, like `println!` — Chapter 21 explains why; until then, enjoy the enthusiasm.

[2] Python proves `str` immutability by showing `replace()` returning a new object with a new `id()`. Rust's `s.replace("real", "mall")` likewise returns a fresh `String` and leaves the original alone — same lesson, no `id()` required, because ownership tells you there are two strings.

[3] The moment this stops feeling like pedantry is the moment a signal name, a file path, or a log message contains its first non-ASCII character. Python 2 programmers can tell you stories; Rust decided at birth never to be in those stories.

[4] The randomization is a deliberate defense — a hash table with predictable layout invites pathological (even malicious) key patterns — but the practical takeaway for us is simpler: if your test passes or fails depending on `HashMap` iteration order, the bug is in the test.

Next: Chapter 9, where `Option` finally introduces itself properly, `Result` retires the exception, and the `?` operator makes honest error handling almost as terse as ignoring errors used to be.

Chapter 9: Result, Option, and the End of Exceptions

> **In the UVM…** error handling depended on the dialect. Python reported failures with exceptions — a workplace metaphor explains them: hit an error, raise it to your boss, who raises it to their boss, up the chain until somebody handles it or it reaches the public as a crash with a traceback. We caught them with `try` / `except` / `finally`. SystemVerilog never had exceptions: a function that failed returned a sentinel value, logged a `uvm_error`, or `$fatal`ed the simulation outright.

Rust does not have exceptions. Not "discourages them," not "has them but calls them something else" — the mechanism does not exist. There is no `raise`, no `try`, no `except`, and nothing that silently unwinds through your function while it's minding its own business. SystemVerilog engineers may feel at home here — but hold the feeling, because SV's alternative was sentinel values nothing forced you to check and log messages nothing forced you to read. Rust's alternative has teeth.

And before Python readers mourn, remember what the boss metaphor was papering over. In Python, when you call a function, *nothing about that function tells you it might raise*. `nice_div()` looks exactly like a function that always returns a number. The fact that it can instead fling a `ZeroDivisionError` through your code is invisible — undocumented control flow that you discover at runtime, in our world usually forty minutes into a simulation.

Rust's answer is almost embarrassingly simple: **errors are ordinary values, and functions that can fail say so in their return type.** Two enums from the standard library do all the work:

- `Option<T>` — a value that might not be there.
- `Result<T, E>` — an operation that might fail, and if so, why.

You already have the tool for consuming both: `match`, from Chapter 4. This chapter is `match` earning its keep.

Option: a value that might not be there

Python has one value for "nothing here": `None`. It shows up as a return value (`players.get(4)` returns `None` when number 4 isn't in the dictionary), as a default, and as a sentinel. And it carries a famous failure mode: `None` is a perfectly good Python object right up until you use it like the thing you expected, at which point you get an `AttributeError` — often far from the line that produced the `None`, which is what makes it such a satisfying bug to chase.[1] SystemVerilog's version is `null`: a handle that types like the real thing and detonates mid-simulation the first time anyone dereferences it.

Rust refuses to let "maybe nothing" hide inside an ordinary type. A `u8` is always a number; a `String` is always a string. When a value might be absent, the type says so:

```
enum Option<T> {
    Some(T),
    None,
}
```

This is just an enum with a payload, exactly like the ones you built in Chapter 7 — it happens to be defined in the standard library and generic over any type `T` (generics get their full treatment in Chapter 11; for now, read `Option<u8>` as "maybe a `u8`").

You met `Option` briefly in Chapter 8, because `HashMap` lookups return one. In Python, `players.get(4)` returns `None` for a missing key, and `players.get(4, "Not in database")` supplies a default. Figure 1 is the same lookup in Rust.

```rust
// Figure 1: A HashMap lookup returns Option

use std::collections::HashMap;

fn main() {
    let mut players = HashMap::new();
    players.insert(7, "Beckham");
    players.insert(10, "Messi");
    players.insert(11, "Salah");

    match players.get(&4) {
        Some(name) => println!("Number 4? {name}"),
        None => println!("Number 4? Not in database"),
    }

    let player = players.get(&4).copied().unwrap_or("Not in
database");
    println!("Number 4? {player}");
}
```

```
--
Number 4? Not in database
Number 4? Not in database
```

The two halves of figure 1 do the same job two ways. The `match` is the fundamental move: an `Option` is an enum, so we take it apart with `match`, and the compiler *requires* both arms. You cannot forget the `None` case — leaving it out is a compile error, not a 2 a.m. discovery. The second half uses `unwrap_or`, one of a family of convenience methods on `Option` that package up the common matches; it is the exact analog of passing `get()` a default in Python.

Here is the part worth slowing down for. In your old languages, the nothing-value from a failed lookup is the same shape as any other value — you can pass a `None` or a `null` along, store it in a transaction field, and only find out three function calls later. In Rust, an `Option<&str>` *is not* a `&str`. You cannot print it as a name, compare it to a name, or hand it to a function expecting a name. The compiler stops you at the line where you forgot to handle absence — which is to say, the far-from-the-bug failure mode — Python's `AttributeError`, SystemVerilog's null-handle crash — is not merely discouraged. It doesn't compile.

When you only care about one arm, `if let` from Chapter 4 reads better than a `match` with an empty arm:

```
if let Some(name) = players.get(&10) {
    println!("Found: {name}");
}
```

Result: an operation that can fail

`Option` says "there might be nothing." `Result` says "this might fail, and here is why":

```
enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

`T` is the type you get on success; `E` is the error type you get instead. Let's walk the classic failure scenarios, starting with the classic: dividing by zero.

```
// Figure 2: You still can't divide by zero

fn main() {
    let divisor: i32 = "0".parse().unwrap();
    println!("3/0 = {}", 3 / divisor);
}
```

```
--
thread 'main' panicked at src/main.rs:3:26:
attempt to divide by zero
note: run with `RUST_BACKTRACE=1` environment variable to display a
backtrace
```

A word about the `parse`: it is there because it has to be. Write `let divisor = 0;` and rustc, able to *see* the zero, refuses to compile the division at all. Only a divisor the compiler cannot predict — one arriving at runtime, as bus values do — gets the chance to panic.

That is a **panic** — Rust's crash. We will come back to panics later in the chapter, because they have a specific job, and "report a divide by zero to the caller" is not it. When the possibility of failure is part of a function's honest contract, the function should return it. The standard library agrees: alongside the `/` operator, integers provide `checked_div`, which returns an `Option` — `None` instead of a crash.[2]

A Python `nice_div()` catches `ZeroDivisionError` and returns `math.inf`. Our integer version can't return infinity, but it can do something better: return a `Result` that names what went wrong. Figure 3 is `nice_div`, Rust edition.

```
// Figure 3: nice_div returns a Result instead of raising

#[derive(Debug)]
enum DivError {
    DivideByZero,
}

fn nice_div(dividend: u32, divisor: u32) -> Result<u32, DivError> {
    match dividend.checked_div(divisor) {
        Some(result) => Ok(result),
        None => Err(DivError::DivideByZero),
    }
}

fn main() {
    match nice_div(33, 2) {
        Ok(result) => println!("nice_div(33, 2) = {result}"),
        Err(err) => println!("You screwed up your division, human:
{err:?}"),
    }
    match nice_div(3, 0) {
        Ok(result) => println!("nice_div(3, 0) = {result}"),
        Err(err) => println!("You screwed up your division, human:
{err:?}"),
    }
}
```

```
--
nice_div(33, 2) = 16
You screwed up your division, human: DivideByZero
```

Read the signature first: `fn nice_div(dividend: u32, divisor: u32) -> Result<u32, DivError>`. That return type is the whole philosophy in one line. In Python, `nice_div`'s ability to fail was a secret between the function body and

whoever read the documentation; in SystemVerilog, it was a status flag you were free to ignore. In Rust it is in the signature, which means the *compiler* knows, which means every caller is forced to acknowledge it. The `match` in `main` is our `except` block — except it cannot be forgotten, because you cannot get the `u32` out of a `Result<u32, DivError>` without going through it.

Notice also who's who in the port: the `Err` arm of the `match` is playing the role of Python's `except ZeroDivisionError` block, and constructing `Err(DivError::DivideByZero)` is playing the role of `raise`. Same drama, but now the error travels as a return value, in plain sight.

The exception that vanished

Python's next classic scenario is `nice_div(3, "zero")` — dividing an `int` by a `str`, which raises a `TypeError`, which requires a second `except` block to catch. Figure 4 ports that call to Rust.

```rust
// Figure 4: The TypeError scenario, ported to Rust

fn main() {
    match nice_div(3, "zero") {
        Ok(result) => println!("nice_div = {result}"),
        Err(err) => println!("Error: {err:?}"),
    }
}
```

```
--
error[E0308]: mismatched types
 --> src/main.rs:4:24
  |
4 |     match nice_div(3, "zero") {
  |           -------- ^^^^^^ expected `u32`, found `&str`
  |           |
  |           arguments to this function are incorrect
```

It doesn't compile. Of the two failure categories Python needed `except` blocks for, one has simply left the building: passing the wrong *type* is not a runtime error to be caught, it is a program the compiler refuses to build. The uncaught `TypeError`, the second `except` block, catching two exception types on one line

— none of it has a Rust equivalent, because the bug it handles cannot occur. Keep a tally of moments like this; the book has several more coming.[3]

The `?` operator: propagation you can see

Back to the boss metaphor. The good idea inside exceptions was *delegation*: a low-level function shouldn't have to decide what a divide-by-zero means for the whole program. It should hand the problem upward. Python's `raise` did that invisibly. Rust does it with one visible character.

Suppose we're writing a little TinyALU-flavored utility: compute a ratio of two accumulated counts as a percentage. It calls `nice_div`, and if the division fails, our function can't succeed either — the error should go up to *our* caller. Written longhand, that's a `match` where the `Err` arm just re-returns the error. Written idiomatically, it's figure 5.

```
// Figure 5: The ? operator sends the error up the stack

fn percent(numerator: u32, denominator: u32) -> Result<u32,
DivError> {
    let ratio = nice_div(numerator * 100, denominator)?;
    Ok(ratio)
}

fn main() {
    match percent(40, 0) {
        Ok(pct) => println!("{pct}%"),
        Err(err) => println!("percent failed: {err:?}"),
    }
}
```

```
--
percent failed: DivideByZero
```

The `?` after `nice_div(...)` means: *if this is `Ok(value)`, unwrap it and keep going; if it is `Err(e)`, return `Err(e)` from this function, right now.* That is exception propagation — the error bubbles up the call stack, each function passing it to its boss — with two differences that change everything:

1. **It's visible at the call site.** Every fallible call in a Rust function is marked with a `?` (or an explicit `match`). Scanning a function body tells you exactly where it can bail out early. A Python function body gives you no such list; any line might throw.
2. **It's visible in the signature.** You can only use `?` in a function that itself returns a `Result` (the compiler enforces this, with a helpful error if you forget). So the ability to fail is contagious *in the types*: if `percent` propagates `nice_div`'s errors, then `percent`'s signature says `Result`, and its callers are on notice too. The chain of bosses is written down.

Python's re-raise pattern — catch an exception, print a snarky message, then `raise` it onward — becomes a `match` (or an `inspect_err` call) that logs and then returns the `Err`. Nothing new to learn, just values. And when the error types along the chain differ, `?` will convert between them automatically if you've told it how; that hook is a trait called `From`, and traits are the very next chapter, so we'll leave that thread hanging deliberately.

One more nicety: `main` itself can return a `Result`. When it returns an `Err`, the program prints the error and exits with a failing status — the last boss in the chain has a sensible default. In Part II you'll see that rustdv tests work the same way: a test is an `async fn` returning `Result<(), TestError>`, and an `Err` fails the test. The `?` operators sprinkled through a testbench are little arrows pointing at everything that can end the test.

Designing an error enum

`DivError` had one variant, which made it a demo. Real error types earn their keep when there are several ways to fail and the caller might care which one happened — exactly the job Python's exception *class hierarchy* did, with `except ZeroDivisionError` and `except TypeError` selecting by type. In Rust, the error type is an enum, the variants are the failure modes, payloads carry the evidence, and `match` does the selecting.

Let's build one the TinyALU will actually need. Think ahead to a testbench utility that decodes an operation field from the DUT into our `Ops` enum from Chapter 7. Two things can go wrong: the bits might not encode any legal

operation, or the DUT might never hand us the value at all before the clock runs out. That's a two-variant enum, one variant carrying the offending byte:

```
// Figure 6: A custom error enum for the TinyALU

use std::fmt;

#[derive(Debug, Clone, Copy, PartialEq)]
enum AluError {
    InvalidOp(u8),
    Timeout,
}

impl fmt::Display for AluError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            AluError::InvalidOp(bits) => {
                write!(f, "invalid ALU op code: {bits:#04x}")
            }
            AluError::Timeout => write!(f, "timed out waiting for
the DUT"),
        }
    }
}

fn decode_op(bits: u8) -> Result<Ops, AluError> {
    match bits {
        1 => Ok(Ops::Add),
        2 => Ok(Ops::And),
        3 => Ok(Ops::Xor),
        4 => Ok(Ops::Mul),
        _ => Err(AluError::InvalidOp(bits)),
    }
}

fn main() {
    for bits in [1, 4, 7] {
        match decode_op(bits) {
            Ok(op) => println!("{bits} decodes to {op:?}"),
            Err(err) => println!("decode failed: {err}"),
        }
    }
}
```

```
--
1 decodes to Add
4 decodes to Mul
decode failed: invalid ALU op code: 0x07
```

Walk through what each piece buys:

- `#[derive(Debug, ...)]` gives us the `{err:?}` developer-facing printout for free — the same derive habit from Chapters 7 and 8.
- **The `Display` implementation** is the human-facing message, the counterpart of the string you'd pass when raising a Python exception or logging a `uvm_error` (`raise ValueError(f"invalid op: {bits}")`). Writing `impl fmt::Display for AluError` is our first hand-written trait implementation; Chapter 10 will explain the machinery you just used. For now: implementing `Display` is what makes `{err}` (no `:?`) work, and it's the conventional courtesy every public error type pays.
- **The payload on `InvalidOp(u8)`** carries the evidence to wherever the error is handled. Python attached this data to the exception object; we attach it to the variant. No fishing it back out of a message string.
- **Callers select with `match`.** A caller who wants to retry on `Timeout` but fail hard on `InvalidOp` writes a two-arm match — the moral equivalent of two `except` blocks, checked for exhaustiveness by the compiler. Add a third variant to `AluError` next month, and every such `match` in the codebase becomes a compile error until it says what to do about the new case. Try getting *that* from an exception hierarchy.

This pattern — an enum of failure modes, `Debug` derived, `Display` implemented, payloads where useful — is the whole craft of error design in application code, and it's the shape rustdv's own errors take (`HandleError` for signal lookups, `TestError` for test outcomes — you'll meet them in Chapter 17).

panic!: for bugs, not for failures

Now we can return to figure 2's crash. `panic!` is Rust's mechanism for *this program has a bug*: it prints a message and location, unwinds the current thread, and by default takes the program down. You can invoke it yourself:

```
panic!("driver state machine reached an impossible state");
```

The panicking family has members you will use daily:

- **`assert!(condition, "message...")`** — panic if the condition is false. `assert_eq!(a, b)` panics if two values differ, and prints both. SystemVerilog engineers know immediate assertions well; the difference is disposition — an SV assertion failure prints and, by default, the simulation soldiers on, while a Rust `assert!` stops the program at the scene.
- **`.unwrap()`** — on an `Option` or `Result`: give me the value, and panic if it's `None` / `Err`.
- **`.expect("message")`** — `unwrap` with a message you choose, which makes it strictly better in code you keep.[4]

Figure 7 puts `assert!` to work in a checksum helper, `xor_bytes`. Rust sharpens the classic example: `xor_bytes` takes `&[u8]`, so a value over 255 can't even reach the function (the `TypeError` disappearance, again). What still *can* go wrong is a claim about our own logic — say, this version that folds in a parity check the surrounding testbench relies on:

```
// Figure 7: assert! guards an invariant

fn xor_bytes(bytes: &[u8]) -> u8 {
    let mut xor = 0;
    for b in bytes {
        xor ^= b;
    }
    xor
}

fn main() {
    let frame = [0x08, 0x09, 0x10];
    let checksum = xor_bytes(&frame);
    println!("checksum: {checksum:#04x}");
    assert!(
        xor_bytes(&[checksum, 0x11]) == 0,
        "checksum self-test failed: {checksum:#04x} ^ 0x11 != 0"
    );
    assert!(
        xor_bytes(&[checksum, 0x12]) == 0,
        "checksum self-test failed: {checksum:#04x} ^ 0x12 != 0"
    );
    println!("all self-tests passed");
}
```

```
--
checksum: 0x11
thread 'main' panicked at src/main.rs:16:5:
checksum self-test failed: 0x11 ^ 0x12 != 0
note: run with `RUST_BACKTRACE=1` environment variable to display a
backtrace
```

The first assertion passes silently; the second one stops the program at the line where the impossible happened, with our message. That is what you want from an invariant check: loud, early, and located.

Python has a famous `assert` trap: `assert (8 < 7, "Obviously false")` never fires, because the parentheses build a two-element tuple, and a non-empty tuple is truthy. I am pleased to report the Rust translation of that bug: `assert!((8 < 7, "Obviously false"))` does not compile, because a tuple is not a `bool`, and `assert!` insists on a `bool`. The trap is not merely avoided; it is unrepresentable.

So when do you panic and when do you return `Err`? The line is intent:

- **`Result::Err` is for failures the design expects.** A signal that might not exist, an operand that might be out of range, a check that might not pass. These are outcomes, and the caller gets to decide what they mean.
- **`panic!` / `assert!` are for states the design promises are impossible.** If one fires, the code — not the input, not the DUT — is wrong, and there is no sensible way to continue.

`.unwrap()` and `.expect()` sit exactly on this line, which is why they need judgment: each one converts an `Err` / `None` into a panic, so each one is a small signed statement that says "I claim this cannot fail here." In playground code and examples, unwrap freely. In a testbench you'll run for a year, prefer `expect` with a message that will make sense to whoever reads the panic — probably you, later, in a worse mood.

One Python comfort has no direct Rust twin: `finally`. Rust's guarantee of "this cleanup runs no matter what" doesn't live in error-handling syntax at all — it lives in ownership. When a scope ends, by `return`, by `?`, or by panic-unwinding, values are dropped and their destructors run, as you saw in Chapter 5. Cleanup-on-any-exit is not something you remember to write in Rust; it's where the `Drop` happens.

The taxonomy this book will live by

Everything above compresses into one convention, and it is load-bearing: the rest of this book — and the design of rustdv itself — assumes it.

> **The failure taxonomy. `Result::Err` is for checks** — the DUT did something wrong. A scoreboard comparing predicted against actual and finding a mismatch produces an `Err`. The test fails, which is the test doing its job. This is *expected fallibility*: finding these is why we come to work. **`panic!` / `assert!` are for testbench bugs** — *we* did something wrong. A driver calling `item_done` twice, a queue that is empty when the protocol guarantees it can't be, a state machine in a state the match arms say is impossible. The testbench is broken, and no result it reports can be trusted until it's fixed.

Both fail the test — rustdv catches panics at the task boundary and scores them as failures, just as cocotb caught a stray exception in any task — but the report distinguishes them, because the reader of the report must react differently: an `Err` sends you to the waveform viewer; a panic sends you to your own source code. It is the difference between the lab reporting that the chip failed and the lab reporting that the thermometer is broken.

Both of your languages blurred this line. In Python, an `AssertionError` from a scoreboard check and an `AttributeError` from a testbench typo were just exceptions rising through the same machinery, distinguished by reading the traceback. In SystemVerilog, `uvm_error` for DUT misbehavior versus `uvm_fatal` for testbench disasters was the right convention — but it was only a convention, nothing enforced it, and the simulation ran on either way until somebody read the log. Rust gives the two categories different mechanisms, different types, and different syntax — so from Chapter 18 on, when you see a scoreboard whose check returns `Result` and a driver studded with `assert!`, you are seeing this chapter's taxonomy at work, not a stylistic accident.

You now hold both halves of Rust's honesty policy: types that admit absence (`Option`), and signatures that admit failure (`Result`). Along the way, you implemented your first trait — that `Display` on `AluError` — mostly on faith. Chapter 10 replaces the faith with understanding: traits are how Rust does

everything Python did with inheritance and dunder methods, and they are the last big idea between you and real testbench components.

[1] In Python, `players.get(4)` returns `None` for a missing player. The bug arrives when you then write `player.upper()` — and Python tells you `'NoneType' object has no attribute 'upper'`, three files away from the lookup.

[2] Python's `nice_div` returned `math.inf` for division by zero, on the theory that it's the mathematically correct answer. Rust's *floating-point* division agrees — `3.0 / 0.0` is `inf`, per IEEE 754, no panic — so on this one point the languages are in complete accord and only the integers object.

[3] Chapter 27 is essentially this moment stretched to a full chapter: every config-database failure mode the UVM ever taught you to debug, replayed as a compile error.

[4] A colleague of mine calls `.unwrap()` "a panic with no commit message."

Chapter 10: Traits

This chapter takes on the biggest piece of machinery yet: object-oriented programming itself. The UVM is built on inheritance in every language it speaks — `uvm_driver` extends `uvm_component`, which extends `uvm_object`, and every constructor dutifully calls its parent's, `super.new()` or `super().__init__()`, up the chain. Both earlier books spent multiple chapters on that machinery, because the methodology cannot run without it.

Rust has no inheritance. None. There is no base class, no child class, no `super`, no method resolution order to print. When I first learned this I assumed Rust must be missing something essential, and I was wrong in an instructive way: inheritance was never the *goal*. It was the mechanism every UVM language shared for two separate goals — sharing behavior across types, and treating different types uniformly. Rust delivers both with a single feature called a **trait**, and once you see the two goals pulled apart, you may find you don't miss the mechanism.

> **In the UVM...** we used inheritance to avoid copying code: a base class holds the shared behavior, a child extends it — `class lion extends animal` (SV) or `class Lion(Animal)` (Python) — and overrides what differs. A `virtual` method lets the child's override win even through a base-class handle, and every constructor's first duty is to invoke its parent's — `super.new(name, parent)`, `super().__init__(name, parent)` — a discipline the UVM enforces by breaking at runtime when you forget.

This chapter rebuilds that material piece by piece: the animal menagerie, the `super` question, multiple inheritance, and then the payoff for verification — how traits replace the machinery every transaction class has always needed: `do_compare()` and `convert2string()` in SystemVerilog, `__eq__` and `__str__` in Python.

Shared behavior without a base class

A trait is a named list of method signatures that a type can opt into. It declares *what* a type can do; a separate `impl` block declares that a particular type does it. Teaching OOP with an animal menagerie is a tradition in this series — the Primer's lion said Roar, the Python book's dog said bow bow — and Rust will not be the book that breaks it.

```
// Figure 1: Shared behavior through a trait, not a base class

trait Animal {
    fn species(&self) -> &str;
    fn sound(&self) -> &str;

    fn make_sound(&self) {
        println!("The {} says '{}'", self.species(), self.sound());
    }
}

struct Dog;
struct Cat;

impl Animal for Dog {
    fn species(&self) -> &str { "dog" }
    fn sound(&self) -> &str { "bow bow" }
}

impl Animal for Cat {
    fn species(&self) -> &str { "cat" }
    fn sound(&self) -> &str { "miāo" }
}

fn main() {
    Dog.make_sound();
    Cat.make_sound();
}
```

```
--
The dog says 'bow bow'
The cat says 'miāo'
```

Read the trait from the bottom up. `species()` and `sound()` are *required* methods — they have signatures but no bodies, so any type implementing `Animal` must provide them. If "a method with a signature but no body, which every child must provide" sounds familiar, it should: it is SystemVerilog's `pure`

`virtual function` in an abstract class, except that a trait's requirements are checked wherever the `impl` is compiled, not when a class finally dares to extend the abstract base. `make_sound()` has a body, which makes it a **default method**: implementors get it for free and may override it. The shared behavior a base class used to hold lives in the default method; the per-type differences the constructors used to hold live in each `impl` block.

Notice what is *not* here. `Dog` does not mention `Animal` in its definition — `struct Dog;` stands alone, and the relationship is declared separately, in `impl Animal for Dog`. `class Dog extends Animal` in SystemVerilog, like `class Dog(Animal)` in Python, fused "what Dog is" with "what Dog can do" into one statement. Rust keeps them apart, and the separation has a consequence you will come to rely on: you can implement a new trait for an existing type without touching the type's definition. When Chapter 32 needs a coverage collector to receive transactions, it will not edit the transaction — it will implement `Subscriber` on the collector, and the transaction never knows.

One more absence: data. A base class could declare a `species` field and let children fill it in. A trait *cannot contain fields* — only method signatures and default bodies. If shared behavior needs data, it asks for the data through a required method, exactly as `make_sound()` asks for `species()`. This will feel like ceremony for about one chapter, and then it will feel like honesty: the trait's signature documents precisely what it needs from you, instead of quietly reaching into your member variables and hoping.

Where did super go?

Every UVM engineer carries a scar shaped like a forgotten `super` call. In SystemVerilog, a component whose `new()` skips `super.new(name, parent)` detaches itself from the component hierarchy and fails somewhere downstream, at runtime, with an error that mentions nothing about constructors. Python's version of the trap is sharper still — *Python for RTL Verification* built a figure around a `SmallDog` that extended `Dog`, overrode `__init__()`, forgot to call `super().__init__()`, and blew up with an `AttributeError`, because in Python data attributes are not inherited; they exist only if some constructor actually ran and created them. In both languages the

fix is the same discipline: *always call up the chain*, and find out at runtime when you don't.

Rust closes this trap at both ends. First: struct fields are declared in the struct, not created by whichever initializer happens to run. There is no execution order that leaves a `SmallDog` half-built, because a struct that is missing a field does not compile. Second: when a `SmallDog` wants to reuse `Dog`'s behavior, it does so by *containing* a `Dog` — composition — and delegating to it explicitly.[1]

```
// Figure 2: SmallDog by composition — delegation replaces super()

struct SmallDog {
    dog: Dog,    // a SmallDog HAS the dog parts
}

impl Animal for SmallDog {
    fn species(&self) -> &str { self.dog.species() }   // delegate
to Dog
    fn sound(&self) -> &str { "yap yap" }              // override
}

fn main() {
    let sd = SmallDog { dog: Dog };
    sd.make_sound();
}
```

```
--
The dog says 'yap yap'
```

Look at `self.dog.species()`. That line is doing the job every `super` call did, but spelled as what it actually is: a call to a specific method on a specific value. There is no method-resolution order to walk, no rule about which class comes next in the search, no way to forget the call and find out at runtime — if `SmallDog`'s `impl` doesn't provide `species()` one way or another, the compiler rejects the `impl` block on the spot, naming the missing method. The half-built object has no Rust equivalent to show you. The program that produces it cannot be built.

Python also offers `Dog.__init__(self)` — calling the parent's method explicitly through the class object — whose advantage is that you know exactly which copy you are calling. Delegation is that idea, promoted from alternative to only

option. Rust decided that knowing which copy you are calling is not an advantage but a requirement.

Wearing many hats

Here the two dialects part ways, and Rust sides with both of them at once. SystemVerilog forbids multiple inheritance outright — one parent per class, no exceptions — so an SV engineer has never had to ask which parent's constructor runs first. Python allows it, and *Python for RTL Verification* modeled Pat, a firefighter with kids, as `class FirefighterWithKids(Parent, Firefighter)`, followed by a careful discussion of the method resolution order and a design rule borrowed from *Highlander* — of `__init__()` methods, there can be only one.

In Rust, "Pat has two roles" is simply "Pat implements two traits" — SystemVerilog's restraint and Python's flexibility in the same feature. There is nothing to diagram.

```
// Figure 3: Multiple roles as multiple trait implementations

trait Parent {
    fn kiss(&self);
}

trait Firefighter {
    fn hose(&self);
}

struct Pat {
    name: String,
}

impl Parent for Pat {
    fn kiss(&self) { println!("{} gives the baby a kiss.",
self.name); }
}

impl Firefighter for Pat {
    fn hose(&self) { println!("{} sprays water.", self.name); }
}

fn main() {
    let pat = Pat { name: String::from("Pat") };
    pat.kiss();
    pat.hose();
}
```

```
--
Pat gives the baby a kiss.
Pat sprays water.
```

A type can implement as many traits as it likes, and because traits carry no data, the classic multiple-inheritance hazards — which parent's field wins, which constructor runs, what order the search visits the parents — never arise. `Pat` has exactly one set of fields, declared in exactly one place, and each trait bolts a capability onto it. The Highlander rule enforced itself.[2]

[1] The Python book credits the SmallDog change to Ron Swanson of Parks and Recreation. I see no reason to withdraw the attribution.

[2] Rust examined the method resolution order and politely declined to have one.

Default methods: the UVM's no-op pattern, formalized

Here is where this chapter starts paying rent for Chapter 24. Think about how the UVM's phases work: `uvm_component` defines `build_phase()`, `connect_phase()`, `run_phase()` and the rest as *empty virtual methods*, and your components override only the phases they care about. The base class's no-op bodies exist so the phase machinery can call every phase on every component without checking what each component bothered to define.

That pattern — "here is the full interface; override what you use" — is exactly what default methods are for, and it is how rustdv's `Component` lifecycle trait is designed. A preview, signatures only (Chapter 24 does this properly):

```
// Figure 4: The shape of rustdv's Component trait (preview —
signatures only)

pub trait Component {
    fn start(&mut self, ctx: &mut RustdvCtx) {}          //
default: nothing to run
    fn check(&mut self, errors: &mut CheckSink) {}    // default:
do nothing
    fn report(&self) {}                                  // default:
do nothing
}
```

A driver overrides `start()`; a scoreboard overrides `check()`; each inherits the empty default for every phase it ignores, and a purely structural component — an env that exists to hold its children — can implement the trait without overriding anything at all. The same division of labor the UVM has always used, with one upgrade. In the UVM, the empty methods live in a base class, so getting them requires joining the family tree rooted at `uvm_object`. In rustdv, they live in a trait you implement, so a component is just a plain struct — its

fields are its children, per Chapter 24 — that opts into the lifecycle. Same methodology, no family tree.

Deriving: the compiler writes the boring impls

Transactions need utility methods, and every UVM dialect makes you write them: equality so the scoreboard can compare a prediction to a result — `do_compare()` in SystemVerilog, `__eq__` in Python — and a string rendering so logs are readable — `convert2string()`, `__str__`. It has always been your job to write each one on every transaction class, field by field. Rust maps each of these jobs to a standard trait, and for most of them the compiler will write the implementation for you. The keyword is `#[derive(...)]`, an attribute you place on a struct or enum, listing traits whose implementations the compiler should generate from the fields.

Let's put the TinyALU's transaction under the microscope.

```
// Figure 5: Derived traits on the TinyALU command transaction

#[derive(Clone, Copy, Debug, PartialEq)]
enum Ops {
    Add = 1,
    And = 2,
    Xor = 3,
    Mul = 4,
}

#[derive(Clone, Debug, PartialEq)]
struct AluCommand {
    a: u8,
    b: u8,
    op: Ops,
}

fn main() {
    let cmd = AluCommand { a: 0xAA, b: 0x55, op: Ops::Xor };
    let copy = cmd.clone();
    println!("equal? {}", cmd == copy);
    println!("{:?}", cmd);
}
```

```
--
equal? true
AluCommand { a: 170, b: 85, op: Xor }
```

One line above the struct bought us three implementations. `Clone` gives `cmd.clone()`, a field-wise copy — `do_copy()`, generated. `PartialEq` gives `==`, a field-wise comparison — this was you writing `do_compare()` or `__eq__` by hand, remembering to compare every field, and remembering again when you added a field. The derived version regenerates from the struct definition on every compile, so it *cannot* fall out of sync with the fields. `Debug` gives the `{:?}` format you see in the output: a machine-ish rendering of the whole struct, the job `sprint()` did for a `uvm_object` and `__repr__` did in Python.[3]

That leaves the human-friendly rendering — `convert2string()`, `__str__`. Its Rust counterpart is the `Display` trait, and here the compiler makes you write it yourself, on purpose: Rust's position is that a machine can guess how to *dump* your type but not how to *present* it. Implementing `Display` is our first hand-written implementation of a standard-library trait, and it looks like every trait impl you've seen this chapter:

```
// Figure 6: Implementing Display by hand — the __str__ of Rust

use std::fmt;

impl fmt::Display for AluCommand {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "cmd: a=0x{:02x} b=0x{:02x} op={:?}", self.a,
self.b, self.op)
    }
}

fn main() {
    let cmd = AluCommand { a: 0xAA, b: 0x55, op: Ops::Xor };
    println!("{}", cmd);
}
```

```
--
cmd: a=0xaa b=0x55 op=Xor
```

Once `Display` exists, `{}` in any format string works, exactly as `convert2string()` fed your `uvm_info` messages and `__str__` fed `print()`. The `write!` macro is `println!`'s cousin that writes into the formatter instead of to the screen, and the `fmt::Result` return is Chapter 9's `Result` making a cameo — formatting can fail, and the signature says so.

Here is the full mapping, the table this chapter exists to give you. Left columns: what your language made you write. Right columns: where it went.

The job	SystemVerilog (`uvm_object`)	Python	Rust trait
compare transactions	`do_compare()`	`__eq__`	`PartialEq`
unambiguous debug dump	`sprint()`	`__repr__`	`Debug` (`{:?}`)
human-readable string	`convert2string()`	`__str__`	`Display` (`{}`)
ordering and sorting	—	`__lt__`, `__le__`, …	`PartialOrd`, `Ord`
hashing / keying	—	`__hash__`	`Hash`
operator overloading	—	`__add__`, `__sub__`, …	`std::ops::Add`, `Sub`, …
duplicate transactions	`do_copy()`	`copy.deepcopy()`	`Clone`

Three notes on the table. The SystemVerilog dashes are not gaps in the table but in the language — SV classes have no operator overloading and no standard ordering hook, which is why your scoreboards sort with hand-written comparison functions. `PartialOrd` and `Ord` split ordering in half because some types have values that refuse to be ordered (floating-point `NaN` is the culprit — hence *partial*); for transaction structs of integers and enums, you derive both and move on. And the operator traits in `std::ops` mean Rust has real operator overloading — `cmd_a + cmd_b` can be made to work — but it is opt-in per trait, per type.

If that table feels like it just dissolved a chapter of whichever book taught you these methods, hold the thought: Chapter 35 shows that it also dissolves most of `uvm_object`. The field-wise copy-and-compare machinery — which SV-UVM

generates with `uvm_field_*` macros that walk the fields at runtime, and pyuvm hand-rolled by walking `__dict__` at runtime — is precisely what `derive` generates at compile time. A rustdv transaction is a plain struct with `#[derive(Clone, Debug, PartialEq)]` on top — no base class required.

[3] The derived `Debug` prints `a: 170` rather than `a: 0xaa` because it renders a `u8` as decimal — another small argument for writing `Display` yourself when humans will read the result.

Two kinds of polymorphism

We now have `Dog`, `Cat`, and `SmallDog`, each implementing `Animal`, and one question left — the question inheritance answered with "they share a base class": how do we write code that works on *any* animal?

Rust gives two answers, and choosing between them is a skill this book will exercise from here to the final chapter.

Answer one: generics. Write a function with a type parameter, and *bound* the parameter by the trait:

```
// Figure 7: Generic function — dispatch resolved at compile time

fn check_in<T: Animal>(animal: &T) {
    animal.make_sound();
}

fn main() {
    check_in(&Dog);
    check_in(&Cat);
}
```

```
--
The dog says 'bow bow'
The cat says 'miāo'
```

`<T: Animal>` reads "for any type T that implements Animal." This is Python's duck typing with the ducks counted before the program runs: where Python said "just call `make_sound()` and hope," the bound *documents* the requirement in the signature and the compiler *checks* it at every call site. Behind the scenes the compiler compiles a separate copy of `check_in` for `Dog` and for `Cat` — a process called **monomorphization** — so each call dispatches directly, as fast as if you had written the two functions by hand. Generic code costs nothing at runtime. Chapter 11 is entirely about this machinery, so I'll leave it warm rather than cooked.

Answer two: trait objects. Sometimes you need one collection holding a mixture of types — a Python list held anything, and a kennel holds whatever shows up. For that, Rust erases the concrete type behind a pointer:

```
// Figure 8: Trait objects — dispatch resolved at runtime

fn main() {
    let kennel: Vec<Box<dyn Animal>> = vec![
        Box::new(Dog),
        Box::new(Cat),
        Box::new(SmallDog { dog: Dog }),
    ];
    for animal in &kennel {
        animal.make_sound();
    }
}
```

```
--
The dog says 'bow bow'
The cat says 'miāo'
The dog says 'yap yap'
```

`dyn Animal` is a **trait object**: "some type, I'm not saying which, that implements Animal." Because the compiler no longer knows the concrete type, two things follow. The value must live behind a pointer — here Chapter 13's `Box` — since different animals have different sizes and a `Vec` needs uniform elements. And each call to `make_sound()` is dispatched at runtime through a **vtable**, a small table of function pointers riding along with the object. If that sounds like how Python method calls or SystemVerilog virtual methods work — yes, exactly, except that Rust makes you *ask* for dynamic dispatch by writing `dyn`, and hands you static dispatch everywhere else.

The trade, in one breath: generics are faster and fully checked but require the concrete types to be knowable where the code is compiled; trait objects accept types chosen at runtime but pay a pointer, an allocation, and an indirect call. The rule of thumb this book follows: **reach for generics first, and reserve `dyn` for collections that must mix types.**

You will see rustdv make both choices, each where it belongs. The driver is `Driver<REQ, RSP>` — generic over its transaction types, so handing the wrong transaction to a driver is a compile error rather than the `$cast` failure SV-UVM debugging is made of, or the runtime type explosion pyuvm checked for by hand. Subscribers are a bound, `T: Subscriber`, the "anything with a `write()` method" of the analysis chapters. And trait objects appear where heterogeneity is the point: the factory hands back every component it builds as a trait object, because a slot a test may override cannot commit to a concrete type (Chapter 29), and a sequencer stores its queued sequences as trait objects because sequences of different types wait in one line (Chapter 36). Known types: generics. One slot, many possible occupants: `dyn`.

Summary

Rust replaces inheritance with traits: named, explicit interfaces that a type opts into with an `impl` block. Required methods state what the type must provide — pure virtual functions whose absence is caught at the `impl`, not at extension time; default methods carry shared behavior, doing the job of base-class methods — including the UVM's empty-phase-method pattern, which becomes default methods on rustdv's `Component` trait. Code reuse comes from composition and delegation, and the delegating call replaces `super` with an ordinary, visible method call. Multiple roles come from implementing multiple traits, with no diamond and nothing to resolve.

The transaction utility methods every UVM dialect demanded map one-for-one onto standard traits — `do_compare()` / `__eq__` to `PartialEq`, `convert2string()` / `__str__` to `Display`, `do_copy()` to `Clone`, debug dumps to `Debug` — and `#[derive(...)]` makes the compiler write most of them from your struct's fields, which is how a rustdv transaction gets by with no base class at all. Finally, "code that works on any implementor" comes in two flavors:

generic functions with trait bounds, monomorphized and free at runtime, and trait objects behind `dyn`, dispatched through a vtable when one collection must hold many types.

We have been leaning on that `<T: Animal>` notation while promising the details later. Later has arrived — Chapter 11 opens up generics: type parameters, bounds, and why `Driver<REQ, RSP>` is the most honest thing a driver has ever said about itself.

Chapter 11: Generics

Chapter 10 ended with a promissory note. We wrote `fn check_in<T: Animal>(animal: &T)`, waved at the angle brackets, and promised that generics — code with type parameters, resolved at compile time — would get their own chapter. This is that chapter. By the end of it you will know what the compiler actually does with `<T: Animal>`, why the result runs exactly as fast as code without it, and why `Driver<REQ, RSP>` is, as promised, the most honest thing a driver has ever said about itself.

Here is the small confession first: you have been using generics since Chapter 8. `Vec<T>` is a generic struct. So are `Option<T>` and `Result<T, E>` — every time you wrote `Vec<AluCommand>` or `Result<u16, AluError>`, you were filling in someone else's type parameters. This chapter teaches you to write your own.

> **In the UVM...** we had two answers to "one definition, many types." SystemVerilog's answer was the parameterized class — `class register #(type T = int);` — the machinery beneath every `uvm_sequence #(REQ, RSP)` and `uvm_sequencer #(REQ)` you have ever declared. Python's answer was to skip the question: functions accept everything, a scoreboard's `write()` took whatever the analysis port delivered, and if the object had the right attributes, everything worked. That philosophy has a name, *duck typing* — if it walks like a duck and quacks like a duck, treat it as a duck — and nobody checks for feathers until the moment of the quack.

Duck typing is pleasant to write. Its cost is *when* the feather-check happens: at every call, at runtime, and only on the paths your test actually exercised. Rust keeps the pleasant part — one function, many types — and moves the check: if it implements the trait bound, it's a duck, and you find out at compile time, for every path, including the ones your test forgot. SystemVerilog readers, meanwhile, should read this chapter as *parameterized classes with a real contract* — your instinct is right, and the differences are the good part.

A function for any type

Suppose we want the largest value in a slice. For the TinyALU's `u8` operands we could write it directly, but the moment we also want the largest `u16` result, or the alphabetically last test name, we are copying the function and changing one type annotation — exactly the copy-and-modify busywork object-oriented programming exists to avoid. Rust's answer is a **type parameter**: a placeholder type, declared in angle brackets, that the caller fills in.

Let's write it the way you would naively write it, because the failure is the lesson.

```
// Figure 1: A generic function, first attempt — the compiler wants
proof

fn largest<T>(list: &[T]) -> &T {
    let mut largest = &list[0];
    for item in list {
        if item > largest {
            largest = item;
        }
    }
    largest
}
```

```
--
error[E0369]: binary operation `>` cannot be applied to type `&T`
 --> src/main.rs:4:17
  |
4 |         if item > largest {
  |            ---- ^ ------- &T
  |            |
  |            &T
  |
help: consider restricting type parameter `T`
  |
1 | fn largest<T: std::cmp::PartialOrd>(list: &[T]) -> &T {
  |             ++++++++++++++++++++++
```

Read the declaration first: `fn largest<T>` says "this function is defined for some type T, to be named later," and then `&[T]` and `&T` use the placeholder as if it were a real type. Python would have shrugged and run this. Rust refuses, and its objection is philosophically the whole chapter: *you said T could be any*

type, and then you compared two of them with `>`*. Not every type can do that.* What do we know about a type we know nothing about? Nothing — so we may call nothing on it.

The fix is in the compiler's own `help` text, and it is Chapter 10's vocabulary: a **trait bound**. `T: PartialOrd` narrows "any type" to "any type that implements `PartialOrd`" — the ordering trait from Chapter 10's dunder table, the one behind `<` and `>`.

```
// Figure 2: The bound is the fix — and the documentation

fn largest<T: PartialOrd>(list: &[T]) -> &T {
    let mut largest = &list[0];
    for item in list {
        if item > largest {
            largest = item;
        }
    }
    largest
}

fn main() {
    let operands: Vec<u8> = vec![0x22, 0xAA, 0x07];
    let results: Vec<u16> = vec![0x0154, 0x7100, 0x00FF];
    let tests = vec!["alu_add_test", "alu_xor_test",
"alu_mul_test"];

    println!("largest operand: 0x{:02x}", largest(&operands));
    println!("largest result:  0x{:04x}", largest(&results));
    println!("last test name:  {}", largest(&tests));
}
```

```
--
largest operand: 0xaa
largest result:  0x7100
last test name:  alu_xor_test
```

One function body, three element types, and no type mentioned at any call site — the compiler infers `T` from the argument, the same way it has been inferring types for you since Chapter 3.[1] Notice too that we return `&T`, a reference, not `T`: Chapter 5 taught us that returning the value itself would mean moving it out of the caller's slice, and the borrow is both cheaper and honest about who still owns the data.

The bound does double duty, and this is worth slowing down for. To the *compiler* it is a permission slip: inside `largest`, exactly the methods of `PartialOrd` may be called on `T`, no more. To the *reader* it is documentation you can trust: the signature `fn largest<T: PartialOrd>(list: &[T]) -> &T` tells you everything this function will ever demand of your type. Python's equivalent contract lived in the docstring, the tribal knowledge, or the stack trace; SystemVerilog's lived in an elaboration error that erupted from the class's guts and named your instantiation instead of the requirement.

More than one requirement

A bound can require several traits at once, joined with `+`. Here is a function a scoreboard might want — compare an expected transaction against an actual one, and complain legibly on a mismatch. Comparing needs `PartialEq`; complaining legibly needs `Debug`. Both go in the contract:

```
// Figure 3: Multiple bounds with a where clause

use std::fmt::Debug;

fn check_match<T>(expected: &T, actual: &T) -> bool
where
    T: PartialEq + Debug,
{
    if expected == actual {
        true
    } else {
        println!("MISMATCH: expected {:?}, actual {:?}", expected,
actual);
        false
    }
}

fn main() {
    check_match(&0x54u16, &0x54u16);
    check_match(&0x54u16, &0x55u16);
}
```

```
--
MISMATCH: expected 84, actual 85
```

We could have written `fn check_match<T: PartialEq + Debug>(...)` and it would mean the same thing; the `where` clause is the same information moved out of the angle brackets so the signature stays readable. Convention, and this book, use the inline form for one short bound and `where` for anything longer. Either way, look at what this function is: the duck-typed checker — "anything with an `__eq__` and a `__repr__`," or in SV terms, anything whose base class promised `do_compare()` and `convert2string()` — with the requirements written down and enforced per type. A transaction type that forgot to derive `PartialEq` doesn't fail in hour three of a regression; it fails to compile, with an error pointing at the missing derive.

Generic structs

Type parameters work on structs the same way, and you already know the syntax from the consumer side — `Vec<T>` — so producing one holds no surprises. Here is a register model wide enough for any leg of the TinyALU:

```
// Figure 4: A generic struct — one definition, many widths

struct Register<T> {
    value: T,
}

impl<T: Copy> Register<T> {
    fn new(value: T) -> Self {
        Register { value }
    }
    fn read(&self) -> T {
        self.value
    }
    fn write(&mut self, value: T) {
        self.value = value;
    }
}

fn main() {
    let mut a_reg: Register<u8> = Register::new(0x00);   // the A
leg is 8 bits
    let mut result: Register<u16> = Register::new(0x0000); // the
result is 16

    a_reg.write(0xAA);
    result.write(0x7100);
    println!("A: 0x{:02x}  result: 0x{:04x}", a_reg.read(),
result.read());
}
```

```
--
A: 0xaa  result: 0x7100
```

Two spellings deserve a comment. `struct Register<T>` declares the parameter; `impl<T: Copy> Register<T>` declares it *again* for the methods, and that is where the bound lives — `read()` hands back a copy of the value, so the methods require `T: Copy`, which every integer satisfies. Keeping the struct definition unbounded and putting requirements on the `impl` is idiomatic: the data doesn't care what `T` can do; the *operations* do.

And note what the type system now knows: `Register<u8>` and `Register<u16>` are **different types**, exactly as SystemVerilog's `register #(byte)` and `register #(shortint)` specializations were. Try to write a `u16` result into the 8-bit A register and the program does not compile — where SV, having kept the widths straight in the declaration, would have quietly truncated at the

assignment, and Python kept widths straight by discipline and masking alone. Here the widths are in the types and the seams are sealed. This — a struct parameterized by the type it carries — is precisely what `Vec<T>` has been all along, and it is the shape rustdv's plumbing takes: when Chapter 31 connects components with typed FIFOs, a FIFO of ALU commands is a `TlmFifo<AluCommand>`, and connecting it to a port expecting results is a compile error, not a 3 a.m. discovery.

Monomorphization, or: where did the ducks go?

Now the question a performance-minded verification engineer should be asking. Python's duck typing has a runtime cost, and it is not small: *every* method call, every attribute access — `item.compare(other)`, `self.scoreboard.write(txn)` — is a lookup by name, at runtime, every single time, because until the moment of the call Python does not know what the object is. Twenty million transactions means twenty million rounds of "does this object have a `write`?" That is the interpreter overhead Chapter 1 put on the ledger.

So what does `largest` cost? It is one function that works on three types — surely *something*, somewhere, is checking which type showed up?

Nothing is, and the reason is the best idea in this chapter. At compile time, the compiler finds every concrete type your program actually uses with `largest`, and generates a separate, specialized copy of the function for each one — a process called **monomorphization**, "making single-formed." Your generic source code is a stencil; the compiler stamps it out once per type. Conceptually, figure 2 compiles as if you had written:

```rust
// Figure 5: What the compiler generates from figure 2
(conceptually — you never see this)

fn largest_u8(list: &[u8]) -> &u8 {
    // ... same body, with T = u8 throughout
}

fn largest_u16(list: &[u16]) -> &u16 {
    // ... same body, with T = u16 throughout
}

fn largest_str(list: &[&str]) -> &&str {
    // ... same body, with T = &str throughout
}
```

Each call site in `main` is wired directly to its own stamped-out copy. When `largest(&operands)` runs, there is no lookup, no vtable, no "which type is this?" — the `u8` version was chosen before the program existed as a binary, and the call is exactly as fast as the hand-written `u8`-only function you refused to copy-paste three times. The same happens to structs: `Register<u8>` and `Register<u16>` compile to two independent struct definitions, each with its own specialized methods. SystemVerilog engineers have seen this movie — each parameterization of an SV class is its own specialization too; Rust's version simply happens before any simulator gets involved. This is what the design of rustdv means by *generic code costs nothing at runtime*: you write the abstraction once and pay for it never.[2]

Set the two models side by side, because this is the chapter's picture. Python answers "which `write()` do I call?" by looking it up when the call happens — maximally flexible, paid for on every transaction of every test of every regression. Rust answers the same question once, at compile time, by generating the exact code each caller needs — and the flexibility you give up is precisely the flexibility of being wrong. Chapter 10's trait objects (`Box<dyn Animal>`, the vtable, the runtime dispatch) remain available for the cases that need runtime choice; monomorphized generics are what you get everywhere else, which in a testbench is almost everywhere.

Honesty requires the other side of the ledger, and it is real but modest: stamping out copies takes compile time and makes the binary larger. A generic function used with thirty types is compiled thirty times. For testbench code — a handful of transaction types, not thirty — you will notice this approximately

never, but now you know why heavily generic Rust code compiles slower than it runs.

The destination: SeqItemPort<REQ, RSP>

Part I keeps promising that these language chapters are load-bearing, so let me show you the load — and give SystemVerilog its due, because SV-UVM got this design *right*: `uvm_driver #(type REQ = uvm_sequence_item, type RSP = REQ)` is a parameterized class, and typing the driver's conversation was the correct instinct. pyuvm, living in dynamic Python, dropped the parameters — what came out of `get_next_item()` was whatever the sequencer sent, and a misconfigured test discovered the mismatch at runtime, usually as an `AttributeError` deep in `run_phase()`.

rustdv keeps SystemVerilog's design and adds Rust's enforcement. Signatures only — Chapters 31 and 36 build this for real:

```
// Figure 6: The shape of rustdv's driver (preview — signatures
only)

pub struct SeqItemPort<REQ, RSP = REQ> { /* channel endpoints —
elided */ }

pub struct AluDriver {                        // your driver: a
plain struct...
    seq_item_port: SeqItemPort<AluCommand>,  // ...that owns a
typed port
    // ...
}
```

Everything in this chapter is in the first line. Two type parameters, because a driver's conversation has two directions: `REQ` is the request transaction it pulls from the sequencer, `RSP` the response it may send back. Type parameters can have *defaults* — `RSP = REQ` says "if you don't name a response type, it's the same as the request," the exact convention `uvm_driver`'s declaration has carried for two decades — so the common no-response port is just `SeqItemPort<AluCommand>`. There is no `uvm_driver` base class to extend — your driver is an ordinary struct that owns a port — but the port, the sequencer on its far end, and every transaction that crosses between them must agree on

the types *or the testbench does not compile*. The wrong-sequence-on-the-wrong-agent bug does not become an error message. It becomes unwritable.

That is the trade this book keeps making, in its purest form yet: runtime flexibility — any port, any transaction, sort it out mid-simulation — exchanged for a signature that is documentation, contract, and proof all at once. It is the design SV-UVM sketched with parameterized classes, finished. And thanks to monomorphization, `SeqItemPort<AluCommand>` compiles to code as direct as a port hand-written for ALU commands alone, because that is literally what the compiler makes of it.

Summary

Generics let one definition serve many types: type parameters in angle brackets stand for a type named later, on functions (`fn largest<T: PartialOrd>(list: &[T]) -> &T`) and on structs (`Register<T>` , and the `Vec<T>` , `Option<T>` , and `Result<T, E>` you have used since Chapter 8). Trait bounds are the contract — `T: PartialOrd` , or several requirements joined with `+` , or a `where` clause when the list grows — and they finish what SystemVerilog's parameterized classes started while replacing Python's duck typing with the same idea checked at compile time: if it implements the bound, it's a duck, and the compiler verifies the feathers before the program runs. Monomorphization is why none of this costs anything at runtime: the compiler stamps out a specialized copy of the generic code for each concrete type used, so every call dispatches directly, in exchange for some compile time and binary size. And `SeqItemPort<REQ, RSP = REQ>` is where the book is taking all of it — a driver whose port carries its transaction types in its type, so mismatched testbench plumbing fails at compile time instead of mid-regression.

We now have types that carry proofs. What we do not yet have is Rust's way of passing *behavior* around — the thing Python did every time it handed a function to `sorted(key=...)` or stored a coroutine to run later, and the thing rustdv's factory will do for a living. Chapter 12 takes up closures and iterators, and if you enjoyed watching the compiler specialize your generics for free, you are going to like what it does to a `for` loop.

[1] When inference can't decide — or you want to be explicit — you can name the type at the call site with `largest::<u8>(&operands)`. The `::<>` operator is universally called the *turbofish*, it is official Rust culture, and no, nobody has come up with a better name. Swim on.

[2] C++ programmers will recognize this as what templates do — and SystemVerilog's parameterized classes inherited the same per-instantiation behavior — with one upgrade worth appreciating: a Rust generic function is type-checked once, against its bounds, when it is *defined*, not re-checked per instantiation with errors erupting from the template's guts. The compiler in figure 1 complained about our signature, in our terms, before any caller existed.

Chapter 12: Closures and Iterators

Python made testbench code shorter and stranger at the same time with two features: comprehensions, which built a whole list in one bracketed line, and generators, which used `yield` to produce values one at a time without ever building the list at all. Both were ways of saying *here is a stream of values and a recipe for making them*. SystemVerilog never had either — nor the feature underneath them — which makes this chapter the newest ground in Part I for half of this book's readers, and worth every minute of it.

Rust has both ideas, reorganized around one feature: the **iterator**. And driving the iterator machinery is a smaller feature that this book has been saving up for eleven chapters, because it is quietly one of the most important in the language: the **closure**. Closures matter far beyond this chapter. When Chapter 29 rebuilds the UVM factory, the values its registry stores — the makers that construct components on demand — will turn out to be exactly this chapter's closures. This is the chapter where you learn why that sentence makes sense.

> **In the UVM...** stimulus recipes were loops. Python's dialect could also write them as generators — `yield` hands a value to the caller and *keeps running* — and as comprehensions like `[nn**2 for nn in range(11) if nn % 2 == 0]`, four parts in square brackets replacing a four-line loop. SystemVerilog's closest analog was a task feeding a mailbox: the stream-of-values idea was there, but functions were never values you could pass around, store, or build streams from.

Closures: functions as values

A closure is a function without a name, written inline, that can use the variables around it. Python had these in two flavors: `lambda x: x + 1` for one-liners, and nested `def` for anything longer. SystemVerilog had nothing of the kind — an SV function has a name, a declaration, and an address in a package somewhere, and it certainly cannot be stored in a variable. Rust has one syntax,

and the whole feature fits in a figure. The parameters go between vertical bars, and the body follows.

```rust
// Figure 1: A closure is an unnamed function in a variable

fn main() {
    let add_one = |x: u32| x + 1;

    let describe = |aa: u8, bb: u8| {
        let sum = aa as u16 + bb as u16;
        format!("{aa} + {bb} = {sum}")
    };

    println!("{}", add_one(41));
    println!("{}", describe(0xFF, 0x01));
}
```

```
--
42
255 + 1 = 256
```

`add_one` holds a function the way a variable holds a number. A single expression needs no braces; a multi-line body takes braces and, like every Rust block, evaluates to its last expression. Notice there is no return type written on either closure — the compiler infers closure types from how you use them, which is why closures usually look lighter than `fn` declarations. In fact the parameter types are usually optional too; I wrote `x: u32` for clarity, but `let add_one = |x| x + 1;` compiles fine once the compiler sees a call that pins the type down.

So far this is `lambda` with different punctuation. The interesting part — the part Python never asked you to think about — is what happens when the closure uses a variable it did not declare.

Capture, through the ownership lens

A Python closure that mentions an outer variable just… uses it. Every Python name is a reference, so the closure captures a reference, silently, and if two pieces of code mutate the same captured object at the same time, that is your

problem to discover at runtime. Chapter 6's shared-variable race — two processes sharing `transaction_data` — was exactly this bug wearing a coroutine costume.

Rust closures also capture outer variables, but here is the difference: *capturing is subject to the ownership rules from Chapters 5 and 6, like everything else.* A closure that reads a variable borrows it with `&T`. A closure that mutates one borrows it with `&mut T`. A closure that consumes one takes ownership. The compiler looks at the closure's body, picks the least drastic mode that works, and then enforces it — visibly, in the type system, at compile time.

Watch the three modes in order. First, a closure that only reads.

```
// Figure 2: A closure that reads captures by shared borrow

fn main() {
    let ops = vec!["ADD", "AND", "XOR", "MUL"];

    let show = || println!("ops under test: {ops:?}");

    show();
    show();
    println!("still mine: {} ops", ops.len());  // ops was only
borrowed
}
```

```
--
ops under test: ["ADD", "AND", "XOR", "MUL"]
ops under test: ["ADD", "AND", "XOR", "MUL"]
still mine: 4 ops
```

`show` borrowed `ops` the way any `&Vec` would, so we can call it repeatedly and still use `ops` afterward. Second, a closure that mutates.

```
// Figure 3: A closure that mutates captures by exclusive borrow

fn main() {
    let mut errors = Vec::new();

    let mut log_error = |msg: &str| errors.push(msg.to_string());

    log_error("ADD result mismatch");
    log_error("XOR result mismatch");

    println!("{} errors: {errors:?}", errors.len());
}
```

```
--
2 errors: ["ADD result mismatch", "XOR result mismatch"]
```

Two things changed. The closure itself must be declared `let mut`, because calling it mutates the captured `errors` — mutation is never invisible in Rust, not even here. And while `log_error` is alive, it holds the *exclusive* borrow of `errors`; if we tried to `println!("{errors:?}")` between the two calls, the compiler would refuse, citing the aliasing-XOR-mutability rule from Chapter 6. One writer, no readers alongside. The race your old languages could only warn about is structurally impossible to write.

Third, a closure that takes ownership. Sometimes a closure must own its captures — most often because it will outlive the scope it was created in, which is precisely the situation when you store a closure in a struct or hand it to another task. The `move` keyword forces the transfer.

```
// Figure 4: A move closure takes ownership of its captures

fn main() {
    let test_name = String::from("alu_smoke_test");

    let banner = move || format!("*** {test_name} ***");

    println!("{}", banner());
    println!("{}", test_name);  // ERROR: test_name moved into the
closure
}
```

```
--
error[E0382]: borrow of moved value: `test_name`
 --> src/main.rs:8:20
  |
4 |     let banner = move || format!("*** {test_name} ***");
  |                  ------- value moved into closure here
...
8 |     println!("{}", test_name);
  |                    ^^^^^^^^^^ value borrowed here after move
```

The error message tells the whole story: `test_name` moved into `banner`, and Chapter 5's rule applies — after a move, the old name is dead. Delete the last `println!` and the program compiles. This is the same monitor-hands-transaction-to-scoreboard reasoning you already know; the only novelty is that the new owner is a closure instead of a function parameter.

Rust names these three capture behaviors with three traits, and you will meet them constantly in documentation: **`Fn`** for closures that can be called any number of times through a shared borrow (figure 2), **`FnMut`** for closures that mutate and need exclusive access to call (figure 3), and **`FnOnce`** for closures that consume something and therefore can only be called once.[1] You do not choose among them by annotation — the compiler classifies each closure from its body — but you will *read* them in every function signature that accepts a closure, and later in this chapter you will write one into a struct field.

[1] The names describe how the closure may be *called*, and they nest: every `Fn` is also an `FnMut`, and every `FnMut` is also an `FnOnce` — a closure you may call many times can certainly be called once. Interviewers love this; day-to-day code mostly just needs you to recognize the three names.

Iterator adapters: comprehensions, unrolled

Now the payoff. Python builds `even_squares` in one famous line:

```
even_squares = [nn**2 for nn in range(11) if nn % 2 == 0]
```

A comprehension has four parts: an expression, a variable, an iterator, and a filter. Rust has no comprehension syntax. Instead it lets you take those same four parts and chain them left to right as method calls on the iterator — each method taking, naturally, a closure.

```rust
// Figure 5: The list comprehension, as an iterator chain

fn main() {
    let even_squares: Vec<u32> = (0..=10)
        .filter(|nn| nn % 2 == 0)
        .map(|nn| nn * nn)
        .collect();

    println!("even squares {even_squares:?}");
}
```

```
--
even squares [0, 4, 16, 36, 64, 100]
```

Read the chain aloud and it is the comprehension in sentence order: take the range zero through ten, *filter* it down to the even numbers, *map* each survivor to its square, and *collect* the results into a `Vec`. The `|nn| ...` closures are the comprehension's expression and filter parts, now explicit values passed as arguments. Where Python's comprehension makes you learn the four positions inside the brackets, the Rust version wears its structure on the outside — and when a chain grows too clever, it splits across lines exactly as figure 5 shows, no special multi-line dispensation required.

Python's dictionary comprehension ports the same way. `{ii : ii**3 for ii in range(4)}` becomes a chain that maps each number to a `(key, value)` pair and collects into a `HashMap`:

```rust
// Figure 6: The dictionary comprehension, collected into a HashMap

use std::collections::HashMap;

fn main() {
    let cubes: HashMap<u32, u32> = (0..4)
        .map(|ii: u32| (ii, ii.pow(3)))
        .collect();

    println!("cubes: {cubes:?}");
}
```

```
--
cubes: {2: 8, 0: 0, 3: 27, 1: 1}
```

Two things to notice. `collect()` is doing something quietly remarkable: the *same method* built a `Vec` in figure 5 and a `HashMap` here, steered by the type annotation on the left — the generics machinery from Chapter 11 earning its keep. (One wrinkle: the closure's parameter carries its own `: u32`, because a method call like `.pow` must know its receiver's concrete type on the spot — inference has not yet flowed backward from the annotation when the closure body is checked.) And look at that output order. Chapter 8 warned you that Rust's `HashMap`, unlike a Python dict or an SV associative array, promises nothing about iteration order, and here is the proof; your run will likely print a different scramble.

One more adapter completes the everyday set. Where `map` transforms each element and `filter` drops some, **`fold`** boils the whole stream down to a single value: it takes a starting accumulator and a closure that combines the accumulator with each element in turn. Here is a scoreboard-flavored example — counting mismatches in a list of (expected, actual) pairs:

```rust
// Figure 7: fold reduces a stream to one value

fn main() {
    let results = [(0x55u16, 0x55u16), (0x100, 0x100), (0x0FE,
0x0FF)];

    let mismatches = results
        .iter()
        .fold(0, |errs, (exp, act)| if exp == act { errs } else {
errs + 1 });

    println!("mismatches: {mismatches}");
}
```

```
--
mismatches: 1
```

In truth you will reach for `fold` less often than you expect, because the standard library pre-packages the common folds: `.sum()`, `.count()`, `.max()`, `.any()`, `.all()`. The chain `results.iter().filter(|(exp, act)| exp != act).count()` does figure 7's job and reads better. But `fold` is the general case the shortcuts are made of, and knowing it makes the shortcuts unmysterious.

Lazy, like a generator

Here is the fact that connects comprehensions to generators, and it deserves its own paragraph: **iterator adapters do nothing until something consumes them.** The chain `(0..=10).filter(...).map(...)` computes no squares. It builds a small struct that *describes* the computation, and only when `collect()` — or a `for` loop, or `sum()` — starts pulling values through does any work happen, one element at a time, no intermediate list anywhere.

If that sounds familiar, it should. It is exactly the property that made Python generators worth a chapter: `my_range(1_000_000)` didn't build a million-entry list, it produced values on demand. In Rust, *every* iterator chain behaves that way by default. Python made laziness an opt-in feature with special syntax; Rust made it the only behavior and never needed the syntax.

Which raises the obvious question: what happened to `yield`?

Where generators went

Rust has no `yield` statement.[2] What it has instead is the `Iterator` trait — one required method, `next()`, which returns `Some(value)` until the stream is exhausted and `None` thereafter. That `Option` should ring a bell from Chapter 9: where Python generators signal exhaustion with a `StopIteration` exception behind the scenes, Rust signals it in the return type, in the open.

Any type that implements `Iterator` works in a `for` loop, chains with every adapter above, and collects into collections. The classic generator is Fibonacci, so let us port it. Where a Python generator kept `lastnumb` and `numb` alive between `yield`s inside a paused function, Rust keeps them as fields in a struct, and `next()` advances the state one step per call.

```
// Figure 8: The Fibonacci generator, as an Iterator implementation

struct Fibonacci {
    curr: u64,
    next: u64,
}

impl Iterator for Fibonacci {
    type Item = u64;

    fn next(&mut self) -> Option<u64> {
        let result = self.curr;
        self.curr = self.next;
        self.next = result + self.next;
        Some(result)
    }
}

fn main() {
    let fib = Fibonacci { curr: 0, next: 1 };
    for numb in fib.take(8) {
        print!("{numb} ");
    }
    println!();
}
```

```
--
0 1 1 2 3 5 8 13
```

The state that Python hid in a suspended stack frame is now a two-field struct you can see, and the resumption that Python performed by magic is now an ordinary method call. Notice that this iterator never returns `None` — it is *infinite*, which is fine, because it is lazy; `.take(8)` is an adapter that cuts the stream off after eight values. An infinite generator was a slightly daring trick in Python. In Rust it is Tuesday.

Writing an `impl Iterator` block for every one-off stream would get old, though, and here Chapter 11's `impl Trait` makes its second appearance — this time in a return position. A function can declare that it returns *some* iterator without naming the struct, and inside, you build that iterator however is clearest — with an ordinary loop, or from ranges and adapters. This is the closest Rust idiom to "a function with `yield` in it," and it is how this book will write value streams from here on.

Suppose a test wants every combination of small A and B operands for the TinyALU. In Python:

```
def operand_pairs(n):
    for aa in range(n):
        for bb in range(n):
            yield (aa, bb)
```

And in Rust, as a function returning `impl Iterator`. The honest first translation keeps the Python's two loops exactly, and changes only one thing:

```rust
// Figure 9: A TinyALU operand-pair stream, replacing a generator
function

fn operand_pairs(n: u8) -> impl Iterator<Item = (u8, u8)> {
    let mut pairs = Vec::new();
    for aa in 0..n {
        for bb in 0..n {
            pairs.push((aa, bb));
        }
    }
    pairs.into_iter()
}

fn main() {
    for (aa, bb) in operand_pairs(3) {
        print!("({aa},{bb}) ");
    }
    println!();
}
```

```
--
(0,0) (0,1) (0,2) (1,0) (1,1) (1,2) (2,0) (2,1) (2,2)
```

Line for line, the body *is* the Python: the same two loops, in the same order. Exactly one thing changed, at the end. Python's `yield` handed each pair back the instant it was made; stable Rust has no `yield`, so instead you fill a `Vec` and hand back its iterator. `pairs.into_iter()` turns the vector into a stream of the same `(u8, u8)` items the signature promised, and the caller's `for` loop cannot tell it apart from a generator. That is the whole lesson of this figure: a function that returns `impl Iterator` is how Rust writes "a function that produces a stream of values."

There is one price, and paying it is the next figure. This version builds the entire vector before it returns a single pair, where Python's generator produced each pair on demand. For nine operand pairs that costs nothing — but the lazy form is worth seeing on its own, both because it is what you will meet in other people's code and because it is where the two `move` keywords from figure 4 stop being a curiosity and start earning their keep:

```rust
// Figure 10: The operand-pair stream, built lazily from adapters

fn operand_pairs(n: u8) -> impl Iterator<Item = (u8, u8)> {
    (0..n).flat_map(move |aa| (0..n).map(move |bb| (aa, bb)))
}

fn main() {
    for (aa, bb) in operand_pairs(3) {
        print!("({aa},{bb}) ");
    }
    println!();
}
```

```
--
(0,0) (0,1) (0,2) (1,0) (1,1) (1,2) (2,0) (2,1) (2,2)
```

The one new idea here is `flat_map`, the nested-loop adapter: for each `aa` it runs the inner closure, which produces a whole stream of `(aa, bb)` pairs, and `flat_map` splices those inner streams end to end — the outer `for aa` and the inner `for bb`, rewritten as adapters. Same nine pairs, same order, but now nothing is computed until the caller asks for the next one. And *that* laziness is what forces the two `move`s. The inner closure must own its copy of `aa`, and the outer must own `n`, because these closures ride out of the function inside the returned iterator and are called long after `operand_pairs`'s own variables are gone. The capture rules you learned through the ownership lens are exactly what make it safe to return a paused computation from a function. Python kept the whole stack frame alive on the heap to manage this; Rust moves in precisely the values the closures need, and the compiler names each one if you forget the `move`.

[2] Generator syntax has been experimented with in unstable Rust for years, but stable Rust — the Rust this book teaches — does not have it, and between adapters and `impl Iterator`, you will rarely feel the gap.

Closures you keep: a seed for Chapter 29

Everything so far has passed closures *downward* — into `map`, into `filter`, used and forgotten. The last idea in this chapter is the one with the longest reach in this book: a closure is a value, and like any value, it can be **stored in a struct field** and called later, by code that has no idea what the closure does inside.

Because every closure has its own unwritable type, storing one takes a trait object — Chapter 10's `dyn`, boxed up: `Box<dyn Fn(u8, u8) -> u16>` is "some heap-allocated callable, taking two `u8`s, returning a `u16`; I don't know or care which one." Here is a toy checker whose *prediction function is data*:

```
// Figure 11: A struct that carries its behavior as a closure

struct Checker {
    predict: Box<dyn Fn(u8, u8) -> u16>,
}

impl Checker {
    fn check(&self, aa: u8, bb: u8, actual: u16) {
        let expected = (self.predict)(aa, bb);
        if expected == actual {
            println!("PASS: ({aa}, {bb}) -> {actual}");
        } else {
            println!("FAIL: ({aa}, {bb}) expected {expected}, got
{actual}");
        }
    }
}

fn main() {
    let adder_check = Checker {
        predict: Box::new(|aa, bb| aa as u16 + bb as u16),
    };
    adder_check.check(0xFF, 0x01, 0x100);

    let and_check = Checker {
        predict: Box::new(|aa, bb| (aa & bb) as u16),
    };
    and_check.check(0x0F, 0x35, 0x0006);  // wrong on purpose
}
```

```
--
PASS: (255, 1) -> 256
FAIL: (15, 53) expected 5, got 6
```

Two `Checker` values, one struct definition, two completely different behaviors — selected not by inheritance, not by overriding a virtual method, but by *which closure was placed in the field at construction time*. Sit with that for a moment, because it is the seed of something large. The UVM factory — the registry, the `type_id::create()` calls, the override tables — exists to answer one question: *how does a test change what the testbench builds without editing the testbench?* rustdv keeps the registry, because create-by-name and overrides installed at a distance need one — and what that registry *stores*, one per component type, is a maker: a constructor as a value, a closure in a box, exactly like `predict` here. Where SystemVerilog manufactures its makers with `type_id` proxy classes and pyuvm with a metaclass, Rust just writes the closure down. You now hold the mechanism the registry stores; Chapter 29 supplies the methodology around it.

Summary

Closures are unnamed functions in variables: `|x| x + 1`, with braces for multi-line bodies and types mostly inferred. They capture surrounding variables under the ordinary ownership rules — shared borrow to read, exclusive borrow to mutate, ownership when `move` d — and the traits `Fn`, `FnMut`, and `FnOnce` name those three calling contracts in signatures. Iterator adapter chains — `filter`, `map`, `flat_map`, `fold`, `collect` — replace Python's list, set, and dictionary comprehensions part for part, and they are lazy by default, doing no work until consumed. Python's generators map to the `Iterator` trait: implement `next()` on a struct for full control, or return `impl Iterator` built from adapters for the everyday case, with `move` closures carrying the captured state out of the function. Finally, closures are values: boxed as `Box<dyn Fn(...)>`, they can live in struct fields and be swapped at construction time — the mechanism Chapter 29 will grow into rustdv's replacement for the UVM factory.

That `Box` in figure 11 was the first time this book put a value on the heap on purpose, and I slipped it past you with one sentence of explanation. It deserves

better — because `Box` has two siblings, `Rc` and `RefCell`, and among them they answer the question every UVM engineer eventually asks here: *how do two components share one scoreboard?* Chapter 13 pays that debt.

Chapter 13: Smart Pointers: Box, Rc, and RefCell

> **In the UVM…** we guarded our class internals as best the language allowed. SystemVerilog gave us real keywords — `local` and `protected`, compiler-enforced. Python could not: its classic story is a `Temperature` class and a user who kept reaching past its methods to poke `tt.temp` directly — deterred first by a single underscore (a convention, unenforced), then by double-underscore name mangling (enforced, grudgingly), and finally civilized by `@property` accessors. Both languages were answering the same question: who gets to touch this value, and on whose terms?

Chapters 5 and 6 handed you two rules and told you the rest of the book stands on them: every value has exactly one owner, and at any moment a value may have many readers or one writer, never both. Since then, you may have been quietly carrying a worry. You have written UVM testbenches. You *know* what one looks like inside: an environment holding an agent, a monitor and a scoreboard both holding the same BFM or the same virtual interface, an analysis port fanning one transaction out to three subscribers. Shared access everywhere, and — be honest — shared *mutable* access more often than the diagrams admit. If Rust's rules are absolute, how does any of that survive the port?

The answer is that the rules are absolute but the *enforcement point* is negotiable. Rust provides a small set of standard-library types — the community calls them **smart pointers** — that let you buy back Python-style sharing, one capability at a time, each purchase visible in your code and each with a price tag attached. This chapter teaches the three you will actually meet: `Box<T>`, `Rc<T>`, and `RefCell<T>`. By the end you will be able to read `Rc<RefCell<T>>` and recognize an old friend wearing a name tag: a Python object reference, made visible.

`Box<T>` : the heap, on request

The simplest smart pointer barely earns the name. `Box<T>` puts a value on the heap and owns it — one owner, moved and dropped exactly like everything in Chapter 5. The only thing that changed is *where the bytes live*.

```
// Figure 1: A Box owns its value on the heap — everything else is
Chapter 5

struct Transaction {
    a: u8,
    b: u8,
    op: u8,
}

fn main() {
    let t = Box::new(Transaction { a: 5, b: 3, op: 1 });
    println!("boxed transaction: {} op {}", t.a, t.b);
}   // t goes out of scope; the Box is dropped; the heap memory is
freed.
    // One owner, one drop, on time — nothing new.
```

```
--
boxed transaction: 5 op 3
```

Notice `t.a` works without ceremony — a `Box` hands through field access as if the box weren't there. So why would you ever ask for one? In Python, and in SystemVerilog's class world, you never chose; every object lived on the heap and every variable was a handle to it. Rust defaults to the stack and lets you opt into the heap, and there are two situations where you must. The first is a type whose size the compiler cannot pin down — a recursive type, like a linked list whose node contains another node; boxing the inner node gives the compiler a fixed-size pointer to reason about. The second you have already met: **trait objects**. Chapter 10 introduced `Box<dyn Component>` as the way to store "some type implementing `Component`, decided at runtime" — and since the compiler cannot know that type's size, onto the heap it goes, behind a `Box`.

That is the whole story. `Box<T>` is single ownership with a heap address. When you see one in rustdv — the boxed component a factory maker returns, say — read it as "an owned value whose concrete type or size wasn't known at compile time" and move on. The interesting escape hatches are the next two.

`Rc<T>` : Python's refcount, made visible

Here is a piece of CPython trivia that is about to stop being trivia: every object you ever created in Python carried a hidden integer — a **reference count** — and every assignment, argument pass, and container insert incremented it, while every scope exit and `del` decremented it. When the count hit zero, the object died.[1] You never saw this machinery; it ran on every single Python statement you ever executed, silently, whether you needed it or not. We can even catch it in the act:

```
# Figure 2: The refcount Python never showed you

import sys

a = ["ADD", 5, 3]
print(sys.getrefcount(a))
b = a
print(sys.getrefcount(a))
```

```
--
2
3
```

(`getrefcount` reports one higher than you'd guess, because the act of calling it lends the list one more temporary reference. Even the inspection tool participates in the scheme.)

Rust's `Rc<T>` — *reference counted* — is exactly this mechanism, with two differences: you opt into it per value instead of paying for it everywhere, and the increments happen only where you can see them. `Rc::new` wraps a value and starts the count at one. `Rc::clone` does *not* copy the value — it copies the *handle* and increments the count, which is precisely what Python's `b = a` did behind your back. When each `Rc` handle is dropped, the count decrements; at zero, the value is dropped. It is CPython's memory management, offered à la carte.

```rust
// Figure 3: Rc::clone increments a refcount — on purpose, where
you can see it

use std::rc::Rc;

fn main() {
    let bfm = Rc::new(String::from("TinyALU BFM"));
    println!("owners: {}", Rc::strong_count(&bfm));

    let driver_handle = Rc::clone(&bfm);
    {
        let monitor_handle = Rc::clone(&bfm);
        println!("owners: {}", Rc::strong_count(&bfm));
        println!("monitor sees: {monitor_handle}");
    }   // monitor_handle dropped here: count decrements

    println!("owners: {}", Rc::strong_count(&bfm));
    println!("driver sees: {driver_handle}");
}
```

```
--
owners: 1
owners: 3
monitor sees: TinyALU BFM
owners: 2
driver sees: TinyALU BFM
```

Read figure 3 as a negotiation with Chapter 5. That chapter's rule — one value, one owner — has not been repealed; it has been *satisfied cleverly*. The `Rc` bookkeeping block is the value with the single owner story; the handles share it, and "who frees this?" is answered mechanically: the last handle out turns off the lights. This is why the method is spelled `Rc::clone(&bfm)` rather than hiding inside an assignment — the Rust convention is to make every refcount bump greppable, so that when you audit a testbench you can find each place ownership got shared and ask whether it earned its keep. Python bumped refcounts on every line and could not tell you where; Rust bumps them only where the code says `Rc::clone`.

[1] Mostly. CPython also runs a cycle-detecting garbage collector to rescue objects that point at each other and hold their mutual counts above zero forever. Hold that thought — `Rc` has the same weakness and no rescue squad, and it is going to matter later in this chapter.

So can several components share a BFM this way? Almost. There is a catch, and it is the same catch Chapter 6 stamped into `&T`: sharing is a *reader's* privilege.

```
// Figure 4: Shared owners are readers — Rc will not hand out write
access

use std::rc::Rc;

struct Scoreboard {
    errors: u32,
}

fn main() {
    let sb = Rc::new(Scoreboard { errors: 0 });
    let handle = Rc::clone(&sb);
    handle.errors += 1;
    println!("errors: {}", sb.errors);
}
```

```
--
error[E0594]: cannot assign to data in an `Rc`
 --> src/main.rs:9:5
  |
9 |     handle.errors += 1;
  |     ^^^^^^^^^^^^^^^^^^ cannot assign
  |
  = help: trait `DerefMut` is required to modify through a
dereference,
          but it is not implemented for `Rc<Scoreboard>`
```

Of course it refused. Two handles alive means two potential accessors, and *many readers or one writer, never both* applies to shared owners exactly as it applied to references. `Rc<T>` gives you Python's sharing but only the reading half of Python's behavior. For the writing half, we need the strangest and most instructive type in this chapter.

`RefCell<T>`: the borrow checker, moved to runtime

Everything the borrow checker has done so far, it has done at compile time, by proving things about your source code. But some sharing patterns are true in

ways a compiler cannot see from the source — *these two components take turns, I promise* — and for those, Rust offers a deal: **`RefCell<T>` keeps the borrowing rules but checks them at runtime.** Same law, different courtroom.

A `RefCell<T>` wraps a value and replaces `&` / `&mut` with two accessor methods: `borrow()` returns a read handle, `borrow_mut()` returns a write handle, and the cell *counts* its outstanding loans. Many readers, or one writer — enforced by a counter at the door instead of a proof in the compiler.

```
// Figure 5: Interior mutability — mutation through an immutable
binding

use std::cell::RefCell;

struct Scoreboard {
    errors: u32,
}

fn main() {
    let sb = RefCell::new(Scoreboard { errors: 0 });   // note: no
`mut`

    sb.borrow_mut().errors += 1;    // the write handle lives for
this line only

    println!("errors: {}", sb.borrow().errors);
}
```

```
--
errors: 1
```

Look hard at the first line of `main`: there is no `mut`. Since Chapter 3, that has meant untouchable — yet we mutated `errors` anyway. This trick has a name, **interior mutability**: from the outside, `sb` is an immutable value you can share freely; on the inside, the cell hands out exclusive write access to one caller at a time, checked at the moment of the call. If that sounds familiar, it should — it is the gatekeeper move from this chapter's opening, replayed at the level of the type system. Python's `Temperature` class hid `__temp` behind properties, and SystemVerilog's `local` fields hid behind accessor functions, so the class could enforce its rules at every access; `RefCell` hides its value behind `borrow` and `borrow_mut` so the *borrowing* rules get enforced at every access. All three

designs answer the same question — who gets to touch this value, and on whose terms? — by making traffic pass through a gatekeeper. The difference is what the gatekeeper checks: your old accessors checked whatever validation you wrote by hand; `RefCell` checks aliasing XOR mutability, the one rule this whole language is built on.

And what happens when the rule is violated? At compile time, a violation was a program that never existed. At runtime, the program exists — it is halfway through a simulation — so the only honest response left is to stop:

```
// Figure 6: The borrow checker at runtime — a panic replaces the
compile error

use std::cell::RefCell;

struct Scoreboard {
    errors: u32,
}

fn main() {
    let sb = RefCell::new(Scoreboard { errors: 0 });

    let reader = sb.borrow();          // a reader is at the
whiteboard...
    let mut writer = sb.borrow_mut();  // ...and a writer grabs the
marker

    writer.errors += 1;
    println!("reader saw: {}", reader.errors);
}
```

```
--
thread 'main' panicked at src/main.rs:12:25:
already borrowed: BorrowMutError
note: run with `RUST_BACKTRACE=1` environment variable to display a
backtrace
```

Set figure 6 next to Chapter 6's figure 3. They are the *same program* — one value, a live reader, a live writer, overlapping. In Chapter 6 the compiler rejected it with error E0502 and three annotated line numbers, before anything ran. Here it compiled without a murmur and then panicked at line 12, at runtime. That is the entire trade, in two figures: `RefCell` does not weaken the rule — a reader and a writer still cannot coexist, and the race Chapter 6 buried stays buried — but it moves the *discovery* of a violation from your desk to the running

program. In a testbench, “the running program” means seed 8,441, forty minutes in, with the license checked out. You have not escaped the borrow checker. You have volunteered to meet it later, at a worse time, with a stack trace instead of a source annotation.[2]

That framing tells you exactly when `RefCell` is legitimate: when you *know* the accesses cannot overlap — because your components take turns at await points, because the write handle lives for one expression as in figure 5 — but the knowledge lives in the design rather than in anything the compiler can verify from the source. Then the runtime check is not a time bomb; it is an assertion, permanently guarding an invariant you believe, and the panic is the assertion firing on the day you turn out to be wrong.

[2] A panic in a rustdv task does not take down the simulator, for the record — it is caught at the task boundary and scored as a test failure, the same way cocotb catches a stray exception per task. Cold comfort at seed 8,441, but comfort.

`Rc<RefCell<T>>`: a Python reference, made visible

Now stack them. `Rc` gave us shared ownership with read-only access; `RefCell` gave us gate-checked mutation of a shared value. Compose them — `Rc<RefCell<T>>` — and you have shared handles, any of which can mutate the value, with the borrow rules enforced at each access. Which is to say: you have rebuilt the Python object reference, the thing every variable in every Python program you ever wrote actually was.

Chapter 5 opened with a Python figure that proved `b = a` gives two names for one mutable object, and then showed Rust refusing to compile the same experiment. We have been in debt to that figure for eight chapters. Time to pay it off:

```rust
// Figure 7: Chapter 5's Python experiment, finally legal in Rust —
with its costs itemized

use std::cell::RefCell;
use std::rc::Rc;

#[derive(Debug)]
struct Transaction {
    op: String,
    a: u8,
    b: u8,
}

fn main() {
    let a = Rc::new(RefCell::new(Transaction {
        op: String::from("ADD"),
        a: 5,
        b: 3,
    }));
    let b = Rc::clone(&a);          // two names...

    b.borrow_mut().op = String::from("MUL");   // ...mutate through
one...

    println!("{:?}", a.borrow());              // ...observe
through the other
    println!("same object: {}", Rc::ptr_eq(&a, &b));
}
```

```
--
Transaction { op: "MUL", a: 5, b: 3 }
same object: true
```

Line for line, this is Chapter 5's figure 1: two names, one object, a mutation through `b` visible through `a`, and `Rc::ptr_eq` standing in for Python's `is`. What Python and SystemVerilog handles gave you invisibly and unconditionally, Rust sells you piecewise, each purchase named in the source: `Rc::new` (this value will be shared), `Rc::clone` (here is another owner), `borrow_mut()` (I want the marker, check me at the door), `borrow()` (just reading, count me). A reviewer can see every one of those decisions. In your old languages there was nothing to see — which is precisely why Python needed conventions begging users not to touch `_temp`, and why SystemVerilog grew the `local` keyword.

The bill

I owe you the honest price list, because "just wrap it in `Rc<RefCell<>>`" is the single most common way for a new Rust programmer to dig a hole.

Runtime checks that can panic. Every `borrow()` and `borrow_mut()` is a check that can fail, and figure 6 showed what failure looks like: a panic, at runtime, in whatever seed happens to trip it. The compile-time guarantee you have enjoyed since Chapter 6 is gone for this value; you are back to *testing* for the bug instead of being proven free of it.

Bookkeeping overhead. Refcount increments, borrow-flag checks — each is tiny, and unlike Python you pay only on the values you wrapped. But "tiny" is a per-access word, and monitors touch shared state per transaction, per seed, per regression. It adds up; budget for it consciously.

Reference cycles leak. Here the footnote from earlier comes due. CPython backs its refcounts with a cycle-collecting garbage collector; `Rc` has no such backstop. If a parent holds an `Rc` to its child and the child holds an `Rc` back to the parent, the counts never reach zero and the memory never frees — a leak, silent and permanent. (The standard library's `Weak` type exists to break such cycles; rustdv's design never needs it, for reasons one section away, so this book leaves it at a mention.)

The temptation. This is the real cost. Once you know `Rc<RefCell<T>>` exists, every borrow-checker error acquires an easy exit: wrap it, clone it, move on. Do that habitually and you will have written Python with worse syntax — every shared value a potential runtime panic, every refcount a small tax, and the compiler's proofs traded away for the debugging sessions this book keeps promising you left behind. The discipline is the one you already know from private attributes, pointed the other way: reach for the escape hatch deliberately, at a designed boundary, and let the default remain the default.

Why rustdv barely needs any of this

Which brings us to the question this chapter has been building toward: how much `Rc<RefCell<T>>` will the testbenches in the rest of this book actually

contain? You know how the UVM is built — in SystemVerilog and pyuvm alike, every child holds a handle to its parent, every parent holds its children, and a global tree (`uvm_root` 's registry, pyuvm's `component_dict`) holds a reference to everything. That is a graph full of exactly the parent-and-child cycles that make `Rc` leak, and it works in your old languages only because a garbage collector untangles what refcounts cannot. Port that structure naively and you would be signing up for `Rc<RefCell<T>>` everywhere, `Weak` back-references to break the cycles, and a runtime borrow check on every component access — the old architecture with Rust's ceremony, the worst of both worlds.

rustdv's design refuses the premise: **the component tree *is* the ownership tree**. An environment is a struct; its children are its fields; the parent owns them the way any struct owns its fields, and drops them at its own drop, in the way you have understood since Chapter 5. There is no back-pointer to a parent, no global registry, no cycle — so there is nothing for a refcount to get wrong and no shared mutation for a `RefCell` to referee. When the monitor needs to hand the scoreboard a transaction, it will not reach through a shared reference to poke scoreboard state; it will send the transaction down a channel, moving ownership the way Chapter 5's baton pass always wanted to. The hierarchy that forced the earlier dialects into pervasive implicit sharing simply is not shaped that way here.

What survives is the truly shared resource — the thing that really does have several users and really is one object. The canonical example, previewed now and built in Chapter 25: one `TinyAluBfm` wrapping the DUT interface, needed by a driver *and* a monitor at once. SystemVerilog engineers should hear "virtual interface in the config database" — the same job, and rustdv solves it the same way: the test creates the BFM once and files an `Rc<TinyAluBfm>` in the ConfigDb, and each component that needs it retrieves a counted handle by name. Notice which half of this chapter that uses: `Rc` alone, no `RefCell` , because the BFM's async methods take `&self` and the sharing is read-shaped from the outside. Shared mutability, where it appears in rustdv at all, is deliberate, boundary-marked, and rare — every use called out in the design rather than ambient in the architecture. The escape hatch exists, you now know exactly what it costs, and the framework's job is to make sure you almost never reach for it.

That completes the Rust you need for values: how they are owned, borrowed, shaped, collected, and — this chapter — shared on purpose. What you do not

yet know is how Rust programs are *organized*: where `use std::rc::Rc` has been coming from all this time, what a crate actually is, and how `cargo` turns a directory of files into the testbench library a simulator can load. Chapter 14 is about modules, crates, and cargo — and it ends with something neither of your languages ever gave you: unit tests that run in milliseconds, no simulator required.

Chapter 14: Modules, Crates, and Cargo

We have spent thirteen chapters writing Rust in single files. Real testbenches do not live in single files: SystemVerilog testbenches grow packages and the `.f` file lists that compile them in the right order; Python testbenches grow a `tinyalu_utils.py` holding the `Ops` enum and the prediction function, imported everywhere. This chapter is about where code *lives* in Rust — modules within a project, crates between projects, and `cargo` orchestrating all of it — and it ends with a payoff I have been promising since Chapter 1: running real unit tests against testbench logic with no simulator anywhere in sight.

> **In the UVM...** we shared code through each language's namespace machinery. SystemVerilog: `import uvm_pkg::*`, `` `include "uvm_macros.svh" ``, and a `.f` file whose compilation order somebody maintains by hand. Python: an `import` statement that *ran* the module's code and added its name to our scope, `from pyuvm import *` so that Python UVM code would look like SystemVerilog UVM code, and — when Python couldn't find `tinyalu_utils` — a helping shove: `sys.path.insert(0, str(Path("..").resolve()))`.

Hold on to those last items — the hand-ordered `.f` file and the `sys.path` shove — because they are what this chapter deletes. Rust has no search path to populate, no compilation order to curate, no import-time code execution, and no possibility of a testbench that works on your machine but not on the farm because of an environment variable. What Rust has instead is more ceremonial — I will not pretend otherwise — and in exchange the compiler knows exactly where every name comes from and exactly who is allowed to use it.

Modules: namespaces inside a crate

A **module** in Rust is a named scope you declare with the `mod` keyword. Unlike Python, where every file automatically *is* a module, a Rust module is something you declare explicitly — and you can declare one right in the middle of a file. Let's start there, because it makes the concept visible before any files get involved.

We will need a home for the TinyALU's prediction logic — the pure function that computes what the DUT *should* produce. This was `tinyalu_utils.py`'s most important resident, and it will be our example for the rest of the chapter.

```
// Figure 1: A module declared inline, in the middle of main.rs

mod predictor {
    #[derive(Clone, Copy, Debug, PartialEq)]
    pub enum Ops { Add = 1, And = 2, Xor = 3, Mul = 4 }

    pub fn alu_prediction(a: u8, b: u8, op: Ops) -> u16 {
        match op {
            Ops::Add => a as u16 + b as u16,
            Ops::And => (a & b) as u16,
            Ops::Xor => (a ^ b) as u16,
            Ops::Mul => a as u16 * b as u16,
        }
    }
}

fn main() {
    let sum = predictor::alu_prediction(0xFF, 0x01,
predictor::Ops::Add);
    println!("0xFF + 0x01 = {sum:#06x}");
}
```

```
--
0xFF + 0x01 = 0x0100
```

Everything between the braces of `mod predictor` lives in the `predictor` namespace, and code outside reaches it with `::` — `predictor::alu_prediction` — exactly the job SystemVerilog's `::` did in `uvm_pkg::uvm_component` (SV engineers may enjoy that Rust agrees with them about the spelling) and Python's `.` did in `pyuvm.FIFO_DEBUG`. Note in passing that the prediction itself is honest about widths in a way the Python version

never had to be: the TinyALU's operands are `u8` and its result bus is sixteen bits wide, so `Add` and `Mul` cast up to `u16` *before* operating. `0xFF + 0x01` carries into bit eight instead of wrapping to zero. Chapter 3 planted that seed; here it flowers.

`use`: bringing paths into scope

Writing `predictor::Ops::Add` at every call site gets old, and Rust's answer is the `use` declaration — the direct descendant of Python's `from ... import ...`.

```
// Figure 2: use brings names into scope, like Python's from-import

mod predictor {
    #[derive(Clone, Copy, Debug, PartialEq)]
    pub enum Ops { Add = 1, And = 2, Xor = 3, Mul = 4 }

    pub fn alu_prediction(a: u8, b: u8, op: Ops) -> u16 {
        match op {
            Ops::Add => a as u16 + b as u16,
            Ops::And => (a & b) as u16,
            Ops::Xor => (a ^ b) as u16,
            Ops::Mul => a as u16 * b as u16,
        }
    }
}

use predictor::{alu_prediction, Ops};

fn main() {
    println!("AND: {:#06x}", alu_prediction(0xF0, 0x3C, Ops::And));
    println!("XOR: {:#06x}", alu_prediction(0xF0, 0x3C, Ops::Xor));
}
```

```
--
AND: 0x0030
XOR: 0x00cc
```

The mapping to your old languages is nearly one-to-one:

SystemVerilog	Python	Rust
—	`import pyuvm`	*(nothing needed — see below)*
—	`import pyuvm as p`	`use pyuvm as p;` (aliasing works the same way)
`import uvm_pkg::uvm_driver`	`from pyuvm import FIFO_DEBUG`	`use pyuvm::FIFO_DEBUG;`
`import uvm_pkg::*`	`from pyuvm import *`	`use pyuvm::*;` (a *glob import*)

The first row deserves a word. In Python, `import` did two jobs: it *ran the module's code* and it added a name to your scope. Rust's `use` does only the second, because there is no first — modules do not "run" when you name them. All the code in every module of your program was compiled together before the program started; `use` is purely a naming convenience, with no import-time side effects, no circular-import deadlocks, and no module-level code sneaking configuration in behind your back.[1]

Rust's style community shuns glob imports for the same reason PEP-8 did — nobody reading the file can tell where a name came from. And it carves out the same exception UVM code has always carved out: crates may export a **prelude**, a curated module explicitly designed to be glob-imported. You have been using one all along — `std`'s prelude is why `String`, `Vec`, and `Option` never needed a `use`. When we reach Part II, `use rustdv::prelude::*;` will open every testbench, doing the job `import uvm_pkg::*` and `from pyuvm import *` have always done — one line, sanctioned by convention, because a testbench that starts by importing its methodology library is a testbench you can read.

[1] Readers who have debugged a cocotb testbench that behaved differently depending on *import order* may pause here for a private moment of celebration.

`pub`: privacy the compiler enforces

You may have noticed the `pub` keywords sprinkled through figures 1 and 2. They are not decoration. In Rust, **everything in a module is private by default** — invisible to code outside the module — and `pub` is how an item opts into being part of the module's public interface.

Recall how your old languages handled this. Python has no private anything: every name in every module is importable by anyone, and an underscore prefix (`_my_helper`) means *please don't* — etiquette, not enforcement; pyuvm's internals are full of underscored names that nothing actually stops you from reaching. SystemVerilog has `local` and `protected` inside classes, but nothing at the package level: every name a package declares is every importer's to take. Rust makes privacy the default at every level and replaces the etiquette with a compile error. Let's provoke one. Suppose the predictor grows a private helper that the outside world has no business calling:

```
// Figure 3: Private by default — the underscore convention,
enforced

mod predictor {
    pub enum Ops { Add = 1, And = 2, Xor = 3, Mul = 4 }

    fn widen(x: u8) -> u16 {     // no pub: private to this module
        x as u16
    }

    pub fn alu_prediction(a: u8, b: u8, op: Ops) -> u16 {
        match op {
            Ops::Add => widen(a) + widen(b),
            Ops::And => widen(a & b),
            Ops::Xor => widen(a ^ b),
            Ops::Mul => widen(a) * widen(b),
        }
    }
}

fn main() {
    let w = predictor::widen(0xFF);  // reaching for a private
helper
    println!("{w}");
}
```

```
--
error[E0603]: function `widen` is private
  --> src/main.rs:20:24
   |
20 |     let w = predictor::widen(0xFF);  // reaching for a private
helper
   |                        ^^^^^ private function
   |
note: the function `widen` is defined here
  --> src/main.rs:5:5
   |
5  |     fn widen(x: u8) -> u16 {     // no pub: private to this
module
   |     ^^^^^^^^^^^^^^^^^^^^^^
```

Delete the offending line and the program compiles; `alu_prediction`, living *inside* the module, calls `widen` freely. This is encapsulation with teeth. When you mark a helper private, you are not requesting politeness — you are making a promise the compiler keeps for you: *nothing outside this module depends on this function, so I may rewrite it tomorrow without checking.* Refactoring the guts of a monitor stops requiring a grep across the testbench for who might have reached in. Note also that privacy applies to struct *fields* individually: a `pub struct` can keep its fields private, forcing outsiders through its methods — Chapter 13's gatekeeper, now standing at module scope.

Modules in files: `mod foo;` finds `foo.rs`

Inline modules taught the concept, but real code puts modules in files. Here Rust asks for one more line of ceremony than Python did — and the line matters, so let's be precise about it.

In Python, creating `tinyalu_utils.py` *was* creating a module; the filesystem was the module system. In Rust, a file does not become part of your program until some module declares it. Writing `mod predictor;` — with a semicolon, no braces — tells the compiler: *there is a module named `predictor`, and its body lives in the file `predictor.rs` next to me.* Our playground project, reorganized:

```
# Figure 4: The file-to-module mapping

alu_playground/
├── Cargo.toml
└── src/
    ├── main.rs          <-- contains the line: mod predictor;
    └── predictor.rs     <-- the body of the module: Ops,
alu_prediction
--
% cargo run
   Compiling alu_playground v0.1.0
    Finished `dev` profile
     Running `target/debug/alu_playground`
AND: 0x0030
XOR: 0x00cc
```

Everything that was inside `mod predictor { ... }` moves into `src/predictor.rs` (dropping the wrapper braces and one level of indentation), `main.rs` declares `mod predictor;`, and nothing else changes — the `use` lines, the calls, the visibility rules are exactly as before. If `predictor` later needs child modules of its own, it graduates to a directory: `src/predictor.rs` alongside `src/predictor/history.rs`, with `mod history;` declared inside `predictor.rs`. (You will also meet an older layout, `predictor/mod.rs`, in existing code; the two styles are equivalent, and the older one's chief legacy is an editor tab bar reading `mod.rs`, `mod.rs`, `mod.rs`.[2])

Why the extra declaration? Because in Rust the *module tree is part of the program*, not an emergent property of whatever files happen to be on a search path. There is no `sys.path` to populate, no `PYTHONPATH` to export on every farm machine, no `.f` file whose ordering someone curates, no "works in this directory, fails in that one." The compiler starts at the crate root — `main.rs` for a program, `lib.rs` for a library — follows the `mod` declarations outward, and compiles precisely those files. A file nobody declares is not compiled at all. It is more ceremony than Python's file-is-a-module simplicity, one honest line more per module, and what the line buys is that your program's structure is written down in the program.

[2] A directory full of files all named `mod.rs` is nobody's favorite piece of Rust history. Use the `predictor.rs`-plus-directory style in new code.

Crates: the unit of compilation and sharing

One level up from modules sits the **crate** — the thing `cargo new` has been making for us all along. A crate is Rust's unit of compilation and distribution: the compiler compiles a crate at a time, and crates are what you publish, version, and depend on. Where Python drew a fuzzy line between "module," "package," and "distribution" (three concepts, two of which share the name *package*), and SystemVerilog's unit of sharing was a package plus the `.f` fragment that compiles it plus whatever tarball a vendor shipped it in, Rust draws one line: a crate is a tree of modules with a single root, and it compiles to a single artifact.

Crates come in two flavors you already half-know. A **binary crate** has a `main.rs` with a `fn main()` and compiles to a program — every playground so far. A **library crate** has a `lib.rs` and no `main`; it compiles to a library for other crates to use. `uvm_pkg`, cocotb, and pyuvm are the analogs of library crates; your testbench was the analog of a binary crate importing them.

When a project outgrows one crate, cargo scales up with a **workspace**: several crates developed side by side in one repository, sharing a build directory and a single lock file. This is not a distant abstraction — it is the shape of the very tools this book builds toward. `rustdv-sim` (the cocotb analog) and `rustdv` (the pyuvm analog) live as separate crates in one workspace, for the same reason cocotb and pyuvm are separate packages: you can write a Part II-style testbench against `rustdv-sim` alone, no UVM machinery in sight, and the dependency arrow only points one way. Your own testbenches, though, stay simple — one crate, depending on published ones. Which raises the question: *how* does a crate depend on another? That is `Cargo.toml`'s job.

Cargo.toml: the manifest

Every `cargo new` wrote a `Cargo.toml` we have been politely ignoring for thirteen chapters. It is the crate's **manifest** — the one file that says what the crate is and what it needs. Time to look inside, and to add our first dependency while we're there. Python's `random` came in the standard library; Rust's standard library is deliberately lean, and random numbers live in a crate called `rand` on **crates.io**, the community registry that plays the role of PyPI.

(SystemVerilog has no registry at all — sharing verification IP means tarballs and vendor portals, which is part of why every company's UVM library drifted apart.) Where Python needed `pip install` (into the right virtual environment, remember) plus a line in `requirements.txt`, cargo does both jobs with one command:

```
# Figure 5: Adding a dependency with cargo add

% cargo add rand
    Updating crates.io index
      Adding rand v0.9.1 to dependencies
--
# Cargo.toml, after:

[package]
name = "alu_playground"
version = "0.1.0"
edition = "2024"

[dependencies]
rand = "0.9.1"
```

Two sections, both readable at sight. `[package]` names and versions the crate itself. `[dependencies]` lists what it needs — and this section *is* the requirements file, living in the project, checked into version control, impossible to forget on the farm machines. The next `cargo build` (or `run`, or `test`) downloads `rand`, compiles it, and links it; there is no separate install step, no environment to activate, and no way to run against a different version than the manifest declares. Alongside the manifest cargo maintains `Cargo.lock`, recording the *exact* versions resolved, so that every machine that builds this crate builds it with identical dependencies — the reproducibility that `pip freeze` approximated, produced automatically and kept current. With `rand` declared, using it is just a path — `rand::random::<u8>()` gives the random operands our tests are about to want. No import dance; the dependency's name is the root of its module tree, available everywhere in your crate.

The payoff: `cargo test`

Now the capability I flagged in Chapter 1 as worth the price of admission. It has been sitting quietly inside cargo the whole time, waiting for us to have code worth testing. We do: `alu_prediction` is a pure function — values in, value out, no DUT, no signals, no simulator. In your old flows, logic like this could only be exercised by running the whole stack against a simulator; SystemVerilog cannot so much as parse your predictor without one. In Rust, testing it is built into the language and the tool. You write functions marked `#[test]`, and `cargo test` finds and runs them. The convention is a `tests` submodule at the bottom of the file whose code it tests:

```
// Figure 6: Unit tests live beside the code they test

// src/predictor.rs

#[derive(Clone, Copy, Debug, PartialEq)]
pub enum Ops { Add = 1, And = 2, Xor = 3, Mul = 4 }

pub fn alu_prediction(a: u8, b: u8, op: Ops) -> u16 {
    match op {
        Ops::Add => a as u16 + b as u16,
        Ops::And => (a & b) as u16,
        Ops::Xor => (a ^ b) as u16,
        Ops::Mul => a as u16 * b as u16,
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn add_carries_into_bit_eight() {
        assert_eq!(alu_prediction(0xFF, 0xFF, Ops::Add), 0x01FE);
    }

    #[test]
    fn and_masks_operands() {
        assert_eq!(alu_prediction(0xF0, 0x3C, Ops::And), 0x0030);
    }

    #[test]
    fn xor_finds_differing_bits() {
        assert_eq!(alu_prediction(0xF0, 0x3C, Ops::Xor), 0x00CC);
    }

    #[test]
    fn mul_needs_the_full_result_bus() {
        assert_eq!(alu_prediction(0xFF, 0xFF, Ops::Mul), 0xFE01);
    }
}
```

Every piece of this figure is machinery you already own. `mod tests` is an inline module, straight from figure 1. `use super::*;` is a glob import whose path `super` means “my parent module” — it pulls `Ops` and `alu_prediction` into the tests’ scope, and it is the second sanctioned use of a glob import, because a test module importing everything it tests is exactly as readable as a testbench importing its methodology library. `#[cfg(test)]` tells the compiler to build this module only when compiling tests, so the shipping artifact carries no test code.

And `assert_eq!` you met in Chapter 9, along with the taxonomy that governs it: assertion failures are for *bugs*, and a failed test is a bug by definition.

```
# Figure 7: Running the unit tests

% cargo test
   Compiling alu_playground v0.1.0
    Finished `test` profile [unoptimized + debuginfo]
     Running unittests src/main.rs
--
running 4 tests
test predictor::tests::add_carries_into_bit_eight ... ok
test predictor::tests::and_masks_operands ... ok
test predictor::tests::mul_needs_the_full_result_bus ... ok
test predictor::tests::xor_finds_differing_bits ... ok

test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 0
filtered out; finished in 0.00s
```

Read that last line the way it deserves to be read: *finished in 0.00s*. Four checks of the TinyALU's prediction logic, on a laptop, in less time than a simulator takes to print its banner. No license checked out, no design elaborated, no waveform dumped — because nothing here needed one. And when a test fails, the report is a diagnosis, not a stack trace. Suppose a future refactor forgets the width lesson and writes the addition as `(a + b) as u16`, wrapping at eight bits:

```
# Figure 8: A failing test names the culprit

% cargo test
--
running 4 tests
test predictor::tests::add_carries_into_bit_eight ... FAILED
test predictor::tests::and_masks_operands ... ok
test predictor::tests::mul_needs_the_full_result_bus ... ok
test predictor::tests::xor_finds_differing_bits ... ok

failures:

---- predictor::tests::add_carries_into_bit_eight stdout ----
assertion `left == right` failed
  left: 254
 right: 510

test result: FAILED. 1 passed; 1 failed; 0 ignored; 0 measured
```

`254` is `0xFF + 0xFF` wrapped to eight bits; `510` is the truth. The bug that would have surfaced as a scoreboard miscompare forty minutes into a regression — with the *DUT* as the initial suspect — instead surfaced in milliseconds, correctly attributed to the predictor, before any simulator ran.

Savor what just changed, because it is a new capability, not a nicer version of an old one. Your testbench's pure logic — predictors, transaction arithmetic, coverage binning, anything that computes without touching a signal — now has its own test suite that runs on every build, for free. Your old stacks kept all testbench verification inside the simulation; the testbench was only ever as tested as your last regression. From here on, this book runs `cargo test` habitually: when the later chapters build scoreboards and coverage collectors, their logic arrives with unit tests beside it, and the simulator's time is spent on the only thing that actually needs a simulator — the DUT.

Summary

In this chapter we gave Rust code a place to live. `mod` declares modules — inline for small things, `mod foo;` pointing at `foo.rs` for real ones — and the module tree is written in the program rather than discovered on a search path. `use` brings paths into scope the way `from ... import` did, glob imports and all, with preludes as the sanctioned exception. Everything is private until `pub` says otherwise, turning Python's underscore etiquette into a compiler-kept promise. Crates are the unit of compilation and distribution — Python's module/package/distribution muddle, resolved into one concept — with workspaces gathering related crates, as rustdv's own crates will be gathered. `Cargo.toml` declares dependencies, `cargo add` fetches them from crates.io, and `Cargo.lock` makes every build reproducible. And `cargo test` runs unit tests against pure testbench logic in milliseconds, no simulator required — a capability we will never stop using.

This chapter also closes Part I, so take a step back and look at what you now own. You came in knowing no Rust. You now hold the whole toolkit this book's testbenches are built from: ownership and borrowing, the responsibility-and-access model that replaces the garbage collector and turns data races into compile errors; structs, enums, and `match`, which gave `Ops` and four-state

logic honest types; `Result` and `Option`, where exceptions and sentinels used to be; traits, doing the work of inheritance, the dunder methods, and `do_compare`; generics, finishing what parameterized classes started; closures and iterators, which will carry the factory's job; the smart pointers that make sharing explicit; and now modules, crates, and cargo to organize and test all of it. Every one of these landed on ground your UVM experience prepared, and every one was chosen because a coming chapter needs it. What you cannot yet do is *wait* — no timer, no rising edge, no way to say "pause this task until something happens in the simulation." That is Part II's business. Chapter 15 takes up the question every testbench language must answer — how does software wait for hardware? — and shows how `async` / `await` works when the language gives you the syntax but hands *you* the engine. The simulator is finally in sight.

Interlude: The Complete TinyALU Testbench

> **In the UVM…** the destination was always the same summit: a TinyALU driven by sequences, checked by a scoreboard, measured by coverage, held together by the methodology — whether *The UVM Primer* built it in SystemVerilog or *Python for RTL Verification* built it in pyuvm, both ended there. This interlude shows you that summit in Rust — the complete, running rustdv testbench — *before* the climb. Nothing here is pseudocode: every line below is the shipped `tinyalu_tb` crate, and it runs on Icarus Verilog to `REGRESSION: PASS`.

Part I handed you fourteen chapters of language and kept saying they were load-bearing. This is the load. Read it the way you would walk through a finished house before studying the blueprints: do not try to understand it — try to *recognize* it. You know this testbench. You have built it in another language. The point of the next few pages is that when you squint, it is the tool you already own, spelled in the language you just learned — and the parts you cannot read yet each have a chapter with their name on it. The map of those chapters closes the interlude. Chapter 40 walks this same code with everything explained.

The transactions

```
// Figure 1: The TinyALU transactions (tinyalu_tb/src/alu_item.rs)

#[derive(Clone, Copy, Debug, PartialEq, Eq, Hash)]
#[repr(u8)]
pub enum Ops {
    Add = 1,
    And = 2,
    Xor = 3,
    Mul = 4,
}

#[derive(Clone, Debug, PartialEq)]
pub struct AluCommand {
    pub a: u8,
    pub b: u8,
    pub op: Ops,
}

#[derive(Clone, Debug, PartialEq)]
pub struct AluResult {
    pub result: u16,
}

/// Golden model (the scoreboard's predictor).
pub fn predict(cmd: &AluCommand) -> AluResult {
    let a = cmd.a as u16;
    let b = cmd.b as u16;
    let result = match cmd.op {
        Ops::Add => a + b,
        Ops::And => a & b,
        Ops::Xor => a ^ b,
        Ops::Mul => a * b,
    };
    AluResult { result }
}
```

This figure you can read completely — it is Chapters 4, 6, 7, and 10 doing their jobs. A transaction is a plain struct; no `uvm_sequence_item` base class, and the jobs the base class did arrive as derives: `Clone` is `do_copy`, `PartialEq` is `do_compare`, `Debug` is the printable form. The predictor is a function and a `match`.

The stimulus

```
// Figure 2: A sequence — stimulus as a program
(tinyalu_tb/src/sequences.rs)

/// How the operands get filled, once the driver is committed.
trait Operands {
    fn set_operands(&mut self, rng: &mut Rng, cmd: &mut
AluCommand);
}

/// Every operation, `n` times each — the walk all three sequences
share.
async fn all_ops<S: Operands>(
    seq: &mut S,
    ctx: &mut SeqCtx<AluCommand, AluResult>,
    n: usize,
) -> Result<(), SeqError> {
    let mut rng = ctx.rng();
    for _ in 0..n {
        for op in Ops::ALL {
            let mut cmd = AluCommand { a: 0, b: 0, op };
            ctx.start_item(&mut cmd).await?;
            // Late generation: the driver is waiting, so decide
now.
            seq.set_operands(&mut rng, &mut cmd);
            // Ownership moves to the driver here. A sequence that
needed the
            // command afterward would clone it first; this one
does not.
            ctx.finish_item(cmd).await?;
        }
    }
    Ok(())
}

/// Random operands across every operation, five times each.
#[derive(Default)]
pub struct RandomSeq;

impl Operands for RandomSeq {
    fn set_operands(&mut self, rng: &mut Rng, cmd: &mut AluCommand)
{
        cmd.a = rng.u8();
        cmd.b = rng.u8();
    }
}

impl Sequence for RandomSeq {
    type Req = AluCommand;
    type Rsp = AluResult;

    async fn body(&mut self, ctx: &mut SeqCtx<AluCommand,
```

```
AluResult>) -> Result<(), SeqError> {
        all_ops(self, ctx, 5).await
    }
}
```

`start_item`, `finish_item`, a body that loops the operations: the sequence idiom you know, and the comment about ownership moving is Chapter 5 speaking. What `SeqCtx` and `Sequence` are, and why the rendezvous has two calls, is Chapter 36's whole subject.

A component

```
// Figure 3: The driver (tinyalu_tb/src/components.rs)

#[derive(Component, Default)]
pub struct Driver {
    #[port(seq_item)]
    seq_item_port: SeqItemPort<AluCommand, AluResult>,
}

impl Component for Driver {
    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let bfm: Rc<TinyAluBfm> = ConfigDb::get(Some(ctx), "",
"BFM")?;
        bfm.reset().await;
        loop {
            let item = self.seq_item_port.get_next_item().await;
            bfm.send_op(item.payload().clone()).await;
            self.seq_item_port.item_done(None);
        }
    }
}
```

`get_next_item`, drive, `item_done` — `uvm_driver`'s loop, recognizable at a glance. Two things to merely notice, not yet understand: the driver takes *no constructor arguments* — the BFM arrives from something called the `ConfigDb`, by name — and its `run` returns a `Result`, with `?` doing what Chapter 9 taught.

The scoreboard

```
// Figure 4: The scoreboard — two streams in, verdicts in check
//           (tinyalu_tb/src/components.rs)

#[derive(Default)]
struct CmdLog {
    cmds: Vec<AluCommand>,
}

impl Subscriber<AluCommand> for CmdLog {
    fn write(&mut self, cmd: &AluCommand) {
        self.cmds.push(cmd.clone());
    }
}

#[derive(Default)]
struct ResultLog {
    results: Vec<AluResult>,
}

impl Subscriber<AluResult> for ResultLog {
    fn write(&mut self, res: &AluResult) {
        self.results.push(res.clone());
    }
}

#[derive(Component, Default)]
pub struct Scoreboard {
    #[port(subscribe)]
    cmd_in: SubscribePort<AluCommand>,
    #[port(subscribe)]
    result_in: SubscribePort<AluResult>,
    cmd_log: RustdvShared<CmdLog>,
    result_log: RustdvShared<ResultLog>,
    compared: usize,
    mismatches: usize,
}

impl Component for Scoreboard {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        self.cmd_in.subscribe(self.cmd_log.clone());
        self.result_in.subscribe(self.result_log.clone());
    }

    fn check(&mut self, ctx: &mut RustdvCtx, errors: &mut
CheckSink) {
        let cmd_log = self.cmd_log.get();
        let result_log = self.result_log.get();

        for (cmd, actual) in
cmd_log.cmds.iter().zip(result_log.results.iter()) {
            let expected = predict(cmd);
```

```
            self.compared += 1;
            if expected != *actual {
                self.mismatches += 1;
                ctx.info(&format!(
                    "scoreboard: in={cmd:?} out={actual:?}
expected={expected:?} check=FAIL"
                ));
                errors.error(format!(
                    "scoreboard mismatch: {cmd:?} -> got
{actual:?}, expected {expected:?}"
                ));
            } else {
                ctx.info(&format!(
                    "scoreboard: in={cmd:?} out={actual:?}
expected={expected:?} check=PASS"
                ));
            }
        }

        if cmd_log.cmds.len() != result_log.results.len() {
            errors.error(format!(
                "scoreboard: saw {} commands and {} results",
                cmd_log.cmds.len(),
                result_log.results.len()
            ));
        }
        if self.compared == 0 {
            errors.error("scoreboard: nothing was
compared".to_string());
        }
    }

    fn report(&mut self, ctx: &mut RustdvCtx) {
        ctx.info(&format!(
            "scoreboard: {} compared, {} mismatches",
            self.compared, self.mismatches
        ));
    }
}
```

A scoreboard that subscribes to two streams — commands and results — predicts with the figure-1 function, compares with `PartialEq`, and files its failures somewhere called a `CheckSink` inside a phase called `check`. Notice, without yet knowing why, that it holds a plain `Vec` for each stream, and that the last two error checks refuse to let “nothing arrived” look like “nothing failed.” There are also two monitors publishing onto the buses the scoreboard reads, and a coverage collector counting ops as a second subscriber on the command stream — the same shapes, not reprinted here.

The environment

```
// Figure 5: The environment — build creates, connect wires
//           (tinyalu_tb/src/env.rs)

#[derive(Component, Default)]
pub struct AluEnv {
    #[component]
    seqr: Sequencer<AluCommand, AluResult>,
    #[component]
    driver: RustdvComp,
    #[component]
    cmd_mon: RustdvComp,
    #[component]
    result_mon: RustdvComp,
    #[component]
    scoreboard: RustdvComp,
    #[component]
    coverage: RustdvComp,
    #[component]
    cmd_bus: AnalysisBus<AluCommand>,
    #[component]
    result_bus: AnalysisBus<AluResult>,
    is_active: bool,
    with_coverage: bool,
}

impl Component for AluEnv {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        let activity: Active = ConfigDb::get(Some(ctx), "",
"IS_ACTIVE").unwrap_or(Active::Active);
        self.is_active = activity == Active::Active;
        self.with_coverage = ConfigDb::get(Some(ctx), "",
"WITH_COVERAGE").unwrap_or(true);

        self.seqr = Sequencer::new();
        ConfigDb::set(None, "*", "SEQR", self.seqr.handle());

        if self.is_active {
            self.driver = Driver::create_comp();
        }
        self.cmd_mon = CmdMonitor::create_comp();
        self.result_mon = ResultMonitor::create_comp();
        self.scoreboard = Scoreboard::create_comp();
        if self.with_coverage {
            self.coverage = Coverage::create_comp();
        }

        self.cmd_bus = AnalysisBus::new();
        self.result_bus = AnalysisBus::new();
    }

    fn connect(&mut self, _ctx: &mut RustdvCtx) {
```

```
        if self.is_active {
            self.seqr
                .seq_item_export()
                .connect(&self.driver, Driver::SEQ_ITEM_PORT);
        }

        self.cmd_bus
            .pub_export()
            .connect(&self.cmd_mon, CmdMonitor::AP);
        self.cmd_bus
            .sub_export()
            .connect(&self.scoreboard, Scoreboard::CMD_IN);
        if self.with_coverage {
            self.cmd_bus
                .sub_export()
                .connect(&self.coverage, Coverage::CMD_IN);
        }

        self.result_bus
            .pub_export()
            .connect(&self.result_mon, ResultMonitor::AP);
        self.result_bus
            .sub_export()
            .connect(&self.scoreboard, Scoreboard::RESULT_IN);
    }

    fn start_of_simulation(&mut self, ctx: &mut RustdvCtx) {
        let bfm: Rc<TinyAluBfm> = ConfigDb::get(Some(ctx), "",
"BFM").expect("the test sets BFM");
        bfm.start_tasks();
    }
}
```

Here is the whole UVM vocabulary on one page: a `build` phase creating children top-down, a `connect` phase wiring them bottom-up, components built through something called a factory (`create_comp()`), an `IS_ACTIVE` knob read from the ConfigDb that decides whether a driver exists at all, and one broadcast bus per observed stream. If you have written a `uvm_env` , every line has a shape you have seen — down to the passive env that simply does not build its driver.

The tests

```
// Figure 6: Two tests, one testbench
(tinyalu_tb/src/tinyalu_tb.rs)

#[derive(Component, Default)]
pub struct BaseTest {
    #[component]
    env: RustdvComp,
}

impl Component for BaseTest {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        let bfm = TinyAluBfm::new(&ctx.dut()).expect("TinyALU
signals");
        ConfigDb::set(None, "*", "BFM", Rc::new(bfm));
        self.env = AluEnv::new_comp();
    }

    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let _obj = ctx.raise_objection("stimulus");

        let seqr: Sequencer<alu_item::AluCommand,
alu_item::AluResult> =
            ConfigDb::get(Some(ctx), "", "SEQR")?;

        let mut seq = create_seq::<BaseSeq>();
        seq.start(&seqr).await?;

        let bfm: Rc<TinyAluBfm> = ConfigDb::get(Some(ctx), "",
"BFM")?;
        bfm.wait_idle().await;

        ctx.info("sequence complete");
        Ok(())
    }
}

#[rustdv::test(timeout_time = 500, timeout_unit = "us")]
#[derive(Component, Default)]
struct RandomTest {
    #[component]
    inner: RustdvComp,
}

impl Component for RandomTest {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        set_seq_override::<BaseSeq, RandomSeq>();
        self.inner = BaseTest::new_comp();
    }
}
```

```rust
#[rustdv::test(timeout_time = 500, timeout_unit = "us")]
#[derive(Component, Default)]
struct MaxTest {
    #[component]
    inner: RustdvComp,
}

impl Component for MaxTest {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        set_seq_override::<BaseSeq, MaxSeq>();
        self.inner = BaseTest::new_comp();
    }
}
```

Two tests, and neither adds a component. Each names a different sequence for the factory to substitute and reuses everything else — the whole point of the methodology, visible in six lines of difference.

```
# Figure 7: The testbench running

      0.00ns INFO     rustdv: found 2 test(s), RUSTDV_RANDOM_SEED=1
      0.00ns INFO     running RandomTest (1/2)
[rustdv/tinyalu_tb/src/tinyalu_tb.rs:77]
     70.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 296 }
     70.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 193, b: 103, op: Add }
     90.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 10 }
     90.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 94, b: 11, op: And }
    110.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 57 }
    110.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 185, b: 128, op: Xor }
    130.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 165, b: 117, op: Mul }
    160.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 19305 }
    180.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 168, b: 150, op: Add }
    180.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 318 }
    200.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 97, b: 254, op: And }
    200.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 96 }
    220.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 192, b: 138, op: Xor }
    220.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 74 }
    240.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 168, b: 59, op: Mul }
    270.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 9912 }
    290.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 340 }
    290.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 99, b: 241, op: Add }
    310.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 8 }
    310.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 238, b: 8, op: And }
    330.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 218 }
    330.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 70, b: 156, op: Xor }
    350.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 205, b: 172, op: Mul }
    380.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 35260 }
    400.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
```

```
AluCommand { a: 159, b: 247, op: Add }
    400.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 406 }
    420.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 53, b: 171, op: And }
    420.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 33 }
    440.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 39, b: 138, op: Xor }
    440.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 173 }
    460.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 132, b: 186, op: Mul }
    490.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 24552 }
    510.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 137 }
    510.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 109, b: 28, op: Add }
    530.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 4 }
    530.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 23, b: 12, op: And }
    550.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 52 }
    550.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 245, b: 193, op: Xor }
    570.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 24, b: 60, op: Mul }
    600.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 1440 }
    630.00ns INFO     [RandomTest.inner]: sequence complete
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 193, b: 103, op: Add } out=AluResult
{ result: 296 } expected=AluResult { result: 296 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 94, b: 11, op: And } out=AluResult {
result: 10 } expected=AluResult { result: 10 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 185, b: 128, op: Xor } out=AluResult
{ result: 57 } expected=AluResult { result: 57 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 165, b: 117, op: Mul } out=AluResult
{ result: 19305 } expected=AluResult { result: 19305 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 168, b: 150, op: Add } out=AluResult
{ result: 318 } expected=AluResult { result: 318 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 97, b: 254, op: And } out=AluResult
{ result: 96 } expected=AluResult { result: 96 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 192, b: 138, op: Xor } out=AluResult
{ result: 74 } expected=AluResult { result: 74 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 168, b: 59, op: Mul } out=AluResult
```

```
{ result: 9912 } expected=AluResult { result: 9912 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 99, b: 241, op: Add } out=AluResult
{ result: 340 } expected=AluResult { result: 340 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 238, b: 8, op: And } out=AluResult {
result: 8 } expected=AluResult { result: 8 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 70, b: 156, op: Xor } out=AluResult
{ result: 218 } expected=AluResult { result: 218 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 205, b: 172, op: Mul } out=AluResult
{ result: 35260 } expected=AluResult { result: 35260 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 159, b: 247, op: Add } out=AluResult
{ result: 406 } expected=AluResult { result: 406 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 53, b: 171, op: And } out=AluResult
{ result: 33 } expected=AluResult { result: 33 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 39, b: 138, op: Xor } out=AluResult
{ result: 173 } expected=AluResult { result: 173 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 132, b: 186, op: Mul } out=AluResult
{ result: 24552 } expected=AluResult { result: 24552 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 109, b: 28, op: Add } out=AluResult
{ result: 137 } expected=AluResult { result: 137 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 23, b: 12, op: And } out=AluResult {
result: 4 } expected=AluResult { result: 4 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 245, b: 193, op: Xor } out=AluResult
{ result: 52 } expected=AluResult { result: 52 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 24, b: 60, op: Mul } out=AluResult {
result: 1440 } expected=AluResult { result: 1440 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: 20 compared, 0 mismatches
    630.00ns INFO     [RandomTest.inner.env.coverage]: coverage:
Add=5 And=5 Mul=5 Xor=5
    630.00ns INFO     RandomTest PASSED
    630.00ns INFO     running MaxTest (2/2)
[rustdv/tinyalu_tb/src/tinyalu_tb.rs:92]
    700.00ns INFO     [MaxTest.inner.env.result_mon]:
result_monitor: AluResult { result: 510 }
    700.00ns INFO     [MaxTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 255, b: 255, op: Add }
    720.00ns INFO     [MaxTest.inner.env.result_mon]:
result_monitor: AluResult { result: 255 }
    720.00ns INFO     [MaxTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 255, b: 255, op: And }
    740.00ns INFO     [MaxTest.inner.env.result_mon]:
result_monitor: AluResult { result: 0 }
    740.00ns INFO     [MaxTest.inner.env.cmd_mon]: cmd_monitor:
```

```
AluCommand { a: 255, b: 255, op: Xor }
    760.00ns INFO     [MaxTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 255, b: 255, op: Mul }
    790.00ns INFO     [MaxTest.inner.env.result_mon]:
result_monitor: AluResult { result: 65025 }
    820.00ns INFO     [MaxTest.inner]: sequence complete
    820.00ns INFO     [MaxTest.inner.env.scoreboard]: scoreboard:
in=AluCommand { a: 255, b: 255, op: Add } out=AluResult { result:
510 } expected=AluResult { result: 510 } check=PASS
    820.00ns INFO     [MaxTest.inner.env.scoreboard]: scoreboard:
in=AluCommand { a: 255, b: 255, op: And } out=AluResult { result:
255 } expected=AluResult { result: 255 } check=PASS
    820.00ns INFO     [MaxTest.inner.env.scoreboard]: scoreboard:
in=AluCommand { a: 255, b: 255, op: Xor } out=AluResult { result: 0
} expected=AluResult { result: 0 } check=PASS
    820.00ns INFO     [MaxTest.inner.env.scoreboard]: scoreboard:
in=AluCommand { a: 255, b: 255, op: Mul } out=AluResult { result:
65025 } expected=AluResult { result: 65025 } check=PASS
    820.00ns INFO     [MaxTest.inner.env.scoreboard]: scoreboard: 4
compared, 0 mismatches
    820.00ns INFO     [MaxTest.inner.env.coverage]: coverage: Add=1
And=1 Mul=1 Xor=1
    820.00ns INFO     MaxTest PASSED
**********************************************************************
***********
** TEST                                          STATUS  SIM TIME (ns)
**
**********************************************************************
***********
** RandomTest                                     PASS         630.00
**
** MaxTest                                        PASS         190.00
**
**********************************************************************
***********
REGRESSION: PASS
```

Twenty-four operations driven, predicted, compared, and counted, with every log line stamped with the path of the component that wrote it — and the run ends in the summary table and `REGRESSION: PASS`.

What you could already read, and where the rest is taught

Tally what Part I just let you read fluently: enums and `match` in the transactions and the predictor (Chapters 4, 7); ownership moving the command into

`finish_item`, with `clone()` as the alternative (Chapter 5); `Rc` where one BFM is truly shared (Chapter 13); `Result` and `?` threading every fallible step (Chapter 9); traits standing in for base classes, derives doing `uvm_object`'s jobs (Chapter 10); `SeqItemPort<AluCommand, AluResult>` and the other typed plumbing (Chapter 11); `Vec` and `HashMap` holding what the subscribers keep (Chapter 8); and a crate you could build and unit-test with `cargo` (Chapter 14).

What you took on faith is exactly the rest of the book. The page after this one — the rustdv Toolkit — names every framework identifier you just squinted at. Then: `async` / `await` and the executor underneath every `run` (Chapter 15), tasks and queues (Chapter 16), the simulator connection and the BFM (Chapters 17–19), the macros behind `#[rustdv::test]` and `#[derive(Component)]` (Chapter 21), tests as components (Chapter 23), the nine phases and the growing tree (Chapter 24), the ConfigDb that delivered the BFM (Chapters 25, 27–28), the factory behind `create_comp` and `create_seq` (Chapter 29), ports, FIFOs and the connect idiom (Chapter 31), the analysis buses and why the scoreboard owns its own `Vec`s (Chapter 32), testbench 6.0 wiring this very architecture (Chapters 33–34), transactions in full (Chapter 35), and the sequencer handshake (Chapter 36) with its response machinery (Chapters 37–38) and virtual sequences (Chapter 39). Chapter 40 then returns here, to this exact crate, and walks it with nothing left on faith.

The climb starts on the next page. It is worth it: at the top, this testbench is yours.

The rustdv Toolkit

Every listing from Chapter 15 on begins with the same line:

```
use rustdv::prelude::*;
```

It is the analog of `import uvm_pkg::*` in SystemVerilog and `from pyuvm import *` in Python, and it brings the testbench API into scope. Part I introduced every Rust concept before using it, and this page keeps that promise for the framework: it is the declaration site for the names the glob import hides. Skim it now to learn the shape of what rustdv provides, then return to it whenever a listing uses a name you have not met. Each entry names the chapter that teaches it properly, and Appendix D holds the complete alphabetical reference.

One crate, four layers

rustdv is a facade: one dependency in `Cargo.toml`, one import in the source. Behind the facade sit four layers, and knowing which layer a name comes from tells you what kind of thing it is.

rustdv-sim is the simulation layer — coroutines, triggers, tasks, queues, and signal handles. It does the job cocotb does for Python: it owns the event loop and talks to the simulator's scheduler. Everything in it makes sense in a testbench with no UVM anywhere.

rustdv-methodology is the UVM analog: components and phases, the ConfigDb, the factory, TLM ports and FIFOs, analysis broadcasting, and sequences. Everything in it corresponds to something you already know by another name.

rustdv-runner finds the registered tests in your compiled testbench and runs them — the job `run_test()` and the plusargs flow do in SystemVerilog, and the job cocotb's test discovery does in Python.

rustdv-gpi speaks VPI to the simulator, several layers beneath anything you write. The name is borrowed from cocotb's GPI deliberately: same job, same position in the stack.

Power users can reach whole layers as `rustdv::sim`, `rustdv::runner`, and `rustdv::gpi`. The prelude curates the surface a testbench needs.

`ctx`: the framework in your hand

rustdv has no globals. There is no `uvm_root`, no singleton pool, no parent pointer to climb — so everything the UVM lets you reach ambiently must be handed to you instead. The handing is done through one argument, conventionally named `ctx`, of type `RustdvCtx`. Every test receives it; every phase method receives it. The nearest UVM analogy is the `uvm_phase phase` argument every phase method already takes — rustdv widens that argument until it carries the whole framework.

`ctx` is also how a component knows *where it is*: it carries the component's path in the tree, which is why every log line arrives stamped with the full path and no component ever stores its own name.

You write	You get	Chapter
`ctx.dut()`	the handle to the top of the design	17
`ctx.info("...")` (and `warn`, `error`, ...)	a log line stamped with time and path	15, 26
`ctx.rng()`	the seeded per-test random generator	20
`ctx.raise_objection("why")`	an `ObjectionGuard` — the run phase ends when every guard is dropped	23

The simulation kit

From rustdv-sim. These are the names of a coroutine testbench, UVM or not.

Name	What it is, and when you reach for it	Chapter
`Timer`	the simulated-time trigger: `Timer::ns(2).await` *(SV: `#2ns`; cocotb: `Timer(2, "ns")`)*	15
`NullTrigger`	the trigger that is ready the next time anyone asks — the smallest possible await	15
`TestError`	the error a failing test returns; `Ok(())` is a pass	15
`spawn`, `spawn_named`	launch a concurrent task *(SV: `fork...join_none`; cocotb: `start_soon`)*; the named form stamps the task's log lines	16
`TaskHandle`	what `spawn` returns — await it for the task's result, or `cancel()` it	16
`Queue`	the sim-aware mailbox: a bounded `Queue` blocks a full `put` and an empty `get`, in simulated time *(SV: `mailbox#(T)`)*	16
`Event`	set once, and everyone waiting wakes *(SV: named `event`)*	16
`Lock`	mutual exclusion with an RAII guard *(SV: a one-key `semaphore`)*	16
`join2`, `first2`, `join!`, `first!`	run futures together and wait for both, or for the first *(SV: `fork...join` / `join_any`)*	16
`Clock`	a software clock driver — taught once and then retired, because rustdv BFMs wait on edges rather than make them	17
`LogicHandle`	a named signal in the design: read it, drive it; asking for a signal that does not exist is an `Err`, not a surprise	17
`Logic`, `LogicArray`	four-state values, kept out of your arithmetic until you decide what x means	17
`BigUint`	arbitrary-width unsigned values for buses beyond primitive integer widths	17
`RealHandle`, `StringHandle`	real and string variables, with scheduled and immediate writes	17
`AggregateHandle`	named members of unpacked structs and unions exposed through VPI	17

Name	What it is, and when you reach for it	Chapter
`HandleChildren`, `HandleEvent`	traits supplying hierarchy/member lookup and logic-edge triggers	17
`read_only`, `read_write`, `next_time_step`	await settled sampling, writable synchronization, or the next simulator time step	17
`service_read_only`	run a synchronous service closure at settled ReadOnly while simulation time is held	40
`HandleError`	what signal access returns instead of a crash	17, 19
`SimDuration`	an amount of simulated time	17
`Rng`	the deterministic random source behind `ctx.rng()` — one seed, one reproducible test	20
`log`	the logging facade the framework routes through `ctx`; policy is set per hierarchy	15, 26

A handful of other names — `with_timeout`, `sim_time_ns`, `Either`, `HierarchyHandle` — are in the prelude for completeness and cataloged in Appendix D. General child lookups return `rustdv::SimHandle`, which is outside the prelude. Name metadata comes from `rustdv::sim::handle::HandleBase`. Optional waveform control lives in `rustdv::sim::simulator_trace`; Chapter 40 explains its use with the Verilator host.

The structure kit

From rustdv-methodology: the component tree and its lifecycle.

Name	What it is, and when you reach for it	Chapter
`Component` (trait)	the lifecycle: `build`, `connect`, and the other phase methods a component may implement	24
`#[derive(Component)]`	writes the tree-traversal plumbing so your struct's children are found by the phases	21, 24
`ComponentNode`	what the derive implements — the thing a tree of components is made of	21, 24

Name	What it is, and when you reach for it	Chapter
`ObjectionGuard`	returned by `ctx.raise_objection` ; the run phase ends when the last one drops	23
`CheckSink`	the collector a `check` phase writes failures into; one error in it fails the test	24
`start_all`	drives a phase across a whole tree — the runner's job, never yours to call (its siblings `build_all` , `connect_all` , and the rest are in Appendix D)	24
`Active`	the active/passive knob an agent reads from the ConfigDb *(pyuvm's `is_active` int, as an enum)*	40

Configuration and the factory

Name	What it is, and when you reach for it	Chapter
`ConfigDb`	path-addressed runtime configuration: `set` by path and key, `get` returns a `Result` that names what went wrong	25, 27
`Factory` , `RustdvComp`	build components through a registry so a test can override *what* gets built — by type, by name, or by instance	29
`create_seq` , `set_seq_override` , `RustdvSeq`	the same idea for sequences: a slot the factory fills	36

The TLM kit

Name	What it is, and when you reach for it	Chapter
`PutPort` , `GetPort` , `PeekPort`	the directional ends a component declares; `connect` wires them at elaboration	31
`TlmFifo`	the FIFO two components share without ever learning each other's names — the point of decoupling	31
`RustdvShared`	a cloneable handle to one shared object — `Rc<RefCell>` wearing the framework's name	32

Name	What it is, and when you reach for it	Chapter
`PortName`, `PortOwner`	how the elaboration check names an unconnected port when it reports the whole tree at once	31

The analysis kit

Name	What it is, and when you reach for it	Chapter
`AnalysisBus`	the broadcast hub — it stores nothing; `write` calls every subscriber and returns	32
`PublishPort`, `SubscribePort`	the publishing and subscribing ends	32
`Subscriber`	the trait a subscriber implements per stream — two streams, two impls, no macros	32

The sequence kit

Name	What it is, and when you reach for it	Chapter
`Sequence`	the trait with one method: `body` — a test program, not a component	36
`Sequencer`	the component that grants sequences their turns and feeds the driver	36
`SeqItem`, `SeqCtx`, `SeqError`	the item's bounds, the sequence's context, and what can go wrong	36
`SeqItemPort`, `SeqItemExport`	the driver's side of the handshake	36
`TxnId`	the ticket `finish_item` returns; `get_response` claims its answer, in order or out of it	37, 38

The macros

Name	What it does	Chapter
`#[rustdv::test]`	registers a test with the runner *(cocotb: `@cocotb.test()`; SV: `+UVM_TESTNAME` machinery)*	15, 21
`#[derive(Component)]`	writes the component plumbing *(SV: the `uvm_component_utils` family)*	21, 24
`vpi_bootstrap!()`	one line per testbench crate: exports the entry points the simulator loads	17
`first!`, `join!`	the variadic forms of `first2` / `join2`	16

That is the toolkit. You do not need to hold it all — you need to know it is here, and that no listing from here on uses a name this page or an earlier chapter has not declared. When one seems to, that is a defect in the book, not in your memory.

Chapter 15: async/await and the Executor

Part I ended with a confession: you owned the whole Rust toolkit and still could not *wait*. No timer, no rising edge, no way to say "pause this task until something happens in the simulation." This chapter fixes that, and it fixes it at a level no earlier book in this series had to: by the end you will have written an event loop with your own hands, because Rust — unlike SystemVerilog or Python — hands you the syntax and lets you keep the engine.

> **In the UVM…** waiting was somebody else's engine. SystemVerilog's `@(posedge clk)` and `#2ns` compiled straight into the simulator's event wheel — the process suspends, the scheduler resumes it, and no testbench author ever sees the machinery. cocotb rebuilt the same experience in Python: coroutines defined with `async def`, the top one marked `@cocotb.test()`, triggers like `Timer(2, units="ns")` awaited against an event loop the module supplied.

Every word of that still applies. Rust's `async` / `await` is the same idea you already know — resumable functions parked until an event fires — and a rustdv test will look strikingly like a cocotb test. What differs is underneath, and the difference is the theme of this chapter: **Python's coroutines push, Rust's futures are pulled**, and Rust ships no event loop at all. cocotb had to write its own event loop because asyncio cannot block on simulator time. rustdv is in exactly the same position, and this time you get to see the machine.

Hello, world, once more

Ceremony first. Here is the simplest possible rustdv test, and your first look at Part II's figure convention: from here to the end of the book, most examples are *simulation directories* — a chapter crate built as a library the simulator loads,

run by a script, with Icarus Verilog doing the simulating.[1] The chapter crates live in the examples repository next to the playground projects you already know.

```
// Figure 1: Hello world as a test

use rustdv::prelude::*;

rustdv::vpi_bootstrap!();

#[rustdv::test]
async fn hello_world(_ctx: RustdvCtx) -> Result<(), TestError> {
    // Say hello!
    log::info("Hello, world.");
    Ok(())
}
```

```
--
      0.00ns INFO     running hello_world (1/2)  [ch15-async-await-
executor/src/ch15_async_await_executor.rs:12]
      0.00ns INFO     Hello, world.
      0.00ns INFO     hello_world PASSED
```

Read it against its Python twin. `@cocotb.test()` became `#[rustdv::test]` — a decorator became an attribute, and Chapter 21 is a whole chapter about what that attribute actually does. `async def hello_world(_)` became `async fn hello_world(_ctx: RustdvCtx)`; the underscore convention for an unused argument survives with a type on it. The docstring became a comment, and the test *returns* `Result<(), TestError>` — Chapter 9's failure taxonomy, now load-bearing: `Ok(())` passes, `Err` fails, and no exception machinery is anywhere involved. The two lines with no Python twin are the imports' big brother `use rustdv::prelude::*;` (the sanctioned glob from Chapter 14) and `rustdv::vpi_bootstrap!()`, a macro that exports the entry points the simulator calls when it loads our compiled testbench. cocotb hid the equivalent plumbing inside its makefiles; Rust puts one visible line in the file, and Chapter 17 explains the loading story it belongs to.

The log line should feel like home: simulated time, level, message — the same format down to the column widths, on purpose.

What `async` actually builds

In your old testbenches you treated suspendable processes as a given and let the simulator — or cocotb — worry about resuming them. That was the right call then, and it would be the wrong call now, because in Rust the resuming machinery is *your* code. So let's look inside.

When the Rust compiler sees `async fn`, it does not create a function that runs your code. It creates a function that returns a **state machine** — a value implementing the `Future` trait, frozen at its starting line. The `Future` trait has one method:

```
fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) ->
Poll<Self::Output>;
```

`poll` means: *make as much progress as you can right now.* The answer is either `Poll::Ready(value)` — finished, here's the result — or `Poll::Pending` — parked at an `await`, ask again later. Nothing about a future is concurrent, threaded, or magical; it is a struct with a resume method, and we can drive one manually:

```
// Figure 2: Polling a future by hand

use std::future::Future;
use std::pin::pin;
use std::task::{Context, Poll, Waker};

async fn add_one(n: u32) -> u32 {
    println!("the future ran");
    n + 1
}

fn main() {
    let mut fut = pin!(add_one(2));
    let mut cx = Context::from_waker(Waker::noop());
    println!("polling...");
    match fut.as_mut().poll(&mut cx) {
        Poll::Ready(v) => println!("poll returned Ready({v})"),
        Poll::Pending => println!("poll returned Pending"),
    }
}
```

```
--
polling...
the future ran
poll returned Ready(3)
```

Two observations, one per output line. First: "the future ran" printed *after* "polling..." — calling `add_one(2)` executed none of its body. An `async fn` runs only when polled, which is Python's lazy-coroutine behavior (`coro` did nothing until the event loop sent into it) with the laziness made structural. Second: this future finished in one poll, because it never awaited anything. The `pin!` and `Waker::noop` incantations are scaffolding we'll justify in a moment; what matters is that you have now personally done the executor's job.

The Python contrast is worth one more sentence, because it is the deepest engine difference between the two books. A cocotb coroutine, resumed with `coro.send(None)`, runs until it *yields a trigger outward* — the coroutine pushes the thing it's waiting for up to the scheduler. A Rust future is *polled from outside* and answers Ready or Pending. Push became pull. Everything else in this chapter follows from that inversion.

Pending, and the waker contract

A future that never says `Pending` never waits, and waiting is our whole business. Here is the smallest future that parks itself — not ready the first time you ask, ready the second:

```rust
// Figure 3: A future that says Pending — the trigger's whole job

use std::future::Future;
use std::pin::{pin, Pin};
use std::task::{Context, Poll, Waker};

/// The simplest possible trigger: not ready the first time you
ask,
/// ready the second time. (rustdv's NullTrigger is exactly this.)
struct YieldOnce {
    yielded: bool,
}

impl Future for YieldOnce {
    type Output = ();

    fn poll(mut self: Pin<&mut Self>, cx: &mut Context<'_>) ->
Poll<()> {
        if self.yielded {
            Poll::Ready(())
        } else {
            self.yielded = true;
            cx.waker().wake_by_ref(); // "poll me again"
            Poll::Pending
        }
    }
}

fn main() {
    let mut fut = pin!(async {
        println!("before the await");
        YieldOnce { yielded: false }.await;
        println!("after the await");
    });
    let mut cx = Context::from_waker(Waker::noop());

    println!("first poll:  {:?}", fut.as_mut().poll(&mut cx));
    println!("second poll: {:?}", fut.as_mut().poll(&mut cx));
}
```

```
--
before the await
first poll:  Pending
after the await
second poll: Ready(())
```

Watch the interleaving: the first poll ran the async block *up to* the `await`, hit `YieldOnce`, got `Pending`, and stopped — mid-function, state saved. The second poll resumed *at the await* and ran to the end. That suspend-and-resume

is exactly what `coro.send(None)` did to a Python coroutine when it yielded a trigger; you are watching the same movie from the projectionist's booth.

Now the one new obligation. Before returning `Pending`, our future called `cx.waker().wake_by_ref()`. That `Waker` — riding along in the `Context` argument — is the pull-model's return address: a cheap handle meaning *this task can make progress again; put it back on the run queue*. The contract is: **a future that returns `Pending` must arrange for the waker to be called when it becomes ready**, or nobody will ever poll it again and it sleeps forever. `YieldOnce` fulfills the contract trivially (wake immediately: "poll me again right away"). A real trigger fulfills it meaningfully: rustdv's `Timer` hands its waker to the simulator with "call this in 2 simulated nanoseconds," and a rising-edge trigger hands its waker to a callback on the signal. If you remember cocotb's `TriggerCallback` — the little object that reschedules a task when its trigger fires — you have already met the waker wearing a Python costume.[2]

The scaffolding, briefly, and then we can stop noticing it. `Pin` is Rust's promise that a self-referential state machine won't be moved in memory between polls (the compiler builds futures whose fields point into themselves; moving one would tear it). The `pin!` macro makes that promise for a local. You will read past `Pin` everywhere in this book after today; the framework carries it where it must. `Waker::noop()` is a do-nothing waker, fine for hand-cranking on a workbench, useless in production, replaced by a real one in about a page.

[1] Instructions live in the examples repository's `README.md`, and each chapter's directory has a run script. The DUT for this chapter is a Verilog module that is deliberately, perfectly empty — we need the simulator's clock of simulated time, not its talent for hardware.

[2] cocotb: `_base_triggers.py`, class `TriggerCallback`. Rust: `std::task::Waker`. Same job, same lifecycle, eleven fewer lines.

The event loop you now get to keep

Here is the sentence this chapter has been building toward: **Rust ships `async` / `await` as pure language, and ships no event loop at all.** There is no asyncio in the standard library. Production Rust services reach for a runtime crate like tokio; we will not, for the same reason cocotb never ran on asyncio's loop — a general-purpose runtime owns its event loop and blocks on OS I/O, and our events come from a simulator that insists on being in charge. An executor that serves a simulator must be a *guest*: wake some tasks, drain the queue, and give control back. cocotb wrote exactly such a loop in 82 lines of Python. rustdv's is about the same size, and the heart of it fits in one figure:

```
// Figure 4: An event loop in a page

use std::cell::RefCell;
use std::collections::VecDeque;
use std::future::Future;
use std::pin::Pin;
use std::task::{Context, Poll, Waker};

thread_local! {
    /// The run queue: tasks that are ready to make progress.
    static RUN_QUEUE: RefCell<VecDeque<usize>> =
RefCell::new(VecDeque::new());
}

/// Yield control: reschedule myself, then say Pending once.
struct YieldNow(bool, usize);

impl Future for YieldNow {
    type Output = ();
    fn poll(mut self: Pin<&mut Self>, _cx: &mut Context<'_>) ->
Poll<()> {
        if self.0 {
            Poll::Ready(())
        } else {
            self.0 = true;
            let id = self.1;
            RUN_QUEUE.with(|q| q.borrow_mut().push_back(id)); //
wake: requeue
            Poll::Pending
        }
    }
}

fn main() {
    let count = |name: &'static str, n: u32, id: usize| async move
{
        for i in 1..=n {
            println!("{name} counts {i}");
            YieldNow(false, id).await;
        }
    };

    // The task arena: every spawned future, boxed and pinned.
    let mut tasks: Vec<Pin<Box<dyn Future<Output = ()>>>> =
        vec![Box::pin(count("The Count", 5, 0)),
Box::pin(count("Mom", 3, 1))];

    // Seed the queue, then drain it to exhaustion: the whole event
loop.
    RUN_QUEUE.with(|q| q.borrow_mut().extend([0, 1]));
    let mut cx = Context::from_waker(Waker::noop());
    while let Some(id) = RUN_QUEUE.with(|q|
q.borrow_mut().pop_front()) {
        let _ = tasks[id].as_mut().poll(&mut cx);
```

```
    }
    println!("run queue empty: the loop returns");
}
```

```
--
The Count counts 1
Mom counts 1
The Count counts 2
Mom counts 2
The Count counts 3
Mom counts 3
The Count counts 4
The Count counts 5
run queue empty: the loop returns
```

The Count and Mom are back,[3] and they are interleaving with no threads, no simulator, and no runtime crate — just a `VecDeque` of task ids and a `while let` that drains it. Follow one bounce: The Count prints, awaits `YieldNow`, which pushes his id back onto the queue and returns `Pending`; the loop pops the next id, which is Mom's; she prints and yields the same way. Cooperative multitasking, four moving parts: a **run queue** of ready tasks, a **task arena** owning the futures, **wake** meaning "push the id onto the queue," and a **drain loop** polling until the queue is empty. When you hear "executor" for the rest of this book, this figure is the whole referent — rustdv's adds bookkeeping (task states, results, names for log messages) but no new ideas.

And the drain loop's exit condition is not a detail — it is the *interface to the simulator*. cocotb's event loop runs "to exhaustion" every time a simulator callback fires, then returns control to the simulator until the next callback. rustdv's `run_until_idle` does precisely the same. The executor is a guest in the simulator's house: it works through whatever became ready, then hands the clock back. A general-purpose runtime could never be persuaded to behave this politely, which is why we own the loop.[4]

[3] From the Python book's figure 9, where The Count counted to five while Mom counted to three, interleaved. There they yielded to the scheduler by awaiting timers; here, by explicit `YieldNow`. Chapter 16 restores their timers.

[4] The full chain, which Chapter 17 walks: simulator callback fires → trigger wakes its subscribers (ids onto the run queue) → `run_until_idle` drains the queue → control returns to the simulator, which advances simulated time to the next event.

Awaiting simulated time

With the engine understood, the rest of the chapter is a homecoming. Every language in this series has a way to say "consume simulated time," and two of them deserve reprinting exactly, because they have not changed and neither has the point:

```
# Figure 5: VHDL waits for 2 nanoseconds

process is
begin
  wait for 2 ns;
  report "I am DONE waiting!";
  wait;
end process;
```

```
# Figure 6: SystemVerilog waits for two nanoseconds

initial begin
  #2ns;
  $display("I am DONE waiting!");
end
```

A VHDL process, a SystemVerilog task, a Python coroutine — and now a Rust future. Time-consuming behavior, expressed as code that suspends mid-body:

```rust
// Figure 7: Rust waits for 2 nanoseconds

#[rustdv::test]
async fn wait_2ns(_ctx: RustdvCtx) -> Result<(), TestError> {
    // Waits for two ns then prints
    Timer::ns(2).await;
    log::info("I am DONE waiting!");
    Ok(())
}
```

```
--
      0.00ns INFO     running wait_2ns (2/2)  [ch15-async-await-
executor/src/ch15_async_await_executor.rs:20]
      2.00ns INFO     I am DONE waiting!
      2.00ns INFO     wait_2ns PASSED
```

The test started at 0.00ns and logged at 2.00ns: two nanoseconds of *simulated* time passed, no wall-clock sleeping involved. `Timer(2, units="ns")` became `Timer::ns(2)` — a constructor per unit rather than a string argument, so `Timer::ns`, `Timer::us`, `Timer::ms` are distinct functions and a typo'd unit string is impossible rather than discovered at runtime. And you now know precisely what that innocent `.await` did: the test's future returned `Pending`, `Timer` handed its waker to the simulator with instructions for 2ns hence, the executor's queue ran dry, control went back to the simulator, simulated time advanced, the callback fired, the waker requeued the test, and the drain loop polled it awake on the far side of the await. Ten steps, all of which you have now either written or watched.

One habit to carry forward: `Timer` is for *modeling time*, never for synchronization. Every dialect has scars behind that rule — SystemVerilog testbenches paced by `#delay` guesses, cocotb's NullTrigger race — and "no sleeps for coordination" survives translation intact. Chapter 16's queues and events are the right tools, exactly as their ancestors were.

Summary

This chapter opened the box every earlier dialect kept sealed. Rust's `async fn` compiles to a state machine implementing `Future`, whose one method `poll`

answers `Ready` or `Pending` — the push-a-trigger-out model of cocotb and the simulator's event wheel, inverted into a pull. A future that returns `Pending` owes the executor a wake-up call, delivered through the `Waker` riding in every poll — cocotb's `TriggerCallback`, standardized into the language. Rust ships no event loop, which stopped being bad news the moment we wrote one in a page: a run queue, a task arena, wake-as-requeue, and a drain loop that returns control to whoever owns the events — for us, always, the simulator. On top of that engine, the user-facing surface came home unchanged: `#[rustdv::test]` marks the top-level coroutine, `Timer::ns(2).await` consumes simulated time, and the log reads like the log always read.

What we built by hand today, `rustdv-sim` provides for keeps: a real spawner, task handles you can await and cancel, and the sim-aware queues that make producer/consumer testbenches safe. That is Chapter 16 — where The Count gets his timer back, and where we meet the one place Rust's task model diverges from cocotb's, in the matter of killing a task.

Chapter 16: Tasks, Channels, and Sim-Aware Queues

Chapter 15 built the engine; this chapter builds the traffic. Testbenches are crowds of concurrent behaviors — a driver wiggling pins, monitors watching them, a scoreboard judging — and this chapter ports the two Python-book chapters that made such crowds manageable: launching coroutines as background *tasks*, and letting tasks talk through *queues*. It ends at the one place where the Rust model diverges from cocotb's, which is what happens when you kill a task.

> **In the UVM…** we ran things in parallel and wired them together. SystemVerilog forked processes with `fork...join_none`, reaped them with `join` / `join_any`, and killed them with `disable`; producers and consumers shared data through a `mailbox #(T)` with blocking `put()` / `get()` and nonblocking `try_put()` / `try_get()`. cocotb spelled the same ideas `start_soon()` — returning a `RunningTask` to await, ignore, or `kill()` — `Combine()` / `First()` for groups, and `cocotb.queue.Queue` for the data.

All of it is here, most of it under a light Rust accent. And thanks to Chapter 15, none of it is magic: you know what a future is and what the executor does, so `spawn` is about to be a very short story.

Starting tasks

`rustdv::spawn()` is `cocotb.start_soon()`: hand it a future, get back a handle, and the task starts running at the next turn of the event loop. Our lab animal is the Python book's counter, returned from Sesame Street duty:

```rust
// Figure 1: counter counts up with a delay

async fn counter(name: &'static str, delay: u64, count: u32) {
    // Counts up to the count argument after delay
    for ii in 1..=count {
        Timer::ns(delay).await;
        log::info(&format!("{name} counts {ii}"));
    }
}
```

Note what `counter` is *not*: not a test, not registered with anything, no attribute above it. It is a plain `async fn` — a function that builds a future — and any task may await it or spawn it. The `&'static str` for the name is Chapter 8's string-slice economics (these names are literals living in the program binary; no `String` allocation needed).

Ignoring a running task

First, the cautionary opener — launch it and walk away:

```rust
// Figure 2: Launching a task and ignoring it

#[rustdv::test]
async fn do_not_wait(_ctx: RustdvCtx) -> Result<(), TestError> {
    // Launch a counter
    log::info("start counting to 3");
    spawn(counter("simple count", 1, 3));
    log::info("ignored the running task");
    Ok(())
}
```

```
--
      0.00ns INFO     start counting to 3
      0.00ns INFO     ignored the running task
      0.00ns INFO     do_not_wait PASSED
```

Well, that was unsatisfying, the second time in two books. The counter never counted: the test returned `Ok(())` at time zero, the test manager ended the test and cancelled its surviving children, and the counter died before its first

timer fired. Same lesson as Python: fire-and-forget is for free-running behavior — BFM loops, clock drivers — not for work you need finished.

Awaiting a running task

`spawn` returns a `TaskHandle`, and a `TaskHandle` is itself a future — awaiting it means "block until that task completes," exactly like `await running_task` in cocotb 2.x:

```
// Figure 3: Waiting for a running task

#[rustdv::test]
async fn wait_for_it(_ctx: RustdvCtx) -> Result<(), TestError> {
    // Launch a counter
    log::info("start counting to 3");
    let running_task = spawn(counter("simple count", 1, 3));
    let _ = running_task.await;
    log::info("waited for running task");
    Ok(())
}
```

```
--
      0.00ns INFO     start counting to 3
      1.00ns INFO     simple count counts 1
      2.00ns INFO     simple count counts 2
      3.00ns INFO     simple count counts 3
      3.00ns INFO     waited for running task
      3.00ns INFO     wait_for_it PASSED
```

The `let _ =` deserves its sentence now, because it is not noise. Awaiting a `TaskHandle<T>` yields `Result<T, TaskError>` — because between spawn and completion, someone might *cancel* the task, and a cancelled task has no value to give. Rust will not let you silently ignore that possibility (the `Result` is `#[must_use]`); `let _ =` is you telling the compiler, visibly, that you accept either outcome. Our counter returns `()` and nobody cancels it, so discarding is right. When the value matters, handle the `Result` — which is figure 7's subject.

Running tasks in parallel

The Count counts to five on a one-nanosecond stride; Mom counts to three on a two-nanosecond stride, with consequences implied. `Combine()` becomes `join2`:

```
// Figure 4: Mom and The Count count in parallel

#[rustdv::test]
async fn counters(_ctx: RustdvCtx) -> Result<(), TestError> {
    // Test that starts two counters and waits for them
    log::info("The Count will count to five.");
    log::info("Mom will count to three.");
    let the_count = spawn(counter("The Count", 1, 5));
    let mom_warning = spawn(counter("Mom", 2, 3));
    let _ = join2(the_count, mom_warning).await;
    log::info("All the counting is finished");
    Ok(())
}
```

```
// Figure 5: Mom and The Count's interleaved output
--
      3.00ns INFO     The Count will count to five.
      3.00ns INFO     Mom will count to three.
      4.00ns INFO     The Count counts 1
      5.00ns INFO     Mom counts 1
      5.00ns INFO     The Count counts 2
      6.00ns INFO     The Count counts 3
      7.00ns INFO     Mom counts 2
      7.00ns INFO     The Count counts 4
      8.00ns INFO     The Count counts 5
      9.00ns INFO     Mom counts 3
      9.00ns INFO     All the counting is finished
```

Interleaved counting, cooperative multitasking, one thread — Chapter 15's run queue doing exactly what you watched it do, now with the simulator supplying the wake-ups.[1] `join2` waits for both tasks and returns both results as a tuple; rustdv also provides `first2` (cocotb's `First()`, SystemVerilog's `join_any`), which returns when the *first* future finishes — and, in a very Rust move, *drops* the loser. Hold that thought two sections.

[1] The timestamps start at 3.00ns because the chapter's tests run back-to-back in one simulation, and figure 3 ended at 3ns. The Python book

trimmed such artifacts from its outputs; we keep them, because a regression is one simulation and the clock does not reset between tests.

Returning values from tasks

Tasks can return values, and the value comes out where cocotb's did — by awaiting the handle:

```
// Figure 6: A coroutine that increments a number
// and returns it after a delay

async fn wait_for_numb(delay: u64, numb: u32) -> u32 {
    // Waits for delay ns and then returns the increment of the
number
    Timer::ns(delay).await;
    numb + 1
}
```

```
// Figure 7: Getting a return value by awaiting the TaskHandle

#[rustdv::test]
async fn inc_test(_ctx: RustdvCtx) -> Result<(), TestError> {
    // Demonstrates spawn() return values
    log::info("sent 1");
    let inc1 = spawn(wait_for_numb(1, 1));
    let nn = inc1.await.expect("task was cancelled");
    log::info(&format!("returned {nn}"));
    log::info(&format!("sent {nn}"));
    let inc2 = spawn(wait_for_numb(10, nn));
    let nn = inc2.await.expect("task was cancelled");
    log::info(&format!("returned {nn}"));
    Ok(())
}
```

```
--
      9.00ns INFO     sent 1
     10.00ns INFO     returned 2
     10.00ns INFO     sent 2
     20.00ns INFO     returned 3
     20.00ns INFO     inc_test PASSED
```

One nanosecond for the first increment, ten for the second, values flowing back through `await` — figure-for-figure with the Python book. The new element is `.expect(...)`: since awaiting a handle yields `Result<u32, TaskError>`, we must say what happens if the task was cancelled out from under us. Here that would be a testbench bug, and Chapter 9's taxonomy says testbench bugs panic — `expect` is the assertion that documents it.

Cancelling a task: the one real divergence

Now the section this chapter has owed you since its title. cocotb kills a task by throwing `CancelledError` *into* the coroutine: the task's `finally` blocks run, it can do last-wish cleanup, it can even (buggily) refuse to die. Rust has no exceptions to throw and no way to run code *inside* a future that is not being polled. Cancellation in Rust is: **the executor drops the future.** The state machine is destroyed where it stands; anything it owned is dropped with it; no task-side code runs after the drop point.

The user-facing surface barely changes:

```
// Figure 8: Cancelling a task — Rust's kill()

#[rustdv::test]
async fn cancel_a_running_task(_ctx: RustdvCtx) -> Result<(),
TestError> {
    // Cancel a running task
    let kill_me = spawn(counter("Kill me", 1, 1000));
    Timer::ns(5).await;
    kill_me.cancel();
    log::info("Cancelled the long-running task.");
    Ok(())
}
```

```
--
     21.00ns INFO     Kill me counts 1
     22.00ns INFO     Kill me counts 2
     23.00ns INFO     Kill me counts 3
     24.00ns INFO     Kill me counts 4
     25.00ns INFO     Cancelled the long-running task.
     25.00ns INFO     cancel_a_running_task PASSED
```

Four counts in five nanoseconds, then silence — indistinguishable from `kill()` at this range. The divergence appears only when the dying task *owned cleanup responsibilities*, and Rust's answer has two halves.

The first half is good news, and it is most of the story: cleanup you would have written in a `finally` block moves into `Drop` implementations on whatever the task holds — Chapter 13's RAII, now applied to task death. A task holding a `LockGuard` releases the lock when cancelled, *automatically*, because dropping the future drops the guard. The whole class of cocotb bugs where a killed task forgot its `finally` — or caught `CancelledError` and failed to re-raise it, which cocotb has dedicated machinery to detect — cannot be written.

The second half is the honest loss: a cancelled Rust task cannot *await* during its last moments. cocotb code that, on kill, drove a bus back to idle over several clock cycles has no direct translation, because dropping is synchronous. The idiom that replaces it is **shutdown by message**: instead of killing the driver, send it a "stop" item through the very queues this chapter teaches (or set an `Event` it checks), and await its handle while it winds down on its own terms. Ask first, rather than shoot and clean up. We will not need the idiom for the TinyALU — its tasks are all stateless loops, safe to drop anywhere — but it is recorded here because someday your DUT will care what the bus does during the funeral.

Task communication: the sim-aware Queue

With tasks running in parallel, they need to share data — in order, without races. The old answers were SystemVerilog's `mailbox #(T)` and cocotb's `Queue`; rustdv's is `sim::Queue<T>`: same blocking `put` / `get`, same nonblocking variants, executor-aware so that a blocked task parks itself with the executor rather than spinning.[2] One Rust twist up front: the queue is typed, always. A `Queue<u32>` carries `u32`s and nothing else; the Python queue carried anything — as did the default, unparameterized SV mailbox — and a producer that put the wrong thing in was the consumer's runtime problem. Here it is the producer's compile error.

```
// Figure 9: A coroutine using a Queue to send data

async fn producer(queue: Queue<u32>, nn: u32, delay: Option<u64>) {
    // Produce numbers from 1 to nn and send them
    for datum in 1..=nn {
        if let Some(d) = delay {
            Timer::ns(d).await;
        }
        queue.put(datum).await;
        log::info(&format!("Producer sent {datum}"));
    }
}
```

```
// Figure 10: A coroutine using a Queue to receive data

async fn consumer(queue: Queue<u32>) {
    // Get numbers and print them to the log
    loop {
        let datum = queue.get().await;
        log::info(&format!("Consumer got {datum}"));
    }
}
```

The optional delay came along as `Option<u64>` — Chapter 9's type for "maybe a delay," where Python used `delay=None`. Cloning a `Queue` clones a *handle* to the same shared queue (like `Rc`, Chapter 13), which is how producer and consumer end up holding the same one.

An infinitely long queue

```
// Figure 11: An infinitely long Queue consumes no time

#[rustdv::test]
async fn infinite_queue(_ctx: RustdvCtx) -> Result<(), TestError> {
    // Show an infinite queue
    let queue = Queue::unbounded();
    spawn(consumer(queue.clone()));
    spawn(producer(queue, 3, None));
    Timer::ns(1).await;
    Ok(())
}
```

```
--
     25.00ns INFO     Producer sent 1
     25.00ns INFO     Producer sent 2
     25.00ns INFO     Producer sent 3
     25.00ns INFO     Consumer got 1
     25.00ns INFO     Consumer got 2
     25.00ns INFO     Consumer got 3
```

With unbounded capacity nothing ever blocks the producer, so it runs to completion in zero simulated time and *then* the consumer drains the queue — all sends, then all receives, the classic unbounded-queue behavior in every dialect.

A Queue of size 1

Bound the capacity at one and the two tasks are forced to alternate:

```rust
// Figure 12: A Queue of size 1 can block when it is full

#[rustdv::test]
async fn queue_max_size_1(_ctx: RustdvCtx) -> Result<(), TestError>
{
    // Show producer and consumer with a capacity of 1
    let queue = Queue::new(Some(1));
    spawn(consumer(queue.clone()));
    spawn(producer(queue, 3, None));
    Timer::ns(1).await;
    Ok(())
}
```

```
--
     26.00ns INFO     Producer sent 1
     26.00ns INFO     Consumer got 1
     26.00ns INFO     Producer sent 2
     26.00ns INFO     Consumer got 2
     26.00ns INFO     Producer sent 3
     26.00ns INFO     Consumer got 3
```

Put one, block; get one, block; ping-pong to the end. `Queue::new(Some(1))` is `Queue(maxsize=1)` with the capacity wrapped in `Option` — `Some(1)` bounded, and `Queue::unbounded()` as the readable spelling of `None`.

Queues and simulated delay

Give the producer a five-nanosecond stride and the pattern holds while time advances:

```
// Figure 13: Demonstrating simulated time delays
// in Queue communication

#[rustdv::test]
async fn producer_consumer_sim_delay(_ctx: RustdvCtx) -> Result<(),
TestError> {
    // Show producer and consumer with simulation delay
    let queue = Queue::new(Some(1));
    spawn(consumer(queue.clone()));
    let ptask = spawn(producer(queue, 3, Some(5)));
    let _ = ptask.await;
    Timer::ns(1).await;
    Ok(())
}
```

```
--
     32.00ns INFO     Producer sent 1
     32.00ns INFO     Consumer got 1
     37.00ns INFO     Producer sent 2
     37.00ns INFO     Consumer got 2
     42.00ns INFO     Producer sent 3
     42.00ns INFO     Consumer got 3
```

[2] "Executor-aware" is why we do not use Rust's ordinary channel types here: `std::sync::mpsc` blocks *threads*, and tokio's channels wake *tokio*. A simulation queue must park a task with *our* executor and wake it on simulated-time events. Same reason cocotb could not use `queue.Queue` from Python's standard library.

Nonblocking communication

Sometimes a task cannot afford to block — the classic example is a loop pacing itself on clock edges, which would miss edges if `get()` parked it. SystemVerilog's escape was `try_put()` / `try_get()`; cocotb's was

`put_nowait()` / `get_nowait()` plus `QueueFull` / `QueueEmpty` exceptions. rustdv keeps SystemVerilog's names, but — no exceptions, no status-integer returns — they answer in Chapter 9's vocabulary: `try_put` returns `Result<(), T>` (your item handed back on failure, so it isn't lost), and `try_get` returns `Option<T>`.

```
// Figure 14: Putting objects in a Queue without blocking

async fn producer_no_wait(queue: Queue<u32>, nn: u32) {
    // Produce numbers from 1 to nn and send them
    for datum in 1..=nn {
        let mut item = datum;
        while let Err(rejected) = queue.try_put(item) {
            log::info("Queue Full, waiting 1ns");
            item = rejected;
            Timer::ns(1).await;
        }
        log::info(&format!("Producer sent {datum}"));
    }
}
```

```
// Figure 15: Getting objects from a Queue without blocking

async fn consumer_no_wait(queue: Queue<u32>) {
    // Get numbers and print them to the log
    loop {
        let datum = loop {
            match queue.try_get() {
                Some(datum) => break datum,
                None => {
                    log::info("Queue Empty, waiting 2 ns");
                    Timer::ns(2).await;
                }
            }
        };
        log::info(&format!("Consumer got {datum}"));
    }
}
```

Compare the shapes with their Python originals. The `try/except QueueFull` block became `while let Err(rejected) = ...` — the failure is a *value*, and the value contains our rejected item, which we put back in `item` and retry. The `try/except QueueEmpty` became a `match` on `Option`, with `break datum` carrying the prize out of the inner loop (loops are expressions; Chapter 4 finally

cashes that check). Nothing is caught, because nothing is thrown; every failure path is spelled in the signatures.

```
// Figure 16: Running our nonblocking test

#[rustdv::test]
async fn producer_consumer_nowait(_ctx: RustdvCtx) -> Result<(),
TestError> {
    // Show producer and consumer not waiting
    let queue = Queue::new(Some(1));
    spawn(consumer_no_wait(queue.clone()));
    producer_no_wait(queue, 3).await;
    Timer::ns(3).await;
    Ok(())
}
```

```
--
     43.00ns INFO     Producer sent 1
     43.00ns INFO     Queue Full, waiting 1ns
     43.00ns INFO     Consumer got 1
     43.00ns INFO     Queue Empty, waiting 2 ns
     44.00ns INFO     Producer sent 2
     44.00ns INFO     Queue Full, waiting 1ns
     45.00ns INFO     Consumer got 2
     45.00ns INFO     Queue Empty, waiting 2 ns
     45.00ns INFO     Producer sent 3
     47.00ns INFO     Consumer got 3
     47.00ns INFO     Queue Empty, waiting 2 ns
     48.00ns INFO     producer_consumer_nowait PASSED
```

Note also that the test *awaits the producer directly* rather than spawning it — a coroutine you need finished before proceeding can simply be awaited, no task required.

Two more synchronizers you'll want

Queues carry data; two lighter primitives carry *timing*, and later chapters use both, so meet them now. `sim::Event` does the job of SystemVerilog's named events and cocotb's `Event`: any number of tasks `wait().await` on it, and one `set()` releases them all — with the same subtlety cocotb documents, that a wait on an already-set event returns immediately. It is the tool for "the reset is

done," "the sequence may start" — every place you were ever warned not to use a sleep. `sim::Lock` is cocotb's `Lock` and the one-key case of SystemVerilog's `semaphore`, a mutex for tasks sharing a resource (two sequences sharing one bus), with the fairness guarantee preserved: acquisition order is request order, first-come first-served. Locking returns a `LockGuard` whose `Drop` releases — which you could have predicted by now: it is the objection guard pattern, the file pattern, the RAII pattern, and before long it will simply be how you assume everything works.

Summary

This chapter put crowds of tasks to work. `spawn()` is `start_soon()`: it schedules a future and returns a `TaskHandle`, which is itself awaitable and yields `Result<T, TaskError>` — the type admitting that tasks can be cancelled before they produce. `join2` and `first2` port `Combine` and `First`. Cancellation is the one real divergence from cocotb: `cancel()` *drops* the future rather than throwing into it, cleanup lives in `Drop` rather than `finally` (usually an upgrade — it cannot be forgotten), and behavior that must consume time while shutting down uses the shutdown-message idiom instead. `sim::Queue<T>` ports the cocotb queue: typed, cloned-by-handle, blocking `put` / `get` that park with the executor, and nonblocking `try_put` / `try_get` that answer in `Result` and `Option` instead of exceptions. `Event` and `Lock` round out the toolbox — set-and-release, and fair mutual exclusion, each with RAII where cocotb had discipline.

We have tasks; we have communication; we have an executor and a simulator underneath it all. What we have not yet touched is the *design*. Chapter 17 finally does: getting a handle to the DUT, reading and writing its signals, and waiting on its clock — simulating with rustdv-sim.

Chapter 17: Simulating with rustdv-sim

Sixteen chapters in, we touch a design. This chapter connects everything Part II has built — futures, the executor, tasks — to an actual DUT in an actual simulator: getting handles to signals, reading and writing values, and waiting on clock edges. By its end you will have verified a piece of hardware in Rust, which means Chapter 18 gets to verify the piece of hardware this book is actually about.

> **In the UVM…** we reached the DUT through a handle. SystemVerilog testbenches got a virtual interface, delivered through the config database: `vif.reset_n <= 0` set a signal, `@(negedge vif.clk)` synchronized with the design. cocotb handed the test the top of the hierarchy as an argument named `dut`: `dut.reset_n.value = 0`, `get_int(dut.count)`, `await FallingEdge(dut.clk)` — same jobs, Python spellings.

One continuity story before the code

Under cocotb sits a C++ layer called the GPI — the *Generic Procedural Interface* — that abstracts the simulators' native APIs (VPI for Verilog simulators, VHPI and FLI for VHDL) behind one set of calls: get a handle by name, read a value, write a value, register a callback. A decade of simulator quirks lives in that layer, and it is the unsung reason cocotb runs everywhere.

rustdv speaks to the simulator through the same procedural interfaces, and its layering copies cocotb's on purpose: a `-sys` crate of raw simulator bindings at the bottom, a safe wrapper crate above it that turns null pointers into `Err` and untracked lifetimes into owned types, and `rustdv-sim` — everything you met in Chapters 15 and 16 — on top.[1] The mechanical difference from cocotb is *what* the simulator loads: where cocotb's makefiles arranged for the simulator to start an embedded Python interpreter that imports your test module, a

rustdv testbench **compiles to a shared library** that the simulator loads directly, the way it would load any VPI plugin. That is the story behind the two ceremony lines from Chapter 15: `vpi_bootstrap!()` exports the entry points the simulator calls at startup, and the chapter run scripts hand `vvp` (Icarus's runtime) our compiled library alongside the compiled design. No interpreter starts, because there is nothing to interpret; the testbench *is* native code, checked before the simulator ever ran.

[1] The supplied Icarus and Verilator backends both bind VPI directly. Icarus provides the four-state reference runs in this book; Verilator provides two-state functional simulation. The cocotb layering explains the names, but it does not imply that rustdv implements cocotb's VHPI or FLI backends.

Verifying a counter

Our DUT for the day, reprinted from the Python book down to the timescale:

```
# Figure 1: A SystemVerilog counter

`timescale 1ns/1ns
module counter(input bit clk,
               input bit reset_n,
               output byte unsigned count);
   always @(posedge clk)
     count <= reset_n ? count + 1 : 'b0;
endmodule
```

Synchronous reset: hold `reset_n` low and `count` clears on each clock; raise it and the counter counts. Two behaviors, two tests.

The dut handle, without the magic

A rustdv test receives a `RustdvCtx`, and `ctx.dut()` returns a `HierarchyHandle` to the top of the hierarchy — the counterpart of cocotb's `dut` argument. Getting at a signal is where the languages part company. In

Python, `dut.reset_n` worked because `__getattr__` invented the attribute on demand by asking the simulator — pure runtime dynamism, and if you typed `dut.rst_n`, you found out via `AttributeError` deep into the run. Rust cannot invent struct fields at runtime, and would not want to: rustdv's spelling is `dut.signal("reset_n")`, and it returns — you knew before you read it — a `Result`:

```
// Figure 2: A typo'd signal name is an Err, not a surprise

#[rustdv::test]
async fn name_lookup(ctx: RustdvCtx) -> Result<(), TestError> {
    // Show what child()/signal() return
    let dut = ctx.dut();
    let good = dut.signal("reset_n");
    let bad = dut.signal("rst_n"); // the classic typo
    log::info(&format!("reset_n -> {good:?}"));
    log::info(&format!("rst_n   -> {bad:?}"));
    Ok(())
}
```

```
--
      0.00ns INFO     reset_n -> Ok(LogicHandle("counter.reset_n"))
      0.00ns INFO     rst_n   -> Err(NotFound { name: "rst_n",
scope: "counter" })
```

In real code nobody matches on these by hand — you write `let reset_n = dut.signal("reset_n")?;` and the `?` from Chapter 9 turns a bad name into an immediate, test-failing error naming the signal *and* the scope, at time zero, before anything subtle has had a chance to happen. `dut.child("name")` is the general form for descending the hierarchy (submodules and all); `signal()` is `child()` plus an is-it-a-signal check.[2]

The launcher selects the DUT root by exporting `RUSTDV_TOP` with its module name. Without that variable, rustdv uses the first top-level module VPI returns. Select the root explicitly when another top-level module also exists, such as a waveform dumper; the shipped launchers already do this.

The prelude brings the handle traits into scope: `HandleChildren` supplies lookup and traversal, and `HandleEvent` supplies edge triggers. A general `child()` lookup returns a `SimHandle`; `as_signal()?` or `as_hierarchy()?`

checks its kind and reports a `HandleError` on a mismatch. To call a handle's `name()` or `full_name()`, also import `rustdv::sim::handle::HandleBase`.

The handle you get back, `LogicHandle`, is typed: it reads and writes logic values and offers edge triggers, and that is all it does. Two figures from the Python chapter dissolve entirely here — the `sys.path` bootstrap (Chapter 14 deleted it with prejudice) and the logger setup (`rustdv::log` initializes itself, and Chapter 26 covers levels). The third `tinyalu_utils` resident, `get_int()`, is worth porting for the pleasure of it:

```
// Figure 3: get_int() ports to one line of unwrap_or

fn get_int(signal: &LogicHandle) -> u64 {
    // x or z becomes 0, as tinyalu_utils decided
    signal.get_u64().unwrap_or(0)
}
```

`get_u64()` returns `Result<u64, ValueError>` because a signal holding `x` or `z` has no integer value — the fact Python expressed by `int()` raising `ValueError`, and SystemVerilog expressed by letting the `x` ride silently into your arithmetic. The Python version needed a four-line `try/except`; Rust's `unwrap_or(0)` says "the value, or zero" in one expression. The policy remains testbench-specific: a testbench that would rather die on `x` writes `get_u64()?` instead, and either way the x-handling decision is visible in the code, per read, instead of ambient in the semantics.

Choosing the value representation

The counter fits in `u64`, but the handle API also provides `get_bool()`, `get_u8()`, `get_u16()`, `get_u32()`, and `get_u128()`, with matching setters such as `set_u8()`. These integer reads are unsigned. They reject X/Z and reject a signal wider than the requested type; `get_bool()` requires one bit. `get_bigint()` and `set_bigint(&value)` use `BigUint` for arbitrary-width buses. A width error and an unknown bit are both errors, so a helper such as `get_int()` above deliberately maps either one to zero. Use `?` when either should fail the test.

To keep all four states, read `get_logic()?` into a `LogicArray` and write with `set_logic(&value)`. `get_binstr()?` and `LogicArray::to_binstr()` produce MSB-first strings, while indexed access is LSB-first: `value[0]` returns the least significant bit as an `Option<Logic>`. `u64::try_from(&value)` rejects X/Z but keeps the low bits of an oversized array; use the handle's typed getter when you want width checking as well.

The same lookup scheme covers variables that are not logic vectors. `dut.real("gain")?` returns a `RealHandle` with `f64` values; `dut.string("label")?` returns a `StringHandle` with owned Rust strings. Both provide `get()`, scheduled `set()`, and immediate `set_now()`; string writes return a `Result` and reject embedded NUL bytes. `dut.aggregate("record")?` returns an `AggregateHandle` whose `child()` and `children()` reach unpacked struct or union members, including nested aggregates. Read and write each member through its typed handle. A packed aggregate exposed as a logic vector uses `LogicHandle`.

Each of these lookups depends on the simulator exposing that object through VPI. Unpacked aggregates require member-level VPI support; the framework's variable-type regression exercises this on Verilator. Verilator is two-state, so it cannot supply the X/Z behavior demonstrated by the book's Icarus runs.

Testing reset

```
// Figure 4: Starting the clock, lowering reset

#[rustdv::test]
async fn no_count(ctx: RustdvCtx) -> Result<(), TestError> {
    // Test no count if reset is 0
    let dut = ctx.dut();
    let clk = dut.signal("clk")?;
    Clock::new(&clk, SimDuration::ns(2)).start();
    let reset_n = dut.signal("reset_n")?;
    reset_n.set_u64(0);
```

`Clock::new(&clk, SimDuration::ns(2)).start()` is `Clock(dut.clk, 2, units="ns")` plus `start_soon` in one breath: it spawns the square-wave task and returns its handle, which we let fall — a free-running clock is the legitimate

fire-and-forget from Chapter 16. `set_u64(0)` is the assignment `dut.reset_n.value = 0`, and it inherits cocotb's careful semantics: the write is *scheduled*, applied at the simulator's next read-write phase, not jammed into the middle of a delta cycle. (An immediate variant, `set_u64_now`, exists for the rare moment you need it — cocotb's `setimmediatevalue`, made greppable.)

This is the only chapter that starts a clock, so it is worth saying why now rather than letting you notice its absence later. The counter here is a bare design with `clk` as an input, and something has to drive it. The TinyALU, from Chapter 18 onward, clocks itself — and every testbench in the rest of the book therefore only ever *waits* on edges. That is a deliberate discipline, not a property of the design: a BFM that waits on edges is the same code whether a simulator or an emulator supplies the edges, while one that drives them can only ever run in a simulator. `Clock` stays in the toolkit for the designs that need it. You will not see it again.

```
// Figure 5: Wait for five clocks and check the output

    for _ in 0..5 {
        clk.rising_edge().await;
    }
    let count = get_int(&dut.signal("count")?);
    log::info(&format!("After 5 clocks count is {count}"));
    assert_eq!(count, 0);
    Ok(())
}
```

```
--
      8.00ns INFO     After 5 clocks count is 0
      8.00ns INFO     no_count PASSED
```

Triggers ride on the handles now — `clk.rising_edge().await`, `clk.falling_edge().await`, `clk.value_change().await` — which is where cocotb 2.x was headed anyway with `signal.rising_edge`. One import vanished in the move: there is no `ClockCycles` in rustdv, because `for _ in 0..5 { clk.rising_edge().await; }` *is* the loop, visible, and needing no `rising=` keyword to flip its polarity — you call the edge you mean. The assertion at the end follows Chapter 9's taxonomy: `assert_eq!` marks a claim whose failure means the test fails, and the runner scores a panicking test as FAILED with the assertion's message in the log.

Checking that the counter counts

Set and sample on the falling edge — the DUT works on the rising edge, so the quiet half-cycle is ours, the discipline every UVM BFM has always followed and every BFM in this book will too:

```
// Figure 6: Testing that the counter counts

#[rustdv::test]
async fn three_count(ctx: RustdvCtx) -> Result<(), TestError> {
    // Test that we count up as expected
    let dut = ctx.dut();
    let clk = dut.signal("clk")?;
    Clock::new(&clk, SimDuration::ns(2)).start();
    let reset_n = dut.signal("reset_n")?;
    reset_n.set_u64(0);
    clk.falling_edge().await;
    reset_n.set_u64(1);
    for _ in 0..3 {
        clk.falling_edge().await;
    }
    let count = get_int(&dut.signal("count")?);
    log::info(&format!("After 3 clocks, count is {count}"));
    assert_eq!(count, 3);
    Ok(())
}
```

```
--
     15.00ns INFO     After 3 clocks, count is 3
     15.00ns INFO     three_count PASSED
```

Reading after the design settles

An edge tells you that a signal changed; it does not promise that all the logic reacting to that edge has finished. `read_write().await` waits for the writable synchronization point, where buffered writes are applied before its waiters resume. `read_only().await` waits until pending writes and combinational logic have settled, which is the point to sample an output that depends on them. The counter tests above sample on the opposite clock edge, so they do not need another synchronization step.

ReadOnly is a sampling region. Driving a signal, or awaiting another ReadOnly or ReadWrite there, is illegal. Wait for a later time step before driving again; `next_time_step().await` waits for the simulator's next time step, while a `Timer` requests a particular delay. Chapter 40 also uses the settled ReadOnly point to explain optional inspection and waveform services.

A common coroutine mistake, demoted

The Python chapter closed with a warning: forget the `await` on a trigger and the simulator will create the coroutine, never run it, and let your test sail on without waiting — a bug you diagnose from a testbench that "doesn't seem to be advancing." Let's make the same mistake in Rust:

```
// Figure 7: Forgetting the await is now a compiler warning

#[rustdv::test]
async fn oops(ctx: RustdvCtx) -> Result<(), TestError> {
    // Demonstrate the coroutine mistake
    let dut = ctx.dut();
    let clk = dut.signal("clk")?;
    Clock::new(&clk, SimDuration::ns(2)).start();
    clk.rising_edge(); // forgot .await
    log::info("Did not await");
    Ok(())
}
```

```
--
warning: unused `Edge` that must be used
  --> ch17-simulating-with-rustdv-
sim/src/ch17_simulating_with_rustdv_sim.rs:77:5
   |
77 |     clk.rising_edge(); // forgot .await
   |     ^^^^^^^^^^^^^^^^^
   |
   = note: triggers do nothing unless you .await them
```

The program still compiles and still exhibits the bug — time does not advance, same as a forgotten `await` in Python. The difference is that *the compiler told you*, at the exact line, with a note written for this exact mistake, before any simulation ran. Chapter 15 taught why the bug exists (a future does nothing

until polled; laziness is structural); Rust's `#[must_use]` machinery turns that structural fact into a diagnostic. Earlier books asked you to keep this mistake in mind. This book asks you to keep your build log clean, which is easier.

Summary

This chapter put rustdv-sim in front of a simulator. The testbench compiles to a shared library the simulator loads — no interpreter, no environment, the continuity with cocotb living one layer down in the procedural interfaces both frameworks speak. `ctx.dut()` yields a `HierarchyHandle`; `dut.signal("name")?` looks up signals dynamically and returns `Result`, converting the classic typo from a mid-run `AttributeError` into a time-zero error with the scope and name attached. `LogicHandle` reads with `get_u64()` (a `Result`, because `x` and `z` are real) and writes with `set_u64()` (scheduled, applied at the read-write phase, as cocotb taught us all). Triggers are methods on handles — `rising_edge()`, `falling_edge()`, `value_change()` — awaited in ordinary loops that replace `ClockCycles`. `Clock` ports the clock generator. And the forgotten-`await` mistake, which Python discovered at runtime by symptom, is now a targeted compiler warning.

The value representation is a choice at the read: unsigned integers, arbitrary-width `BigUint`, or four-state `LogicArray`. Real/string variables and aggregate members use their own typed handles. The prelude supplies traversal and edge methods, `RUSTDV_TOP` selects the root, and ReadOnly provides a settled sampling point.

The counter was practice. Next chapter, the TinyALU comes back — same DUT, same `start`/`done` protocol, same timing diagram — and gets its first Rust testbench: version 1.0, a single loop with random operands, a prediction function, and a check. The climb proper begins.

Chapter 18: Basic Testbench: 1.0

The TinyALU is back. From here to the end of the book we do what both earlier books did: write increasingly modular and maintainable versions of one testbench, for one DUT, with the version numbers meaning the same architectural steps they have always meant. This chapter is version 1.0 — a single loop, easy to follow, deliberately naive — and its private teaching agenda is watching `Result`, `?`, and `match`-hardened enums shape testbench code.

> **In the UVM...** both earlier books began exactly here. The TinyALU: two 8-bit legs `A` and `B`, an `op` bus, a 16-bit `result`. ADD, AND, and XOR take one clock; MUL takes three. The user drives the operands and raises `start`; the DUT raises `done` with the result. `reset_n` is active-low and synchronous. And testbench 1.0 was one loop on the falling clock edge, reading `start` and `done` to decide whether to send a command or check a result.

The DUT is the same `tinyalu.sv` the earlier books verified — same protocol, same timing. Its port list is the whole contract:

```
# Figure 1: The TinyALU's interface

module tinyalu (input [7:0] A,
                input [7:0] B,
                input [2:0] op,
                input clk,
                input reset_n,
                input start,
                output done,
                output [15:0] result);
```

The rules that matter for the loop: a command is *sent* by raising `start` when both `start` and `done` are 0; a multi-cycle operation is *in flight* while `start` is 1 and `done` is 0; the *result* is valid on a falling edge where both are 1; and `done` while `start` is low is something the DUT must never do.

The Ops enumeration

Every version of this testbench has had an `Ops` enum doing double duty: naming the operations and mapping each to its opcode. The Rust `Ops` has been with us since Chapter 7, and here is its final, working form:

```
// Figure 2: The operation enumeration

// Legal ops for the TinyALU
#[derive(Clone, Copy, Debug, PartialEq, Eq, Hash)]
#[repr(u8)]
pub enum Ops {
    Add = 1,
    And = 2,
    Xor = 3,
    Mul = 4,
}

impl Ops {
    pub const ALL: [Ops; 4] = [Ops::Add, Ops::And, Ops::Xor,
Ops::Mul];
}
```

Three details earn their keep today. `#[repr(u8)]` with explicit discriminants gives each op its opcode, so `op as u64` produces the value the `op` bus wants — the `IntEnum` job. `Hash` joins the derive list because this chapter's coverage lives in a `HashSet<Ops>` (Chapter 8). And `Ops::ALL` replaces Python's `list(Ops)` — Rust enums do not iterate themselves, so we say what "all of them" means, once, next to the definition.

The alu_prediction() function

Constrained-random verification stands on prediction: generate random stimulus, predict the result, compare. Here is the golden model, and the figure where `match` retires the `if/elif` chain for good:

```rust
// Figure 3: The prediction function for the scoreboard

pub fn alu_prediction(a: u8, b: u8, op: Ops) -> u16 {
    // Rust model of the TinyALU
    let (a, b) = (a as u16, b as u16);
    match op {
        Ops::Add => a + b,
        Ops::And => a & b,
        Ops::Xor => a ^ b,
        Ops::Mul => a * b,
    }
}
```

Two whole classes of defensive code from the Python version are gone, not moved. The opening `assert isinstance(op, Ops)` vanished because the signature *is* the assertion — nothing but an `Ops` can arrive. And the unwritten fifth branch (what if op is none of the four?) vanished because Chapter 4's exhaustiveness rule proves there is no fifth `Ops`. The widths, meanwhile, got honest: `u8` legs, `u16` result, casts before arithmetic so ADD carries into bit eight and MUL fills the bus — details Python's unbounded integers let the book gloss over, and unit-test bait we already collected in Chapter 14.

Setting up the TinyALU test

```
// Figure 4: The start of the TinyALU test. Reset the DUT

#[rustdv::test]
async fn alu_test(ctx: RustdvCtx) -> Result<(), TestError> {
    let dut = ctx.dut();
    let mut rng = ctx.rng();
    // The RTL self-clocks (tinyalu.sv); the BFM only waits on
edges.
    let clk = dut.signal("clk")?;

    let mut passed = true;
    let mut cvg: HashSet<Ops> = HashSet::new(); // functional
coverage

    let reset_n = dut.signal("reset_n")?;
    let start = dut.signal("start")?;
    clk.falling_edge().await;
    reset_n.set_u64(0);
    start.set_u64(0);
    clk.falling_edge().await;
    reset_n.set_u64(1);
```

The shape is the classic 1.0 exactly — `passed` flag, coverage set, reset sequence on falling edges — with Chapter 17's spellings. One newcomer: `ctx.rng()`. Python reached for the global `random` module, SystemVerilog for `$urandom` and a simulator seed flag; rustdv hands each test a seeded generator, and the runner prints the seed on every run (`RUSTDV_RANDOM_SEED=1` in this chapter's transcript), so a failure reproduces by exporting one variable. Randomness you cannot replay is a bug-report you cannot act on.

Sending commands

```
// Figure 5: Creating one transaction for each operation

    let mut cmd_count = 1;
    let mut op_list: Vec<Ops> = Ops::ALL.to_vec();
    let num_ops = op_list.len();
    let (mut aa, mut bb) = (0u8, 0u8);
    let mut op = Ops::Add;
    while cmd_count <= num_ops {
        clk.falling_edge().await;
        let st = get_int(&start);
        let dn = get_int(&dut.signal("done")?);
```

(One honest Rust tax, visible above: `aa`, `bb`, and `op` are declared *before* the loop, because they are written in one falling-edge iteration — command sent — and read in a later one — result checked. Python let the book conjure `aa` into existence inside the `if` and trust it would still be there four clocks later; Rust makes the loop-spanning lifetime explicit. The compiler would have caught the case where a result arrives before any command set them — which is exactly the kind of protocol accident 1.0-style testbenches suffer.)

With the falling-edge values of `start` and `done` in hand, the loop dispatches on the protocol state, one figure per state:

```
// Figure 6: Creating a TinyALU command

        if st == 0 && dn == 0 {
            aa = rng.u8();
            bb = rng.u8();
            op = op_list.remove(0);
            cvg.insert(op);
            dut.signal("A")?.set_u64(aa as u64);
            dut.signal("B")?.set_u64(bb as u64);
            dut.signal("op")?.set_u64(op as u64);
            start.set_u64(1);
        }
```

`rng.u8()` is `random.randint(0, 255)` with the range in the type. `op_list.remove(0)` is `op_list.pop(0)` — pop-from-the-front keeps the op order deterministic even as operands randomize.

```
// Figure 7: Erroring on a state that must never happen

        if st == 0 && dn == 1 {
            return Err(TestError::from("DUT Error: done set to 1
without start"));
        }
```

The Python version raised `AssertionError` here. We return an `Err` instead, and the distinction is Chapter 9's taxonomy applied with a straight face: this is not a testbench bug, it is the *DUT misbehaving* — a check, and checks fail tests through `Result`. Panics stay reserved for our own broken code.

```
// Figure 8: If we are in an operation, continue

        if st == 1 && dn == 0 {
            continue;
        }
```

Checking the result

```
// Figure 9: The operation is complete

        if st == 1 && dn == 1 {
            start.set_u64(0);
            cmd_count += 1;
            let result = get_int(&dut.signal("result")?) as u16;

// Figure 10: Checking results against the prediction

            let pr = alu_prediction(aa, bb, op);
            if result == pr {
                log::info(&format!("PASSED: {aa:02x} {op:?}
{bb:02x} = {result:04x}"));
            } else {
                log::error(&format!(
                    "FAILED: {aa:02x} {op:?} {bb:02x} =
{result:04x} - predicted {pr:04x}"
                ));
                passed = false;
            }
        }
    }
```

Format strings do the Python's job with the Python's syntax, near enough: `{aa:02x}` for hex operands, `{result:04x}` for the sixteen-bit result, and `{op:?}` — the `Debug` derive — where Python used `op.name`.

Finishing the test

Belt and suspenders, ported: did we actually exercise every operation?

```
// Figure 11: Checking functional coverage using a set

    let missed: HashSet<Ops> =
        Ops::ALL.iter().filter(|op|
!cvg.contains(op)).copied().collect();
    if !missed.is_empty() {
        log::error(&format!("Functional coverage error. Missed:
{missed:?}"));
        passed = false;
    } else {
        log::info("Covered all operations");
    }
```

Python spelled it `set(Ops) - cvg`; Rust spells it as an iterator chain — filter `ALL` down to the ops the coverage set is missing, collect the survivors (Chapter 12's adapters, on duty). And where the Python test ended with `assert passed`, ours ends by *constructing its return value*:

```
// Figure 12: The final check relays pass/fail to rustdv

    if passed {
        Ok(())
    } else {
        Err(TestError::from("alu_test saw failing comparisons"))
    }
}
```

The test's last expression is the test's verdict — no exception mechanism carrying a boolean by proxy, just the `Result` the signature promised in figure 4.

```
# Figure 13: A successful test
--
     40.00ns INFO     PASSED: c1 Add 67 = 0128
     60.00ns INFO     PASSED: 5e And 0b = 000a
     80.00ns INFO     PASSED: b9 Xor 80 = 0039
    130.00ns INFO     PASSED: a5 Mul 75 = 4b69
    130.00ns INFO     Covered all operations
    130.00ns INFO     alu_test PASSED
```

Four operations, random operands, all compared against prediction, all covered — the same happy transcript as the Python book's figure 15, sixty-five microseconds of that book's simulated time compressed to 130 nanoseconds mostly because our clock is faster and our reset shorter. MUL takes its three cycles (watch the 50ns gap before it); the others take one.

Summary

Testbench 1.0 verified the TinyALU with one loop: reset, then a falling-edge state machine that sends a random command when the bus is idle, errors if `done` fires without `start`, waits out multi-cycle operations, and predicts-and-compares when `done` arrives — with a coverage set confirming every op ran. The Rust-specific texture: `Ops` carries its opcodes in its `repr` and its completeness in the type; `alu_prediction` lost its runtime type guard to the signature and its missing-branch anxiety to `match`; DUT misbehavior returns `Err` while panics stay reserved for testbench bugs; and the run is reproducible by seed, printed on every transcript.

And the earlier books' closing judgment of 1.0 needs no translation: this testbench works because the TinyALU is tiny. Everything is jammed in one loop — stimulus, protocol, checking, coverage — and no team could grow it. The first step out, then as now, is to split the *signal-level* work from the *testbench-level* work. That split has a name: the Bus Functional Model, and it is Chapter 19.

Chapter 19: TinyAluBfm

Testbench 1.0 worked, and it is unmaintainable — one loop doing five jobs: signal-level communication, stimulus, checking, coverage, reporting. The diagnosis both earlier books delivered holds without translation: copy-and-modify testbenches lead "only to frustration and tears." The cure starts here, by extracting the lowest layer — everything that touches a pin — into a **bus functional model**.

> **In the UVM…** the BFM owned the pins. SystemVerilog built it as an interface with tasks — the Primer's move, made in its third chapter. Python built `TinyAluBfm` as a *singleton* class holding three queues and three forever-loops on the falling clock edge: `cmd_driver` drove commands from a queue, `cmd_mon` captured `(A, B, op)` tuples when `start` rose, and `result_mon` captured `result` when `done` rose. Either way, tests talked only to `reset()`, `send_op()`, `get_cmd()`, and `get_result()`, and never touched a signal again.

Same design here — the three loops, the three queues, the four-method surface — with one architectural change we should discuss up front, because it is this chapter's Rust lesson.

Where the singleton went

The Python `TinyAluBfm` was a singleton for a sensible reason: one TinyALU, therefore one BFM, and Python's easiest way to guarantee "the same object wherever you ask for it" was `metaclass=Singleton`. But look at what the singleton was really *doing*: providing shared access to a single owner of the DUT pins. That is an ownership sentence, and Rust has ownership in the type system. The rustdv BFM is a plain struct; the test creates exactly one and shares it as `Rc<TinyAluBfm>` — Chapter 13's counted handle, passed to whoever needs it. Sharing is explicit in the type instead of ambient in a global, which

pays off the day your *next* DUT has two identical bus interfaces: two BFMs, two `Rc`s, no singleton to un-design.[1]

The BFM lives in `tinyalu_utils`, an ordinary library crate that every remaining testbench version lists in its `[dependencies]` — where Python smuggled the equivalent module in through `sys.path` (Chapter 14's payoff, part two: `Ops`, `alu_prediction`, and `get_int` moved in with it).

[1] pyuvm's own documentation reached a similar conclusion over the years; singletons make testbenches easy to write and hard to reuse. Rust simply makes the reusable version the path of least resistance.

Living on the clock edge

Digital systems act on clock edges. The TinyALU works on the rising edge, so — the discipline from Chapter 17 — the BFM's loops all set and sample on the *falling* edge, and every one of them is a variation on one skeleton:

```
// Figure 1: Every BFM loop lives on the falling edge

loop {
    clk.falling_edge().await;
    // ... check signals and do the work
}
```

The struct and its queues

```
// Figure 2: The TinyAluBfm struct — one owner of the pins

pub struct TinyAluBfm {
    clk: LogicHandle,
    reset_n: LogicHandle,
    start: LogicHandle,
    done: LogicHandle,
    a: LogicHandle,
    b: LogicHandle,
    op: LogicHandle,
    result: LogicHandle,
    driver_queue: Queue<(u8, u8, Ops)>,
    cmd_mon_queue: Queue<CmdTuple>,
    result_mon_queue: Queue<u64>,
}
```

```
// Figure 3: Initializing the TinyAluBfm

impl TinyAluBfm {
    pub fn new(dut: &HierarchyHandle) -> Result<TinyAluBfm,
HandleError> {
        Ok(TinyAluBfm {
            clk: dut.signal("clk")?,
            reset_n: dut.signal("reset_n")?,
            start: dut.signal("start")?,
            done: dut.signal("done")?,
            a: dut.signal("A")?,
            b: dut.signal("B")?,
            op: dut.signal("op")?,
            result: dut.signal("result")?,
            driver_queue: Queue::new(Some(1)),
            cmd_mon_queue: Queue::unbounded(),
            result_mon_queue: Queue::unbounded(),
        })
    }
```

Read the queue capacities against the Python `__init__`, because they carry the same design decisions: the **driver queue** holds one command (`maxsize=1`), so `send_op` blocks until the driver has taken the previous command — backpressure straight from Chapter 16's size-1 queue figure. The two **monitor queues** are unbounded, because monitors must never stall the bus they observe; they publish with the nonblocking `try_put` and let the consumer catch up on its own schedule. And note the eight `?` marks: the BFM's constructor resolves every signal it will ever touch, so a renamed port

fails the test at time zero, by name, before a single edge — testbench 1.0 scattered those lookups through the loop; the BFM concentrates them where they can fail early.

The queue types make one more Python-invisible decision visible. The driver queue carries `(u8, u8, Ops)` — inputs the *testbench* creates, so they are typed at the source. The command-monitor queue carries `CmdTuple`, an alias for `(u64, u64, u64)` — values read *off the buses*, whose interpretation is the consumer's job, exactly as the Python monitor's tuples were raw integers. The moment where raw becomes typed is in the test, and it is figure 16's punchline.

reset()

```
// Figure 4: Centralizing the reset function

    pub async fn reset(&self) {
        self.clk.falling_edge().await;
        self.reset_n.set_u64(0);
        self.start.set_u64(0);
        self.a.set_u64(0);
        self.b.set_u64(0);
        self.op.set_u64(0);
        self.clk.falling_edge().await;
        self.reset_n.set_u64(1);
        self.clk.falling_edge().await;
    }
```

Line for line the Python `reset()`. Every future test resets the DUT with one await, and when the reset sequence someday grows a step, it grows in one place.

The three loops

The BFM's loops all share the classic skeleton — `loop { clk.falling_edge().await; ...work... }` — living on the falling edge because the DUT lives on the rising one. The monitors first, since they are simpler. Each

is a private method returning the future its loop runs; keep an eye on how the loops get *access* to the pins:

```
// Figure 5: Monitoring the result bus

    fn result_mon(&self) -> impl std::future::Future<Output = ()> +
use<> {
        let (clk, done, result) = (self.clk, self.done,
self.result);
        let queue = self.result_mon_queue.clone();
        async move {
            let mut prev_done = 0;
            loop {
                clk.falling_edge().await;
                let dn = get_int(&done);
                if prev_done == 0 && dn == 1 {
                    let _ = queue.try_put(get_int(&result));
                }
                prev_done = dn;
            }
        }
    }
```

The protocol logic is the classic monitor's exactly: remember `prev_done`, and when `done` goes 0→1 across two falling edges, `result` is valid — capture it, publish it nonblockingly. The Rust texture is in the two lines before `async move`. A spawned task must own what it uses (`'static`, Chapter 12's `move` closures), and it cannot borrow `self`, which the executor might outlive. The `+ use<>` bound says that the returned future captures none of the method's input lifetimes. The method *copies out* what the loop needs instead: `LogicHandle`s are cheap `Copy` types (they are IDs into the simulator), and cloning a `Queue` clones a handle to the shared queue (Chapter 16). The loop owns its working set outright — which is also why no other task can race it for `prev_done`.

```
// Figure 6: Monitoring the command signals

    fn cmd_mon(&self) -> impl std::future::Future<Output = ()> +
use<> {
        let (clk, start, a, b, op) = (self.clk, self.start, self.a,
self.b, self.op);
        let queue = self.cmd_mon_queue.clone();
        async move {
            let mut prev_start = 0;
            loop {
                clk.falling_edge().await;
                let st = get_int(&start);
                if st == 1 && prev_start == 0 {
                    let cmd_tuple = (get_int(&a), get_int(&b),
get_int(&op));
                    let _ = queue.try_put(cmd_tuple);
                }
                prev_start = st;
            }
        }
    }
```

Same shape, opposite edge of the protocol: `start` going 0→1 means the command buses are valid.

The driver is the 1.0 loop's send-side, verbatim in spirit:

```
// Figure 7: Driving commands on the falling edge of clk

    fn cmd_driver(&self) -> impl std::future::Future<Output = ()> +
use<> {
        let (clk, start, done) = (self.clk, self.start, self.done);
        let (a, b, op) = (self.a, self.b, self.op);
        let queue = self.driver_queue.clone();
        async move {
            start.set_u64(0);
            a.set_u64(0);
            b.set_u64(0);
            op.set_u64(0);
            loop {
                clk.falling_edge().await;
                let st = get_int(&start);
                let dn = get_int(&done);
                if st == 0 && dn == 0 {
                    // Figure 8: Drive a command when the bus is
idle

                    match queue.try_get() {
                        Some((aa, bb, opr)) => {
                            a.set_u64(aa as u64);
                            b.set_u64(bb as u64);
                            op.set_u64(opr as u64);
                            start.set_u64(1);
                        }
                        None => continue,
                    }
                } else if st == 1 {
                    // Figure 9: If start is 1 check done

                    if dn == 1 {
                        start.set_u64(0);
                    }
                }
            }
        }
    }
```

Python's `try: get_nowait() ... except QueueEmpty: continue` became the `match` on `Option` — same protocol, no exception. And notice what the driver *doesn't* do: it never reads `result`. Driving is its whole job; results belong to `result_mon`. Being able to ignore the rest of the testbench is what modularity buys.

Starting the loops, and talking to them

```
// Figure 10: Start the BFM tasks

    pub fn start_tasks(&self) {
        spawn_named(self.cmd_driver(), "bfm.cmd_driver");
        spawn_named(self.cmd_mon(), "bfm.cmd_mon");
        spawn_named(self.result_mon(), "bfm.result_mon");
    }
```

`start_tasks` is an ordinary function, not an `async fn` — like Python's `start_tasks()`, it consumes no simulated time; it just enqueues three tasks (`spawn_named` is `spawn` with a name for the log). These are the legitimate fire-and-forget loops from Chapter 16.

The public face is four awaitables:

```
// Figure 11: The get_cmd() coroutine returns the next command

    pub async fn get_cmd(&self) -> CmdTuple {
        self.cmd_mon_queue.get().await
    }

// Figure 12: The get_result() coroutine returns the next result

    pub async fn get_result(&self) -> u64 {
        self.result_mon_queue.get().await
    }

// Figure 13: send_op puts the command into the command Queue

    pub async fn send_op(&self, aa: u8, bb: u8, op: Ops) {
        self.driver_queue.put((aa, bb, op)).await;
    }
```

Every method takes `&self` — the BFM's whole public surface is read-shaped borrowing, the interior queues handling the mutation. That is the property that lets `Rc<TinyAluBfm>` work without a `RefCell` in sight, as Chapter 13 promised it would.

The test, rewritten on top

```
// Figure 14: Starting a test by resetting the DUT
// and starting the BFM tasks

#[rustdv::test]
async fn test_alu(ctx: RustdvCtx) -> Result<(), TestError> {
    // Test all TinyALU operations through the BFM
    let mut rng = ctx.rng();
    let mut passed = true;
    // The RTL self-clocks (tinyalu.sv); the BFM only waits on
edges.
    let bfm = Rc::new(TinyAluBfm::new(&ctx.dut())?);
    bfm.reset().await;
    bfm.start_tasks();

    let mut cvg: HashSet<Ops> = HashSet::new();
```

```
// Figure 15: Creating a command and sending it

    for op in Ops::ALL {
        let aa = rng.u8();
        let bb = rng.u8();
        bfm.send_op(aa, bb, op).await;

// Figure 16: Wait to get the command from the DUT
// and store it in the coverage set

        let seen_cmd = bfm.get_cmd().await;
        let seen_op = Ops::from_u64(seen_cmd.2).expect("illegal op
on the bus");
        cvg.insert(seen_op);

// Figure 17: Wait for the result, then create a prediction

        let result = bfm.get_result().await as u16;
        let pr = alu_prediction(aa, bb, op);

// Figure 18: Check the result against the predicted result

        if result == pr {
            log::info(&format!("PASSED: {aa:02x} {op:?} {bb:02x} =
{result:04x}"));
        } else {
            log::error(&format!(
                "FAILED: {aa:02x} {op:?} {bb:02x} = {result:04x} -
predicted {pr:04x}"
            ));
            passed = false;
        }
    }
```

Compare this loop with Chapter 18's. No falling edges, no `start` / `done` bookkeeping, no protocol states — a `for` over the ops, a send, two gets, a compare. The signal-level grime is *gone from the test*, which reads at the level a test should: operations, predictions, verdicts.

Two Python-book points survive intact and deserve their reprise. First, we read the command back from the monitor even though we just created it — because an independent read catches driver bugs, and because a future testbench may watch a TinyALU it doesn't drive. Second, the monitor gave us raw integers, and `Ops::from_u64(seen_cmd.2)` is the boundary where the raw bus value must prove it is a legal operation — Python's `Ops(seen_cmd[2])` raising on garbage, become an `Option` we `expect` on, since an illegal opcode on the bus here means our own driver broke: testbench bug, panic, per the taxonomy.

The coverage check and final `Result` close the test exactly as 1.0 did, and:

```
# Figure 19: Another successful test
--
     50.00ns INFO     PASSED: c1 Add 67 = 0128
     70.00ns INFO     PASSED: 5e And 0b = 000a
     90.00ns INFO     PASSED: b9 Xor 80 = 0039
    140.00ns INFO     PASSED: a5 Mul 75 = 4b69
    140.00ns INFO     Covered all operations
    140.00ns INFO     test_alu PASSED
```

Same seed, same operands, same results as Chapter 18's transcript — `c1 Add 67` and friends — ten nanoseconds later apiece, the cost of the queue hop between test and driver. Two testbenches, one behavior, and the second one you could hand to a teammate.

Summary

This chapter extracted testbench 1.0's signal-level layer into `TinyAluBfm`: three falling-edge loops (driver, command monitor, result monitor) around three queues (bounded driver queue for backpressure, unbounded monitor queues so observation never stalls the bus), fronted by `reset()`, `send_op()`, `get_cmd()`, and `get_result()`. The Python singleton became a plain struct shared as `Rc` — ownership machinery doing the "exactly one, available to all" job the metaclass did, without foreclosing on a second instance. Spawned loops copy out the handles they need and own their state; the constructor front-loads every signal lookup behind `?`; and the test now reads as stimulus-predict-compare with no pins in it.

The test is still doing three jobs, though — generating, checking, covering — and version 2.0 splits *those* apart: driver, monitors, scoreboard as structs with methods, wired by hand, so we can feel precisely the plumbing pain that the UVM chapters exist to remove. Onward.

Chapter 20: Struct-Based Testbench: 2.0

Version 1.0 mixed everything in one loop; Chapter 19 pulled the pins out into the BFM. Version 2.0 takes the step both earlier books took next: break the *testbench* functionality — stimulus, checking, coverage — into separate pieces with names, so different tests reuse them instead of copying them. In SystemVerilog and Python alike, those pieces were classes related by inheritance. Here they are structs and a trait, and this chapter is where Part I's inheritance-versus-traits argument (Chapter 10) stops being an argument and starts being a testbench.

> **In the UVM...** we drew a UML diagram. A `BaseTester` defined `execute()` and left the operand supply undefined — a pure virtual method in SystemVerilog, an "ask forgiveness" abstract method in Python — while `RandomTester` and `MaxTester` extended it, each supplying one small method. A `Scoreboard` gathered commands and results into lists through two tasks and checked them all at the end. And the test wired everything together with *the tester's class itself* as the variation point.

The Tester trait

Both earlier books opened this step with a UML diagram; the Rust structure diagram is smaller, because there is no base class — only a trait and two implementors:

```
# Figure 1: Tester structure

              trait Tester
        fn get_operands(&mut self)   <- required (the "abstract
method")
        async fn execute(&mut self)  <- provided (the shared
behavior)
              /            \
     RandomTester         MaxTester
     random bytes         0xFF, 0xFF
```

The Python design's heart was a hole: `BaseTester.execute()` called `self.get_operands()`, a method that did not exist, trusting a subclass to fill it in — and trusting every future teammate to notice. Rust expresses "shared behavior with one deliberate hole" as a trait with a default method, and the hole is *declared*:

```
// Figure 2: Common behavior across all testers

#[allow(async_fn_in_trait)]
pub trait Tester {
    fn get_operands(&mut self) -> (u8, u8);

    async fn execute(&mut self, bfm: &TinyAluBfm) {
        for op in Ops::ALL {
            let (aa, bb) = self.get_operands();
            bfm.send_op(aa, bb, op).await;
        }
        // send two dummy operations to allow
        // the last real operation to complete
        bfm.send_op(0, 0, Ops::Add).await;
        bfm.send_op(0, 0, Ops::Add).await;
    }
}
```

Chapter 10's default-method machinery, load-bearing at last: `execute` is written once, in the trait, and calls `self.get_operands()` — which every implementor is *required* to provide, checked at compile time. A tester that forgets `get_operands` does not run and fail; it does not compile. What Python called an abstract base class and enforced by runtime `AttributeError`, Rust calls a required method and enforces before the simulator starts.[2]

Two smaller notes. `execute` takes the BFM as a parameter rather than conjuring the singleton — Chapter 19 removed the singleton, so dependencies

now arrive through arguments, a small habit that Chapter 25 grows into a methodology. And the two dummy operations at the end are an old trick, preserved: they keep the pipeline moving so the last real operation completes before the test stops generating stimulus. (Version 2.0 is honest, not elegant. Hold that thought.)

The concrete testers are as small as their Python originals:

```
// Figure 3: RandomTester overrides get_operands()

pub struct RandomTester {
    pub rng: Rng,
}

impl Tester for RandomTester {
    fn get_operands(&mut self) -> (u8, u8) {
        (self.rng.u8(), self.rng.u8())
    }
}
```

```
// Figure 4: MaxTester overrides get_operands()

pub struct MaxTester;

impl Tester for MaxTester {
    fn get_operands(&mut self) -> (u8, u8) {
        (0xFF, 0xFF)
    }
}
```

`RandomTester` carries its own `Rng` — the seeded generator is state, and state lives in the struct, visibly. `MaxTester` has no state at all, so it is a *unit struct*, a type with no fields whose only job is to select an implementation. One thing does one thing.

[2] Python presents an abstract method's absence as a feature of dynamic typing, and it is — the same feature, viewed from the other side, that let a typo'd override silently define a *new* method instead of overriding anything. SystemVerilog's `pure virtual` closes the first door, at the price of joining a class hierarchy. The trait closes both doors with one key.

The Scoreboard

Same definition as ever: a scoreboard gathers data from the DUT, predicts results, and compares. Same structure, too — two gathering tasks feeding storage, and a check function that runs after stimulus ends. The Rust version's storage types deserve a hard look, because they are this chapter's honest pain:

```
// Figure 5: Initializing the Scoreboard

pub struct Scoreboard {
    bfm: Rc<TinyAluBfm>,
    cmds: Rc<RefCell<Vec<CmdTuple>>>,
    results: Rc<RefCell<Vec<u64>>>,
    cvg: HashSet<Ops>,
}

impl Scoreboard {
    pub fn new(bfm: Rc<TinyAluBfm>) -> Scoreboard {
        Scoreboard {
            bfm,
            cmds: Rc::new(RefCell::new(Vec::new())),
            results: Rc::new(RefCell::new(Vec::new())),
            cvg: HashSet::new(),
        }
    }
```

`Rc<RefCell<Vec<...>>>`. In Python, the scoreboard's spawned tasks appended to `self.cmds` and nobody thought twice — every Python reference is shared and mutable. Rust makes us say it: the lists are mutated by *spawned tasks* while also being read later by *the scoreboard* — shared ownership (`Rc`) of mutable state (`RefCell`), Chapter 13's escape hatch, deployed exactly as advertised. It works, and the compiler holds us to single-writer discipline at runtime. But feel the friction, because the friction is the lesson: hand-wiring shared mutable lists between tasks is work the methodology should be doing for you. Chapter 32 gives this exact pattern a home — the subscriber that owns its storage, with the framework's `RustdvShared` marking the one sanctioned seam — and version 6.0 shows it managed instead of hand-rolled.

```
// Figure 6: The Scoreboard's data-gathering tasks

    pub fn start_tasks(&self) {
        let (bfm, cmds) = (self.bfm.clone(), self.cmds.clone());
        spawn_named(
            async move {
                loop {
                    let cmd = bfm.get_cmd().await;
                    cmds.borrow_mut().push(cmd);
                }
            },
            "scoreboard.get_cmd",
        );
        let (bfm, results) = (self.bfm.clone(),
self.results.clone());
        spawn_named(
            async move {
                loop {
                    let result = bfm.get_result().await;
                    results.borrow_mut().push(result);
                }
            },
            "scoreboard.get_result",
        );
    }
```

Python's `get_cmd` / `get_result` coroutines plus `start_tasks`, fused: each task clones its handles (the Chapter 19 pattern) and loops forever appending. Commands and results correlate by arrival order, as in the Python original — order is the contract, which works precisely as long as the DUT completes operations in order. (It does. The TinyALU remains obligingly tiny.)

```
// Figure 7: The check_results() phase

    pub fn check_results(&mut self) -> bool {
        let mut passed = true;
        let mut results = self.results.borrow_mut();
        for cmd in self.cmds.borrow().iter() {
            let (aa, bb, op_int) = *cmd;
            let op = Ops::from_u64(op_int).expect("illegal op
captured");
            self.cvg.insert(op);
            let actual = results.remove(0) as u16;
            let prediction = alu_prediction(aa as u8, bb as u8,
op);
            if actual == prediction {
                log::info(&format!("PASSED: {aa:02x} {op:?}
{bb:02x} = {actual:04x}"));
            } else {
                passed = false;
                log::error(&format!(
                    "FAILED: {aa:02x} {op:?} {bb:02x} =
{actual:04x} - predicted {prediction:04x}"
                ));
            }
        }

// Figure 8: The Scoreboard checks functional coverage

        if Ops::ALL.iter().any(|op| !self.cvg.contains(op)) {
            log::error("Functional coverage error: missed
operations");
            passed = false;
        } else {
            log::info("Covered all operations");
        }
        passed
    }
}
```

Why does the scoreboard bother with coverage when the tester loops over all ops? The classic answer stands: the scoreboard must work with *any* tester, including future ones that don't. The scoreboard checks what happened, not what the stimulus promised.

execute_test(): one wiring for all tests

```
// Figure 9: The execute_test coroutine starts the tasks

async fn execute_test(ctx: &RustdvCtx, tester: &mut impl Tester) ->
Result<bool, TestError> {
    // The RTL self-clocks (tinyalu.sv); the BFM only waits on
edges.
    let bfm = Rc::new(TinyAluBfm::new(&ctx.dut())?);
    let mut scoreboard = Scoreboard::new(bfm.clone());
    bfm.reset().await;
    bfm.start_tasks();
    scoreboard.start_tasks();

// Figure 10: Execute the tester

    tester.execute(&bfm).await;
    let passed = scoreboard.check_results();
    Ok(passed)
}
```

Python's `execute_test(tester_class)` took a *class* and instantiated it — runtime dynamism at its most Pythonic. Rust's takes `&mut impl Tester`: any type implementing the trait, resolved at compile time (Chapter 11's generics — this function is monomorphized once per tester type, at zero runtime cost). The caller constructs the tester; `execute_test` neither knows nor cares which one it got. That is the same test-writing ergonomics, minus the ability to pass a class that turns out not to be a tester at all.

The tests

```
// Figure 11: The tests launch execute_test with a tester

#[rustdv::test]
async fn random_test(ctx: RustdvCtx) -> Result<(), TestError> {
    // Random operands
    let mut tester = RandomTester { rng: ctx.rng() };
    let passed = execute_test(&ctx, &mut tester).await?;
    if passed {
        Ok(())
    } else {
        Err(TestError::from("random_test saw failing comparisons"))
    }
}
```

```
// Figure 12: The max test differs only in its tester

#[rustdv::test]
async fn max_test(ctx: RustdvCtx) -> Result<(), TestError> {
    // Maximum operands
    let mut tester = MaxTester;
    let passed = execute_test(&ctx, &mut tester).await?;
    if passed {
        Ok(())
    } else {
        Err(TestError::from("max_test saw failing comparisons"))
    }
}
```

Two tests, differing in one constructed value — the components did all the work, exactly as this testbench has always been designed. The transcript, both tests in one regression:

```
# Figure 13: Two tests, one testbench
--
    150.00ns INFO     PASSED: c1 Add 67 = 0128
    150.00ns INFO     PASSED: 5e And 0b = 000a
    150.00ns INFO     PASSED: b9 Xor 80 = 0039
    150.00ns INFO     PASSED: a5 Mul 75 = 4b69
    150.00ns INFO     Covered all operations
    150.00ns INFO     random_test PASSED
    300.00ns INFO     PASSED: ff Add ff = 01fe
    300.00ns INFO     PASSED: ff And ff = 00ff
    300.00ns INFO     PASSED: ff Xor ff = 0000
    300.00ns INFO     PASSED: ff Mul ff = fe01
    150.00ns INFO     Covered all operations
    300.00ns INFO     max_test PASSED
```

Note the timestamps: unlike Chapters 18 and 19, where PASSED lines trickled out as results arrived, all four comparisons print at once — the scoreboard checks *after* the run, batch-style. Chapter 24 will give that timing a name (`check`, a lifecycle phase) and a guarantee (it runs after the run phase ends). Note also `ff Xor ff = 0000` and `ff Mul ff = fe01` doing what max-operand tests exist to do: probing the corners where a lazier predictor would have wrapped, zeroed, or overflowed.

Summary

Testbench 2.0 broke the test's remaining jobs into named pieces. The `Tester` trait holds the shared `execute` loop and declares the `get_operands` hole; `RandomTester` and `MaxTester` fill it in a line apiece, with the compiler enforcing what Python's abstract base class could only hope for. The `Scoreboard` gathers commands and results through spawned tasks into shared lists and batch-checks them with prediction and coverage — paying openly, in `Rc<RefCell<Vec>>` plumbing, for the shared mutable state that Python's references hid. `execute_test` wires it all once, generically over `impl Tester`, so each new test is a few lines constructing a different tester.

And with that, Part II's promise is kept: language, executor, tasks, queues, signals, and two working testbench architectures. What we have *not* kept doing is pretending the wiring scales — the by-hand construction in `execute_test`, the shared-list scoreboard, the tester passed around by argument. Making

structure, configuration, and communication into reusable methodology is precisely the UVM's business, and Chapters 22 through 39 rebuild it in Rust. But between here and there stands one load-bearing chapter of language: macros — how `#[rustdv::test]` has been finding our tests all along, and how code that writes code replaces decorators and metaclasses. Chapter 21.

Chapter 21: Macros: Code That Writes Code

Every `#[rustdv::test]` you have typed since Chapter 15 has been an IOU. This chapter pays it. The chapters ahead lean on two pieces of compile-time machinery — the test attribute and `#[derive(Component)]` — and you deserve to know exactly what replaced the machinery you left behind: SystemVerilog's `` `uvm_component_utils `` macros, pyuvm's decorators and metaclasses. That is a language question with a twist, because Python's tools ran at *import time* with the full power of a running program, SystemVerilog's ran in the preprocessor as text substitution, and Rust has neither an import time nor a text preprocessor.

> **In the UVM...** the framework organized our code for us, each dialect in its own way. SystemVerilog used macros: `` `uvm_component_utils(my_driver) `` expanded — textually, before the compiler proper ever saw it — into the factory-registration boilerplate nobody wanted to hand-write. Python used runtime hooks: `@cocotb.test()` received our coroutine as an argument, registered it in a global list, and handed it back, and pyuvm went further, with a *metaclass* registering every component class with the factory as a side effect of the `class` statement itself.

What a decorator actually did

Strip `@cocotb.test()` to its skeleton and it is ten lines of Python:

```python
# Figure 1: What @cocotb.test() really does — a registering
decorator

test_registry = []

def test():
    def wrapper(coro):
        test_registry.append(coro)   # side effect at import time
        return coro
    return wrapper

@test()
def hello_world():
    print("Hello, world.")

@test()
def wait_2ns():
    print("I am DONE waiting!")

print(f"registered {len(test_registry)} tests:")
for t in test_registry:
    print(f"  {t.__name__}")
```

```
--
registered 2 tests:
  hello_world
  wait_2ns
```

The decorator is an ordinary function that runs *when the module is imported*, mutating a global list. That is the whole trick, and it is a good trick — cocotb's regression manager later walks `test_registry` (in real life, with options and filters) and runs what it finds. The trick's precondition is the part Rust lacks: **a moment when code runs because source code was loaded.** Rust programs are compiled, then executed; nothing happens in between, and `use` (Chapter 14) triggers nothing. So Rust splits the decorator's two jobs — *transforming code* and *registering it* — into two different mechanisms, and this chapter takes them in turn.

Declarative macros: patterns in, code out

Rust's entry-level macro is `macro_rules!` — the family `println!`, `format!`, `assert_eq!`, and `vec!` come from, and the reason they carry an exclamation point: the `!` warns you that *arguments are being rewritten, not passed*. A declarative macro is a set of match arms over source-code patterns:

```
// Figure 2: A declarative macro — patterns in, code out

macro_rules! check {
    ($actual:expr, $expected:expr) => {
        if $actual == $expected {
            println!("PASSED: {} = {:04x}", stringify!($actual),
$actual);
        } else {
            println!(
                "FAILED: {} = {:04x} — predicted {:04x}",
                stringify!($actual),
                $actual,
                $expected
            );
        }
    };
}

fn main() {
    let result: u16 = 0xFF + 0x01;
    check!(result, 0x0100);
    check!(result, 0x0000);
}
```

```
--
PASSED: result = 0100
FAILED: result = 0100 — predicted 0000
```

`$actual:expr` binds any expression; the right side is the code that replaces the call, with the bindings spliced in. Notice what no function could do: `stringify!($actual)` prints the *source text* of the argument — the reason `assert_eq!` failures can show you the expression that failed, and something Python decorators managed only by introspecting live objects. Declarative macros are pure rewriting, hygienic, and resolved entirely inside the compiler. rustdv uses them sparingly — `first!`, `join!`, `vpi_bootstrap!` — and the book's advice is

the community's: reach for a macro only after a function or generic has failed you.

Procedural macros: the compiler takes arguments

The heavier tool — and the true decorator replacement — is the **procedural macro**: a Rust function that runs *inside the compiler*, receives your code as a stream of tokens, and returns the token stream to compile instead. Where a decorator transformed a live function object at import time, a proc macro transforms source code at compile time. They come in the two flavors this book cares about: **attribute macros** (`#[rustdv::test]`), which replace the item they decorate, and **derive macros** (`#[derive(Clone)]` , `#[derive(Component)]`), which read a type's definition and append new code alongside it.

So: what does `#[rustdv::test]` actually emit? Here is the expansion, lightly tidied, for `async fn hello_world(...)` :

```
// Figure 3: What #[rustdv::test] expands to (tidied)

// 1. Your function, untouched — the attribute adds, never
rewrites:
async fn hello_world(_ctx: RustdvCtx) -> Result<(), TestError> { /*
your body */ }

// 2. A shim with a uniform signature, so the runner can hold
//    every test in one list (Box<dyn Future>, Chapter 13):
fn __rustdv_shim(ctx: RustdvCtx) -> Pin<Box<dyn Future<Output =
Result<(), TestError>>>> {
    Box::pin(hello_world(ctx))
}

// 3. The registration, contributed to a distributed slice:
#[linkme::distributed_slice(TEST_REGISTRATIONS)]
static __RUSTDV_TEST_REG: TestRegistration = TestRegistration {
    name: "hello_world",
    module: module_path!(),
    file: file!(),
    line: line!(),
    run: __rustdv_shim,
    timeout: None,
    skip: false,
    expect_fail: false,
};
```

Parts 1 and 2 are the decorator's *wrapping* job, done with types: your `async fn` stays exactly as you wrote it, and the shim adapts it to the one shape the regression runner stores. The attribute's arguments — `timeout_time`, `timeout_unit`, `expect_fail`, `skip`, `name` — are cocotb's `Test` options, parsed by `syn` into Rust syntax and emitted with `quote`; a typo'd option is a compile error pointing at the attribute. `file!()` and `line!()` capture the source location the runner prints in `running hello_world (1/2) [ch15_...src/ch15_async_await_executor.rs:12]` — you have been reading this macro's output in every transcript since Chapter 15.

Part 3 is the *registering* job, and it needs its own section, because there is no global list and no import time to fill one.

Registration without a runtime: the linker as registry

The trick is old, standard, and delightful: **let the linker build the array.** The `linkme` crate packages it as a *distributed slice*: declare the slice once, then contribute static values from any file or crate in the build. `linkme` places those values into the platform's linker sections and presents the result as an ordinary shared slice. In miniature, and runnable:

```
// Figure 4: Link-time registration in miniature

use linkme::distributed_slice;

/// What a registration carries: a name and a function to run.
struct Registration {
    name: &'static str,
    run: fn(),
}

#[distributed_slice]
static TESTS: [Registration];

fn hello() {
    println!("Hello, world.");
}
#[distributed_slice(TESTS)]
static REG_HELLO: Registration = Registration { name: "hello", run:
hello };

fn goodbye() {
    println!("Goodbye, world.");
}
#[distributed_slice(TESTS)]
static REG_GOODBYE: Registration = Registration { name: "goodbye",
run: goodbye };

fn collect() -> &'static [Registration] {
    &TESTS
}

fn main() {
    let tests = collect();
    println!("found {} registered tests:", tests.len());
    for t in tests {
        print!("  {} -> ", t.name);
        (t.run)();
    }
}
```

```
--
found 2 registered tests:
  goodbye -> Goodbye, world.
  hello -> Hello, world.
```

Read the output closely: `goodbye` came out *first*. Link order is the linker's business, not yours — which is why the real rustdv runner sorts its collected registrations by `(file, line)` before running, so a regression's order is the order tests appear in your source. `linkme` owns the platform-specific section names, retention attributes, boundary symbols and pointer work; rustdv iterates the resulting slice through safe Rust.[1]

This is the moment to bank a comparison the rest of the book builds on. cocotb discovers tests because importing your module *runs* registration code. rustdv discovers tests because compiling your crate *emits* registrations into the binary. Both are "the framework finds your tests by name" — same user experience, and command-line selection by test name works the same way — but the Rust version happens entirely before execution, cannot be affected by import order, and works in a `cdylib` the simulator loads, where "import time" would be a meaningless phrase.

[1] The technique has platform texture — ELF section symbols on Linux differ from Mach-O and Windows spellings — which is precisely why rustdv delegates it to `linkme` instead of maintaining those spellings itself.

Derive macros: field lists instead of `__dict__`

The other half of Python's runtime magic was introspection: pyuvm's `do_copy` and `do_compare` walked `self.__dict__` at runtime to copy and compare whatever fields a transaction happened to have, and its factory metaclass registered classes by watching them be defined. Rust cannot look up a struct's fields at runtime — the names are gone by then — but a **derive macro** can look at them at *compile time*, which is how `#[derive(Clone, Debug, PartialEq)]` has been writing your `do_copy`, `convert2string`, and `do_compare` since Chapter 10: the derive receives the struct definition as

tokens, iterates the fields, and emits field-by-field implementations. Same field-walking idea as `__dict__`, run once, in the compiler, with the results type-checked.

rustdv ships exactly one derive of its own, and the rest of the book uses it everywhere: `#[derive(Component)]`. Given a component struct, it reads the fields you mark as children and writes the boilerplate that a UVM-ish framework needs to walk your hierarchy:

```
// Figure 5: What #[derive(Component)] writes for you (tidied)

#[derive(rustdv::Component)]
pub struct AluEnv {
    seqr: Sequencer<AluCommand>,
    #[component]
    driver: Option<Driver>,
    #[component]
    scoreboard: Scoreboard,
}

// ...expands to (hand-writable, if you ever prefer):
impl ComponentNode for AluEnv {
    fn node_name(&self) -> &'static str { "AluEnv" }
    fn visit_children(&mut self, f: &mut dyn FnMut(&str, &mut dyn
ComponentNode)) {
        if let Some(__c) = &mut self.driver { f("driver", __c); }
        f("scoreboard", &mut self.scoreboard);
    }
}
```

Three things to notice, all previews of Chapter 24. The hierarchy's names come from your *field names* — `driver`, `scoreboard` — synthesized at compile time, where pyuvm passed name strings to every constructor. `Option<Driver>` is understood: a `None` child (the passive agent's missing driver) is simply skipped, and `Vec<T>` children get indexed names like `drivers[0]`. And the tidied expansion above shows the traversal only — the full one *also registers the component by name*, contributing to a second `linkme` distributed slice. That is pyuvm's factory metaclass, kept: every component can be created by name or overridden with no separate registration step, and Chapter 29 collects on it. Both metaclass jobs — test discovery and component registration — turned out to be the linker's.

When not to write a macro

A chapter that hands you power tools owes you the safety lecture. Macro-generated code is code you didn't write and can't click into; error messages inside a macro expansion point at generated text; and every macro is a small language your teammates must learn. The bar this book applies — and applied to rustdv itself — is: a macro must delete user-visible boilerplate *and* be explainable in one paragraph. `#[rustdv::test]` clears the bar (it deletes a shim and a registration you should never hand-write). `#[derive(Component)]` clears it (field-walking is the machine's job — as true here as it was for the `uvm_field_*` macros and pyuvm's `__dict__` walk). Everything else in rustdv — the lifecycle, the ConfigDb, TLM, sequences — is plain code, on purpose, so that when you read the UVM chapters you are reading Rust, not incantations.

Summary

Python organized testbenches at import time: decorators wrapped and registered functions; metaclasses registered classes; `__dict__` introspection copied and compared objects. Rust has no import time, and this chapter met its replacements. Declarative macros (`macro_rules!`) rewrite patterns into code — the `!` family you have used all book. Procedural macros run inside the compiler: the `#[rustdv::test]` attribute leaves your function intact and emits a uniform shim plus a registration; `#[derive(...)]` reads field lists at compile time and writes the member-wise code pyuvm wrote by runtime reflection, including rustdv's own `#[derive(Component)]` — hierarchy traversal plus by-name registration. Registration without a runtime rides in the linker: `linkme` gathers distributed statics into slices, and rustdv sorts tests by source location — import-order bugs structurally impossible. And the governing taste: macros where dynamism used to be, plain code everywhere else.

That closes the language's account, and it was the last debt outstanding. You now know how tests are found and how hierarchies will be walked and registered. The chapters ahead can finally ask this series' biggest question in its new language: what is the UVM *for*? Chapter 22: Why UVM?

Chapter 22: Why UVM?

The Universal Verification Methodology is the most successful verification methodology in the history of the world, and the story of how it got that way is by now well told — many of you lived it, and both earlier books in this series retell it: eRM, VMM, AVM, and OVM came first; the UVM came last and survived, blessed by all three big EDA vendors and stewarded since by a committee of vendors and users. What a methodology *is* has not changed either — a standard set of answers to the questions every testbench developer faces. What has changed is the language we will answer them in, and that raises this chapter's one real question: **does a statically-typed, compiled language change what the UVM is for?**

The answer is no — and the reasons are worth a page, because they are the frame for the eighteen chapters ahead.

The UVM's value was never really its mechanisms. It was the *agreements*: that testbenches have a standard shape (tests own environments, environments own the working components); that stimulus is separated from structure; that components communicate through standard ports rather than by reaching into each other; that one engineer's testbench is legible to the next engineer because both learned the same methodology. Those agreements are language-independent. A team writing Rust needs them exactly as much as a team writing SystemVerilog or Python — which is to say, needs them the moment the testbench outgrows one file or one author.

And one part of that answer runs so hard against a Rust programmer's instincts that it gets said plainly now. The UVM's central mechanisms — the two-stage build, the configuration database, the factory, TLM connection — are *runtime* machinery on purpose. Each one exists to defer a decision: what to build, what value to use, which type to substitute, what connects to what. Deferring decisions is what lets one closed environment serve a hundred tests, and a decision deferred to run time is one the compiler cannot check, in any language. The UVM's designers had a statically-typed language in hand and chose runtime indirection three separate times — typed classes and yet a factory, parameterized classes and yet a config DB, `mailbox#(T)` and yet TLM. That judgment was right, and rustdv keeps all three mechanisms rather than

“fixing” them into something a compiler can see through — spending its checking on data and ownership instead, and working to make the late failures *loud*.

Here are the methodology’s questions, each with a note on where its answer lives.

How do we define tests? cocotb used `@cocotb.test()`; pyuvm layered `@pyuvm.test()` over a `uvm_test` class. rustdv keeps both shapes — the attribute on a function since Chapter 15, and on a struct that *is a component*, which is where the methodology lives. Chapter 23.

How do we build testbenches? Testbench 2.0’s `execute_test()` built everything by hand, one way among many. The UVM standardizes construction: a `build` phase that grows the tree top-down and a `connect` phase that wires it bottom-up, with the gap between a component existing and its children existing hosting everything else on this list. Chapters 24 and 25.

How do we reuse testbench components? Vertical reuse — the TinyALU testbench living on inside a larger design’s — still motivates environments and the active/passive distinction, which rides the configuration mechanism. Chapters 24, 25, and 40.

How do we create verification IP? Same answer as ever: standard shapes make protocol testbenches shareable. A rustdv environment is a crate you can publish, with `cargo` handling what tarballs and READMEs once did.

How do multiple components monitor the DUT? The Scoreboard hogging `get_cmd()` is still the problem; analysis broadcasting — one-to-many, fire-and-forget — is still the answer. Chapter 32.

How do we share common data? The config database: a runtime store, path-addressed, with wildcards and precedence, ported whole — and the place rustdv spends its types is the failure path, where SystemVerilog’s `get()` returns a silent zero and rustdv’s returns a `Result` naming the cause. Chapters 25, 27, and 28.

How do we modify the testbench’s structure in each test? The factory’s job, and the factory’s answer: build through `create`, override by type, name, or

instance, with registration ridden in on the derive so nobody forgets it. Chapters 29 and 30.

How do components pass data to each other? TLM: ports, exports, and the FIFO between them that lets two components trade transactions without ever meeting. Chapter 31.

How do we create stimulus? Separating stimulus from structure is the sequence machinery, ported handshake-for-handshake. Chapters 36 through 39.

How do we log messages? Hierarchical, level-controlled logging, mapped onto the component tree. Chapter 26.

How do we pass data around the testbench? Testbench 2.0's `(A, B, op)` tuple, where `op` is "the thing at index 2," still grates. The UVM's answer was `uvm_object` with copy/compare/print conventions; Rust's answer is a plain struct with derives doing those same jobs. Chapter 35.

One more thing carries over from the earlier books, because it is the real thesis: you do not *have* to use the UVM. You do have to answer every one of these questions anyway, for every testbench beyond a certain size — and teams that answer them differently cannot share code, or engineers. A methodology's deepest feature is that it makes your testbench boring in all the places where boring is a compliment, so the interesting effort goes where it belongs: into verifying the DUT.

The tradition for starting that journey is also unchanged. Chapter 23 writes "Hello, world" as a UVM test — and then rebuilds testbench 2.0 as testbench 3.0, with the runner driving the test and objections deciding when it ends.

Chapter 23: uvm_test Testbench: 3.0

Testbench 2.0 was modular; now we make it methodological. As is tradition, we start writing UVM tests by creating a `HelloWorldTest`, to examine the mechanics of defining and running a test before wiring one to the TinyALU.

In the UVM... the test was a class extending `uvm_test`, with an objection-guarded `run_phase()` that raised, said hello, and dropped — `class hello_world extends uvm_test` selected by `run_test()` in SystemVerilog; `class HelloWorldTest(uvm_test)` marked `@pyuvm.test()` in Python — and the framework instantiated it under the name `uvm_test_top` and drove its phases. Then testbench 3.0 refactored 2.0: `BaseTest` carried the shared `run_phase()`, while `RandomTest` and `MaxTest` overrode `build_phase()` to pick a tester.

Hello, world, in the methodology

```
// Chapter 23, Figure 1: The basic rustdv-UVM use model in
hello_world

#[rustdv::test]
#[derive(Component, Default)]
struct HelloWorldTest;

impl Component for HelloWorldTest {
    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let _obj = ctx.raise_objection("saying hello");
        ctx.info("Hello, world.");
        Ok(())
    } // the guard drops here: the objection is released
}
```

Here is the minimum needed to define and run a rustdv UVM test:

- `#[rustdv::test]` — the attribute you have used since Chapter 15, now on a *struct*. It registers the test with the runner under its type name, verbatim — `HelloWorldTest`, not `hello_world_test` — so the runner can select it by the name you see in the source. This is the UVM's `run_test()`: the framework instantiates your test and drives it.
- `#[derive(Component)]` — the test *is a component*, exactly as `uvm_test` extends `uvm_component`. Everything a component can do, a test can do, and Chapter 24 leans on that hard.
- `impl Component for HelloWorldTest` — the test overrides the one phase it uses, `run`. It has no children, so no `build`; the trait's defaults cover every phase you don't write.
- `ctx.raise_objection("saying hello")` — the UVM's objection, as a guard. The run phase continues until every objection is released, and releasing happens by *dropping the guard* — here, at the closing brace. Note what that deletes: the forgot-to-drop bug, which hangs a pyuvm run phase until the timeout fires, is unwritable. Scope ends, objection drops. (Chapter 16's `LockGuard`, Chapter 13's RAII — third verse.)
- `ctx.info("Hello, world.")` — and this is the first place `ctx.info` earns its keep over the bare `log::info` of Part II, because the context knows *who is talking*:

```
# Figure 2: Hello, world!

      0.00ns INFO     rustdv: found 3 test(s), RUSTDV_RANDOM_SEED=1
      0.00ns INFO     running HelloWorldTest (1/3)  [ch23-uvm-test-
testbench-3.0/src/ch23_uvm_test_testbench_3_0.rs:149]
      0.00ns INFO     [HelloWorldTest]: Hello, world.
      0.00ns INFO     HelloWorldTest PASSED
```

The `[HelloWorldTest]` between the brackets is the component's path in the testbench hierarchy — a hierarchy that is, so far, one component deep. One divergence from tradition worth a sentence: the UVM names the root `uvm_test_top` no matter which test is running, and rustdv names it after the test you registered. When something three components deep logs a message in Chapter 26, its path will start with the name of the test that built it, which tells you at a glance which test's universe the message came from.

A note for readers keeping score against Chapter 15: `#[rustdv::test]` accepts *two* shapes, and both are first-class. On a free `async fn`, it is cocotb's model —

`@cocotb.test()` on a coroutine — and every test in Part II was one. On a struct implementing `Component`, it is pyuvm's model — `@pyuvm.test()` on a class — and it is what a test that owns a component tree needs to be. The function form is not training wheels; a function-shaped test remains the right spelling for a function-shaped job. This book's testbenches are about to grow trees, so from here on the struct form carries the story.

Where the tower went

Both earlier books paused here for the UML tower every UVM engineer has climbed. It is worth reprinting, with each floor's rustdv forwarding address:

```
# Figure 3: The uvm_test tower, and its rustdv equivalent

pyuvm                           rustdv
─────                           ──────
uvm_void                        (no common ancestor needed;
registration
                                 rides the derive — Ch. 29)
uvm_object                      plain structs + derives (Ch. 35)
uvm_report_object               ctx.info() — logging rides the
context (Ch. 26)
uvm_component                   the Component trait (Ch. 24)
uvm_test                        #[rustdv::test] on a struct
```

The tower's *jobs* all survive; the inheritance chain that delivered them does not, because Rust composes capabilities instead of stacking them. What `uvm_object` gave you arrives as derives on your transaction structs; what `uvm_report_object` gave you rides in on `ctx`; what `uvm_component` gave you is the `Component` trait; and `uvm_test` — the class whose only real job was "this is the one the framework starts" — is the attribute.

Refactoring testbench 2.0

Testbench 2.0's classes — the `Tester` trait, `RandomTester`, `MaxTester`, and the `Scoreboard` — return in this chapter's example file under a banner comment reading *copied from testbench 2.0*, and that convention deserves a sentence

because the book will use it from here to the end. Chapter examples repeat the classes they use rather than importing them, exactly as the earlier books re-showed code, because these classes *evolve*: the tester is a plain trait object today and a component in Chapter 25, the scoreboard checks in a function today and in a `check` phase tomorrow. Watching them change is the point, and an import would hide the change. (The definitions are unchanged from Chapter 20's figures 1 through 8; we will not re-read them here.)

What 3.0 actually changes is who runs them. pyuvm expressed base-and-variants as `BaseTest` providing `run_phase`, extended by `RandomTest` and `MaxTest` overriding `build_phase`. Rust has no inheritance, so the shared body is a *function*, and each test hands it a tester:

```
// Chapter 23, Figure 4: alu_test — the shared run phase of every
ALU test

async fn alu_test(ctx: &mut RustdvCtx, tester: &mut impl Tester) ->
Result<(), TestError> {
    let _obj = ctx.raise_objection("alu_test stimulus");

    let bfm = Rc::new(TinyAluBfm::new(&ctx.dut())?);
    let mut scoreboard = Scoreboard::new(bfm.clone());

    bfm.reset().await;
    bfm.start_tasks();
    scoreboard.start_tasks();

    tester.execute(&bfm).await;

    if scoreboard.check_results() {
        Ok(())
    } else {
        Err(TestError::from("scoreboard saw failing comparisons"))
    }
}
```

This is Chapter 20's `execute_test`, promoted to a run phase. Three things to notice, and one to notice by its absence:

- **The test is the only component.** The BFM and the scoreboard are ordinary local values inside `run`; the tester is a plain value passed in. No tree, no phases for them — that is Chapters 24 and 25's business, and this chapter refuses to get ahead of itself.

- The BFM comes from `ctx.dut()` — the same handle-then-check pattern as Chapter 19, with `?` propagating a missing signal as a named failure.
- The objection guards the whole body: stimulus, then checking, then `Ok` or a `TestError` that fails the test with the scoreboard's verdict.
- And no clock-starting line, because there is none to start: the RTL self-clocks, and the BFM only ever waits on edges — Chapter 19's design rule, still paying rent.

pyuvm kept `BaseTest` abstract by convention — nothing but discipline stopped a teammate from running it, since its only protection was the missing decorator. `alu_test` is abstract by *signature*: a function that requires a `&mut impl Tester` argument cannot be registered as a test, because there is nothing to fill the argument with. The two real tests are exactly as thin as pyuvm's:

```
// Chapter 23, Figure 5: The tests choose a tester and share
alu_test

#[rustdv::test]
#[derive(Component, Default)]
struct RandomTest;

impl Component for RandomTest {
    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let mut tester = RandomTester { rng: ctx.rng() };
        alu_test(ctx, &mut tester).await
    }
}

#[rustdv::test]
#[derive(Component, Default)]
struct MaxTest;

impl Component for MaxTest {
    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let mut tester = MaxTester;
        alu_test(ctx, &mut tester).await
    }
}
```

Where pyuvm's `RandomTest.build_phase()` set `self.tester = RandomTester()`, our `RandomTest::run` builds a `RandomTester` and passes it in — and note the seed's route: `ctx.rng()` hands the tester the per-test seeded generator, so a failing run reproduces. The variation point is an argument

today. In Chapter 25 it becomes a `build`-phase decision, and in Chapter 30 a factory slot; the shape — shared body, per-test variation at a designed point — is the methodology, and it never changes again.

```
# Figure 6: RandomTest passes

      0.00ns INFO     running RandomTest (2/3)  [ch23-uvm-test-
testbench-3.0/src/ch23_uvm_test_testbench_3_0.rs:188]
    150.00ns INFO     PASSED: ce Add 42 = 0110
    150.00ns INFO     PASSED: 2f And 64 = 0024
    150.00ns INFO     PASSED: 29 Xor b3 = 009a
    150.00ns INFO     PASSED: 86 Mul 83 = 4492
    150.00ns INFO     Covered all operations
    150.00ns INFO     RandomTest PASSED
```

```
# Figure 7: MaxTest maxes all the operands

    150.00ns INFO     running MaxTest (3/3)  [ch23-uvm-test-
testbench-3.0/src/ch23_uvm_test_testbench_3_0.rs:199]
    300.00ns INFO     PASSED: ff Add ff = 01fe
    300.00ns INFO     PASSED: ff And ff = 00ff
    300.00ns INFO     PASSED: ff Xor ff = 0000
    300.00ns INFO     PASSED: ff Mul ff = fe01
    300.00ns INFO     Covered all operations
    300.00ns INFO     MaxTest PASSED
```

Same behavior as testbench 2.0 — and one detail in these transcripts quietly measures how far the testbench has to go. The `PASSED` lines carry no `[path]`, because the scoreboard prints them with plain `log::info`: it is not a component, so it has no path to be stamped with. Compare `[HelloWorldTest]` in figure 2. When the scoreboard becomes a component in Chapter 25, its lines pick up their address, and you will be able to read a log line's provenance without grepping for its format string.

Summary

Testbench 3.0 brings the UVM's test discipline to rustdv. `#[rustdv::test]` on a struct is `@pyuvm.test()` on a class: the test is a component, registered under its type name verbatim, instantiated by the runner, its phases driven for it, its path named after itself rather than `uvm_test_top`. Objections are RAII guards

— raise returns a guard, scope-exit drops it, and the forgot-to-drop hang cannot be written. The base-class pattern crossed the no-inheritance gap as a shared `async fn` taking `&mut impl Tester`, abstract by signature rather than by convention, with each test choosing its tester and its seed source in three lines.

At 3.0 the test is the only component in the testbench, and everything it uses is a local. The next chapter grows the tree: `uvm_component`, the nine phases, and the answer to how a testbench gets *structure*.

Chapter 24: Components

Testbench 3.0 gave us a UVM test, and the test was the only component in it: the BFM and the scoreboard were ordinary local values inside its `run`. That works at TinyALU scale and stops working shortly after — a real testbench is a *tree* of single-purpose components, each with its own job, sharing one lifecycle so that everything gets built, wired, run, and checked in a dependable order. The class that provides all of that in the UVM is `uvm_component`. This chapter ports it.

> **In the UVM…** every testbench class extends `uvm_component`, whose phase methods the framework calls in a fixed order — `build`, `connect`, `end_of_elaboration`, `start_of_simulation`, `run` (the only task, objection-gated), `extract`, `check`, `report`, `final`. We built the hierarchy in `build_phase()` by instantiating children with a *name* and a *parent* — `mc = middle_comp::type_id::create("mc", this)` in SystemVerilog, `self.mc = MiddleComp("mc", self)` in pyuvm — and the framework wove the references into a tree with paths like `uvm_test_top.mc.bc`.

The nine phases

Figure 1 is a test that overrides every phase method just to prove the order. It is a struct test, as in Chapter 23 — and that choice now pays off, because a struct test *is a component*: `#[rustdv::test]` registers it, and the runner drives its phases exactly the way `@pyuvm.test()` hands a class to pyuvm's phaser.

```rust
// Chapter 24, Figure 1: A uvm_test demonstrating the phase methods

#[rustdv::test]
#[derive(Component, Default)]
struct PhaseTest;

impl Component for PhaseTest {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        ctx.info("1 build");
    }
    fn connect(&mut self, ctx: &mut RustdvCtx) {
        ctx.info("2 connect");
    }
    fn end_of_elaboration(&mut self, ctx: &mut RustdvCtx) {
        ctx.info("3 end_of_elaboration");
    }
    fn start_of_simulation(&mut self, ctx: &mut RustdvCtx) {
        ctx.info("4 start_of_simulation");
    }
    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let _obj = ctx.raise_objection("run");
        ctx.info("5 run");
        Ok(())
    }
    fn extract(&mut self, ctx: &mut RustdvCtx) {
        ctx.info("6 extract");
    }
    fn check(&mut self, ctx: &mut RustdvCtx, errors: &mut
CheckSink) {
        let _ = errors;
        ctx.info("7 check");
    }
    fn report(&mut self, ctx: &mut RustdvCtx) {
        ctx.info("8 report");
    }
    fn final_phase(&mut self, ctx: &mut RustdvCtx) {
        ctx.info("9 final");
    }
}
```

The pieces, in source order:

- `#[derive(Component)]` — the derive from Chapter 21. It writes the tree-traversal plumbing so the phaser can walk this component's children. `PhaseTest` has none, so the derive's work here is small; it earns its keep in figure 4.
- `impl Component for PhaseTest` — the `Component` trait carries all nine phase methods, every one with a default no-op body. A component

overrides only the phases it uses; this one overrides all nine only because the order is the demonstration.
- `ctx: &mut RustdvCtx` — every phase receives the context, and its log lines are stamped with the path the phase walk derived. No phase method takes a name; no component stores one.
- `async fn run` — the one phase that takes simulated time, so the one that is `async`. It raises an objection the moment it starts and holds it as a guard: the run phase ends when every guard in the testbench has dropped. Dropping happens here at the end of the function, the way any Rust value drops.
- `fn final_phase`, not `fn final` — `final` is a Rust keyword, so this is the one phase whose rustdv name differs by necessity.

Figure 2 is the run.

```
# Figure 2: The lifecycle runs in order

      0.00ns INFO     rustdv: found 2 test(s), RUSTDV_RANDOM_SEED=1
      0.00ns INFO     running PhaseTest (1/2)  [ch24-
components/src/ch24_components.rs:38]
      0.00ns INFO     [PhaseTest]: 1 build
      0.00ns INFO     [PhaseTest]: 2 connect
      0.00ns INFO     [PhaseTest]: 3 end_of_elaboration
      0.00ns INFO     [PhaseTest]: 4 start_of_simulation
      0.00ns INFO     [PhaseTest]: 5 run
      0.00ns INFO     [PhaseTest]: 6 extract
      0.00ns INFO     [PhaseTest]: 7 check
      0.00ns INFO     [PhaseTest]: 8 report
      0.00ns INFO     [PhaseTest]: 9 final
      0.00ns INFO     PhaseTest PASSED
      ... (TestTop, the second test in this crate, follows)
**********************************************************************
***********
** TEST                                        STATUS  SIM TIME (ns)
**
**********************************************************************
***********
** PhaseTest                                     PASS           0.00
**
** TestTop                                       PASS           0.00
**
**********************************************************************
***********
REGRESSION: PASS
```

Nine phases, in the UVM's order, driven by the framework — nobody in the listing called any of them. The `[PhaseTest]` between the brackets is the component's path, derived by the walk rather than stored anywhere.

One divergence to note now, because it matters to SystemVerilog readers checking this against muscle memory: rustdv follows *pyuvm's* traversal directions, not the SystemVerilog UVM's. `build` runs top-down and `connect` bottom-up in all three frameworks, but `end_of_elaboration`, `start_of_simulation`, `extract`, `check`, and `report` run top-down here, where the SystemVerilog UVM runs them bottom-up. If your testbench depends on a child's `report` running before its parent's, that assumption does not carry over.

Why build and connect exist

A fair question from a Rust point of view: a struct's constructor can build its children, so why have a `build` phase at all? Build the children in `new()`, take the connections as constructor arguments, and the framework gets simpler — the design almost writes itself.

It is also wrong, and the reason it is wrong is the most important paragraph in this chapter. The gap between *a component existing* and *its children existing* is not dead time to be optimized away — it is where every late-binding mechanism in the UVM lives. Configuration must be able to reach a component *before* it decides what children to make: that is how one environment builds an active agent in one test and a passive one in another (Chapter 25). The factory must be able to substitute a child's type *before* the child is constructed: that is what a factory override is (Chapter 29). And connection must happen *after* everything below exists: that is why `connect` runs bottom-up (Chapter 31). Fold building into constructors and all three mechanisms lose the moment they operate in. The UVM's designers had typed classes, parameters, and constructors in hand and still built a two-stage lifecycle — three frameworks in three languages kept it — because deferring those decisions is the point, not an accident of class-based construction.

So in rustdv, `build` and `connect` are real phase methods, and the directions are load-bearing: `build` runs top-down so a parent decides what to create

before its children exist, and `connect` runs bottom-up so wiring happens over a finished subtree.

Growing the tree

Figure 3 is the hierarchy this section builds — the same three-level tower the earlier books used.

```
# Figure 3: The three-level hierarchy

    TestTop                (a test — the root)
       └── mc: MiddleComp
              └── bc: BottomComp
```

Each parent creates its child *in its own build phase*, and the phaser descends into whatever `build` created, so the tree grows top-down as it is walked. Figures 4 through 6 are the three components, from the top down.

```
// Chapter 24, Figure 4: the test at the top. Its build phase
constructs the
// middle component; the phaser descends into the tree that build
creates.
#[rustdv::test]
#[derive(Component, Default)]
struct TestTop {
    #[component]
    mc: Option<MiddleComp>,
}

impl Component for TestTop {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        ctx.info("build phase");
        self.mc = Some(MiddleComp::default());
    }
    fn final_phase(&mut self, ctx: &mut RustdvCtx) {
        ctx.info("final phase");
    }
}
```

Three lines carry the design:

- `#[component]` — this attribute tells the derive which fields are children. The phase walk visits exactly the marked fields, in declaration order.
- `mc: Option<MiddleComp>` — the child is declared as an `Option` because before `build` runs there *is no child*. `None` is the type-level spelling of "declared but not yet built" — the state every UVM component is in between its own construction and its `build_phase`. The struct definition names what the tree can hold; `build` decides what it does hold.
- `self.mc = Some(MiddleComp::default())` — building the child is an assignment. Compare `self.mc = MiddleComp("mc", self)`: no name string, because the field is named `mc` and the walk derives the path; no parent handle, because ownership already says whose field this is.

```
// Chapter 24, Figure 5: the middle component builds the bottom
component in
// its own build phase, and announces itself at end of elaboration.
#[derive(Component, Default)]
struct MiddleComp {
    #[component]
    bc: Option<BottomComp>,
}

impl Component for MiddleComp {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        self.bc = Some(BottomComp::default());
    }
    fn end_of_elaboration(&mut self, ctx: &mut RustdvCtx) {
        ctx.info("end of elaboration phase");
    }
}
```

`MiddleComp` is not a test — no `#[rustdv::test]` — just a component that both is built and builds. When the top-down walk reaches it, its `build` runs and `bc` comes into existence; the walk then descends into `bc`. Top-down construction, exactly as `build_phase` has always worked.

```
// Chapter 24, Figure 6: the bottom component. Only a run phase,
which
// objects, logs under its path (uvm_test_top.mc.bc in UVM;
TestTop.mc.bc
// here), and drops.
#[derive(Component, Default)]
struct BottomComp;

impl Component for BottomComp {
    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let _obj = ctx.raise_objection("bc run");
        ctx.info("run phase");
        Ok(())
    }
}
```

`BottomComp` overrides one phase and is the pattern for every leaf that does work: raise the objection, do the job, and let the guard drop when the function ends. Every component's `run` gets this same deal — each raises its own objection for its own work, and the run phase of the whole testbench ends when the last guard anywhere has dropped. What the objection buys becomes vivid in Chapter 31, where a parent and its children run *at the same time* and components that never finish on their own — monitors, responders — stop exactly when the objecting components are done.

Figure 7 is the run.

```
# Figure 7: The walk derives every path

      0.00ns INFO     running TestTop (2/2)  [ch24-
components/src/ch24_components.rs:118]
      0.00ns INFO     [TestTop]: build phase
      0.00ns INFO     [TestTop.mc]: end of elaboration phase
      0.00ns INFO     [TestTop.mc.bc]: run phase
      0.00ns INFO     [TestTop]: final phase
      0.00ns INFO     TestTop PASSED
```

Four log lines, four different phase methods, three different components — and each line carries the right path. `[TestTop.mc.bc]` is the path the UVM would spell `uvm_test_top.mc.bc`, synthesized from field names by the walk as it descends: `TestTop.build` created `mc`, the phaser recursed, `mc.build` created `bc`, and each component logged under the path the traversal accumulated. Nothing stored a path and nobody typed one. Move a component

to a different place in the tree and its path follows, because there is no string anywhere that could go stale.

What this costs

Two prices, stated plainly.

Phase discipline is not checked at compile time. There is one context type, `RustdvCtx`, and every phase receives it whole — the compiler does not know that raising an objection makes no sense in `build`, or that a value configured during `run` is too late for a `build` that already ran. An operation performed in the wrong phase is a run-time failure with a good message, exactly as it is in every UVM. A family of per-phase context types could push some of this to compile time; it would also mean eight signatures for every helper that takes a context, and rustdv declines the trade: the lifecycle is runtime machinery, and the type system is not pretending otherwise.

`async fn` in a trait is a live edge of Rust. `Component::run` is an `async fn` in a trait, and Rust has not finished smoothing that feature: a trait with an `async fn` cannot be made into a `dyn` trait object directly. The framework deals with it by keeping a dyn-safe mirror of the trait internally — machinery you never see and never write, which is why no listing in this book mentions it. A framework author feels that edge so that a testbench author does not.

Summary

`uvm_component` ported whole. The `Component` trait carries the nine phases with default no-op bodies — override what you use — and the runner drives them in pyuvm's order and directions: `build` top-down, `connect` bottom-up, run objection-gated, the elaboration and post-run phases top-down (a divergence from the SystemVerilog UVM's bottom-up, worth checking against old habits). A child is a struct field, `Option`-wrapped because it does not exist until its parent's `build` creates it; the phase walk descends into what `build` made, deriving every component's path from field names as it goes. Build and connect

are real phases because the gap they occupy — after a component exists, before its children do — is where configuration, factory overrides, and connection all operate; Chapters 25, 29, and 31 each collect on that argument in turn.

Version 4.0 is next: the machinery of this chapter, put to work on the TinyALU — an environment component, a scoreboard that checks in `check`, and the BFM delivered through the ConfigDb instead of reached for.

Chapter 25: uvm_env Testbench: 4.0

Chapter 24 built the machinery; testbench 4.0 moves in. At 3.0 the test was the only component, with the tester and scoreboard as ordinary locals inside its `run`. This version makes each of them a real component with its own phases, and gathers them into an **environment** — the container that keeps a tester and its scoreboard together, and the reusable unit the UVM is built around. The earlier books did this in three steps and so do we: componentize the testers and the scoreboard, instantiate them in an environment, instantiate the environment in tests.

In the UVM... we made `BaseTester` a `uvm_component` that drove stimulus in an objection-guarded `run_phase()`; the `Scoreboard` launched its gathering tasks in `start_of_simulation_phase()` and compared in `check_phase()`; `BaseEnv` built the scoreboard, `RandomEnv` / `MaxEnv` added the right tester; and `RandomTest` / `MaxTest` did nothing but build the right env. The BFM reached everyone as a virtual interface through the config database — `uvm_config_db#(virtual tinyalu_bfm)::set(null, "*", "bfm", bfm)` in every `top.sv`.

Converting the testers to components

Chapter 20's testers varied in exactly one behavior — how they choose operands. That stays true as they become components:

```
// Chapter 25, Figure 1: What varies between testers is only the
operands
pub trait Operands {
    fn get_operands(&mut self, rng: &mut Rng) -> (u8, u8);
}
```

```rust
// Chapter 25, Figure 2: RandomOperands and MaxOperands
#[derive(Default)]
pub struct RandomOperands;

impl Operands for RandomOperands {
    fn get_operands(&mut self, rng: &mut Rng) -> (u8, u8) {
        (rng.u8(), rng.u8())
    }
}

#[derive(Default)]
pub struct MaxOperands;

impl Operands for MaxOperands {
    fn get_operands(&mut self, _rng: &mut Rng) -> (u8, u8) {
        (0xFF, 0xFF)
    }
}
```

SystemVerilog and Python express “same tester, different operands” with an abstract base class and one overridden method. Rust expresses it as a trait with one required method — and then, in the next figure, as a *type parameter* on the component that uses it:

```rust
// Chapter 25, Figure 3: BaseTester implements the phases common to
all testers
#[derive(Component, Default)]
pub struct BaseTester<T: Operands + Default + 'static> {
    operands: T,
    rng: Option<Rng>,
}

impl<T: Operands + Default + 'static> Component for BaseTester<T> {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        // The seeded RNG comes from the context, so a run is
reproducible.
        self.rng = Some(ctx.rng());
    }

    fn start_of_simulation(&mut self, ctx: &mut RustdvCtx) {
        let bfm: Rc<TinyAluBfm> = ConfigDb::get(Some(ctx), "",
"BFM").expect("the test sets BFM");
        bfm.start_tasks();
    }

    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let _obj = ctx.raise_objection("tester stimulus");
        let bfm: Rc<TinyAluBfm> = ConfigDb::get(Some(ctx), "",
"BFM")?;
        let rng = self.rng.as_mut().expect("build phase did not
run");

        bfm.reset().await;

        for op in Ops::ALL {
            let (aa, bb) = self.operands.get_operands(rng);
            bfm.send_op(aa, bb, op).await;
        }
        // send two dummy operations to allow
        // the last real operation to complete
        bfm.send_op(0, 0, Ops::Add).await;
        bfm.send_op(0, 0, Ops::Add).await;
        Ok(())
    }
}
```

Stop at the `ConfigDb` lines, because they are this chapter's other new idea and the code cannot be read without them. The tester is created by its parent's `build` phase, and `build` passes nothing — no name, no parent, and no BFM. A component that needs something it was not handed *asks for it by name*, and something above it in the tree put it there. Two lines are enough to read every listing in this chapter:

```
ConfigDb::set(None, "*", "BFM", bfm)   // the test: everyone gets
this one
ConfigDb::get(Some(ctx), "", "BFM")    // a component: give me mine
```

Chapter 27 is the full treatment — paths, wildcards, precedence, and what happens when the name is wrong. Until then, three observations carry you. First, this is exactly how SystemVerilog delivers the virtual interface, for exactly the reason: siblings built by a parent cannot be handed things through constructors, so a named store above the tree does it. Second, the `get` returns a `Result` — `?` in a phase that returns one, `expect` in a phase that does not — so a wrong name announces itself instead of returning a silent zero. Third, and worth a sentence because pyuvm readers will look for the alternative: there is no BFM singleton, anywhere, from here to the end of the book. A singleton asserts there is exactly one BFM in the world, which is false for any testbench with two interfaces. The database asserts only that there is one *under this name, for this subtree* — which scales, and which a test can re-point without touching a component. That is what the mechanism is for.

The honest cost: you have just met a database, a path glob, and a `Result` while still learning what an environment is. The UVM front-loads this too — every SystemVerilog engineer's first testbench has that `set(null, "*", ...)` line in `top.sv` before they can explain it — and the two-line reading above is all this chapter needs.

```
// Chapter 25, Figure 4: The two testers are type aliases over the
base
pub type RandomTester = BaseTester<RandomOperands>;
pub type MaxTester = BaseTester<MaxOperands>;
```

Where the Python book subclassed `BaseTester` twice to override one method, the generic base is written once and the two testers are *names for specializations*. What varies is a type parameter, not an override. (Hold this thought loosely: it is the right tool when the variation is chosen at compile time, as it is here. Chapter 30 meets the variation that must be chosen at *run* time, and generics will not survive the encounter.)

The Scoreboard as a component

```
// Chapter 25, Figure 5: The Scoreboard as a component
#[derive(Component, Default)]
pub struct Scoreboard {
    cmds: Rc<RefCell<Vec<CmdTuple>>>,
    results: Rc<RefCell<Vec<u64>>>,
    cvg: HashSet<Ops>,
}
```

```
// Chapter 25, Figure 6: Launching the monitoring tasks in
start_of_simulation
impl Component for Scoreboard {
    fn start_of_simulation(&mut self, ctx: &mut RustdvCtx) {
        let bfm: Rc<TinyAluBfm> = ConfigDb::get(Some(ctx), "",
"BFM").expect("the test sets BFM");
        let (cmd_bfm, cmds) = (bfm.clone(), self.cmds.clone());
        spawn_named(
            async move {
                loop {
                    let cmd = cmd_bfm.get_cmd().await;
                    cmds.borrow_mut().push(cmd);
                }
            },
            "scoreboard.get_cmds",
        );

        let (result_bfm, results) = (bfm, self.results.clone());
        spawn_named(
            async move {
                loop {
                    let result = result_bfm.get_result().await;
                    results.borrow_mut().push(result);
                }
            },
            "scoreboard.get_results",
        );
    }
```

Why `start_of_simulation` and not `run`? Because this phase does not *consume time* — `spawn_named` schedules a task and returns, exactly as `cocotb.start_soon` did in the Python original — and the lifecycle guarantees `start_of_simulation` finishes across the *whole tree* before any `run` begins. The monitors are provably listening before the first stimulus. Spawn them in `run` instead and you are racing the tester for the first transaction. (Resist the urge to "correct" this on the grounds that the UVM starts processes in

`run_phase` — pyuvm's version of this scoreboard makes the same choice for the same reason.)

One ownership note, because it is Chapter 5's rule surfacing in framework clothes: a spawned task must own everything it touches — that is the `'static` bound Chapter 16 taught. The tasks here cannot borrow the scoreboard, so they clone `Rc` handles: each task owns a reference count, not a borrow of `self`.

```
// Chapter 25, Figure 7: Checking results in the check phase
    fn check(&mut self, ctx: &mut RustdvCtx, errors: &mut
CheckSink) {
        let mut results = self.results.borrow_mut();
        for cmd in self.cmds.borrow().iter() {
            let (aa, bb, op_int) = *cmd;
            let op = Ops::from_u64(op_int).expect("illegal op
captured");
            self.cvg.insert(op);
            let actual = results.remove(0) as u16;
            let prediction = alu_prediction(aa as u8, bb as u8,
op);
            if actual == prediction {
                ctx.info(&format!("PASSED: {aa:02x} {op:?} {bb:02x}
= {actual:04x}"));
            } else {
                errors.error(format!(
                    "FAILED: {aa:02x} {op:?} {bb:02x} =
{actual:04x} - predicted {prediction:04x}"
                ));
            }
        }

        if Ops::ALL.iter().any(|op| !self.cvg.contains(op)) {
            errors.error("Functional coverage error: missed
operations".to_string());
        } else {
            ctx.info("Covered all operations");
        }
    }
}
```

Chapter 20's `check_results()` returned a `bool` and somebody had to remember to call it. This `check` is a *phase*: the framework calls it after the run phase ends, and failures reported to the `CheckSink` fail the test. Nothing calls anything by hand — the sequencing the earlier testbenches did with discipline, the lifecycle now does with guarantees.

Using an environment

```
// Chapter 25, Figure 8: The environment builds the scoreboard and
a tester
#[derive(Component, Default)]
pub struct AluEnv<T: Operands + Default + 'static> {
    #[component]
    scoreboard: Option<Scoreboard>,
    #[component]
    tester: Option<BaseTester<T>>,
}

impl<T: Operands + Default + 'static> Component for AluEnv<T> {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        self.scoreboard = Some(Scoreboard::default());
        self.tester = Some(BaseTester::<T>::default());
    }
}
```

```
// Chapter 25, Figure 9: RandomEnv and MaxEnv are type aliases
pub type RandomEnv = AluEnv<RandomOperands>;
pub type MaxEnv = AluEnv<MaxOperands>;
```

The environment is Chapter 24's pattern doing real work: `Option` children filled in by `build`, the phaser descending into the subtree as it comes into existence. And where Python needed `BaseEnv`, `RandomEnv`, and `MaxEnv` as three classes — the subclasses calling `super().build_phase()` and adding their tester — one generic env replaces all three, with the variants as aliases again.

```
// Chapter 25, Figure 10: Each test builds the environment it wants
#[rustdv::test]
#[derive(Component, Default)]
struct RandomTest {
    #[component]
    env: Option<RandomEnv>,
}

impl Component for RandomTest {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        let bfm = TinyAluBfm::new(&ctx.dut()).expect("TinyALU
signals");
        ConfigDb::set(None, "*", "BFM", Rc::new(bfm));
        self.env = Some(RandomEnv::default());
    }
}

#[rustdv::test]
#[derive(Component, Default)]
struct MaxTest {
    #[component]
    env: Option<MaxEnv>,
}

impl Component for MaxTest {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        let bfm = TinyAluBfm::new(&ctx.dut()).expect("TinyALU
signals");
        ConfigDb::set(None, "*", "BFM", Rc::new(bfm));
        self.env = Some(MaxEnv::default());
    }
}
```

The test builds the BFM — it is the only component that holds the DUT handle — and files it from the top: `None` context means “from the root,” `"*"` means every path, so one line serves the whole tree. Then it builds the env it wants, and it is done: **the test has no run phase at all.** Stimulus lives in the tester now, and the objection the tester raises is what holds the run phase open. Compare this against Chapter 23’s test, which did everything; the methodology is redistributing the work into the tree, one version at a time.

```
# Figure 11: Testbench 4.0 running

      0.00ns INFO     rustdv: found 2 test(s), RUSTDV_RANDOM_SEED=1
      0.00ns INFO     running RandomTest (1/2)  [ch25-uvm-env-
testbench-4.0/src/ch25_uvm_env_testbench_4_0.rs:256]
    150.00ns INFO     [RandomTest.env.scoreboard]: PASSED: c1 Add
67 = 0128
    150.00ns INFO     [RandomTest.env.scoreboard]: PASSED: 5e And
0b = 000a
    150.00ns INFO     [RandomTest.env.scoreboard]: PASSED: b9 Xor
80 = 0039
    150.00ns INFO     [RandomTest.env.scoreboard]: PASSED: a5 Mul
75 = 4b69
    150.00ns INFO     [RandomTest.env.scoreboard]: Covered all
operations
    150.00ns INFO     RandomTest PASSED
    150.00ns INFO     running MaxTest (2/2)  [ch25-uvm-env-
testbench-4.0/src/ch25_uvm_env_testbench_4_0.rs:271]
    300.00ns INFO     [MaxTest.env.scoreboard]: PASSED: ff Add ff =
01fe
    300.00ns INFO     [MaxTest.env.scoreboard]: PASSED: ff And ff =
00ff
    300.00ns INFO     [MaxTest.env.scoreboard]: PASSED: ff Xor ff =
0000
    300.00ns INFO     [MaxTest.env.scoreboard]: PASSED: ff Mul ff =
fe01
    300.00ns INFO     [MaxTest.env.scoreboard]: Covered all
operations
    300.00ns INFO     MaxTest PASSED
**********************************************************************
***********
** TEST                                          STATUS  SIM TIME (ns)
**
**********************************************************************
***********
** RandomTest                                      PASS         150.00
**
** MaxTest                                         PASS         150.00
**
**********************************************************************
***********
REGRESSION: PASS
```

Chapter 23 promised this transcript: the `PASSED` lines now carry `[RandomTest.env.scoreboard]`, because the scoreboard is a component with an address, and the address was derived from field names by the walk. Same results as 3.0; every line of output now says who produced it.

Summary

Testbench 4.0 turns the 2.0 classes into components and houses them in an environment. The testers become one generic `BaseTester<T>` with the varying behavior as a type parameter and the variants as type aliases; the scoreboard collects in `start_of_simulation` — spawned tasks, owning `Rc` handles rather than borrowing, listening before any `run` begins — and compares in `check`, where failures reach the `CheckSink` instead of a hand-checked `bool`. The BFM travels by name through the ConfigDb, set once at the top by the test and retrieved by whoever needs it: the mechanism SystemVerilog uses for virtual interfaces, met here in two lines and treated fully in Chapter 27. The tests shrink to two `build` lines each.

Before the ConfigDb gets its chapter, one comfort of the old testbenches deserves restoring: log messages you can filter, route, and silence per component. Logging is Chapter 26.

Chapter 26: Logging

With a hierarchy to hang them on, we can tour the features that live on it, starting with the one you have been reading all book: logging. Large testbenches generate more output than humans can read, so the game is filtering and directing — per component, per subtree, per destination — and the pyuvm surface for that game ports over nearly method-for-method.

In the UVM... we logged through the framework. SystemVerilog: `` `uvm_info(get_type_name(), "msg", UVM_MEDIUM) ``, verbosities from `UVM_NONE` to `UVM_DEBUG`, opened per subtree with `set_report_verbosity_level_hier()`. pyuvm: `self.logger`, inherited from `uvm_report_object` — levels from DEBUG to CRITICAL, INFO the default threshold, `set_logging_level_hier(DEBUG)` for a subtree, and handlers to say where messages went. The path between square brackets, `[uvm_test_top.comp]`, told us who was talking.

Creating log messages

```
// Chapter 26, Figure 1: Logging messages of all levels
#[derive(Component, Default)]
struct LogComp;

impl Component for LogComp {
    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let _obj = ctx.raise_objection("logging");
        ctx.debug("This is debug");
        ctx.info("This is info");
        ctx.warning("This is warning");
        ctx.error("This is error");
        ctx.critical("This is critical");
        Ok(())
    }
}
```

Five levels, one method each, all on the context — `debug`, `info`, `warning`, `error`, `critical`, the same ladder pyuvm inherited from Python's `logging` module. The default threshold is Info, so when this component runs unconfigured, the debug line is filtered out. And notice what the component does *not* pass anywhere: a name. `ctx.info` is attributed to whoever called it, because the context knows.

One component, four logging policies

The Python book demonstrates logging configuration by writing `LogTest` and then subclassing it three times, each subclass overriding `end_of_elaboration_phase` to configure differently. No inheritance here, so the varying part becomes — the same move as Chapter 25's testers — a type parameter:

```
// Chapter 26, Figure 2: The logging policy is the only thing that
varies
pub trait LogPolicy: Default {
    /// Called in `end_of_elaboration`, where pyuvm configures
logging:
    /// the hierarchy is final, and nothing has run yet.
    fn configure(&self, ctx: &mut RustdvCtx);
}

#[derive(Component, Default)]
struct LogTest<P: LogPolicy + 'static> {
    #[component]
    comp: Option<LogComp>,
    policy: P,
}

impl<P: LogPolicy + 'static> Component for LogTest<P> {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        self.comp = Some(LogComp::default());
    }

    fn end_of_elaboration(&mut self, ctx: &mut RustdvCtx) {
        self.policy.configure(ctx);
    }
}
```

`end_of_elaboration` is where logging configuration belongs, for the reason Chapter 28 will use it for database dumps: the hierarchy is final and nothing

has run, so the policy governs every message the run phase will produce. The four policies:

```
// Chapter 26, Figure 3: The default — no configuration at all
#[derive(Default)]
pub struct DefaultLogging;

impl LogPolicy for DefaultLogging {
    fn configure(&self, _ctx: &mut RustdvCtx) {}
}
```

```
// Chapter 26, Figure 4: Setting the logging level for a hierarchy
#[derive(Default)]
pub struct DebugLogging;

impl LogPolicy for DebugLogging {
    fn configure(&self, ctx: &mut RustdvCtx) {
        ctx.set_logging_level_hier(Level::Debug);
    }
}
```

`_hier` means what it means in pyuvm: this component and everything under it. Which component? The one holding the context — no path argument, and none possible to typo.

```
// Chapter 26, Figure 5: Writing log entries to a file
#[derive(Default)]
pub struct FileLogging;

impl LogPolicy for FileLogging {
    fn configure(&self, ctx: &mut RustdvCtx) {
        ctx.add_file_handler_hier("rustdv_ch26_log.txt", false)
            .expect("could not open the log file");
        ctx.remove_console_hier();
    }
}
```

Two handler operations, both hierarchy-scoped: add a file destination for this subtree, and take the subtree off the console. The file path is deliberately relative — the log lands beside wherever you ran the simulation. A shared absolute path like `/tmp/rustdv_log.txt` is a file some other user, or a leftover from an earlier run under a different account, can own and lock you out of; a

test that writes outside its own working directory can be broken by something it has never heard of.

```
// Chapter 26, Figure 6: Disabling logging for a hierarchy
#[derive(Default)]
pub struct NoLogging;

impl LogPolicy for NoLogging {
    fn configure(&self, ctx: &mut RustdvCtx) {
        ctx.disable_logging_hier();
    }
}
```

```
// Chapter 26, Figure 7: The four tests are type aliases over one
base
#[rustdv::test]
type LogTestDefault = LogTest<DefaultLogging>;

#[rustdv::test]
type DebugTest = LogTest<DebugLogging>;

#[rustdv::test]
type FileTest = LogTest<FileLogging>;

#[rustdv::test]
type NoLog = LogTest<NoLogging>;
```

```
# Figure 8: Four tests, four logging behaviors

      0.00ns INFO     rustdv: found 4 test(s), RUSTDV_RANDOM_SEED=1
      0.00ns INFO     running LogTestDefault (1/4)  [ch26-
logging/src/ch26_logging.rs:124]
      0.00ns INFO     [LogTestDefault.comp]: This is info
      0.00ns WARNING  [LogTestDefault.comp]: This is warning
      0.00ns ERROR    [LogTestDefault.comp]: This is error
      0.00ns CRITICAL [LogTestDefault.comp]: This is critical
      0.00ns INFO     LogTestDefault PASSED
      0.00ns INFO     running DebugTest (2/4)  [ch26-
logging/src/ch26_logging.rs:127]
      0.00ns DEBUG    [DebugTest.comp]: This is debug
      0.00ns INFO     [DebugTest.comp]: This is info
      0.00ns WARNING  [DebugTest.comp]: This is warning
      0.00ns ERROR    [DebugTest.comp]: This is error
      0.00ns CRITICAL [DebugTest.comp]: This is critical
      0.00ns INFO     DebugTest PASSED
      0.00ns INFO     running FileTest (3/4)  [ch26-
logging/src/ch26_logging.rs:130]
      0.00ns INFO     FileTest PASSED
      0.00ns INFO     running NoLog (4/4)  [ch26-
logging/src/ch26_logging.rs:133]
      0.00ns INFO     NoLog PASSED
**********************************************************************
***********
** TEST                                        STATUS  SIM TIME (ns)
**
**********************************************************************
***********
** LogTestDefault                               PASS           0.00
**
** DebugTest                                    PASS           0.00
**
** FileTest                                     PASS           0.00
**
** NoLog                                        PASS           0.00
**
**********************************************************************
***********
REGRESSION: PASS
```

Read the transcript against the policies: the default test filters debug; the debug test shows it; the file test prints *nothing* — its subtree went off the console — and the no-log test is silent. Where did `FileTest`'s messages go?

```
# Figure 9: rustdv_ch26_log.txt receives what the console did not

      0.00ns INFO     [FileTest.comp]: This is info
      0.00ns WARNING  [FileTest.comp]: This is warning
      0.00ns ERROR    [FileTest.comp]: This is error
      0.00ns CRITICAL [FileTest.comp]: This is critical
```

The file has one more thing to teach, by omission: `DebugTest` ran immediately before `FileTest` and set its level to Debug, yet the file holds no `This is debug` line. The runner resets logging configuration between tests, the way pyuvm's `run_test` does, so no test can leave the console switched off — or the level opened up — for the next one. No cleanup phase needed, and none to forget.

What the figures do not contain

The chapter's real lesson is a thing missing from every listing: **a path.** `LogComp` is one type, written once, and it logged as `[LogTestDefault.comp]`, `[DebugTest.comp]`, `[FileTest.comp]` — whichever was true for the test it was built under, with nothing in the component saying so. `ctx.info` is attributed to its caller, and `ctx.set_logging_level_hier` addresses its caller's subtree, because the context carries the path the phase walk derived in Chapter 24. The alternative — a logger constructed with `"uvm_test_top.comp"` and a level set by path string — is a hand-typed name that keeps compiling, and keeps lying, after the component is renamed or moved. rustdv never asks you to type a path the tree already knows.

Summary

Logging rides the context: five levels with Info as the default gate, `set_logging_level_hier` to open a subtree, file handlers and console removal per hierarchy, and `disable_logging_hier` for silence — pyuvm's surface, with the paths supplied by the framework instead of the engineer. Configuration happens in `end_of_elaboration`, applies to the run that follows, and is reset

between tests by the runner. The four demonstrations share one generic test with the policy as a type parameter, the by-now-familiar spelling of a base class with one overridden method.

Chapter 25 introduced the ConfigDb in two lines and promised the rest. Time to pay: paths, wildcards, globals, precedence — configuration, in full.

Chapter 27: Configuration

Chapter 25 used the ConfigDb twice and promised the full story later: the test did `ConfigDb::set`, the driver did `ConfigDb::get`, and the BFM crossed the testbench without ever appearing in a constructor. This chapter is the full story. The subject is one of verification's permanent problems — *a test must parameterize components buried in a hierarchy the test did not write* — and the UVM's answer to it, a path-addressed runtime database, ported whole.

> **In the UVM...** we stored values with `uvm_config_db#(string)::set(this, "env.loga", "MSG", ...)` in SystemVerilog or `ConfigDB().set(self, "env.loga", "MSG", ...)` in pyuvm — a context object, a path string, a key — and components retrieved them with `get()`. Wildcards (`"env.t*"`) configured groups at a stroke; a null context made globals; and when two writers hit the same path, a precedence rule decided.

Notice what that mechanism *is*: late binding, deliberately. The component that reads a value and the test that writes it never meet — not in a constructor, not in a call chain, nowhere the compiler can see. That is not a weakness the UVM's designers failed to engineer away. It is the feature: one environment, closed and finished, serves a hundred tests because the tests reach into it by *name* at run time. A configuration mechanism the compiler could fully check would be one whose decisions were already made at compile time — which is to say, not a configuration mechanism. rustdv keeps the runtime database, keeps the paths, keeps the wildcards, and spends its type system on making the *failures* loud. You will see that at the first `get`.

Reading a value

The lab animal, as in the earlier books, is a `MsgLogger`: a component whose one behavior — the message it logs — comes from outside.

```
// Chapter 27, Figure 1: Logging a message we get from the ConfigDb

#[derive(Component, Default)]
struct MsgLogger;

impl Component for MsgLogger {
    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let _obj = ctx.raise_objection("logging the configured
message");
        let msg: String = ConfigDb::get(Some(ctx), "", "MSG")?;
        ctx.info(&msg);
        Ok(())
    }
}
```

Notice first what this component does *not* have: a constructor argument, a field holding the message, any knowledge of who set it. It asks the database. The interesting line is the `get`, and it repays a slow read.

- **Read the three arguments as context, offset, field.** The store is ambient; the *identity* asking is passed in. The `""` is not a keyword meaning "me" — it is an empty *offset* from the context, which happens to land on the caller. A non-empty offset asks on behalf of another component:

  ```
  ConfigDb::get(Some(ctx), "",     "MSG")   // me
  ConfigDb::get(Some(ctx), "loga", "MSG")   // what loga will
  see
  ConfigDb::get(None,      "",     "SEQR")  // no context at all
  ```

  That third form matters more than it looks. A sequence is not a component and has no path, so when Chapter 36 needs one to find its sequencer, asking with no context is the only way it can ask. Both source books keep exactly this signature, and in *The UVM Primer*'s SystemVerilog the null-context form is the majority idiom.

- **The offset is relative for the same reason log paths are derived, not stored.** An absolute path is a hand-typed string that keeps compiling and starts lying the moment a component moves.

- **A value in the database must either implement `Clone`, so everyone gets their own copy, or ride in an `Rc`, so everyone gets a handle to one copy.** `MSG` is a `String` and a copy is what each logger wants. A sequencer is the opposite case: when Chapter 36 files one in the database, every sequence must reach the *same* sequencer, so it goes in as an `Rc`.

- **`get` returns a `Result`, and the `?` propagates it.** SystemVerilog's `get()` has four distinct ways to disappoint you — the value was never set, the path did not match, the field name was typo'd, the type parameter disagreed with the `set` — and collapses all four into `return 0` with your variable untouched. The failure mode is not the miss; it is that the miss is *silent*, and indistinguishable from a legitimate zero. rustdv's `get` names which of the four happened, and the `Result` is `#[must_use]`: you can propagate it with `?` or handle it, but you cannot quietly drop it. The lookup is still a runtime lookup — that is the design — and when it misses, everyone finds out.

One more thing worth saying about that SystemVerilog signature, because a typed-language reader will assume types would have prevented the mess: `uvm_config_db#(T)` *is* typed — heavily — and the type parameter is part of the lookup. What that bought is a bug class: `set` with `int`, `get` with `uvm_bitstream_t`, and the two calls never meet — a mismatch invisible in the code and silent at run time. pyuvm dropped the type parameter deliberately, and dropping it *removed* that failure mode outright. rustdv follows pyuvm: one key, no type in the address, and the type check happens at the single point of retrieval, loudly. This is the book's recurring lesson in miniature — where a type lives matters more than how many there are.

Writing a value

The environment holds two loggers; the test configures each by path.

```rust
// Chapter 27, Figure 2: Two loggers in the environment
#[derive(Component, Default)]
struct MsgEnv {
    #[component]
    loga: Option<MsgLogger>,
    #[component]
    logb: Option<MsgLogger>,
}

impl Component for MsgEnv {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        self.loga = Some(MsgLogger::default());
        self.logb = Some(MsgLogger::default());
    }
}
```

```rust
// Chapter 27, Figure 3: Giving loga and logb different messages
#[rustdv::test]
#[derive(Component, Default)]
struct MsgTest {
    #[component]
    env: Option<MsgEnv>,
}

impl Component for MsgTest {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        self.env = Some(MsgEnv::default());
        ConfigDb::set(Some(ctx), "env.loga", "MSG",
String::from("LOG A msg"));
        ConfigDb::set(Some(ctx), "env.logb", "MSG",
String::from("LOG B msg"));
    }
}
```

The paths are relative to the *setter*: `"env.loga"` resolves against the test's own position, exactly as `this` anchored the SystemVerilog `set`. And note the timing, because Chapter 24's argument is collecting its first payment: the test writes these values in its `build`, *before* `env` has built its children. Build runs top-down, so by the time `loga` exists and its `run` asks the database, the answer is waiting. The test reaches two components it never touches, through a mechanism the compiler never sees — which is precisely the job.

```
# Figure 4: The loga and logb components have different things to
say

      0.00ns INFO     running MsgTest (1/4)  [ch27-
configuration/src/ch27_configuration.rs:99]
      0.00ns INFO     [MsgTest.env.loga]: LOG A msg
      0.00ns INFO     [MsgTest.env.logb]: LOG B msg
      0.00ns INFO     MsgTest PASSED
```

Wildcards

pyuvm configured a family of components at a stroke with `ConfigDB().set(self, "env.t*", ...)`. rustdv keeps the glob. First, an environment with something worth matching:

```
// Chapter 27, Figure 5: Adding talka and talkb to the environment
#[derive(Component, Default)]
struct MultiMsgEnv {
    #[component]
    loga: Option<MsgLogger>,
    #[component]
    logb: Option<MsgLogger>,
    #[component]
    talka: Option<MsgLogger>,
    #[component]
    talkb: Option<MsgLogger>,
}

impl Component for MultiMsgEnv {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        self.loga = Some(MsgLogger::default());
        self.logb = Some(MsgLogger::default());
        self.talka = Some(MsgLogger::default());
        self.talkb = Some(MsgLogger::default());
    }
}
```

The Python version made `MultiMsgEnv` by subclassing and `super().build_phase()`. Rust has no inheritance, and the difference between the two envs is *structural* — which children exist — so the env is written out. Four fields is cheaper to read than a mechanism for sharing two of them.

```rust
// Chapter 27, Figure 6: Using a wildcard to configure both talkers
at once
#[rustdv::test]
#[derive(Component, Default)]
struct MultiMsgTest {
    #[component]
    env: Option<MultiMsgEnv>,
}

impl Component for MultiMsgTest {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        self.env = Some(MultiMsgEnv::default());
        ConfigDb::set(Some(ctx), "env.loga", "MSG",
String::from("LOG A msg"));
        ConfigDb::set(Some(ctx), "env.logb", "MSG",
String::from("LOG B msg"));
        ConfigDb::set(Some(ctx), "env.t*", "MSG",
String::from("TALK TALK"));
    }
}
```

`set` takes a glob; `get` takes a concrete path. pyuvm enforces the same asymmetry, and it is the right way round: you write to a *pattern* of components, but you always read as *one* component.

```
# Figure 7: The "talk" components get the same message

      0.00ns INFO     running MultiMsgTest (2/4)  [ch27-
configuration/src/ch27_configuration.rs:150]
      0.00ns INFO     [MultiMsgTest.env.loga]: LOG A msg
      0.00ns INFO     [MultiMsgTest.env.logb]: LOG B msg
      0.00ns INFO     [MultiMsgTest.env.talka]: TALK TALK
      0.00ns INFO     [MultiMsgTest.env.talkb]: TALK TALK
      0.00ns INFO    MultiMsgTest PASSED
```

Global data

One more logger, `gtalk`, which nobody configures by name:

```
// Chapter 27, Figure 8: Adding gtalk, which nobody configures by
name
#[derive(Component, Default)]
struct GlobalEnv {
    #[component]
    loga: Option<MsgLogger>,
    #[component]
    logb: Option<MsgLogger>,
    #[component]
    talka: Option<MsgLogger>,
    #[component]
    talkb: Option<MsgLogger>,
    #[component]
    gtalk: Option<MsgLogger>,
}

impl Component for GlobalEnv {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        self.loga = Some(MsgLogger::default());
        self.logb = Some(MsgLogger::default());
        self.talka = Some(MsgLogger::default());
        self.talkb = Some(MsgLogger::default());
        self.gtalk = Some(MsgLogger::default());
    }
}
```

```
// Chapter 27, Figure 9: Storing a message for everybody
#[rustdv::test]
#[derive(Component, Default)]
struct GlobalTest {
    #[component]
    env: Option<GlobalEnv>,
}

impl Component for GlobalTest {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        self.env = Some(GlobalEnv::default());
        ConfigDb::set(Some(ctx), "env.loga", "MSG",
String::from("LOG A msg"));
        ConfigDb::set(Some(ctx), "env.logb", "MSG",
String::from("LOG B msg"));
        ConfigDb::set(Some(ctx), "env.t*", "MSG",
String::from("TALK TALK"));
        ConfigDb::set(None, "*", "MSG", String::from("GLOBAL"));
    }
}
```

The last line is the port of pyuvm's `ConfigDB().set(None, ...)` and SystemVerilog's `set(null, ...)`: with no context to offset from, the glob is

absolute, and `"*"` matches every path. It is the fallback for any component no more specific rule names — here, `gtalk`.

Which raises the obvious question: `loga`'s path matches `env.loga`, `env.t*` does not match it, but `*` does — so which value does `loga` get? **Resolution is most-specific-first**: `env.loga` beats `env.t*` beats `*`. That is pyuvm's rule, and rustdv adopts it. SystemVerilog readers should note their UVM does *not* sort by specificity — it gathers every match and takes the one with the highest precedence, which is why a stray global in an SV testbench can shadow a specific setting in ways that surprise people. Chapter 28 is about seeing what actually resolved, for exactly such moments.

```
# Figure 10: The default is matched only where nothing overrides it

      0.00ns INFO     running GlobalTest (3/4)  [ch27-
configuration/src/ch27_configuration.rs:207]
      0.00ns INFO     [GlobalTest.env.loga]: LOG A msg
      0.00ns INFO     [GlobalTest.env.logb]: LOG B msg
      0.00ns INFO     [GlobalTest.env.talka]: TALK TALK
      0.00ns INFO     [GlobalTest.env.talkb]: TALK TALK
      0.00ns INFO     [GlobalTest.env.gtalk]: GLOBAL
      0.00ns INFO     GlobalTest PASSED
```

The parent/child conflict

The database's most instructive scenario. The env configures its own child; the test configures the same component by a longer path; both writes land on exactly the same component. Which message prints?

```
// Chapter 27, Figure 11: The env configures its own child...
#[derive(Component, Default)]
struct ConflictEnv {
    #[component]
    loga: Option<MsgLogger>,
}

impl Component for ConflictEnv {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        self.loga = Some(MsgLogger::default());
        ConfigDb::set(Some(ctx), "loga", "MSG", String::from("CHILD
RULES!"));
    }
}
```

```
// Chapter 27, Figure 12: ...and the test configures the same
component
#[rustdv::test]
#[derive(Component, Default)]
struct ConflictTest {
    #[component]
    env: Option<ConflictEnv>,
}

impl Component for ConflictTest {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        self.env = Some(ConflictEnv::default());
        ConfigDb::set(Some(ctx), "env.loga", "MSG",
String::from("PARENT RULES!"));
    }
}
```

The parent wins — and it is worth understanding why, because the rule is not arbitrary and it is not "first write wins." Under a naive last-write-wins, the parent would *lose*: build is top-down, so an ancestor always writes before its descendants. If recency decided, every child could silently overrule the test that instantiated it, and configuring a testbench from the top — the entire point of the mechanism — would be impossible. So a build-phase write carries a precedence that *decreases with the setter's depth*: the shallower the writer, the stronger the write, regardless of order. The test outranks the env; `PARENT RULES!` prints. (A write made after build, from a run phase, carries full precedence and outranks every build-time write — by then, whoever is writing is doing so on purpose.) This is the same rule the UVM applies and the same reasoning behind it; the difference is Chapter 28's, where you can ask the database to *show you* the contest instead of memorizing its outcome.

```
# Figure 13: The parent wins

      0.00ns INFO     running ConflictTest (4/4)  [ch27-
configuration/src/ch27_configuration.rs:256]
      0.00ns INFO     [ConflictTest.env.loga]: PARENT RULES!
      0.00ns INFO     ConflictTest PASSED
```

Summary

The problem — tests parameterizing components they never touch — is permanent, and rustdv answers it the way all three UVMs do: a runtime database addressed by hierarchical path. `set` takes context, glob, and field; `get` takes context, offset, and field, and returns a `Result` that names which of the four possible misses happened — the one place this chapter spends types, because the lookup itself is late binding and is supposed to be. Values are `Clone`d in or shared by `Rc`. Wildcards write to patterns; reads are always concrete; resolution is most-specific-first; and in the parent/child conflict the shallower writer wins so that configuration from the top stays possible.

pyuvm needed a second chapter to teach *debugging* the ConfigDB, and honesty requires the same here: a runtime lookup can miss, and the next chapter is the debugger's toolkit that comes with it — printing the database, tracing its decisions, and testing the failure paths on purpose.

Chapter 28: Configuration Debugging

Chapter 27 showed configuration working. This chapter shows it failing, which is the more useful skill — both earlier books devoted a chapter to exactly this, because the config database's failure modes are quiet, remote from their causes, and rank among the UVM's most common support questions in every dialect. The database is addressed by strings resolved at run time, so the compiler cannot help. That is not a flaw to apologize for; it is the cost of late binding, stated plainly in Chapter 27 — and it is why the database ships with a debugger's toolkit. This chapter is that toolkit: an error that names its cause, a dump that shows the competition, and a tracer that films every operation.

> **In the UVM...** we learned the database's classic mistakes one painful demonstration at a time: a path that matched nothing, a key spelled two ways, a wildcard shadowing a specific setting, a parent and child fighting over one component. SystemVerilog's `get()` reports them all the same way — `return 0`, variable untouched — and both books taught `print_config(1)` and `+UVM_CONFIG_DB_TRACE` as the way out.

Missing data

The lab animal is Chapter 27's logger, unchanged: it asks for `"MSG"` and propagates the error with `?`.

```
// Chapter 28, Figure 1: The logger that propagates the error
#[derive(Component, Default)]
struct MsgLogger;

impl Component for MsgLogger {
    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let _obj = ctx.raise_objection("logging the configured
message");
        let msg: String = ConfigDb::get(Some(ctx), "", "MSG")?;
        ctx.info(&msg);
        Ok(())
    }
}
```

Now break it. The environment holds `loga` and `logb`; the test configures only one:

```
// Chapter 28, Figure 2: A message for only one of two loggers
#[rustdv::test(expect_error = "config_not_found")]
#[derive(Component, Default)]
struct MsgTest {
    #[component]
    env: Option<MsgEnv>,
}

impl Component for MsgTest {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        self.env = Some(MsgEnv::default());
        ConfigDb::set(Some(ctx), "env.loga", "MSG",
String::from("LOG A msg"));
    }
}
```

`logb` finds nothing, its `?` fails the test, and the failure carries the name `config_not_found`. Look at the attribute: `expect_error = "config_not_found"` says this test passes *only if it fails that way*. This is sharper than a bare "expected to fail" flag — a test that failed for some other reason is still reported as a failure, and the report says what you got instead:

```
MsgTest FAILED: expected error 'config_type_mismatch', got
config_not_found: ...
```

(That line is real output, produced by deliberately changing the expectation to the wrong kind.) `expect_error` is how this book demonstrates failures without faking transcripts, and it is how you can pin down a testbench's error behavior in a regression.

The second classic:

```
// Chapter 28, Figure 3: Misspelling a key
#[rustdv::test(expect_error = "config_not_found")]
#[derive(Component, Default)]
struct MsgTestAlmostFixed {
    #[component]
    env: Option<MsgEnv>,
}

impl Component for MsgTestAlmostFixed {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        self.env = Some(MsgEnv::default());
        ConfigDb::set(Some(ctx), "env.loga", "MSG",
String::from("LOG A msg"));
        ConfigDb::set(Some(ctx), "env.logb", "MESG",
String::from("LOG B msg"));
    }
}
```

Both loggers are now configured — except one value was stored under `MESG`. The failure is identical to figure 2's, because a key that was never written and a key written under another name are the same thing to the database. Nothing here is a compile error; `"MESG"` is a perfectly good string. This is the bug the toolkit exists for, and figures 5 through 7 will catch it.

Handling a missing value

Sometimes "not found" is not a bug — a component with a sensible default should use it. The variant matters:

```
// Chapter 28, Figure 4: A logger that copes
#[derive(Component, Default)]
struct NiceMsgLogger;

impl Component for NiceMsgLogger {
    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let _obj = ctx.raise_objection("logging the configured
message");
        let msg: String = match ConfigDb::get(Some(ctx), "", "MSG")
{
            Ok(msg) => msg,
            Err(ConfigError::NotFound { .. }) => {
                ctx.warning("Could not find MSG. Setting to
default");
                String::from("No message for you!")
            }
            Err(other) => return Err(other.into()),
        };
        ctx.info(&msg);
        Ok(())
    }
}
```

Matching on the variant is the point. `NotFound` is recoverable — fall back and *say so*, with a warning that leaves a trail in the log. Any other failure is not: a type mismatch means the value is there and you asked for it wrongly, and defaulting past it would bury a real bug under a polite default. Chapter 27 explained why SystemVerilog cannot draw this line — its `get()` collapses every failure into `return 0` — and pyuvm's `except UVMConfigItemNotFound` draws it the same way this `match` does. The difference is the last arm: the compiler makes you decide what happens to the errors you did *not* name.

Printing the database

```
// Chapter 28, Figure 5: Printing the ConfigDb
#[rustdv::test]
#[derive(Component, Default)]
struct NiceMsgTest {
    #[component]
    env: Option<NiceMsgEnv>,
}

impl Component for NiceMsgTest {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        self.env = Some(NiceMsgEnv::default());
        ConfigDb::set(Some(ctx), "env.loga", "MSG",
String::from("LOG A msg"));
    }

    fn end_of_elaboration(&mut self, _ctx: &mut RustdvCtx) {
        ConfigDb::print();
    }
}
```

`end_of_elaboration` is the right home for the dump, and knowing why is knowing the lifecycle: the hierarchy is final and nothing has run yet, so what you see is exactly what the run phase will resolve against. (The same reasoning put `print_config` calls there in both source books.)

```rust
// Chapter 28, Figure 6: Debugging the misspelled key by printing
#[rustdv::test]
#[derive(Component, Default)]
struct NiceMsgTestAlmostFixed {
    #[component]
    env: Option<NiceMsgEnv>,
}

impl Component for NiceMsgTestAlmostFixed {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        self.env = Some(NiceMsgEnv::default());
        ConfigDb::set(Some(ctx), "env.loga", "MSG",
String::from("LOG A msg"));
        ConfigDb::set(Some(ctx), "env.logb", "MESG",
String::from("LOG B msg"));
    }

    fn end_of_elaboration(&mut self, _ctx: &mut RustdvCtx) {
        ConfigDb::print();
    }
}
```

```
# Figure 7: The dump puts MSG and MESG side by side

      0.00ns INFO     running NiceMsgTestAlmostFixed (4/7)  [ch28-
config-debugging/src/ch28_config_debugging.rs:179]
      0.00ns INFO     PATH                          : KEY       :
DATA
      0.00ns INFO     NiceMsgTestAlmostFixed.env.loga: MSG       :
{1000: "LOG A msg"}
      0.00ns INFO     NiceMsgTestAlmostFixed.env.logb: MESG      :
{1000: "LOG B msg"}
      0.00ns INFO     [NiceMsgTestAlmostFixed.env.loga]: LOG A msg
      0.00ns WARNING  [NiceMsgTestAlmostFixed.env.logb]: Could not
find MSG. Setting to default
      0.00ns INFO     [NiceMsgTestAlmostFixed.env.logb]: No message
for you!
      0.00ns INFO     NiceMsgTestAlmostFixed PASSED
```

The dump is what finds the figure-3 bug: two entries under `env.logb`-shaped paths, one keyed `MSG` and one keyed `MESG`, and the mismatch is visible in a way it never is at the point of failure. A misspelling is invisible in the place you wrote it and obvious in a table.

For contrast, the wildcard configuration from Chapter 27, working — worth running with the dump on simply to see what *healthy* looks like, since you will be reading these tables on bad days:

```
// Chapter 28, Figure 8: Wildcards behaving, for contrast
#[rustdv::test]
#[derive(Component, Default)]
struct MultiMsgTest {
    #[component]
    env: Option<MultiMsgEnv>,
}

impl Component for MultiMsgTest {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        self.env = Some(MultiMsgEnv::default());
        ConfigDb::set(Some(ctx), "env.loga", "MSG",
String::from("LOG A msg"));
        ConfigDb::set(Some(ctx), "env.logb", "MSG",
String::from("LOG B msg"));
        ConfigDb::set(Some(ctx), "env.t*", "MSG",
String::from("TALK TALK"));
    }
}
```

Debugging a parent/child conflict

Chapter 27 ended with the parent winning the write to `env.loga` and a promise: here, you can watch the contest instead of memorizing its outcome.

```
// Chapter 28, Figure 9: Both the env and the test configure
env.loga
#[derive(Component, Default)]
struct ConflictEnv {
    #[component]
    loga: Option<MsgLogger>,
}

impl Component for ConflictEnv {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        self.loga = Some(MsgLogger::default());
        ConfigDb::set(Some(ctx), "loga", "MSG", String::from("CHILD
RULES!"));
    }
}

#[rustdv::test]
#[derive(Component, Default)]
struct ConflictTest {
    #[component]
    env: Option<ConflictEnv>,
}

impl Component for ConflictTest {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        self.env = Some(ConflictEnv::default());
        ConfigDb::set(Some(ctx), "env.loga", "MSG",
String::from("PARENT RULES!"));
    }

    fn end_of_elaboration(&mut self, _ctx: &mut RustdvCtx) {
        ConfigDb::print();
    }
}
```

```
# Figure 10: The dump shows the competition, with precedences

PATH                        : KEY       : DATA
ConflictTest.env.loga       : MSG       : {1000: "PARENT RULES!",
999: "CHILD RULES!"}
```

This is where the dump earns its keep. A resolved value tells you who won; it does not tell you anyone else was competing. The dump lists *every* value stored at a path with the precedence each was written at — the losing entry is right there, and the numbers explain the outcome instead of asking you to trust Chapter 27's rule. The test wrote at depth 0, precedence 1000; the env at depth 1, precedence 999; shallower wins, and now you can see by how much.

Tracing

The dump is a snapshot; tracing is the film.

```
// Chapter 28, Figure 11: Tracing every ConfigDb operation
#[rustdv::test]
#[derive(Component, Default)]
struct GlobalTest {
    #[component]
    env: Option<GlobalEnv>,
}

impl Component for GlobalTest {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        ConfigDb::set_tracing(true);
        self.env = Some(GlobalEnv::default());
        ConfigDb::set(Some(ctx), "env.loga", "MSG",
String::from("LOG A msg"));
        ConfigDb::set(Some(ctx), "env.logb", "MSG",
String::from("LOG B msg"));
        ConfigDb::set(Some(ctx), "env.t*", "MSG",
String::from("TALK TALK"));
        ConfigDb::set(None, "*", "MSG", String::from("GLOBAL"));
    }
}
```

```
# Figure 12: Every set and get, as it happens

CFGDB/SET context=GlobalTest offset="env.loga" ->
GlobalTest.env.loga MSG="LOG A msg"
CFGDB/SET context=<none> offset="*" -> * MSG="GLOBAL"
CFGDB/GET context=GlobalTest.env.gtalk offset="" ->
GlobalTest.env.gtalk MSG="GLOBAL"
```

Turn it on before building the hierarchy and every operation logs with the context, the offset, and the path they resolved to. Read the trace's structure: each line shows the *inputs* and the resolution. When a lookup misses, the thing you got wrong is almost always the resolved path — a context you did not expect, an offset that anchored somewhere else — and the trace shows exactly the resolution the database performed, not the one you imagined. This is the port of `+UVM_CONFIG_DB_TRACE`, as a call rather than a plusarg, so a test can scope it to the region under suspicion.

Summary

Late binding traded away compile-time checking; this chapter is what it bought instead. A failed `get` is a `Result` whose variants distinguish the recoverable miss (`NotFound` — default and warn) from the genuine bugs (a type mismatch is never something to default past), and `expect_error` turns a deliberate failure into a regression asset that fails if it fails any *other* way. `ConfigDb::print()` at `end_of_elaboration` shows the database as the run phase will see it — misspellings side by side, conflicts with the precedence numbers that decide them — and `ConfigDb::set_tracing(true)` films every set and get with the resolution that the point of failure never shows you.

The ConfigDb carries values to components that nobody passed them to. The factory, next, does the same for *types*: it builds components a test can substitute without touching the environment that asks for them.

Chapter 29: The Factory

The UVM factory answers a question every test writer eventually asks: *how do I change what the testbench does without editing the testbench?* One environment, closed and finished, should serve many tests — and configuration alone only changes *values*. To change what a slot in the hierarchy is *built as*, you need construction itself to be interceptable. That is the factory, and rustdv has one, working the way the factory you know works: build a component through it, and code above you can substitute a different type without touching the code that built it.

> **In the UVM…** we instantiated components through the factory — `tiny_component::type_id::create("tc", this)` in SystemVerilog, `TinyComponent.create("tc", self)` in pyuvm — instead of calling the constructor; then `set_type_override_by_type(...)` made every subsequent create of a Tiny produce a Medium, and an instance override targeted one path. Registration happened behind our backs — the `` `uvm_component_utils `` macro in SV, a metaclass at import time in Python — and `factory.print()` listed the overrides in force.

Everything in that box has a direct rustdv counterpart, and this chapter walks them in the same order the Python book's factory chapter does. The differences are under the floor, and the chapter will point at each as it goes by.

Creating a component through the factory

```
// Chapter 29, Figure 1: A tiny example component
#[derive(Component, Default)]
struct TinyComponent;

impl Component for TinyComponent {
    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let _obj = ctx.raise_objection("tiny");
        ctx.info("I'm so tiny!");
        Ok(())
    }
}
```

Nothing here mentions the factory, and that is the first difference worth noticing: **registration is universal and automatic.** `#[derive(Component)]` enrolls every component by name, so `TinyComponent` can be created by type or by the string `"TinyComponent"`, and can be the target of an override, with no separate registration step and no "did I remember the utils macro?" This is the same promise pyuvm's metaclass makes, kept by the derive you were already writing.

Now, two ways to build one:

```
// Chapter 29, Figure 2: Building the component the normal way
#[rustdv::test]
#[derive(Component, Default)]
struct TinyTest {
    #[component]
    tc: RustdvComp,
}

impl Component for TinyTest {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        self.tc = TinyComponent::new_comp();
    }
}
```

```
# Figure 3: The normal way builds what it says

      0.00ns INFO     running TinyTest (1/7)  [ch29-
factory/src/ch29_factory.rs:71]
      0.00ns INFO     [TinyTest.tc]: I'm so tiny!
```

`new_comp()` is rustdv's plain constructor — the analog of UVM's `new`. Note what it does *not* take: no name, no parent. Both come from the tree, as they have since Chapter 24. A component built this way is fixed; nobody upstream can swap it, because it never went through the factory.

```
// Chapter 29, Figure 4: Building the component through the factory
#[rustdv::test]
#[derive(Component, Default)]
struct TinyFactoryTest {
    #[component]
    tc: RustdvComp,
}

impl Component for TinyFactoryTest {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        self.tc = TinyComponent::create_comp();
    }
}
```

```
# Figure 5: The factory way builds the same thing — until someone
objects

      0.00ns INFO     running TinyFactoryTest (2/7)  [ch29-
factory/src/ch29_factory.rs:90]
      0.00ns INFO     [TinyFactoryTest.tc]: I'm so tiny!
```

Put figures 2 and 4 side by side: identical structs, one `RustdvComp` field each, and exactly one line different — `new_comp()` versus `create_comp()`. With no override in force they even log the same output. The difference is invisible here and total later: `create_comp()` flags the slot, and the build walk checks flagged slots for an override and swaps in the substitute if one is installed. This is the `new` versus `create` distinction every UVM engineer already carries, transcribed — and it puts a real decision in the block author's hands: **overridability is the build line, not the field type.** A `RustdvComp` field says nothing about whether its occupant can be swapped; the line that fills it says

everything. Write `create_comp()` for the slots a reuser may replace, `new_comp()` for the ones they may not.

The string form completes the set:

```
// Chapter 29, Figure 6: Building a component from a string name
#[rustdv::test]
#[derive(Component, Default)]
struct CreateByNameTest {
    #[component]
    tc: RustdvComp,
}

impl Component for CreateByNameTest {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        self.tc = Factory::create_by_name("TinyComponent");
    }
}
```

The name is data — here a literal, in a bigger testbench a line from a command file. The universal registry is what turns the string back into a constructor, and this test logs exactly what figures 3 and 5 did. An unregistered name is a testbench bug and fails at this call — names are data, and no compiler checks data.

Overriding a type

```
// Chapter 29, Figure 7: The component we substitute in
#[derive(Component, Default)]
struct MediumComponent;

impl Component for MediumComponent {
    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let _obj = ctx.raise_objection("medium");
        ctx.info("I'm medium size.");
        Ok(())
    }
}
```

```rust
// Chapter 29, Figure 8: Overriding TinyComponent with
MediumComponent, by type
#[rustdv::test]
#[derive(Component, Default)]
struct MediumFactoryTest {
    #[component]
    tc: RustdvComp,
}

impl Component for MediumFactoryTest {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        Factory::set_type_override::<TinyComponent,
MediumComponent>();
        self.tc = TinyComponent::create_comp();
    }
}
```

```
# Figure 9: The same create line builds something else

      0.00ns INFO     running MediumFactoryTest (4/7)  [ch29-
factory/src/ch29_factory.rs:151]
      0.00ns INFO     [MediumFactoryTest.tc]: I'm medium size.
```

Read the build closely, because its two lines are doing Chapter 24's argument one more time. The create line is unedited — it still says `TinyComponent::create_comp()`, and in a real testbench it would live in an environment that never learns it was overridden. The override is installed *above*, before the create runs, and it is build's top-down direction that guarantees the ordering: the test's `build` runs before the walk descends to the slot, so the override is in force by the time it matters. This is the gap between existing and having children, doing exactly the job Chapter 24 promised the factory would need it for.

One check does happen at compile time, and it is worth being exact about which: the type pair. `set_type_override::<TinyComponent, MediumComponent>()` requires the substitute to *be* a component — the maker generated from it must produce a tree node — so overriding with a non-component does not build. SystemVerilog catches that analog at run time in `$cast`. Which slot gets overridden, though, and whether anyone creates a `TinyComponent` at all — those resolve at run time, by design, because deferring them is what the factory is *for*.

When even the types are data, the override is too:

```
// Chapter 29, Figure 10: The same override, by string name
#[rustdv::test]
#[derive(Component, Default)]
struct MediumNameTest {
    #[component]
    tc: RustdvComp,
}

impl Component for MediumNameTest {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        Factory::set_type_override_by_name("TinyComponent",
"MediumComponent");
        self.tc = TinyComponent::create_comp();
    }
}
```

```
# Figure 11: Same substitution, by name

      0.00ns INFO     running MediumNameTest (5/7)  [ch29-
factory/src/ch29_factory.rs:171]
      0.00ns INFO     [MediumNameTest.tc]: I'm medium size.
```

Overriding one instance

A type override hits every flagged slot that asks for the type. Sometimes you want just one:

```
// Chapter 29, Figure 12: An environment with two components of the
same type
#[derive(Component, Default)]
struct TwoCompEnv {
    #[component]
    tc1: RustdvComp,
    #[component]
    tc2: RustdvComp,
}

impl Component for TwoCompEnv {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        self.tc1 = TinyComponent::create_comp();
        self.tc2 = TinyComponent::create_comp();
    }
}
```

```
// Chapter 29, Figure 13: Overriding only env.tc1
#[rustdv::test]
#[derive(Component, Default)]
struct TwoCompTest {
    #[component]
    env: Option<TwoCompEnv>,
}

impl Component for TwoCompTest {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        Factory::set_inst_override::<TinyComponent,
MediumComponent>(ctx, "env.tc1");
        self.env = Some(TwoCompEnv::default());
    }
}
```

```
# Figure 14: The path picks the instance

      0.00ns INFO     running TwoCompTest (6/7)  [ch29-
factory/src/ch29_factory.rs:218]
      0.00ns INFO     [TwoCompTest.env.tc1]: I'm medium size.
      0.00ns INFO     [TwoCompTest.env.tc2]: I'm so tiny!
```

Notice what the env did *not* do: it built `tc1` and `tc2` with two identical `create_comp()` calls, no names typed. The factory can still tell them apart because the *fields* are named — when the walk reaches each slot, it checks the override against the path it landed at, and `env.tc1` matches while `env.tc2` does not. The path in `set_inst_override` is a string, but it is a string doing the

same job `ConfigDb::set`'s path does: addressing a component elsewhere in the tree, from a place that has no other way to point at it. It does not duplicate a name the field already carries.

(And the demonstration is not vacuous: delete the `set_inst_override` line and both children log "I'm so tiny!"; restore it and only `tc1` changes. The override drives the outcome — a rerun anyone can do.)

Debugging the factory

Chapter 28 gave the ConfigDb a dump because a resolved value doesn't show the competition. The factory has the same need — a resolved build tells you what got built, not *why* — and the same answer:

```
// Chapter 29, Figure 15: Printing the overrides in force
#[rustdv::test]
#[derive(Component, Default)]
struct PrintOverridesTest {
    #[component]
    tc: RustdvComp,
}

impl Component for PrintOverridesTest {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        Factory::set_type_override_by_name("TinyComponent",
"MediumComponent");
        self.tc = TinyComponent::create_comp();
    }

    fn end_of_elaboration(&mut self, _ctx: &mut RustdvCtx) {
        Factory::print();
    }
}
```

```
# Figure 16: The overrides in force, listed

      0.00ns INFO     running PrintOverridesTest (7/7)  [ch29-
factory/src/ch29_factory.rs:241]
      0.00ns INFO     Factory overrides:
      0.00ns INFO       *                           : TinyComponent
-> MediumComponent
      0.00ns INFO     [PrintOverridesTest.tc]: I'm medium size.
```

`Factory::print()` at `end_of_elaboration`, for Chapter 28's reason: the hierarchy is final, nothing has run, and what you see is what every `create_comp()` resolved against. (Under the floor it is the same store the ConfigDb dumps, seen through the factory's window — a fact you can enjoy and never need.)

Summary

The factory, ported whole and working as the one you know: `new_comp()` is `new` and fixed, `create_comp()` is `create` and overridable, and the choice between them is the block author deciding what a reuser may swap — per build line, not per type. Registration is universal via the derive, so create-by-name and override-by-name need no bookkeeping; type overrides check at compile time only that the substitute is a component, and everything else — which slots, which paths, whether the override fires at all — resolves during the top-down build walk, which is the moment Chapter 24's gap exists to provide. Instance overrides tell twins apart by the paths their field names created, and `Factory::print()` shows the standing orders when a build surprises you.

Testbench 5.0 puts the factory to work: one environment with a variation point, and two tests that fill it differently without touching a line of the env.

Chapter 30: Variation-Point Testbench: 5.0

Testbench 4.0 had a flaw both earlier books flagged the moment it shipped: two tests needed two environments — `AluEnv<RandomOperands>` and `AluEnv<MaxOperands>` in our version, `RandomEnv` and `MaxEnv` before — even though the environments differed in exactly one component. Version 5.0 fixes it the way the factory always promised: **one environment**, with the difference carried in from the tests, through the machinery Chapter 29 just built.

> **In the UVM…** we kept one `AluEnv` that created its tester through the factory — `base_tester::type_id::create("tester", this)` in SystemVerilog, `BaseTester.create("tester", self)` in pyuvm — and each test registered an override in `build_phase`: `set_type_override_by_type(BaseTester, RandomTester)`. Three lines that changed what the env built without the env knowing.

Before the code, the design question this version answers. Testbench 4.0's `AluEnv<T>` chose its tester with a *type parameter* — a compile-time decision, which meant `RandomEnv` and `MaxEnv` were **different types**. That is the right tool when the variation is fixed at build time. But the whole point of a variation point is that a *test* chooses at *run* time — and a factory override cannot reach a type parameter: by the time any code runs, `AluEnv<RandomOperands>` simply *is* what it is, monomorphized and sealed. Runtime choice needs a runtime slot. So 5.0's env is one concrete type with a `create_comp()` line where the type parameter used to be, and the generic form survives for what it is good at: variation chosen at compile time, like Chapter 26's logging policies. Know which kind of variation you have, and you know which tool to reach for.

The testers

The testers are the ones you have had since testbench 2.0. Chapter 20 wrote them as a trait with one required method and one provided one, which is how Rust says “abstract base class with a single overridden method”:

```
// Chapter 30, Figure 1: The Tester trait — one method varies, the
rest is shared
trait Tester {
    fn get_operands(&mut self) -> (u8, u8);

    async fn execute(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let _obj = ctx.raise_objection("tester stimulus");
        let bfm: Rc<TinyAluBfm> = ConfigDb::get(Some(ctx), "",
"BFM")?;

        bfm.reset().await;

        for op in Ops::ALL {
            let (aa, bb) = self.get_operands();
            bfm.send_op(aa, bb, op).await;
        }
        // send two dummy operations to allow
        // the last real operation to complete
        bfm.send_op(0, 0, Ops::Add).await;
        bfm.send_op(0, 0, Ops::Add).await;
        Ok(())
    }
}
```

Only the opening has changed since Chapter 20. A tester is now a component, so `execute` takes the context rather than a BFM handle: it raises the objection that holds the run phase open, asks the ConfigDb for the BFM nobody handed it, and does its own reset — setup that Chapter 20’s `execute_test` handled before calling it. `get_operands` is untouched, and it is still the only thing a tester has to write.

```rust
// Chapter 30, Figure 2: The abstract base and the two testers that
fill its slot
#[derive(Component, Default)]
struct BaseTester;

impl Component for BaseTester {
    async fn run(&mut self, _ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        panic!("BaseTester is abstract — override it with
RandomTester or MaxTester");
    }
}

#[derive(Component, Default)]
struct RandomTester {
    rng: Option<Rng>,
}

impl Tester for RandomTester {
    fn get_operands(&mut self) -> (u8, u8) {
        let rng = self.rng.as_mut().expect("build phase did not
run");
        (rng.u8(), rng.u8())
    }
}

impl Component for RandomTester {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        self.rng = Some(ctx.rng());
    }

    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        self.execute(ctx).await
    }
}

#[derive(Component, Default)]
struct MaxTester;

impl Tester for MaxTester {
    fn get_operands(&mut self) -> (u8, u8) {
        (0xFF, 0xFF)
    }
}

impl Component for MaxTester {
    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        self.execute(ctx).await
    }
}
```

Each tester now wears two traits. `Tester` gives it stimulus, exactly as before; `Component` gives it phases; and `run` is the one line that joins them — the phaser calls `run`, `run` calls `execute`, and `execute` calls back into `get_operands`. `RandomTester` picks up its seeded `Rng` in `build` because a component is created by its parent with nothing passed in.

`BaseTester` is the type the environment names and the factory overrides — the analog of the Python book's abstract `BaseTester`, which raises an error if run un-overridden. Here that is a `panic!`: a test that forgets its override builds a `BaseTester`, and running one *is* the bug, reported in its own words. It implements `Component` but not `Tester`, because there is no stimulus it could sensibly run. The derive registers all three types, so any of them can stand in the tester slot.

The environment

The scoreboard is testbench 4.0's, re-shown in the chapter's file and deliberately untouched — the reader is meant to see it unchanged while the tester's *selection* changes around it. The env is where 5.0 differs:

```
// Chapter 30, Figure 3: The environment builds its tester through
the factory
#[derive(Component, Default)]
struct AluEnv {
    #[component]
    scoreboard: RustdvComp,
    #[component]
    tester: RustdvComp,
}

impl Component for AluEnv {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        self.scoreboard = Scoreboard::new_comp();
        self.tester = BaseTester::create_comp();
    }

    fn start_of_simulation(&mut self, ctx: &mut RustdvCtx) {
        let bfm: Rc<TinyAluBfm> = ConfigDb::get(Some(ctx), "",
"BFM").expect("the test sets BFM");
        bfm.start_tasks();
    }
}
```

Two build lines, and they encode the block author's whole policy. The scoreboard is `new_comp()` — fixed, not a variation point, no test may swap it. The tester is `create_comp()` — the one slot a reuser may fill differently. Same field type on both (`RustdvComp` says nothing about overridability); the build line carries the decision, exactly as Chapter 29 taught. Note also that the env is what starts the BFM's tasks — once, for the whole environment, matching the Python book's `AluEnv` .

The tests

```
// Chapter 30, Figure 4: random_test overrides BaseTester with
RandomTester
#[rustdv::test]
#[derive(Component, Default)]
struct RandomTest {
    #[component]
    env: RustdvComp,
}

impl Component for RandomTest {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        let bfm = TinyAluBfm::new(&ctx.dut()).expect("TinyALU
signals");
        ConfigDb::set(None, "*", "BFM", Rc::new(bfm));
        Factory::set_type_override::<BaseTester, RandomTester>();
        self.env = AluEnv::new_comp();
    }
}
```

```
// Chapter 30, Figure 5: max_test differs only in the tester it
installs
#[rustdv::test]
#[derive(Component, Default)]
struct MaxTest {
    #[component]
    env: RustdvComp,
}

impl Component for MaxTest {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        let bfm = TinyAluBfm::new(&ctx.dut()).expect("TinyALU
signals");
        ConfigDb::set(None, "*", "BFM", Rc::new(bfm));
        Factory::set_type_override::<BaseTester, MaxTester>();
        self.env = AluEnv::new_comp();
    }
}
```

The build-order guarantee from Chapters 24 and 29, working: the test's `build` installs the override before the walk descends into `env`, so by the time the factory resolves `env.tester`, the substitution is in force. The env is not edited between the two tests, and does not know which tester it got.

```
# Figure 6: One env, two behaviors

      0.00ns INFO     rustdv: found 2 test(s), RUSTDV_RANDOM_SEED=1
      0.00ns INFO     running RandomTest (1/2)  [ch30-variation-
point-testbench-5.0/src/ch30_variation_point_testbench_5_0.rs:245]
    150.00ns INFO     [RandomTest.env.scoreboard]: PASSED: c1 Add
67 = 0128
    150.00ns INFO     [RandomTest.env.scoreboard]: PASSED: 5e And
0b = 000a
    150.00ns INFO     [RandomTest.env.scoreboard]: PASSED: b9 Xor
80 = 0039
    150.00ns INFO     [RandomTest.env.scoreboard]: PASSED: a5 Mul
75 = 4b69
    150.00ns INFO     [RandomTest.env.scoreboard]: Covered all
operations
    150.00ns INFO     RandomTest PASSED
    150.00ns INFO     running MaxTest (2/2)  [ch30-variation-point-
testbench-5.0/src/ch30_variation_point_testbench_5_0.rs:262]
    300.00ns INFO     [MaxTest.env.scoreboard]: PASSED: ff Add ff =
01fe
    300.00ns INFO     [MaxTest.env.scoreboard]: PASSED: ff And ff =
00ff
    300.00ns INFO     [MaxTest.env.scoreboard]: PASSED: ff Xor ff =
0000
    300.00ns INFO     [MaxTest.env.scoreboard]: PASSED: ff Mul ff =
fe01
    300.00ns INFO     [MaxTest.env.scoreboard]: Covered all
operations
    300.00ns INFO     MaxTest PASSED
**********************************************************************
***********
** TEST                                          STATUS  SIM TIME (ns)
**
**********************************************************************
***********
** RandomTest                                      PASS         150.00
**
** MaxTest                                         PASS         150.00
**
**********************************************************************
***********
REGRESSION: PASS
```

Compare against Chapter 25's transcript: the operands and results are *identical*, bit for bit, same seed. The same stimulus, selected by a runtime factory override instead of a compile-time type parameter — which is the entire chapter, demonstrated by two transcripts agreeing.

Summary

Testbench 5.0 replaces 4.0's type-parameter variation with a factory slot: one concrete `AluEnv` whose tester is built with `create_comp()`, an abstract `BaseTester` whose run is a self-describing `panic!`, and tests that differ only in the override they install before building the env. The dividing rule is worth keeping: a type parameter serves variation chosen at compile time; a `create_comp()` slot serves variation a test chooses at run time, because an override cannot reach into a monomorphized type. The scoreboard, deliberately untouched, is the control group.

The env's components still share data the pre-UVM way, though — everything funnels through the BFM's queues, and the scoreboard hogs them. Giving components a standard way to talk to *each other* is TLM, and it is next.

Chapter 31: Component Communications

Testbench 5.0 ended with an environment that builds its components through the factory — and with those components still doing their talking through the BFM's queues, as they have since version 1.0. That worked because our components sat directly on the DUT's traffic. It stops working the moment one *testbench* component needs to hand data to another: the tester to a driver, a monitor to a scoreboard. Wire them directly and each must know the other's type, which un-does everything the factory just bought us. The UVM's answer is transaction-level connection — TLM — and this chapter ports its point-to-point half: ports, exports, and the FIFO between them.

> **In the UVM…** we connected components with TLM-1 machinery: `uvm_blocking_put_port` and its export on the far side, `uvm_get_port`, the nonblocking and peek variants, each wired with `connect()` calls in `connect_phase` — and a runtime error (pyuvm's `UVMTLMConnectionError`, SV's elaboration-time connection fatal) when the wiring was wrong. A `uvm_tlm_fifo` sat between a producer's put port and a consumer's get port.

The model, in one paragraph, because every listing in this chapter is an instance of it. A **port** is the thing a component *calls*: `put`, `get`, `peek`, each in a blocking form that waits and a `try_` form that does not. The port forwards to an **export** on the far side. The export belongs to a **FIFO**, which owns the queue and implements every capability against it. And here is the part that makes it architecture rather than plumbing: the producer connects to the FIFO, the consumer connects to the FIFO, and *neither ever learns the other exists*. The FIFO is the point of decoupling. Swap the consumer through a factory override and the producer neither knows nor cares — which is precisely why this chapter had to come after the factory.

Blocking put, get, and peek

```
// Chapter 31, Figure 1: A producer holds a put port and blocks on
a full FIFO
#[derive(Component, Default)]
struct Producer {
    #[port(put)]
    put_port: PutPort<u32>,
}

impl Component for Producer {
    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let _obj = ctx.raise_objection("producing");
        for n in 0..3 {
            self.put_port.put(n).await; // blocks while the FIFO is
full
            ctx.info(&format!("put {n}"));
        }
        Ok(())
    }
}
```

Two new pieces of vocabulary:

- `#[port(put)]` — the attribute declares this field to the derive as a port, which does two jobs at once: it makes the port reachable *by name* when the environment wires it (you will see how in figure 3), and it enrolls the port in the end-of-elaboration connection check (figure 14 shows what that buys).
- `self.put_port.put(n).await` — a blocking put, in the coroutine sense both source frameworks use: the producer's run phase is suspended, in simulated time, until the FIFO has room.

```rust
// Chapter 31, Figure 2: A consumer that peeks, then gets
#[derive(Component, Default)]
struct Consumer {
    #[port(peek)]
    peek_port: PeekPort<u32>,
    #[port(get)]
    get_port: GetPort<u32>,
}

impl Component for Consumer {
    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let _obj = ctx.raise_objection("consuming");
        for _ in 0..3 {
            let seen = self.peek_port.peek().await; // blocks while
empty; no consume
            ctx.info(&format!("peeked {seen}"));
            let got = self.get_port.get().await; // consumes the
peeked item
            assert_eq!(seen, got, "peek must not consume the
item");
            ctx.info(&format!("got {got}"));
        }
        Ok(())
    }
}
```

The consumer declares *two* ports and wires both to the same FIFO: `peek` returns the next item without consuming it, blocking until one is there, and `get` then consumes that same item. The `assert_eq!` makes the “peek does not consume” contract executable rather than documentary.

```rust
// Chapter 31, Figure 3: The env builds the two components and a
FIFO, then
// wires them in `connect`
#[rustdv::test]
#[derive(Component, Default)]
struct PutGetPeekTest {
    #[component]
    producer: RustdvComp,
    #[component]
    consumer: RustdvComp,
    #[component]
    fifo: TlmFifo<u32>,
}

impl Component for PutGetPeekTest {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        self.producer = Producer::new_comp();
        self.consumer = Consumer::new_comp();
        self.fifo = TlmFifo::new(1); // depth 1 — forces the
producer to wait
    }

    fn connect(&mut self, _ctx: &mut RustdvCtx) {
        self.fifo.put_export().connect(&self.producer,
Producer::PUT_PORT);
        self.fifo.peek_export().connect(&self.consumer,
Consumer::PEEK_PORT);
        self.fifo.get_export().connect(&self.consumer,
Consumer::GET_PORT);
    }
}
```

This is `connect` doing the job Chapter 24 gave it, and each line deserves unpacking, because the shape is subtler than it looks.

- The children are `RustdvComp` — factory-built, type-erased, exactly as testbench 5.0 left them. Which means the parent *cannot* write `self.producer.put_port`: that field does not exist on a `RustdvComp`, and Rust has no `$cast`-to-concrete to recover it. Something else must make the port reachable.
- That something is the port name. `#[port(put)]` on `Producer` generated an associated constant, `Producer::PUT_PORT`, and the derive taught `ComponentNode` to answer for it — through the same trait-object surface everything else uses. The export initiates: `fifo.put_export().connect(&self.producer, Producer::PUT_PORT)` says *this FIFO's put side serves that component's port of this name*.

- The name is typed, not a string. `Producer::PUT_PORT` carries the port's interface in its type, so misspelling it does not compile, and aiming a `get` export at a `put` port does not compile either. What resolves at elaboration is the wiring; what the wiring *means* was settled earlier.
- The FIFO itself is a `#[component]` child — concrete, not factory-erased, so its exports are reachable to call `connect` on. That is a deliberate carve-out, and it is the model closest to the UVM, where `uvm_tlm_fifo` is a real component with a path: a FIFO is plumbing. You will never override one through the factory, so it never pays the erasure that makes overriding possible.

There is one more thing to notice: the *same* `connect` call works when a component wires its own port. Figure 12 will show `connect(self, MathTest::X_OUT)` — `self`, not a child. That is not luck; it rules out a whole family of designs. A registry keyed by hierarchical path, say, could reach a child but never the connecting component itself, because a component does not know its own path. A mechanism that works for a child but needs a second spelling for `self` has broken the UVM's uniformity — the property that `uvm_test` *is* a `uvm_component`, no cases, no exceptions. A trait method answers for whoever implements the trait, which is every component including the one doing the connecting. That uniformity is why connection is a trait method and not a lookup.

```
# Figure 4: Alternating through a depth-1 FIFO

      0.00ns INFO     running PutGetPeekTest (1/4)  [ch31-
component-communications/src/ch31_component_communications.rs:124]
      0.00ns INFO     [PutGetPeekTest.producer]: put 0
      0.00ns INFO     [PutGetPeekTest.consumer]: peeked 0
      0.00ns INFO     [PutGetPeekTest.consumer]: got 0
      0.00ns INFO     [PutGetPeekTest.producer]: put 1
      0.00ns INFO     [PutGetPeekTest.consumer]: peeked 1
      0.00ns INFO     [PutGetPeekTest.consumer]: got 1
      0.00ns INFO     [PutGetPeekTest.producer]: put 2
      0.00ns INFO     [PutGetPeekTest.consumer]: peeked 2
      0.00ns INFO     [PutGetPeekTest.consumer]: got 2
      0.00ns INFO     PutGetPeekTest PASSED
```

Read the interleaving. The FIFO holds one item, so the producer *cannot* run ahead: put, peek, get, put, peek, get. Two run phases are taking turns in simulated time — which means two run phases are running *at once*. This is the

payoff Chapter 24 promised: a producer blocked on a full FIFO can only proceed because the consumer's run phase is live to drain it. A phaser that ran components to completion one at a time would deadlock on this listing — the simplest one in the chapter.

Nonblocking put and get

The blocking forms wait; the `try_` forms answer immediately and let the component decide what to do about "no." The examples switch from `u32` to a transaction with something to lose:

```
// A transaction, not an integer
#[derive(Debug)]
struct Packet {
    n: u32,
    label: String,
}

impl Packet {
    fn new(n: u32) -> Packet {
        Packet { n, label: format!("pkt{n}") }
    }
}
```

`Packet` owns a `String`, so it is not `Copy` — like every real transaction you will ever put on a port. The figures below are written against it deliberately; a `u32` would let a retry loop compile that falls apart the moment you substitute your own command type.

```
// Chapter 31, Figure 5: A non-blocking producer never waits
#[derive(Component, Default)]
struct NbProducer {
    #[port(put)]
    put_port: PutPort<Packet>,
}

impl Component for NbProducer {
    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let _obj = ctx.raise_objection("producing (nb)");
        for n in 0..3 {
            let mut packet = Packet::new(n);
            while let Err(back) = self.put_port.try_put(packet) {
                ctx.info("FIFO full, retrying");
                Timer::ns(1).await;
                packet = back; // the FIFO gave it back; try again
with it
            }
            ctx.info(&format!("put {n}"));
        }
        Ok(())
    }
}
```

Read the `Err` binding slowly, because this signature is the chapter's best lesson in how ownership shapes an API. The UVM's `try_put` returns a *bit*. It can, because SystemVerilog passes a class handle and the caller still holds its own; a refused put costs nothing. rustdv's `try_put` takes the packet **by value** — it must, since a successful put hands the packet to whoever gets it next — and so a bare "no" would have *eaten* a packet that was never delivered. The signature has to be `Result<(), T>`: `Err(back)` is the packet coming home, and `packet = back` is the retry loop taking it back for the next attempt.

Two things to be clear about. First, this is not Rust catching a bug SystemVerilog has. SystemVerilog has no bug here; it has a different ownership model, and each signature is the correct one for its model. Second, the tempting way to write the loop — `while self.put_port.try_put(packet).is_err()` — does not compile, because `packet` moved into the first attempt and is gone by the second. The compiler is not being difficult; it is asking the question the design already answered: *if the put failed, who has the packet?* The `Err` binding is the answer.

```
// Chapter 31, Figure 6: A non-blocking consumer
#[derive(Component, Default)]
struct NbConsumer {
    #[port(get)]
    get_port: GetPort<Packet>,
}

impl Component for NbConsumer {
    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let _obj = ctx.raise_objection("consuming (nb)");
        let mut seen = 0;
        while seen < 3 {
            match self.get_port.try_get() {
                Some(packet) => {
                    ctx.info(&format!("got {} (n={})",
packet.label, packet.n));
                    seen += 1;
                }
                None => Timer::ns(1).await,
            }
        }
        Ok(())
    }
}
```

`try_get` returns `Option<Packet>` for the mirror-image reason: a get either hands you the whole packet — label and all, the consumer now owns it and the FIFO no longer does — or hands you nothing. There is no third state in which a packet exists but nobody owns it.

```rust
// Chapter 31, Figure 7: Same wiring, non-blocking components
#[rustdv::test]
#[derive(Component, Default)]
struct NonBlockingTest {
    #[component]
    producer: RustdvComp,
    #[component]
    consumer: RustdvComp,
    #[component]
    fifo: TlmFifo<Packet>,
}

impl Component for NonBlockingTest {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        self.producer = NbProducer::new_comp();
        self.consumer = NbConsumer::new_comp();
        self.fifo = TlmFifo::new(1);
    }

    fn connect(&mut self, _ctx: &mut RustdvCtx) {
        self.fifo.put_export().connect(&self.producer,
NbProducer::PUT_PORT);
        self.fifo.get_export().connect(&self.consumer,
NbConsumer::GET_PORT);
    }
}
```

The FIFO now carries `Packet` rather than `u32`, and not one connect line changed shape — the generic FIFO doing its job. (And the typed port name doing its job too: connect this `TlmFifo<Packet>` to a port that wants `u32` and the mistake fails to compile.)

```
# Figure 8: Nonblocking components spend time instead of waiting

      0.00ns INFO     running NonBlockingTest (2/4)  [ch31-
component-communications/src/ch31_component_communications.rs:247]
      0.00ns INFO     [NonBlockingTest.producer]: put 0
      0.00ns INFO     [NonBlockingTest.producer]: FIFO full,
retrying
      0.00ns INFO     [NonBlockingTest.consumer]: got pkt0 (n=0)
      1.00ns INFO     [NonBlockingTest.producer]: put 1
      1.00ns INFO     [NonBlockingTest.producer]: FIFO full,
retrying
      1.00ns INFO     [NonBlockingTest.consumer]: got pkt1 (n=1)
      2.00ns INFO     [NonBlockingTest.producer]: put 2
      2.00ns INFO     [NonBlockingTest.consumer]: got pkt2 (n=2)
      2.00ns INFO     NonBlockingTest PASSED
```

Same three packets, but now the clock moves: each retry burns a nanosecond of `Timer` instead of suspending on the FIFO. Blocking components let the FIFO schedule them; nonblocking components schedule themselves.

The parent runs too: a three-stage pipeline

Everything so far had a parent that only built and connected. The next test is the chapter's centerpiece, and the reason is architectural: **the test itself is a stage**.

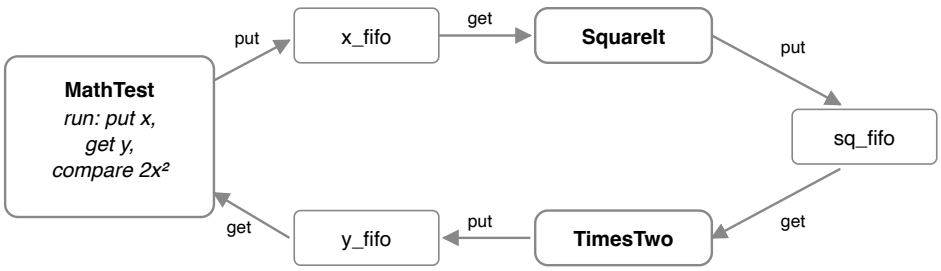


Figure 9: A processing pipeline — y = 2x². The test is the first and last stage; the workers know only their FIFOs.

The test chooses x, sends it into the pipeline, and waits for y to come back around, comparing against 2x². Two worker components do the arithmetic, each connected only to FIFOs; neither knows the other exists — or that the thing feeding them is the test itself.

```
// Chapter 31, Figure 10: The first stage squares its input
#[derive(Component, Default)]
struct SquareIt {
    #[port(get)]
    input: GetPort<u32>,
    #[port(put)]
    output: PutPort<u32>,
}

impl Component for SquareIt {
    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        loop {
            let x = self.input.get().await; // waits for the test
to send x
            ctx.info(&format!("{x}² = {}", x * x));
            self.output.put(x * x).await; // waits for TimesTwo to
take it
        }
    }
}
```

```
// Chapter 31, Figure 11: The second stage doubles what the first
produced
#[derive(Component, Default)]
struct TimesTwo {
    #[port(get)]
    input: GetPort<u32>,
    #[port(put)]
    output: PutPort<u32>,
}

impl Component for TimesTwo {
    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        loop {
            let sq = self.input.get().await; // waits for SquareIt
            ctx.info(&format!("2 × {sq} = {}", 2 * sq));
            self.output.put(2 * sq).await; // waits for the test to
read it
        }
    }
}
```

Note the `loop` with no exit and no objection. The workers are *responders*: they serve forever, and they get to, because ending the run phase is not their job — it belongs to the component doing the work that matters. That is what makes

the objection load-bearing rather than ceremonial: when the test's guard drops, the phase ends, and the workers' unfinished loops are dropped with it.

```
// Chapter 31, Figure 12: The test drives the pipeline and checks
the answer
#[rustdv::test]
#[derive(Component, Default)]
struct MathTest {
    #[component]
    square_it: RustdvComp,
    #[component]
    times_two: RustdvComp,
    #[component]
    x_fifo: TlmFifo<u32>,
    #[component]
    sq_fifo: TlmFifo<u32>,
    #[component]
    y_fifo: TlmFifo<u32>,
    #[port(put)]
    x_out: PutPort<u32>,
    #[port(get)]
    y_in: GetPort<u32>,
}

impl Component for MathTest {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        self.square_it = SquareIt::new_comp();
        self.times_two = TimesTwo::new_comp();
        self.x_fifo = TlmFifo::new(1);
        self.sq_fifo = TlmFifo::new(1);
        self.y_fifo = TlmFifo::new(1);
    }

    fn connect(&mut self, _ctx: &mut RustdvCtx) {
        // test -> square_it
        self.x_fifo.put_export().connect(self, MathTest::X_OUT);
        self.x_fifo.get_export().connect(&self.square_it,
SquareIt::INPUT);
        // square_it -> times_two
        self.sq_fifo.put_export().connect(&self.square_it,
SquareIt::OUTPUT);
        self.sq_fifo.get_export().connect(&self.times_two,
TimesTwo::INPUT);
        // times_two -> test
        self.y_fifo.put_export().connect(&self.times_two,
TimesTwo::OUTPUT);
        self.y_fifo.get_export().connect(self, MathTest::Y_IN);
    }

    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let _obj = ctx.raise_objection("driving the pipeline");
        for x in 1..=4u32 {
            self.x_out.put(x).await; // into the pipeline
            let y = self.y_in.get().await; // ...and back out
            let expected = 2 * x * x;
            if y == expected {
```

```
                ctx.info(&format!("PASSED: x={x}, y={y}"));
            } else {
                return Err(TestError::from(format!(
                    "FAILED: x={x}, y={y}, expected {expected}"
                )));
            }
        }
        Ok(())
    }
}
```

Three things this one listing does that nothing before it could.

First, **the parent holds its own ports** — `x_out` and `y_in` are fields of the test — and connects them with `connect(self, MathTest::X_OUT)`: the same call shape that wires a child, wiring the caller. The uniformity rule, made visible.

Second, **the round trip is closed**: put x, await y. Every prior example streamed data one way; this is the first request/response loop through the component tree, and it is the DUT-free rehearsal for testbench 7.0, where a test's `run` starts a sequence while a driver waits for items.

Third, **every run phase must make progress together**. The test blocks waiting for its answer; `SquareIt` blocks waiting for x; `TimesTwo` blocks waiting for x^2. Nothing completes unless everything runs at once — and the parent is one of the things that must run. A phaser that finished the children before starting the parent could not execute this test at all: the children would wait forever for an x the parent never got to send. This test is *why* rustdv's run phases are concurrent; the shape drove the design.

```
# Figure 13: The pipeline checks itself

      2.00ns INFO     running MathTest (3/4)  [ch31-component-
communications/src/ch31_component_communications.rs:345]
      2.00ns INFO     [MathTest.square_it]: 1² = 1
      2.00ns INFO     [MathTest.times_two]: 2 × 1 = 2
      2.00ns INFO     [MathTest]: PASSED: x=1, y=2
      2.00ns INFO     [MathTest.square_it]: 2² = 4
      2.00ns INFO     [MathTest.times_two]: 2 × 4 = 8
      2.00ns INFO     [MathTest]: PASSED: x=2, y=8
      2.00ns INFO     [MathTest.square_it]: 3² = 9
      2.00ns INFO     [MathTest.times_two]: 2 × 9 = 18
      2.00ns INFO     [MathTest]: PASSED: x=3, y=18
      2.00ns INFO     [MathTest.square_it]: 4² = 16
      2.00ns INFO     [MathTest.times_two]: 2 × 16 = 32
      2.00ns INFO     [MathTest]: PASSED: x=4, y=32
      2.00ns INFO     MathTest PASSED
```

A broken pipeline fails loudly rather than passing quietly — the comparison is in the test's own `run`, which is what "the parent is a stage" buys.

What declared ports make checkable

Declaring ports with `#[port(...)]` gave the framework a complete inventory of what must be wired. Here is what it does with it.

```
// Chapter 31, Figure 14: A port left unconnected is an elaboration
error
#[rustdv::test(expect_error = "tlm_unconnected_port")]
#[derive(Component, Default)]
struct UnconnectedTest {
    #[component]
    producer: RustdvComp,
    #[component]
    fifo: TlmFifo<u32>,
}

impl Component for UnconnectedTest {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        self.producer = Producer::new_comp();
        self.fifo = TlmFifo::new(1);
    }
    // No `connect` — Producer::put_port is declared but never
wired.
}
```

```
# Figure 15: Every missing connection, named, before anything runs

these TLM ports were declared but never connected:
  UnconnectedTest.producer.put_port (put)
```

At the end of elaboration, rustdv walks the tree and reports **every** declared-but-unconnected port at once, by path, before any run phase starts. This is a timing difference worth being precise about. It is not a compile error — the wiring is decided by `connect` code, so elaboration is exactly as early as the check can run. But it is earlier than pyuvm, which discovers a missing connection lazily, at the first `put` that has nowhere to go — one at a time, mid-simulation. An elaboration sweep that names all of them before the DUT ticks is one of the places rustdv hands you something real, and it costs nothing at the call site. (The `expect_error` in the test's attribute is there only because this listing *wants* the failure, to prove it happens.)

One last thing to know about `TlmFifo`, held for later: like `uvm_tlm_fifo`, it also carries a pair of publisher ports, reached with `TlmFifo::put_ap()` and `TlmFifo::get_ap()`, which announce each item the FIFO accepts and releases. They belong to the analysis machinery, and Chapter 32 explains them.

Summary

TLM-1 point-to-point, ported whole: a port is what a component calls, an export is what serves it, and the FIFO between them owns the queue and the decoupling — two components share a FIFO and never learn each other's names. Blocking `put` / `get` / `peek` suspend in simulated time; `try_put` gives a refused transaction *back* (`Result<(), T>`, because taking by value means a bare "no" would lose the packet) and `try_get` answers `Option<T>` for the mirror-image reason. Connection goes through typed port names generated by `#[port(...)]` — reachable through factory-erased children and through `self` alike — and the declared-port inventory buys an elaboration sweep that names every unwired port before anything runs. The pipeline stands as the chapter's thesis in working form: parents run concurrently with their children, responders serve forever, and the objection decides when everyone is done.

Ports, exports, and FIFOs move data from one component to *one* other component. A monitor has the opposite problem — one observation, delivered to a scoreboard, a coverage collector, and anyone else who cares, none of whom may slow it down. That is broadcasting, and it is Chapter 32.

Chapter 32: Analysis Ports

Chapter 31 moved data from one component to one other component, with back-pressure: a full FIFO makes the producer wait, because a command that is not yet driven must not be dropped. A monitor lives in the opposite world. It observes traffic that has already happened, and it must tell *everyone who cares* — the scoreboard, the coverage collector, a logger — without being slowed by any of them, and without caring whether anyone is listening at all. That is broadcasting, the other TLM shape, and the UVM gives it its own machinery.

> **In the UVM…** a monitor holds a `uvm_analysis_port` and calls `ap.write(txn)`. Every connected subscriber's `write()` method runs — a subscriber extends `uvm_subscriber` and overrides `write()` — and the call returns immediately, in zero simulation time, no matter how many subscribers are connected, including none. Scoreboards typically route each incoming stream into a `uvm_tlm_analysis_fifo`, and a component that needed two streams reached for the `uvm_analysis_imp_decl` macros.

Analysis is a different mechanism from put/get, not a mode of it: one-to-many, non-blocking, no return value, no back-pressure. Be reassured up front: **rustdv's analysis layer is a copy of the UVM's.** The subscribers implement `write()`, the publisher calls it, and delivery obeys the UVM's strictest rule — `write` takes no simulation time *(SV: `write` is a `function`, never a `task`; pyuvm: a plain `def`)*. What you know carries over. This chapter builds the layer up in order — the idea, the two ports, the `write` contract, one shared-state helper — and then runs it, because the parts arrive together in every listing and are easier to read once each has been met alone.

One publisher, many subscribers

Start with the shape, because everything else in the chapter is machinery for it. One component — the **publisher** — has something to announce. In a real

testbench it is a monitor: it has just decoded a bus transaction, and its job is to say so. Several others — the **subscribers** — want to hear it: a scoreboard to compare it against a prediction, a coverage collector to bin it, perhaps a logger to file it. Each does something *different* with the *same* item.

Two properties define the relationship. The publisher does not know its subscribers — not how many there are, not what they do, not whether there are any at all. It announces and moves on. And no subscriber can slow the publisher down or dictate to another: each is handed the item, does its own thing, and has no channel back. That is why analysis has no back-pressure and no return value; a broadcast is not a conversation.

Everything below is the rustdv spelling of that shape, and the spelling is the UVM's: a publisher port, subscriber ports, and a `write()` that fans out in zero time.

The two ports

The publisher declares a `PublishPort<T>`; each subscriber declares a `SubscribePort<T>`. Both are `#[port(...)]` fields, exactly as in Chapter 31, and the attribute does the same two jobs: it generates the typed name constant the wiring uses, and it enrolls the port in the elaboration report.

```
#[port(publish)]
ap: PublishPort<CmdTuple>,        // the publisher announces here

#[port(subscribe)]
input: SubscribePort<CmdTuple>,   // a subscriber listens here
```

The publisher's side is all there is on the publisher: it calls `self.ap.write(&item)` and moves on. Everything else belongs to the subscriber, and a rustdv subscriber is always the same three pieces. The next three sections introduce them one at a time.

`Subscriber`: what an arriving item does

The first piece is the subscriber's data: a small struct holding whatever this subscriber keeps. For one subscriber that might be a count; for another, a `Vec` of the items themselves; for Chapter 34's scoreboard, the two lists it will compare. This chapter calls it the **state struct**. It is a plain struct, and the subscriber's real work happens in it.

The struct's `write()` lives there too. `Subscriber<T>` is a trait with a single method, `fn write(&mut self, item: &T)` — the port of `uvm_subscriber`'s `write()`, the same name doing the same job: it says what an arriving item does. You implement `Subscriber` **on the state struct** — the count's `write` increments the count, the `Vec`'s `write` pushes the item. That struct *is* the subscriber, and this is the one place the vocabulary shifts under a UVM engineer: `uvm_subscriber` is a component, while rustdv's `Subscriber` is plain data that a component hosts — the same lesson this chapter keeps teaching about where storage lives.

The instinct is to put `write()` on the hosting component itself, because that is where the UVM puts it. In rustdv it cannot go there, and the reason is ownership. Think about the moment of delivery: the *publisher* is in the middle of its `run` phase, and the item has to land in a *sibling* component's data, right now, in zero time. Chapter 24's tree gives nobody `&mut` access to a sibling — the hosting component is simply unreachable at the moment the item arrives. The state struct is the answer: it lives *outside* the component, so it can be reached at delivery time. What makes that sharing safe is the second piece.

A digression: `RustdvShared`

`RustdvShared<T>` is how the subscriber component and its port both hold the same state struct. It is Chapter 13's `Rc<RefCell<T>>` wrapped in a framework type: `clone()` produces a second handle to the *same* data — not a copy of it — and `get()` / `get_mut()` borrow the data to read or modify, checked at run time as `RefCell` always is.

The use never varies. The component declares its state as a field — `tally: RustdvShared<ItemCount>` — and so holds one handle. During setup it clones a second handle and gives the clone to its port. After that, the two ends never touch each other: the port pours arriving items into the state through its handle, in zero time, without going anywhere near the component; and the component reads through its own handle whenever it likes — usually in `check` or `report`, once the traffic is over. One habit keeps it friction-free: take `get()`'s borrow for a line at a time, never across an `await`.

About the name: it wears the `Rustdv` prefix for the same reason `RustdvComp` and `RustdvCtx` do — it is the framework's type, not the language's. A reader who goes looking for `Shared<T>` in the standard library will find nothing; the name says where to look instead.

`subscribe` and `connect`

The third piece is the setup, and it is two calls made by two different components:

```
// the subscriber, in its own build phase:
let my_subscriber = self.tally.clone();   // a second handle to the
state struct
self.input.subscribe(my_subscriber);       // "pour arriving items
into this"

// the parent, in its connect phase:
bus.sub_export().connect(&self.counter, Counter::INPUT);
```

`subscribe` supplies the receiver. The hosting component hands its port the cloned handle, so the port knows what to pour arriving items into. Only that component can make the call — nobody else holds a handle to its state — which is why it happens in its own `build`.

`connect` chooses the stream. It is the parent wiring topology, with exactly the move used for every connection in Chapter 31: it attaches the hosted port to one particular broadcast. The parent decides who hears what; it neither knows nor cares what any subscriber does with an item.

So the two calls answer two different questions. `connect`: *which items come here?* `subscribe`: *what happens when they do?* Keep the split straight by who makes each call: the parent `connect`s, the component `subscribe`s. Miss one and the failures differ: a port that was never `connect`ed is legal and silent — the elaboration report lists it as unbound and the subscriber hears nothing — while `connect`ing a port whose component never called `subscribe` fails loudly, naming the port and the `build` phase that owes the call.

Every subscriber from here to the end of the book is these three pieces — a state struct with a `write`, a `RustdvShared` holding it, and the `subscribe`/`connect` pair — so the first listing repays a slow read.

A counter and a collector

The chapter's example is deliberately not a TinyALU testbench. It is the publisher/subscriber shape with nothing else in the room: a number generator that publishes `0, 1, 2`, and two components that hear the same three numbers and do different things with them — a **counter** that keeps a tally, and a **collector** that keeps the values. Chapter 33 will put monitors and scoreboards in these roles; today the data is plain `u32`s so the machinery has your whole attention.

Here is the counter, all three pieces of the pattern in one place:

```rust
// Chapter 32, Figure 1: A subscriber counts what it sees
#[derive(Default)]
struct ItemCount {
    count: u32,
}

impl Subscriber<u32> for ItemCount {
    fn write(&mut self, _item: &u32) {
        self.count += 1;
    }
}

#[derive(Component, Default)]
struct Counter {
    #[port(subscribe)]
    input: SubscribePort<u32>,
    tally: RustdvShared<ItemCount>,
}

impl Component for Counter {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        let my_subscriber = self.tally.clone();
        self.input.subscribe(my_subscriber);
    }

    fn report(&mut self, ctx: &mut RustdvCtx) {
        let tally = self.tally.get();
        ctx.info(&format!("counted {} items", tally.count));
    }
}
```

All three pieces are here. `ItemCount` is the state struct and the subscriber — `Subscriber` is implemented there, and its `write` bumps the count, instantly, nothing awaited. `Counter` is the component hosting it: it declares the `SubscribePort`, keeps one `RustdvShared` handle in `tally`, and in `build` hands a clone of that handle to `subscribe`. In `report`, it reads the same state back through `get()`. The component never sees an item arrive; arrival goes straight into `ItemCount`, and the component and the port simply share it.

The collector is the same pattern with different state — a `Vec` where the counter had a number:

```
// Chapter 32, Figure 2: A second subscriber on the same stream
#[derive(Default)]
struct SeenList {
    items: Vec<u32>,
}

impl Subscriber<u32> for SeenList {
    fn write(&mut self, item: &u32) {
        self.items.push(*item);
    }
}

#[derive(Component, Default)]
struct Collector {
    #[port(subscribe)]
    input: SubscribePort<u32>,
    seen: RustdvShared<SeenList>,
}

impl Component for Collector {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        let my_subscriber = self.seen.clone();
        self.input.subscribe(my_subscriber);
    }

    fn report(&mut self, ctx: &mut RustdvCtx) {
        let seen = self.seen.get();
        ctx.info(&format!("collected {:?}", seen.items));
    }
}
```

A tally in one, a `Vec` in the other. Keep that difference in mind; it is about to become the chapter's thesis.

The source and the broadcast

```
// Chapter 32, Figure 3: A source holds an analysis port and writes
to it
#[derive(Component, Default)]
struct NumberGen {
    #[port(publish)]
    ap: PublishPort<u32>,
}

impl Component for NumberGen {
    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let _obj = ctx.raise_objection("generating");
        for n in 0..3 {
            self.ap.write(&n); // broadcast; non-blocking
            ctx.info(&format!("wrote {n}"));
        }
        Ok(())
    }
}
```

`write` takes a reference, returns immediately, and is not `async` — there is nothing to await, because delivery is synchronous and takes zero simulation time. The publisher does not block, does not learn how many subscribers heard it, and does not care.

What connects a publisher to its subscribers is an `AnalysisBus` — the hub that brokers the broadcast:

```
// Chapter 32, Figure 4: One publisher, two subscribers, one hub
#[rustdv::test]
#[derive(Component, Default)]
struct BroadcastTest {
    #[component]
    source: RustdvComp,
    #[component]
    counter: RustdvComp,
    #[component]
    collector: RustdvComp,
    #[component]
    bus: AnalysisBus<u32>,
}

impl Component for BroadcastTest {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        self.source = NumberGen::new_comp();
        self.counter = Counter::new_comp();
        self.collector = Collector::new_comp();
        self.bus = AnalysisBus::new();
    }

    fn connect(&mut self, _ctx: &mut RustdvCtx) {
        self.bus.pub_export().connect(&self.source, NumberGen::AP);
        self.bus.sub_export().connect(&self.counter,
Counter::INPUT);
        self.bus.sub_export().connect(&self.collector,
Collector::INPUT);
    }
}
```

The wiring reads exactly like Chapter 31's: a concrete `#[component]` child, a named export, `connect(component, PORT_NAME)`. The publisher's port goes to `pub_export()`; every subscriber goes to the *same* `sub_export()`, and connecting several is what makes the write fan out. One connection idiom for both TLM shapes is a deliberate rustdv choice — the UVM broadcasts straight from port to subscribers with no intermediary, but with both endpoints factory-erased, neither side could drive the call, and one idiom for the reader to learn beats two.

```
# Figure 5: One write, every subscriber hears it — all in zero time

      0.00ns INFO     running BroadcastTest (1/4)  [ch32-analysis-
ports/src/ch32_analysis_ports.rs:193]
      0.00ns INFO     [BroadcastTest.source]: wrote 0
      0.00ns INFO     [BroadcastTest.source]: wrote 1
      0.00ns INFO     [BroadcastTest.source]: wrote 2
      0.00ns INFO     [BroadcastTest.counter]: counted 3 items
      0.00ns INFO     [BroadcastTest.collector]: collected [0, 1,
2]
      0.00ns INFO     BroadcastTest PASSED
```

Every line is at `0.00ns`. Three writes, both subscribers fully served, and the simulation clock never moved — that is the contract `write` keeps. And note where the results came from when `report` ran: the counter read its `ItemCount` and the collector its `SeenList`, each through the same `RustdvShared` handle whose clone its port had been delivering into all along. The shared state is the join between the zero-time world of `write` and the component that eventually wants the answer.

The bus stores nothing

Despite living in a `#[component]` slot, **an `AnalysisBus` is not a FIFO and stores no items.** It is a subscriber list and nothing more: `write` calls every enrolled subscriber and returns, connecting function calls rather than holding data, and a datum broadcast to nobody is *gone*.

```rust
// Chapter 32, Figure 6: A hub with no subscribers is legal
#[rustdv::test]
#[derive(Component, Default)]
struct NoSubscribersTest {
    #[component]
    source: RustdvComp,
    #[component]
    bus: AnalysisBus<u32>,
}

impl Component for NoSubscribersTest {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        self.source = NumberGen::new_comp();
        self.bus = AnalysisBus::new();
    }

    fn connect(&mut self, _ctx: &mut RustdvCtx) {
        self.bus.pub_export().connect(&self.source, NumberGen::AP);
        // no sub_export() connection — legal for analysis
    }
}
```

```
# Figure 7: Broadcasting into the void

      0.00ns INFO     running NoSubscribersTest (2/4)  [ch32-
analysis-ports/src/ch32_analysis_ports.rs:231]
      0.00ns INFO     [NoSubscribersTest.source]: wrote 0
      0.00ns INFO     [NoSubscribersTest.source]: wrote 1
      0.00ns INFO     [NoSubscribersTest.source]: wrote 2
      0.00ns INFO     NoSubscribersTest PASSED
```

Where Chapter 31's unconnected put port failed elaboration, an analysis `sub_export()` has minimum cardinality zero: broadcasting to nobody is a valid state — a monitor in a block-level environment reused at chip level may well have no one listening — so this elaborates and runs clean. And the storing-nothing rule is the mechanism, not a limitation of it: a broadcast hub that stored what nobody wanted would grow forever, with the monitor paying for listeners it does not have.

So "where does the traffic go?" has a simple answer: **wherever the subscriber decides to put it.** A tally (figure 1), a `Vec` (figure 2), a comparison against a prediction (Chapter 34's scoreboard) — the subscriber owns its storage, held in its `RustdvShared` state and shaped to its job. If you find yourself looking for the analysis FIFO, this paragraph is the answer: there isn't one, and nothing is missing.

It is worth being precise about what that replaces, because the UVM's scoreboards buffer for a reason that is real *there*. A SystemVerilog class gets exactly one `write()` method. A scoreboard watching two streams — commands and results — therefore needs the `uvm_analysis_imp_decl` macros to mint two differently-named writes, and routing each stream into its own `uvm_tlm_analysis_fifo` is the standard way around the whole problem; pyuvm, with one `write` per class, routes into FIFOs for the same reason. A rustdv component declares two `SubscribePort`s and hosts two `Subscriber` impls, one per stream — you will see it done in Chapter 34's scoreboard — so the workaround has nothing to work around, and the buffer that lived in every UVM scoreboard is simply absent. (Hence the name `AnalysisBus`: the type is the broadcast hub, a thing the UVM has no class for at all — emphatically not an analysis FIFO, which in the UVM names the subscriber-side buffer this design does without.)

When the subscriber needs time

One legitimate reason to buffer remains, and it has nothing to do with `imp_decl`: **`write` cannot take simulation time.** It is synchronous, called from the publisher's `run`, and it is not `async` — no `await` is possible inside it. Bumping a counter fits. Consulting a slow reference model, driving a bus, waiting on the DUT does not. A subscriber whose real work takes time splits the job in two:

```
// Chapter 32, Figure 8: When the subscriber needs *time*
#[derive(Default)]
struct Inbox {
    queue: TlmFifo<u32>,
}

impl Subscriber<u32> for Inbox {
    fn write(&mut self, item: &u32) {
        let _ = self.queue.try_put(*item);
    }
}

#[derive(Component, Default)]
struct SlowChecker {
    #[port(subscribe)]
    input: SubscribePort<u32>,
    inbox: RustdvShared<Inbox>,
}

impl Component for SlowChecker {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        self.inbox = RustdvShared::new(Inbox { queue:
TlmFifo::unbounded() });
        let my_inbox = self.inbox.clone();
        self.input.subscribe(my_inbox);
    }

    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let _obj = ctx.raise_objection("checking");
        let my_queue = self.inbox.get().queue.handle();
        for _ in 0..3 {
            let n = my_queue.get().await;
            Timer::ns(5).await; // the slow work `write` could not
have done
            ctx.info(&format!("checked {n}"));
        }
        Ok(())
    }
}
```

`write` does the one thing it can do instantly — `try_put` into a `TlmFifo` the subscriber owns — and the component's `run`, which may await all it likes, takes it from there. Three details repay attention:

- **The FIFO is connected to no port.** A reader fresh from Chapter 31 will expect every `TlmFifo` to be wired in `connect`; this one is an ordinary handoff *inside* one component, between a synchronous method and an asynchronous one. It is not part of the testbench topology.

- **The inbox must be unbounded.** `write` has no way to wait for space, and analysis has no back-pressure to push back with — so a *bounded* inbox here would be a bug that could only drop items. `TlmFifo::unbounded()` is the honest declaration of what analysis traffic is.
- **The handle is taken once, before the loop.** Holding the shared borrow (`self.inbox.get()`) across an `await` would keep the state locked exactly when `write` needs it; taking a queue handle first keeps the two halves out of each other's way.

```
// Chapter 32, Figure 9: The publisher does not wait for the slow
subscriber
#[rustdv::test]
#[derive(Component, Default)]
struct SlowSubscriberTest {
    #[component]
    source: RustdvComp,
    #[component]
    checker: RustdvComp,
    #[component]
    bus: AnalysisBus<u32>,
}

impl Component for SlowSubscriberTest {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        self.source = NumberGen::new_comp();
        self.checker = SlowChecker::new_comp();
        self.bus = AnalysisBus::new();
    }

    fn connect(&mut self, _ctx: &mut RustdvCtx) {
        self.bus.pub_export().connect(&self.source, NumberGen::AP);
        self.bus.sub_export().connect(&self.checker,
SlowChecker::INPUT);
    }
}
```

Nothing in the wiring says this subscriber buffers — the same `sub_export()` as any other. That is the checker's own business, which is the point.

```
# Figure 10: Writes at 0ns; checks at 5, 10, 15

      0.00ns INFO     running SlowSubscriberTest (3/4)  [ch32-
analysis-ports/src/ch32_analysis_ports.rs:325]
      0.00ns INFO     [SlowSubscriberTest.source]: wrote 0
      0.00ns INFO     [SlowSubscriberTest.source]: wrote 1
      0.00ns INFO     [SlowSubscriberTest.source]: wrote 2
      5.00ns INFO     [SlowSubscriberTest.checker]: checked 0
     10.00ns INFO     [SlowSubscriberTest.checker]: checked 1
     15.00ns INFO     [SlowSubscriberTest.checker]: checked 2
     15.00ns INFO     SlowSubscriberTest PASSED
```

The transcript is the argument. All three writes land at `0.00ns` — the publisher is never held up by what a subscriber does with an item — and the checker's results come out at 5, 10, and 15ns as it works through its own queue in its own time.

The FIFO's built-in taps

One piece of analysis machinery was left unexplained in Chapter 31, because it could not be explained before subscribers were: every `TlmFifo` carries a pair of publisher ports of its own. `put_ap()` announces each item the FIFO accepts; `get_ap()` announces each item it releases. They are the port of `uvm_tlm_fifo`'s built-in analysis ports — the same two names there — and they exist for the same reason: the components on a FIFO's data path are not the only ones with an interest in its traffic. A scoreboard may want to see every command a driver will eventually consume; a coverage collector may want to bin items as they pass through. The taps let them watch without joining the queue.

There is nothing new to learn to use one. A watcher on a tap is the same shape as figure 1's counter — a plain struct implementing `Subscriber`, a `SubscribePort`, and a `subscribe` call in the build phase:

```
// Chapter 32, Figure 11: A watcher on a FIFO's tap is an ordinary
subscriber
#[derive(Default)]
struct TapLog {
    items: Vec<u32>,
}

impl Subscriber<u32> for TapLog {
    fn write(&mut self, item: &u32) {
        self.items.push(*item);
    }
}

#[derive(Component, Default)]
struct TapWatcher {
    #[port(subscribe)]
    input: SubscribePort<u32>,
    seen: RustdvShared<TapLog>,
}

impl Component for TapWatcher {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        let my_subscriber = self.seen.clone();
        self.input.subscribe(my_subscriber);
    }

    fn report(&mut self, ctx: &mut RustdvCtx) {
        let seen = self.seen.get();
        ctx.info(&format!("tap saw {:?}", seen.items));
    }
}
```

To give the tap something to watch, the test reuses Chapter 31's `Producer` and `Consumer` verbatim — the producer that blocks on a full FIFO, the consumer that peeks and then gets. They are not reprinted here; the data path is Chapter 31's, unchanged. What is new is one line of wiring:

```
// Chapter 32, Figure 12: A tap is wired like any other
subscription
#[rustdv::test]
#[derive(Component, Default)]
struct FifoTapTest {
    #[component]
    producer: RustdvComp,
    #[component]
    consumer: RustdvComp,
    #[component]
    watcher: RustdvComp,
    #[component]
    fifo: TlmFifo<u32>,
}

impl Component for FifoTapTest {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        self.producer = Producer::new_comp();
        self.consumer = Consumer::new_comp();
        self.watcher = TapWatcher::new_comp();
        self.fifo = TlmFifo::new(1);
    }

    fn connect(&mut self, _ctx: &mut RustdvCtx) {
        // the data path: producer -> queue -> consumer
        self.fifo.put_export().connect(&self.producer,
Producer::PUT_PORT);
        self.fifo.peek_export().connect(&self.consumer,
Consumer::PEEK_PORT);
        self.fifo.get_export().connect(&self.consumer,
Consumer::GET_PORT);
        // the observation tap: the watcher sees every item put,
and takes none
        self.fifo.put_ap().connect(&self.watcher,
TapWatcher::INPUT);
    }
}
```

Note what is and is not mixed here. The FIFO's **data path** is still a queue: one consumer takes each item, and the producer blocks when it is full — this one holds a single item, so the transcript below alternates put and got. The taps are **observation** running alongside: every subscriber sees every item, nothing is consumed, and nobody is delayed. Two different jobs in one component, exactly as the UVM has it. And there is no `AnalysisBus` in the wiring, because the FIFO already is the hub for its own taps — `put_ap()` takes the watcher's port directly, in the same export-owner-name idiom as every other connection on the page.

```
# Figure 13: Every item put, observed and not consumed

     15.00ns INFO     running FifoTapTest (4/4)  [ch32-analysis-
ports/src/ch32_analysis_ports.rs:444]
     15.00ns INFO     [FifoTapTest.producer]: put 0
     15.00ns INFO     [FifoTapTest.consumer]: got 0
     15.00ns INFO     [FifoTapTest.producer]: put 1
     15.00ns INFO     [FifoTapTest.consumer]: got 1
     15.00ns INFO     [FifoTapTest.producer]: put 2
     15.00ns INFO     [FifoTapTest.consumer]: got 2
     15.00ns INFO     [FifoTapTest.watcher]: tap saw [0, 1, 2]
     15.00ns INFO     FifoTapTest PASSED
```

The consumer got each item exactly once — the queue's contract, intact. The watcher's report says `tap saw [0, 1, 2]`: every item put, observed on the way in, and none of them taken. Had the test connected `get_ap()` instead, the log would read the same for this traffic — items released rather than accepted — and a component with an interest in both edges can subscribe to both.

Summary

Analysis is one publisher and many subscribers: the publisher `write`s, every subscriber's `write()` runs, delivery is synchronous and free, and zero listeners is legal — the UVM's analysis layer, carried over whole, including the rule that `write` takes no time. A subscriber is a plain struct implementing `Subscriber` on the state its `write` updates — `uvm_subscriber` is a component; rustdv's subscriber is data a component hosts — shared between component and port with a `RustdvShared` handle and attached twice: `subscribe` in the component's own `build` says what an arriving item does, `connect` in the parent's `connect` phase says whose traffic it hears. Two streams means two ports and two `Subscriber` impls, no macros. The `AnalysisBus` brokers the fan-out with the same connect idiom as every other wiring in the book, and it stores nothing: the subscriber owns the storage, shaped to its job, and the only reason to make that storage a queue is time — `write` cannot await, so a slow subscriber front-ends its `run` with an unbounded inbox and lets the transcript show writes at zero and checks at leisure. And every `TlmFifo` publishes on two taps of its own, `put_ap()` and `get_ap()` — observation running alongside a

data path that still blocks when full and still hands each item to exactly one consumer.

Testbench 6.0 now has everything it needs: components that talk point-to-point, monitors that broadcast, and a scoreboard that subscribes to two streams at once. Chapter 33 builds those components; Chapter 34 wires them to the TinyALU.

Chapter 33: Components in Testbench 6.0

Configuration, the factory, logging, ports, broadcasting — the toolbox is full, and testbench 6.0 spends it. The 6.0 principle, quoted from the Python book because it cannot be improved: each component "either creates data and writes it to a port or gets data from a port and processes it." One job each. This chapter refactors the components to that standard — and only defines them. They connect to FIFOs and buses, not to each other, so a chapter of definitions has nothing it can run; Chapter 34 wires them up and does the running for both.

In the UVM... we split the testbench into a `BaseTester` that put command tuples into a `uvm_put_port`, a `Driver` that pulled from a `uvm_get_port` and drove the BFM, monitors publishing on analysis ports, a `Coverage` subscriber, and a `Scoreboard` buffering two `uvm_tlm_analysis_fifo`s for the check phase. The Python version had a flourish: one `Monitor` class taking a *method name* string, with `getattr` fetching `get_cmd` or `get_result` at runtime.

Stimulus: the Tester and the Driver

```
// Chapter 33, Figure 1: The Tester puts commands into a FIFO
#[derive(Component, Default)]
struct Tester {
    #[port(put)]
    cmd_port: PutPort<Command>,
    rng: Option<Rng>,
}

impl Component for Tester {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        self.rng = Some(ctx.rng());
    }

    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let _obj = ctx.raise_objection("stimulus");
        let rng = self.rng.as_mut().expect("build ran");
        for op in Ops::ALL {
            self.cmd_port.put((rng.u8(), rng.u8(), op)).await;
        }
        // `put` returns as soon as the FIFO takes the command, not
when the
        // DUT has answered it — so dropping the objection here
would end the
        // phase with commands still in the pipeline and results in
flight, and
        // the scoreboard would silently check fewer results than
it saw
        // commands. Hold the objection for a flush, as the Python
testbench
        // does. It waits ten clocks; this waits twenty, because
the multiply
        // is the last operation and takes the longest to come
back.
        let bfm: Rc<TinyAluBfm> = ConfigDb::get(Some(ctx), "",
"BFM")?;
        for _ in 0..20 {
            bfm.clk().falling_edge().await;
        }
        Ok(())
    }
}
```

The Tester creates data and writes it to a port — its whole job, per the principle. `Command` is a type alias for the `(u8, u8, Ops)` tuple, still; transactions get their upgrade in Chapter 35. Two things to note. The Tester never touches the BFM to *drive* — it only borrows the clock for the flush at the end, and the comment

above that loop is the most important one in the chapter: `put` returning means *accepted*, not *answered*, and an objection dropped too early does not fail the testbench — it silently checks less. Chapter 34 returns to this when the objection becomes the thing that ends the whole run.

```
// Chapter 33, Figure 2: The Driver gets commands and drives the
BFM
#[derive(Component, Default)]
struct Driver {
    #[port(get)]
    cmd_port: GetPort<Command>,
}

impl Component for Driver {
    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let bfm: Rc<TinyAluBfm> = ConfigDb::get(Some(ctx), "",
"BFM")?;
        bfm.reset().await;
        loop {
            let (aa, bb, op) = self.cmd_port.get().await; // blocks
until a command
            bfm.send_op(aa, bb, op).await;
        }
    }
}
```

The Driver is the mirror image: gets data from a port, processes it. It is a responder in Chapter 31's sense — an infinite loop, no objection, blocking on an empty FIFO until the Tester supplies work, running for exactly as long as anyone still objects. Neither component knows the other exists; both know only their ends of a FIFO that Chapter 34 will put between them.

Observation: two monitors

```
// Chapter 33, Figure 3: The command monitor watches the bus and
broadcasts
#[derive(Component, Default)]
struct CmdMonitor {
    #[port(publish)]
    ap: PublishPort<CmdTuple>,
}

impl Component for CmdMonitor {
    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let bfm: Rc<TinyAluBfm> = ConfigDb::get(Some(ctx), "",
"BFM")?;
        loop {
            let cmd = bfm.get_cmd().await;
            self.ap.write(&cmd);
        }
    }
}
```

```
// Chapter 33, Figure 4: The result monitor broadcasts results
#[derive(Component, Default)]
struct ResultMonitor {
    #[port(publish)]
    ap: PublishPort<u64>,
}

impl Component for ResultMonitor {
    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let bfm: Rc<TinyAluBfm> = ConfigDb::get(Some(ctx), "",
"BFM")?;
        loop {
            let result = bfm.get_result().await;
            self.ap.write(&result);
        }
    }
}
```

Four lines of body each: watch the BFM's queue, `write` what appears, forever. The Python book made one `Monitor` class serve both jobs by passing the BFM method's *name* as a string and fetching it with `getattr` at runtime — a lovely trick in a language with runtime attribute lookup, and not one Rust offers. Two small components, written out, do the same work; the types differ anyway

(`CmdTuple` versus `u64`), so nothing is duplicated but the shape. Neither monitor knows who is listening — that is the point of publishing.

Checking: the Scoreboard, on two streams

```
// Chapter 33, Figure 5: The Scoreboard subscribes to BOTH streams
#[derive(Default)]
struct CmdLog {
    cmds: Vec<CmdTuple>,
}

impl Subscriber<CmdTuple> for CmdLog {
    fn write(&mut self, cmd: &CmdTuple) {
        self.cmds.push(*cmd);
    }
}

#[derive(Default)]
struct ResultLog {
    results: Vec<u64>,
}

impl Subscriber<u64> for ResultLog {
    fn write(&mut self, result: &u64) {
        self.results.push(*result);
    }
}

#[derive(Component, Default)]
struct Scoreboard {
    #[port(subscribe)]
    cmd_in: SubscribePort<CmdTuple>,
    #[port(subscribe)]
    result_in: SubscribePort<u64>,
    cmd_log: RustdvShared<CmdLog>,
    result_log: RustdvShared<ResultLog>,
    cvg: HashSet<Ops>,
}

impl Component for Scoreboard {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        let my_cmds = self.cmd_log.clone();
        self.cmd_in.subscribe(my_cmds);

        let my_results = self.result_log.clone();
        self.result_in.subscribe(my_results);
    }

    fn check(&mut self, ctx: &mut RustdvCtx, errors: &mut
CheckSink) {
        let cmd_log = self.cmd_log.get();
        let result_log = self.result_log.get();
        for (cmd, result) in
cmd_log.cmds.iter().zip(result_log.results.iter()) {
            let (aa, bb, op_int) = *cmd;
            let op = Ops::from_u64(op_int).expect("legal op");
```

```
            self.cvg.insert(op);
            let actual = *result as u16;
            let prediction = alu_prediction(aa as u8, bb as u8,
op);
            if actual == prediction {
                ctx.info(&format!("PASSED: {aa:02x} {op:?} {bb:02x}
= {actual:04x}"));
            } else {
                errors.error(format!(
                    "FAILED: {aa:02x} {op:?} {bb:02x} =
{actual:04x} - predicted {prediction:04x}"
                ));
            }
        }

        if Ops::ALL.iter().any(|op| !self.cvg.contains(op)) {
            errors.error("Functional coverage error: missed
operations".to_string());
        } else {
            ctx.info("Covered all operations");
        }
    }
}
```

Here is Chapter 32's pattern at full size: **two streams, two `SubscribePort`s, two `Subscriber` impls** — one `write` per subscriber, each stream landing in the `Vec` it chose to keep, no macros minted and no buffer FIFOs routed. Compare the Chapter 25 scoreboard this replaces: gone are the spawned collector tasks and their `Rc<RefCell>` lists — delivery is synchronous now, so the subscribers just push — and gone is any contact with the BFM at all. The scoreboard's inputs are *ports*. It would work unchanged against any DUT whose monitors publish these two types, which is what "single job, standard connections" buys.

The comparison itself is unchanged since 4.0: zip commands against results, predict, compare, tally coverage, and report failures to the `CheckSink` in the `check` phase.

Coverage: a second subscriber

```
// Chapter 33, Figure 6: Coverage subscribes to the command stream
only
#[derive(Default)]
struct OpsSeen {
    ops: HashSet<Ops>,
}

impl Subscriber<CmdTuple> for OpsSeen {
    fn write(&mut self, cmd: &CmdTuple) {
        if let Some(op) = Ops::from_u64(cmd.2) {
            self.ops.insert(op);
        }
    }
}

#[derive(Component, Default)]
struct Coverage {
    #[port(subscribe)]
    cmd_in: SubscribePort<CmdTuple>,
    seen: RustdvShared<OpsSeen>,
}

impl Component for Coverage {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        let my_subscriber = self.seen.clone();
        self.cmd_in.subscribe(my_subscriber);
    }

    fn report(&mut self, ctx: &mut RustdvCtx) {
        let seen = self.seen.get();
        ctx.info(&format!("coverage saw {} of {} ops",
seen.ops.len(), Ops::ALL.len()));
    }
}
```

A second subscriber on the command stream. The monitor does not know Coverage exists; the scoreboard does not either; adding it to the testbench will cost Chapter 34 exactly one `connect` line. That is the decoupling the analysis hub buys, demonstrated by a component whose entire footprint is one line of wiring.

Summary

Six components, one job each, and not a single one holds a reference to another: the Tester puts, the Driver gets and drives, two monitors publish, and the Scoreboard and Coverage subscribe — the scoreboard on two streams with two `Subscriber` impls, the pattern that needs no `imp_decl` machinery and no analysis FIFOs. Every input and output is a declared port; the BFM arrives by name; the flush comment in the Tester is a debt the objection story pays next chapter. Nothing here can run, because nothing here is connected.

Chapter 34 builds the environment that introduces them all to each other — seven connect lines, one idiom — and runs testbench 6.0.

Chapter 34: Connections in Testbench 6.0

Chapter 33 built six components that do not know each other — that was the point of building them that way, and it left the chapter with nothing it could run. This chapter introduces them to each other. One environment builds all six, wires every connection in one `connect` method, and runs the first fully-decoupled TinyALU testbench: version 6.0.

> **In the UVM...** we wired testbench 6.0 in `connect_phase()`: the tester's put port to one side of a `uvm_tlm_fifo`, the driver's get port to the other, and the monitors' analysis ports fanned out to the scoreboard and coverage — a diagram's worth of `connect()` calls.

The diagram is worth having before the calls:

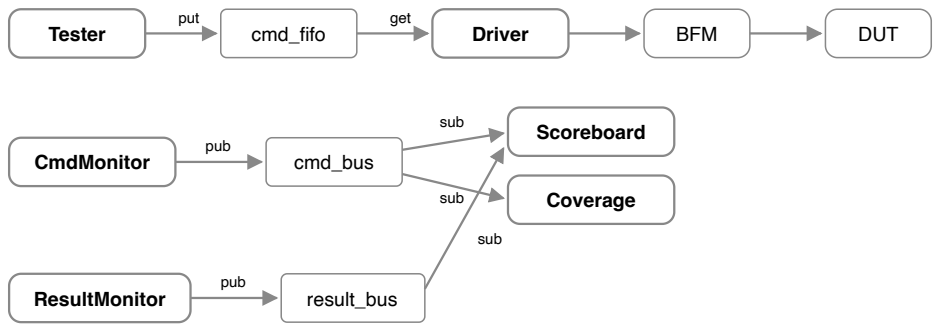


Figure 1: Testbench 6.0. One point-to-point lane for stimulus; two broadcast lanes for observation.

Three lanes, two mechanisms. The stimulus lane is Chapter 31: Tester puts, Driver gets, and the `cmd_fifo` between them means neither knows the other exists. The observation lanes are Chapter 32: each monitor publishes into a bus that stores nothing, and the subscribers own what they keep. Coverage listens to the command bus alongside the scoreboard — and neither the scoreboard nor the monitor knows Coverage is there, which is the decoupling the hub buys.

The environment

```
// Chapter 34, Figure 2: Build the components and the FIFOs;
connect in one place
#[derive(Component, Default)]
struct AluEnv {
    #[component]
    tester: RustdvComp,
    #[component]
    driver: RustdvComp,
    #[component]
    cmd_mon: RustdvComp,
    #[component]
    result_mon: RustdvComp,
    #[component]
    scoreboard: RustdvComp,
    #[component]
    coverage: RustdvComp,
    #[component]
    cmd_fifo: TlmFifo<Command>,
    #[component]
    cmd_bus: AnalysisBus<CmdTuple>, // the command broadcast, two
subscribers
    #[component]
    result_bus: AnalysisBus<u64>, // the result broadcast, one
subscriber
}

impl Component for AluEnv {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        self.tester = Tester::new_comp();
        self.driver = Driver::new_comp();
        self.cmd_mon = CmdMonitor::new_comp();
        self.result_mon = ResultMonitor::new_comp();
        self.scoreboard = Scoreboard::new_comp();
        self.coverage = Coverage::new_comp();
        self.cmd_fifo = TlmFifo::new(1);
        self.cmd_bus = AnalysisBus::new();
        self.result_bus = AnalysisBus::new();
    }

    fn connect(&mut self, _ctx: &mut RustdvCtx) {
        // stimulus: Tester --put--> cmd_fifo --get--> Driver
        self.cmd_fifo.put_export().connect(&self.tester,
Tester::CMD_PORT);
        self.cmd_fifo.get_export().connect(&self.driver,
Driver::CMD_PORT);

        // commands: CmdMonitor publishes; Scoreboard and Coverage
subscribe
        self.cmd_bus.pub_export().connect(&self.cmd_mon,
CmdMonitor::AP);
        self.cmd_bus.sub_export().connect(&self.scoreboard,
```

```
Scoreboard::CMD_IN);
        self.cmd_bus.sub_export().connect(&self.coverage,
Coverage::CMD_IN);

        // results: ResultMonitor publishes; only the Scoreboard
subscribes
        self.result_bus.pub_export().connect(&self.result_mon,
ResultMonitor::AP);
        self.result_bus.sub_export().connect(&self.scoreboard,
Scoreboard::RESULT_IN);
    }

    fn start_of_simulation(&mut self, ctx: &mut RustdvCtx) {
        let bfm: Rc<TinyAluBfm> = ConfigDb::get(Some(ctx), "",
"BFM").expect("the test sets BFM");
        bfm.start_tasks();
    }
}
```

Read the `connect` method as a whole before reading any line of it: seven connections, and **every one is the same shape** — a concrete FIFO, a named export, `connect(component, PORT_NAME)`. Point-to-point traffic through a `TlmFifo`, broadcast through an `AnalysisBus`, and the idiom does not change between them; the only visible difference is that two subscribers connect to the same `sub_export()`. Nothing reaches into an erased child; every endpoint is a `RustdvComp`, and every connection resolves through the trait method that answers the same way for a child slot and for `self`. Chapter 33's six definitions plus these seven lines *are* testbench 6.0 — the figure-1 diagram, transcribed.

Two smaller notes. The `cmd_fifo` has depth 1, so the Tester cannot run ahead of the Driver — the same back-pressure lesson as Chapter 31's first transcript, now doing real work. And `start_of_simulation` starts the BFM's monitoring tasks after the whole tree is built and wired, in the phase whose position in the lifecycle exists for exactly this kind of "everything is ready, nothing has run" work.

The test

```
// Chapter 34, Figure 3: The test is just the env
#[rustdv::test]
#[derive(Component, Default)]
struct AluTest {
    #[component]
    env: RustdvComp,
}

impl Component for AluTest {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        let bfm = TinyAluBfm::new(&ctx.dut()).expect("TinyALU
signals");
        ConfigDb::set(None, "*", "BFM", Rc::new(bfm));
        self.env = AluEnv::new_comp();
    }
}
```

The test constructs the BFM from the DUT handle, files it in the ConfigDb for every component that needs it, and builds the env. It has no `run` at all — for the first time in this book, the test contributes nothing to the run phase, because stimulus is the Tester's job now. Which raises a question the transcript is about to make sharp: if the test doesn't hold the run phase open, who does?

Who ends the run phase

The Tester does — and *when* it does is the subtlest line in the testbench. Chapter 33's Tester puts four commands and then does something that looks like padding:

```
    let bfm: Rc<TinyAluBfm> = ConfigDb::get(Some(ctx), "", "BFM")?;
    for _ in 0..20 {
        bfm.clk().falling_edge().await;
    }
```

`put` returns when the FIFO accepts a command — not when the DUT has answered it. If the Tester dropped its objection right after its last `put`, the run phase would end with commands still in the pipeline and results still in flight, and the scoreboard would check fewer results than it saw commands. Note

carefully what that failure looks like: *nothing*. The zip in the scoreboard's `check` pairs what arrived; it cannot miss what never came. The testbench does not fail — it silently checks less, which is worse. So the Tester holds its objection for a flush: the Python testbench waits ten clocks, and this one waits twenty, because the multiply is the last operation sent and the slowest to come back.

The monitors and the Driver, meanwhile, are infinite loops that never object — responders, in Chapter 31's terms. When the Tester's guard drops, the phase ends around them, and then `extract`, `check`, and `report` walk the tree. That ordering is not free — it is a property the framework has to guarantee. Race the objection against the run phase as a whole and the consensus would take the *components* down with it: `check` would never run, and the test would pass with its scoreboard never executing. So the rule is the one Chapter 24 stated: each component races the objection event individually, the tree outlives the race, and the post-run phases always get their walk. A test that can pass with its checker dead is the UVM's oldest trap, in any language.

```
# Figure 4: Testbench 6.0 running

      0.00ns INFO     rustdv: found 1 test(s), RUSTDV_RANDOM_SEED=1
      0.00ns INFO     running AluTest (1/1)  [ch34-connections-
testbench-6.0/src/ch34_connections_testbench_6_0.rs:346]
    240.00ns INFO     [AluTest.env.scoreboard]: PASSED: c1 Add 67 =
0128
    240.00ns INFO     [AluTest.env.scoreboard]: PASSED: 5e And 0b =
000a
    240.00ns INFO     [AluTest.env.scoreboard]: PASSED: b9 Xor 80 =
0039
    240.00ns INFO     [AluTest.env.scoreboard]: PASSED: a5 Mul 75 =
4b69
    240.00ns INFO     [AluTest.env.scoreboard]: Covered all
operations
    240.00ns INFO     [AluTest.env.coverage]: coverage saw 4 of 4
ops
    240.00ns INFO     AluTest PASSED
**********************************************************************
***********
** TEST                                          STATUS  SIM TIME (ns)
**
**********************************************************************
***********
** AluTest                                       PASS         240.00
**
**********************************************************************
***********
REGRESSION: PASS
```

All the scoreboard lines carry timestamp 240ns — after the flush, in the `check` phase, where Chapter 33 put the comparison. Four commands driven, four results checked against predictions, coverage complete, and two components reporting on the same command stream without either knowing about the other.

One paragraph on the scoreboard, because Chapter 32 promised it here: it subscribes to **two** streams with two `SubscribePort`s and two `Subscriber` impls — `CmdLog` for commands, `ResultLog` for results — and no macros anywhere. This is the multiple-analysis-input problem that SystemVerilog needs the `uvm_analysis_imp_decl` macros for, because a class gets one `write` method; and it works identically when both streams carry the *same* type, which is precisely the case those macros exist to solve. The `uvm_tlm_analysis_fifo`s that would sit inside a UVM scoreboard are absent for Chapter 32's reason: the subscriber owns its storage, and these two own a `Vec` each.

Summary

Testbench 6.0 assembles Chapters 31 through 33 into one machine: a stimulus lane with back-pressure, two broadcast lanes without it, and seven connections in one `connect` method sharing a single idiom. The test shrinks to construction — build the BFM, file it, build the env — and the objection becomes the load-bearing end-of-test mechanism, held by the Tester through a twenty-clock flush because a scoreboard that zips streams cannot complain about results that never arrived. The two-stream scoreboard cashes Chapter 32's promise: one impl per stream, no macros, no buffer FIFOs.

The structure is now complete, and it never changes again — every remaining testbench in this book, including the shipped one, wires this same shape. What changes next is the *data*: the command tuple has been `(u8, u8, Ops)` long enough, and Chapter 35 gives transactions the treatment the UVM gives `uvm_object`.

Chapter 35: Transactions

The 6.0 testbench passes `(u8, u8, Ops)` tuples, and everyone is tired of remembering that `op` is the thing at index 2. The UVM's answer was `uvm_object`: named-field transaction classes with standard copy, compare, and print machinery. This chapter gives the TinyALU its real transactions, by way of the same warm-up the Python book used — a person with an ID, then a student with a list of grades — because the grades list is what makes copying *visible*, and three scalars cannot show it.

> **In the UVM...** we extended `uvm_sequence_item` and got the machinery of `uvm_object`: `clone()` backed by a `do_copy()` that walked the fields — with the "always call `super().do_copy(other)` first" discipline; equality backed by `do_compare()`; printing via `convert2string()` (`__str__()` in pyuvm), overridden by hand for every class; plus the long tail — pack, unpack, record — that the specification demands and most testbenches quietly ignore.

Before the first listing, one adjustment of mental furniture, because it decides how every figure below reads. **A rustdv transaction is not an object. It is a location in memory, referred to by a name.** `AluCommand { a, b, op }` is a layout — three fields side by side — and `cmd` is a binding to that place. Nothing is wrapped around it, nothing points at it, and it has no identity separate from its bytes. That is why there is no base class in this chapter: there is no object for a base class to be part of. And it is why there is no `super()`: no inheritance chain means no chain to copy up. What the base class *gave* you — printing, comparing, copying — arrives instead as traits attached from the outside, which is why they are derives rather than inherited methods. You have known this since Chapter 7; this chapter is the reminder aimed at the UVM habit, which will otherwise keep looking for the object.

Two string forms

```
// Figure 1: A transaction is a plain struct with two string forms

#[derive(Debug)]
struct PersonRecord {
    name: String,
    id_number: u32,
}

// `Display` is `convert2string()` / `__str__()`. It is **not**
derivable —
// you write it, because only you know which fields are worth
reading.
impl fmt::Display for PersonRecord {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "Name: {}, ID: {}", self.name, self.id_number)
    }
}

fn main() {
    let xx = PersonRecord { name: String::from("Joe Shmoe"),
id_number: 37 };

    // `{}` asks for Display: the form you chose.
    println!("Printing the record: {xx}");
    println!("Logging the record:  {}", xx.to_string());

    // `{:?}` asks for Debug: every field, mechanically, for free.
    println!("Debug form:          {xx:?}");
}
```

```
--
Printing the record: Name: Joe Shmoe, ID: 37
Logging the record:  Name: Joe Shmoe, ID: 37
Debug form:          PersonRecord { name: "Joe Shmoe", id_number:
37 }
```

Rust gives a transaction *two* string forms, and they are different jobs. `Debug` (`{:?}`) is the developer field-dump: every field, mechanically formatted, generated free by the derive — the thing you want when debugging the testbench. `Display` (`{}`) is `convert2string()`: the readable form, and it is *not derivable* — you write it, because only the author knows that a command reads best as operand-operator-operand. Reach for `Display` in log messages and

`Debug` in despair, and do not confuse the two: the derive cannot write your `convert2string()` for you, and does not try.

Equality is a decision

```
// Figure 2: You decide what "the same" means

// `#[derive(PartialEq)]` compares **every field**.
#[derive(Debug, PartialEq)]
struct StrictRecord {
    name: String,
    id_number: u32,
}

// But "the same" is a decision, not a fact. Here two records are
the same
// person when the ID matches, whatever the name says.
#[derive(Debug)]
struct PersonRecord {
    name: String,
    id_number: u32,
}

impl PartialEq for PersonRecord {
    fn eq(&self, other: &PersonRecord) -> bool {
        self.id_number == other.id_number
    }
}

// ... (the Display impl from Figure 1, unchanged)

fn main() {
    println!("-- derived: every field must match --");
    let a = StrictRecord { name: String::from("Batman"), id_number:
27 };
    let b = StrictRecord { name: String::from("Bruce Wayne"),
id_number: 27 };
    println!("batman == bruce_wayne? {}", a == b);

    println!("-- written by hand: only the ID counts --");
    let batman = PersonRecord { name: String::from("Batman"),
id_number: 27 };
    let bruce_wayne = PersonRecord { name: String::from("Bruce
Wayne"), id_number: 27 };
    println!("batman == bruce_wayne? {}", batman == bruce_wayne);

    if batman == bruce_wayne {
        println!("Batman is really Bruce Wayne!");
    }
}
```

```
--
-- derived: every field must match --
batman == bruce_wayne? false
-- written by hand: only the ID counts --
batman == bruce_wayne? true
Batman is really Bruce Wayne!
```

`PartialEq` is `do_compare()`, and it lands in the same two flavors the UVM gives it. Derived, it compares every field — the undemanding default, usually right for a transaction. Written by hand, it encodes a *policy*: these two records are the same person because only the ID counts, whatever the name field says. The UVM makes you write `do_compare()` for exactly these cases, and rustdv puts the policy in the same place the UVM does — **on the transaction**. Equality is the data type's own statement about itself, not something a scoreboard improvises per comparison.

Why equality comes in two traits

```
// Figure 3: Why equality comes in two traits

// `PartialEq` promises symmetry and transitivity. It does **not**
promise
// that a == a. That sounds like hair-splitting until you meet a
float:
// IEEE 754 says NaN is equal to nothing, including itself.
#[derive(Debug, PartialEq)]
// #[derive(Eq)]  // <-- will not compile: f64 is not Eq
struct Measurement {
    delay_ns: f64,
}

// `Eq` adds the missing promise — every value equals itself — and
carries no
// methods. `HashSet` and `HashMap` require it, because a key that
does not
// equal itself could never be found again.
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
#[repr(u8)]
enum Ops {
    Add = 1,
    And = 2,
    Xor = 3,
    Mul = 4,
}

fn main() {
    let m = Measurement { delay_ns: f64::NAN };
    println!("a measured delay is not equal to itself: NaN == NaN?
{}", m == m);

    // `Ops` is `Eq`, so it can be a coverage key.
    let mut seen: HashSet<Ops> = HashSet::new();
    for op in [Ops::Add, Ops::Mul, Ops::And, Ops::Xor, Ops::Add] {
        seen.insert(op); // the last one is already in the set
    }
    println!("ops seen: {}", seen.len());
}
```

```
--
a measured delay is not equal to itself: NaN == NaN? false
ops seen: 4
```

New material, with a verification-shaped bite. Rust splits equality into two traits because `PartialEq` does not promise `a == a` — IEEE 754 forbids it for NaN —

while `Eq` adds that promise and nothing else. Where this catches a verification engineer is the coverage bin: `HashSet` requires `Eq`, so a transaction carrying a *measured* value — a float delay, a sampled analog level — cannot be a coverage key, and the compiler says so at the derive. Uncomment the `Eq` on `Measurement` and the error names `f64` as the reason. The ops enum, all integers underneath, derives `Eq` and `Hash` and has been serving as a coverage key since Chapter 18.

Copying: deep for what you own

```
// Figure 4: Clone is deep for what you own

// A student is a person who also owns a list of grades. The `Vec` is the
// interesting part: it is the field that would make a shallow copy visible.
#[derive(Debug, Clone)]
struct StudentRecord {
    name: String,
    id_number: u32,
    grades: Vec<u32>,
}

impl fmt::Display for StudentRecord {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "Name: {}, ID: {} Grades: {:?}", self.name, self.id_number, self.grades)
    }
}

fn main() {
    let mary = StudentRecord {
        name: String::from("Mary"),
        id_number: 33,
        grades: vec![97, 82],
    };

    let mut mary_copy = mary.clone();

    println!("mary:      {mary}");
    println!("mary_copy: {mary_copy}");

    println!("-- add a grade to the copy --");
    mary_copy.grades.push(100);

    println!("mary:      {mary}");
    println!("mary_copy: {mary_copy}");
}
```

```
--
mary:      Name: Mary, ID: 33 Grades: [97, 82]
mary_copy: Name: Mary, ID: 33 Grades: [97, 82]
-- add a grade to the copy --
mary:      Name: Mary, ID: 33 Grades: [97, 82]
mary_copy: Name: Mary, ID: 33 Grades: [97, 82, 100]
```

`Clone` is `do_copy()`, generated from the field list, and cloning a field you *own* clones its contents: `mary_copy` got its own `Vec`, not a second name for Mary's, and the pushed grade proves it. Python needs both `copy.copy()` and `copy.deepcopy()` because assignment shares references and shallow is the accident you start from. Rust has one `clone()`, and how deep it goes is already written *in the type*: owned data is copied. There is no way for figure 4 to have gone the other way — the compiler would not let two records share a `Vec` without your saying so. Which brings us to how you say so:

```
// Figure 5: If you want the shallow copy, you ask for it

// Same record, one field changed: the grades now live behind an
`Rc`, so a
// clone copies the *handle* and both records point at one list.
#[derive(Debug, Clone)]
struct SharedStudent {
    name: String,
    grades: Rc<RefCell<Vec<u32>>>,
}

fn main() {
    let mary = SharedStudent {
        name: String::from("Mary"),
        grades: Rc::new(RefCell::new(vec![97, 82])),
    };

    let mary_copy = mary.clone();

    println!(
        "before: {} {:?} / {} {:?}",
        mary.name, mary.grades.borrow(), mary_copy.name,
mary_copy.grades.borrow()
    );

    println!("-- add a grade through the copy --");
    mary_copy.grades.borrow_mut().push(100);

    println!(
        "after:  {} {:?} / {} {:?}",
        mary.name, mary.grades.borrow(), mary_copy.name,
mary_copy.grades.borrow()
    );

    println!("handles to the one list: {}",
Rc::strong_count(&mary.grades));
}
```

```
--
before: Mary [97, 82] / Mary [97, 82]
-- add a grade through the copy --
after:  Mary [97, 82, 100] / Mary [97, 82, 100]
handles to the one list: 2
```

This is Python's `copy.copy()` — except that in Python sharing is what you get by default, and here you had to write `Rc` to ask for it, with `RefCell` along because shared-and-mutable moves the borrow check to run time (Chapter 13). The deep/shallow decision is settled once, in the type definition, where a reviewer can see it — not at every call site, where the Python book had to warn you about it.

copy() and clone(), by their own names

```
// Figure 6: clone_from is the UVM's copy(): fill an object you
already have

#[derive(Debug, Clone, Default)]
struct StudentRecord {
    name: String,
    id_number: u32,
    grades: Vec<u32>,
}

impl fmt::Display for StudentRecord {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "Name: {}, ID: {} Grades: {:?}", self.name,
self.id_number, self.grades)
    }
}

fn main() {
    let mary = StudentRecord {
        name: String::from("Mary"),
        id_number: 33,
        grades: vec![97, 82],
    };

    // uvm_object.copy(other)  ->  Clone::clone_from(&mut self,
source)
    // uvm_object.clone()      ->  Clone::clone(&self)
    let mut mary_copy = StudentRecord::default();
    println!("before: {mary_copy}");

    mary_copy.clone_from(&mary);
    println!("after:  {mary_copy}");

    println!("-- and clone() is clone(): a new one, returned --");
    let fresh = mary.clone();
    println!("fresh:  {fresh}");
}
```

```
--
before: Name: , ID: 0 Grades: []
after:  Name: Mary, ID: 33 Grades: [97, 82]
-- and clone() is clone(): a new one, returned --
fresh:  Name: Mary, ID: 33 Grades: [97, 82]
```

The UVM has two copying calls and Rust has the same two, nearly under the same names: `copy(other)` fills an object you already have, and so does

`clone_from`; `clone()` hands back a new one in both worlds. What has no counterpart is `do_copy()` itself, and the discipline that came with it — "always call `super().do_copy(other)` first," which the UVM needs because a copy must walk up an inheritance chain. The derive walks the field list instead; there is no first step to forget.

One false friend before the payoff, because the words collide: Rust's `Copy` trait is unrelated to the UVM's `copy()`. `Copy` means "duplicating this is a memcpy and the original stays usable" — it is a statement about ownership, not about transactions, and a transaction owning a `String` or a `Vec` cannot be `Copy` at all. Chapter 31 showed what goes wrong when a `Copy` stand-in lets the wrong retry loop compile.

The TinyALU transactions, final form

```
// Figure 7: The TinyALU transactions, final form

// `Eq` and `Hash` because coverage puts an op in a `HashSet`
(Figure 3).
// `Copy` because an op is one byte and copying it is free — the
one place in
// the testbench where `Copy` is the right answer.
#[derive(Clone, Copy, Debug, PartialEq, Eq, Hash)]
#[repr(u8)]
pub enum Ops {
    Add = 1,
    And = 2,
    Xor = 3,
    Mul = 4,
}

impl Ops {
    pub const ALL: [Ops; 4] = [Ops::Add, Ops::And, Ops::Xor,
Ops::Mul];
}

// A command is plain data: no base class, no framework trait,
nothing to
// extend. Not `Copy`: a transaction is something you hand over.
#[derive(Clone, Debug, PartialEq, Eq)]
pub struct AluCommand {
    pub a: u8,
    pub b: u8,
    pub op: Ops,
}

// `Display` is the one you write.
impl fmt::Display for AluCommand {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "A: {:02x} {:?} B: {:02x}", self.a, self.op,
self.b)
    }
}

#[derive(Clone, Debug, PartialEq, Eq)]
pub struct AluResult {
    pub result: u16,
}

fn main() {
    let cmd = AluCommand { a: 0xA5, b: 0x75, op: Ops::Mul };

    println!("cmd:        {cmd}");
    println!("debug:      {cmd:?}");

    let copy = cmd.clone();
```

```
    println!("clone == cmd? {}", copy == cmd);

    let mut tweaked = cmd.clone();
    tweaked.a = 0;
    println!("tweaked == cmd? {}", tweaked == cmd);

    let mut covered: HashSet<Ops> = HashSet::new();
    for op in Ops::ALL {
        covered.insert(op);
    }
    println!("ops covered: {} of {}", covered.len(),
Ops::ALL.len());

    let _ = AluResult { result: 0x4B69 };
}
```

```
--
cmd:        A: a5 Mul B: 75
debug:      AluCommand { a: 165, b: 117, op: Mul }
clone == cmd? true
tweaked == cmd? false
ops covered: 4 of 4
```

Everything the chapter covered, applied. These three definitions replace the tuples the testbench has carried since Chapter 19, and they serve every remaining chapter of the book: `Ops` is `Copy` because a one-byte op is the testbench's one honest `Copy` case, and `Eq + Hash` because coverage keys demand it; `AluCommand` is *not* `Copy`, because a transaction is something you hand over — the fact `finish_item` will lean on in the next chapter — and its derived `Clone`, `PartialEq`, and `Debug` are `do_copy`, `do_compare`, and the field dump, each regenerating itself whenever you add a field. The one method written by hand is `Display`, because it always was the one method that needed an author.

Summary

A transaction is a place, not an object: no base class to extend, no identity beyond its bytes, no `super()` chain to remember. The `uvm_object` services arrive as traits — `Debug` free from the derive, `Display` written by hand because `convert2string()` always was, `PartialEq` derived when every field

counts and hand-written when “the same” is a policy, `Clone` deep for owned fields with sharing spelled `Rc` in the type, and `clone_from` / `clone` matching `copy()` / `clone()` name for name. `Eq` is the extra Rust asks so a value can promise it equals itself, and coverage keys are where you will feel it.

Because a transaction is a place and not a handle, handing one to `finish_item` will hand over its *contents* — and the sequence testbench that runs on that rule is next.

Chapter 36: Sequence Testbench: 7.0

Testbench 6.0 is a fine machine with one design flaw left: a new stimulus pattern means a new *component*. Want maximum operands instead of random ones? Override the Tester through the factory — rebuild part of the structure to change what the structure carries. *The UVM Primer* puts the objection best: overriding the tester to change stimulus is "like swapping out your car's steering wheel whenever you chose a different destination." The UVM's answer is the sequence system, and it is the methodology's crown jewel: the testbench structure holds still, and the **program** changes. This chapter builds testbench 7.0 around it.

> **In the UVM...** a `uvm_sequence` holds a `body()` task that creates `uvm_sequence_item`s and sends them with `start_item()` / `finish_item()`. A `uvm_sequencer` arbitrates among running sequences; the driver pulls with `seq_item_port.get_next_item()`, drives the DUT, and releases with `item_done()`. A test creates a sequence and calls `seq.start(sequencer)`.

The cast, before the code:

- A **sequence** is *not a component*. It has no place in the tree, no path, no phases — one method, `body`, and a context to run it against. It is a test program.
- The **sequencer** *is* a component: it holds the arbitration machinery and hands out one export. The environment files a handle to it in the ConfigDb so that a test levels above can start sequences on it without knowing where it lives.
- The **driver** pulls. Testbench 6.0's `cmd_fifo` is gone; the sequencer is the decoupling point now.

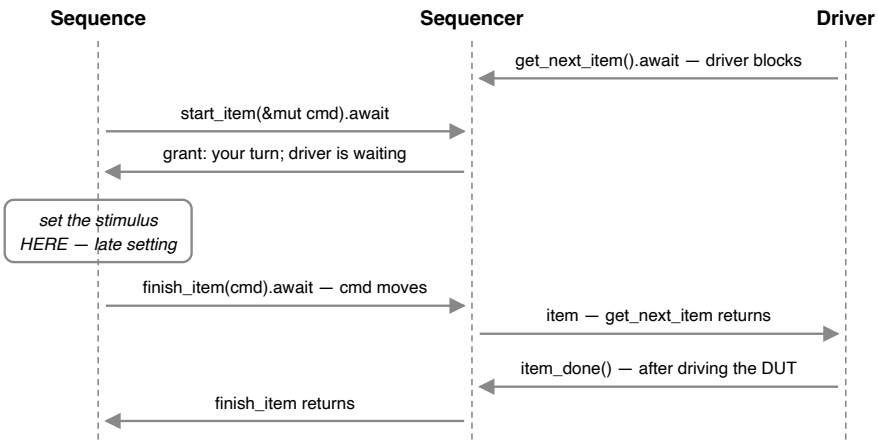


Figure 1: The sequencer handshake. Everything between the grant and finish_item happens with the driver committed and holding still.

Study the window in the middle of figure 1, because it is the answer to the question every newcomer asks about this protocol: *why two calls?* Why `start_item` then `finish_item`, when a single `send(cmd).await` looks like it would do? Because `start_item` returns at a very particular moment — the sequencer has granted this item its turn, *and the driver is blocked waiting for its contents*. Everything the sequence does between the two calls happens with the driver committed and holding still. That is where **late stimulus setting** lives: a sequence can look at the state of the testbench and decide what to send *now*, at the moment of delivery, rather than when it queued the item. A single `send` fixes the values before arbitration runs; the two-call rendezvous fixes them after. SystemVerilog had `mailbox#(T)` in the language and built this two-phase rendezvous anyway; pyuvm simplified nearly everything else about sequences and kept both phases. The gap between the calls is the feature.

The driver

```
// Chapter 36, Figure 2: The driver pulls items instead of being
pushed them
#[derive(Component, Default)]
struct Driver {
    #[port(seq_item)]
    seq_item_port: SeqItemPort<AluCommand, AluResult>,
}

impl Component for Driver {
    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let bfm: Rc<TinyAluBfm> = ConfigDb::get(Some(ctx), "",
"BFM")?;
        bfm.reset().await;
        loop {
            let item = self.seq_item_port.get_next_item().await;
            let cmd = item.payload();
            bfm.send_op(cmd.a, cmd.b, cmd.op).await;
            self.seq_item_port.item_done(None);
        }
    }
}
```

The difference from 6.0's driver is not the direction of the data — it is *who decides when*. `get_next_item()` returns only when a sequence has an item ready **and** the driver asked for it: a rendezvous, not a queue. The port is a `SeqItemPort<AluCommand, AluResult>` — request and response types, the same `#(REQ, RSP)` convention `uvm_driver` uses — declared with `#[port(seq_item)]` and wired in `connect` like every port since Chapter 31. And note `item_done(None)`: testbench 7.0 fires and forgets, no answer travels back, which is why the test will hold its objection for a flush at the end. Chapter 38's driver answers, and the flush goes away.

The transactions are Chapter 35's, re-shown as always rather than imported:

```
#[derive(Clone, Debug, PartialEq, Eq)]
pub struct AluCommand {
    pub a: u8,
    pub b: u8,
    pub op: Ops,
}

#[derive(Clone, Debug, PartialEq, Eq, Default)]
pub struct AluResult {
    pub result: u16,
}
```

The sequences

The Python book writes a `BaseSeq` whose `body()` loops the operations and calls `self.set_operands(tr)`, then subclasses it twice to override that one method. Rust has no inheritance, so the same design splits along a different line: the part that *varies* is a trait, and the part that *does not* is a function.

```
// Chapter 36, Figure 3: One body, three stimulus patterns
trait Operands {
    fn set_operands(&mut self, rng: &mut Rng, cmd: &mut
AluCommand);
}

async fn all_ops<S: Operands>(
    seq: &mut S,
    ctx: &mut SeqCtx<AluCommand, AluResult>,
) -> Result<(), SeqError> {
    let mut rng = ctx.rng();
    for op in Ops::ALL {
        let mut cmd = AluCommand { a: 0, b: 0, op };
        ctx.start_item(&mut cmd).await?; // the driver is now
waiting for us
        seq.set_operands(&mut rng, &mut cmd); // decide the
stimulus HERE
        ctx.finish_item(cmd).await?; // hand it over; wait for
item_done
    }
    Ok(())
}
```

Three lines in `all_ops` are figure 1 as code, and the middle one is deliberately placed: `set_operands` runs *between* `start_item` and `finish_item`, in the late-

setting window, which is the whole reason the two calls exist.

The third line is also this book's Chapter 5 collecting a payoff. `finish_item(cmd)` takes the command **by value** — the sequence hands over the *contents*, not a reference to something it still holds. After that line, `cmd` is gone: use it and the compiler's error names the move. If a sequence needs the command afterward — to log it, or to build the next command from it — `finish_item(cmd.clone())` is equally legal, and which one you write is a decision about ownership the code now states instead of implying. Both source books write the result *into* the sequence item the sequence still holds — not a capability rustdv lacks, but what handles look like when two names point at one object. Rust has one owner, so nothing is shared and nothing is missing.

```
// Chapter 36, Figure 4: The base sequence sends zeros
#[derive(Default)]
struct BaseSeq;

impl Operands for BaseSeq {
    fn set_operands(&mut self, _rng: &mut Rng, _cmd: &mut
AluCommand) {
        // zeros: whatever `all_ops` built the command with
    }
}

impl Sequence for BaseSeq {
    type Req = AluCommand;
    type Rsp = AluResult;

    async fn body(&mut self, ctx: &mut SeqCtx<AluCommand,
AluResult>) -> Result<(), SeqError> {
        all_ops(self, ctx).await
    }
}
```

`Sequence` is the trait with one method — `body` — plus the request and response types. Implementing it is what makes `BaseSeq` startable and, as figure 8 will show, factory-overridable.

```rust
// Chapter 36, Figure 5: Random and maximum operands
#[derive(Default)]
struct RandomSeq;

impl Operands for RandomSeq {
    fn set_operands(&mut self, rng: &mut Rng, cmd: &mut AluCommand)
{
        cmd.a = rng.u8();
        cmd.b = rng.u8();
    }
}

impl Sequence for RandomSeq {
    type Req = AluCommand;
    type Rsp = AluResult;

    async fn body(&mut self, ctx: &mut SeqCtx<AluCommand,
AluResult>) -> Result<(), SeqError> {
        all_ops(self, ctx).await
    }
}

#[derive(Default)]
struct MaxSeq;

impl Operands for MaxSeq {
    fn set_operands(&mut self, _rng: &mut Rng, cmd: &mut
AluCommand) {
        cmd.a = 0xFF;
        cmd.b = 0xFF;
    }
}

impl Sequence for MaxSeq {
    type Req = AluCommand;
    type Rsp = AluResult;

    async fn body(&mut self, ctx: &mut SeqCtx<AluCommand,
AluResult>) -> Result<(), SeqError> {
        all_ops(self, ctx).await
    }
}
```

One detail with regression consequences: the random numbers come from `ctx.rng()`, the seeded generator every other part of the testbench uses, so a failing run reproduces from its seed. (pyuvm's sequences draw from Python's global `random` module, which sits outside cocotb's seeding.)

The environment

```
// Chapter 36, Figure 6: The env owns the sequencer and files its
handle
#[derive(Component, Default)]
struct AluEnv {
    #[component]
    seqr: Sequencer<AluCommand, AluResult>,
    #[component]
    driver: RustdvComp,
    #[component]
    cmd_mon: RustdvComp,
    #[component]
    result_mon: RustdvComp,
    #[component]
    scoreboard: RustdvComp,
    #[component]
    cmd_bus: AnalysisBus<CmdTuple>,
    #[component]
    result_bus: AnalysisBus<u64>,
}

impl Component for AluEnv {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        self.seqr = Sequencer::new();
        ConfigDb::set(None, "*", "SEQR", self.seqr.handle());

        self.driver = Driver::new_comp();
        self.cmd_mon = CmdMonitor::new_comp();
        self.result_mon = ResultMonitor::new_comp();
        self.scoreboard = Scoreboard::new_comp();
        self.cmd_bus = AnalysisBus::new();
        self.result_bus = AnalysisBus::new();
    }

    fn connect(&mut self, _ctx: &mut RustdvCtx) {
        // stimulus: sequences --> [seqr] --> Driver
        self.seqr.seq_item_export().connect(&self.driver,
Driver::SEQ_ITEM_PORT);

        // observation, unchanged from Chapter 34
        self.cmd_bus.pub_export().connect(&self.cmd_mon,
CmdMonitor::AP);
        self.cmd_bus.sub_export().connect(&self.scoreboard,
Scoreboard::CMD_IN);
        self.result_bus.pub_export().connect(&self.result_mon,
ResultMonitor::AP);
        self.result_bus.sub_export().connect(&self.scoreboard,
Scoreboard::RESULT_IN);
    }

    fn start_of_simulation(&mut self, ctx: &mut RustdvCtx) {
        let bfm: Rc<TinyAluBfm> = ConfigDb::get(Some(ctx), "",
```

```
        "BFM").expect("the test sets BFM");
                bfm.start_tasks();
            }
        }
```

Three things:

- `#[component]` is the same carve-out `TlmFifo` got in Chapter 31: both endpoints of a connection are erased `RustdvComp` slots, so something concrete has to make the call, and a sequencer — like a FIFO — is infrastructure you will never factory-override. The connect line has the shape every connection since Chapter 31 has had.
- `ConfigDb::set(None, "*", "SEQR", self.seqr.handle())` files the sequencer where any test can find it. This is pyuvm's idiom, and the reason it beats searching the tree by path string is Chapter 27's: a hand-typed path goes stale, and the ConfigDb is how things that must find each other do. Note the sequencer goes in as a *handle* — every sequence must reach the same sequencer, the `Rc` case from Chapter 27's Clone-or-`Rc` rule.
- The monitors, the two `AnalysisBus` buses, and the two-stream scoreboard are Chapters 33–34's, unchanged. That is the chapter's claim about structure made visible: sequences arrived, and the observation side did not move.

The tests

```
// Chapter 36, Figure 7: The test starts a sequence on the
sequencer
#[rustdv::test]
#[derive(Component, Default)]
struct BaseTest {
    #[component]
    env: RustdvComp,
}

impl Component for BaseTest {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        let bfm = TinyAluBfm::new(&ctx.dut()).expect("TinyALU
signals");
        ConfigDb::set(None, "*", "BFM", Rc::new(bfm));
        self.env = AluEnv::new_comp();
    }

    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let _obj = ctx.raise_objection("running the sequence");
        let seqr: Sequencer<AluCommand, AluResult> =
ConfigDb::get(Some(ctx), "", "SEQR")?;

        let mut seq = create_seq::<BaseSeq>();
        seq.start(&seqr).await?;

        let bfm: Rc<TinyAluBfm> = ConfigDb::get(Some(ctx), "",
"BFM")?;
        for _ in 0..20 {
            bfm.clk().falling_edge().await;
        }
        Ok(())
    }
}
```

The test finds the sequencer in the ConfigDb — it neither knows nor cares where in the tree it lives — and `start` is a method on the *sequence*, taking the sequencer, exactly as both source books write it. Three details:

- The lookup happens in `run`, not in an elaboration phase. pyuvm does it in `end_of_elaboration_phase` because a Python phase cannot return an error; here `?` works, and a missing `SEQR` is a named failure.
- `create_seq::<BaseSeq>()` builds the sequence *through the factory* — a second registry, parallel to Chapter 29's, because a sequence is not a

component and cannot ride the first one. That line is what makes the next figure possible.

- After `start` returns, the last commands are still in flight — accepted, not yet answered. The test holds its objection for twenty falling edges: twenty, not ten, because the multiply is the last operation and the slowest, and a shorter flush would let the scoreboard silently check fewer results than it saw commands.

```
// Chapter 36, Figure 8: Two more tests, one testbench, no new
components
#[rustdv::test]
#[derive(Component, Default)]
struct RandomTest {
    #[component]
    inner: RustdvComp,
}

impl Component for RandomTest {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        set_seq_override::<BaseSeq, RandomSeq>();
        self.inner = BaseTest::new_comp();
    }
}

#[rustdv::test]
#[derive(Component, Default)]
struct MaxTest {
    #[component]
    inner: RustdvComp,
}

impl Component for MaxTest {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        set_seq_override::<BaseSeq, MaxSeq>();
        self.inner = BaseTest::new_comp();
    }
}
```

This is what sequences bought. In Chapter 30, a new stimulus pattern meant a new component and a factory override on a component slot. Here it is a different *program* run through an unchanged structure: `RandomTest` is `BaseTest` plus one `set_seq_override` line, and nothing in `AluEnv` knows either sequence exists. The steering wheel stays; only the destination changes.

```
# Figure 9: Testbench 7.0 running

      0.00ns INFO     rustdv: found 3 test(s), RUSTDV_RANDOM_SEED=1
      0.00ns INFO     running BaseTest (1/3)  [ch36-sequence-
testbench-7.0/src/ch36_sequence_testbench_7_0.rs:385]
    280.00ns INFO     [BaseTest.env.scoreboard]: PASSED: 00 Add 00
= 0000
    280.00ns INFO     [BaseTest.env.scoreboard]: PASSED: 00 And 00
= 0000
    280.00ns INFO     [BaseTest.env.scoreboard]: PASSED: 00 Xor 00
= 0000
    280.00ns INFO     [BaseTest.env.scoreboard]: PASSED: 00 Mul 00
= 0000
    280.00ns INFO     [BaseTest.env.scoreboard]: Covered all
operations
    280.00ns INFO     BaseTest PASSED
    280.00ns INFO     running RandomTest (2/3)  [ch36-sequence-
testbench-7.0/src/ch36_sequence_testbench_7_0.rs:426]
    560.00ns INFO     [RandomTest.inner.env.scoreboard]: PASSED: c1
Add 67 = 0128
    560.00ns INFO     [RandomTest.inner.env.scoreboard]: PASSED: 5e
And 0b = 000a
    560.00ns INFO     [RandomTest.inner.env.scoreboard]: PASSED: b9
Xor 80 = 0039
    560.00ns INFO     [RandomTest.inner.env.scoreboard]: PASSED: a5
Mul 75 = 4b69
    560.00ns INFO     [RandomTest.inner.env.scoreboard]: Covered
all operations
    560.00ns INFO     RandomTest PASSED
    560.00ns INFO     running MaxTest (3/3)  [ch36-sequence-
testbench-7.0/src/ch36_sequence_testbench_7_0.rs:440]
    840.00ns INFO     [MaxTest.inner.env.scoreboard]: PASSED: ff
Add ff = 01fe
    840.00ns INFO     [MaxTest.inner.env.scoreboard]: PASSED: ff
And ff = 00ff
    840.00ns INFO     [MaxTest.inner.env.scoreboard]: PASSED: ff
Xor ff = 0000
    840.00ns INFO     [MaxTest.inner.env.scoreboard]: PASSED: ff
Mul ff = fe01
    840.00ns INFO     [MaxTest.inner.env.scoreboard]: Covered all
operations
    840.00ns INFO     MaxTest PASSED
**********************************************************************
***********
** TEST                                          STATUS  SIM TIME (ns)
**
**********************************************************************
***********
** BaseTest                                        PASS         280.00
**
** RandomTest                                      PASS         280.00
**
** MaxTest                                         PASS         280.00
**
```

```
**********************************************************************
**********
REGRESSION: PASS
```

Summary

Testbench 7.0 separates what to send from what sends it. A sequence is a program — no tree, no path, no phases, one `body` — started on a sequencer, which is the component that arbitrates turns and feeds the driver through the same connect idiom as every other wiring in the book. The two-call handshake is the design's heart: `start_item` returns with the driver committed and waiting, the window between the calls is where stimulus is decided at the last responsible moment, and `finish_item(cmd)` hands over the command by value — clone it first if you still need it, and the compiler will hold you to whichever answer you gave. Sequences build through their own factory registry, so `create_seq::<BaseSeq>()` plus `set_seq_override` gives tests the same substitution power over programs that Chapter 29 gave them over structure.

Version 7.0 fires and forgets, which is why its test counts clocks at the end instead of knowing when the work is done. The next two chapters close the loop: first a driver that answers out of order, where a ticket claims the answer you asked for, then Fibonacci on the TinyALU, where each command cannot be written until the previous one is answered.

Chapter 37: Out-of-Order Transactions: Testbench 7.1

In testbench 7.0 a sequence sent commands and never heard back. Real stimulus often needs the answer — and the answer does not always arrive in the order the requests were sent. That second clause is the hard part, and this chapter teaches it on a device built to make it plain, before Chapter 38 needs it on the TinyALU.

The whole testbench is one idea. Each request carries a number: how many ticks the driver should spend on it before answering. The sequence sends four — 10, 7, 4, 1 — one after another, without waiting for any of them. The driver takes them all in and works on them at once, so the request sent last is finished first and the request sent first is finished last. Every answer still reaches the sequence that asked for it, because each one comes back under its request's own ticket.

That ticket is the `TxnId` the sequencer assigned when the item was sent, and this is the first chapter where it earns its keep. Until now, one item was in flight at a time, and "give me whatever comes next" always found the right answer without being told which one to look for. (If you have written an AXI testbench, you have met all of this: `ARID` and `RID` exist so a slave may answer out of order.)

And the detail that makes the whole system work: **the driver releases the sequence when it accepts the request, not when it has the answer.** `item_done` ends the handshake as soon as the request is taken in, so the sequence can send the next one; the answer comes back later, through `put_response`, under the request's ticket. Hold the handshake open until the work is done and only one request is ever outstanding — and then there is nothing for a ticket to disambiguate.

> **In the UVM…** the driver called `set_id_info(req)` on its response so the sequencer could route it, then `put_response(rsp)`; the sequence called `get_response(rsp, req.get_transaction_id())` to claim a specific

answer. Forgetting `set_id_info` was a classic run-time failure: the response went back with no identity, and the sequence waiting for it waited forever.

One note on the cast: there is no TinyALU in this chapter. The DUT is the empty `playground` module, because the interesting behavior is *in the driver* — it takes a request, spends the time that request asked for, and gets on with the next one. The TinyALU runs one operation at a time, so none of its answers can ever overtake another; a chapter about overtaking needs a device that permits it.

The transactions

```
// Chapter 37, Figure 1: A request that says how long it takes, and
its answer
#[derive(Clone, Debug, Default, PartialEq, Eq)]
struct Req {
    /// How many ticks the driver should spend before answering.
    delay: u32,
}

#[derive(Clone, Debug, Default, PartialEq, Eq)]
struct Rsp {
    /// Echoed back so the sequence can see it got the answer it
asked for.
    delay: u32,
}
```

Plain structs with derives, as Chapter 35 left them. The request carries what the driver needs in order to do the work; the answer carries what the sequence wants to know. Neither mentions a ticket — identity lives in the envelope the sequencer wraps around them, not in your data.

Putting the latency *in the request* is what makes this chapter's transcript readable. The driver does not choose how long to take, so nothing here depends on a random seed: the same four numbers go in every run, and the same reversal comes out.

The driver

```
// Chapter 37, Figure 2: A driver that accepts work and answers it
later
#[derive(Component, Default)]
struct Driver {
    #[port(seq_item)]
    seq_item_port: SeqItemPort<Req, Rsp>,
    /// Accepted but not yet answered.
    outstanding: Vec<InFlight>,
}

/// One request the driver has taken in and owes an answer for.
#[derive(Clone, Copy)]
struct InFlight {
    ticket: TxnId,
    delay: u32,
    left: u32,
}

impl Component for Driver {
    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        loop {
            // --- accept whatever is waiting ----------------------
---------
            Timer::ns(5).await;
            if let Some(item) = self.seq_item_port.try_next_item()
{
                let ticket = item.txn_id();
                let delay = item.payload().delay;
                ctx.info(&format!("accepted {ticket}, delay
{delay}"));
                // Release the sequence now. The answer comes
later.
                self.seq_item_port.item_done(None);
                self.outstanding.push(InFlight { ticket, delay,
left: delay });
            }

            // --- age the outstanding work, and answer what is due
--------
            Timer::ns(5).await;
            let mut still_working = Vec::new();
            for job in self.outstanding.drain(..) {
                if job.left > 1 {
                    still_working.push(InFlight { left: job.left -
1, ..job });
                } else {
                    ctx.info(&format!("answering {}", job.ticket));
                    // The answer goes back under the request's own
ticket.
                    self.seq_item_port.put_response(job.ticket, Rsp
```

```
  { delay: job.delay });
                  }
              }
              self.outstanding = still_working;
          }
      }
  }
```

The loop does one thing in each half of a tick, and that split is what keeps it simple: first half, accept a request if one is waiting; second half, age everything outstanding and answer whatever is due. A driver that tried to wait for a request *and* count time simultaneously would need two loops and a shared list; one sequential loop does both, and nothing races. (A driver on a real bus would use its clock's two edges; the playground is empty, so the tick is simulated time.)

Three lines to dwell on:

- **`try_next_item()` is what keeps the loop moving.** `get_next_item()` blocks. On a tick with an empty sequencer, a blocking accept would sit there forever and never reach the second half of the loop — the requests already accepted would never age, and the sequence waiting for them would never be answered. The non-blocking accept is the UVM's own answer to exactly this situation (`try_next_item` has been in the standard since 1.1d; pyuvm does not carry it, and rustdv follows the UVM here because this driver cannot be written without it).
- **`item_done(None)` is what puts four requests in flight at once.** It ends the handshake with no answer attached, which is the point: the answer does not exist yet. A driver that can answer immediately passes `Some(rsp)` here and never needs a ticket.
- **`put_response(job.ticket, ...)` returns the answer under the request's own ticket**, so the sequence that asked about `#1` gets `#1`, however many others were finished first. The ticket is a required argument, not a separate call: the UVM driver that forgot `set_id_info` sent back a response with no identity, and here that mistake has no spelling.

`InFlight` is the driver's whole memory: a ticket, the delay the request asked for, and how many ticks are left on it. Note what it does *not* keep — the `SeqItem` envelope itself. Once the ticket is out of the envelope, the envelope has done its job.

The sequence

```
// Chapter 37, Figure 3: Send four requests, then collect four
answers
#[derive(Default)]
struct Seq;

impl Sequence for Seq {
    type Req = Req;
    type Rsp = Rsp;

    async fn body(&mut self, ctx: &mut SeqCtx<Req, Rsp>) ->
Result<(), SeqError> {
        let mut tickets = Vec::new();
        for delay in [10, 7, 4, 1] {
            let mut req = Req { delay };
            ctx.start_item(&mut req).await?;
            let ticket = ctx.finish_item(req).await?;
            ctx.info(&format!("sent {ticket}, delay {delay}"));
            tickets.push(ticket);
        }

        // Go round the outstanding tickets again and again, taking
whichever
        // answers are ready and dropping those tickets from the
list, until
        // none are left. Same drain-and-rebuild shape as the
driver's loop.
        let mut waiting = tickets;
        while !waiting.is_empty() {
            Timer::ns(10).await;
            let mut still_waiting = Vec::new();
            for ticket in waiting {
                match ctx.try_get_response(Some(ticket)) {
                    Some(rsp) => ctx.info(&format!("got {ticket},
delay {}", rsp.delay)),
                    None => still_waiting.push(ticket),
                }
            }
            waiting = still_waiting;
        }
        Ok(())
    }
}
```

The sequence sends all four requests before asking about any of them — that is the whole trick. Had it waited for each answer before sending the next, only one request would ever be outstanding, and the driver's varying latency would be invisible. `finish_item` returns the ticket, and the sequence keeps them all.

Then it collects by **polling**, and the polling is not laziness — it is what makes the out-of-order work visible. Each time round, the sequence asks every outstanding ticket whether its answer is ready yet, keeps the ones that are not, and prints the ones that are. Blocking on ticket `#1`, then `#2`, then `#3` would collect the answers in the order they were *sent*, no matter what the driver did; the reordering would be real and the log would not show it. `try_get_response` collects them in the order they were *finished*, which is what the ticket is for.

The collection loop is deliberately the same shape as the driver's: walk the list, keep what is not done, rebuild it. Both sides of the handshake are managing a set of outstanding transactions, and they manage it the same way.

One failure mode deserves its paragraph, because no figure can show it: **asking for an answer that will never exist.** A sequence that calls `get_response` for a ticket the driver never answers waits forever. Nothing can tell "not ready yet" from "never coming"; that is inherent to asking for a specific answer, in every UVM. It is the sequence writer's job to ask only for answers that are owed, and the only way to demonstrate the mistake is a test that hangs, which is why this book does not run one.

The environment and the test

```
// Chapter 37, Figure 4: A sequencer and a driver, and the test
that runs them
#[derive(Component, Default)]
struct Env {
    #[component]
    seqr: Sequencer<Req, Rsp>,
    #[component]
    driver: RustdvComp,
}

impl Component for Env {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        self.seqr = Sequencer::new();
        ConfigDb::set(None, "*", "SEQR", self.seqr.handle());
        self.driver = Driver::new_comp();
    }

    fn connect(&mut self, _ctx: &mut RustdvCtx) {
        self.seqr.seq_item_export().connect(&self.driver,
Driver::SEQ_ITEM_PORT);
    }
}

#[rustdv::test]
#[derive(Component, Default)]
struct ResponseTest {
    #[component]
    env: RustdvComp,
}

impl Component for ResponseTest {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        self.env = Env::new_comp();
    }

    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let _obj = ctx.raise_objection("requests outstanding");
        let seqr: Sequencer<Req, Rsp> = ConfigDb::get(Some(ctx),
"", "SEQR")?;
        let mut seq = Seq::default();
        seq.start(&seqr).await?;
        Ok(())
    }
}
```

The environment is two components — a sequencer and a driver — because the chapter is about the handshake, not the architecture. Note what the test

does *not* have: a flush. `Seq`'s body does not return until every answer is collected, so when `start` returns, nothing is in flight and the objection can drop immediately.

```
# Figure 5: Later requests are answered first; every ticket still
claims its own

      0.00ns INFO     rustdv: found 1 test(s), RUSTDV_RANDOM_SEED=1
      0.00ns INFO     running ResponseTest (1/1)  [ch37-out-of-
order-transaction-testbench-
7.1/src/ch37_out_of_order_transaction_testbench_7_1.rs:231]
     15.00ns INFO     [ResponseTest.env.driver]: accepted #1, delay
10
     15.00ns INFO     [Seq]: sent #1, delay 10
     35.00ns INFO     [ResponseTest.env.driver]: accepted #2, delay
7
     35.00ns INFO     [Seq]: sent #2, delay 7
     55.00ns INFO     [ResponseTest.env.driver]: accepted #3, delay
4
     55.00ns INFO     [Seq]: sent #3, delay 4
     75.00ns INFO     [ResponseTest.env.driver]: accepted #4, delay
1
     75.00ns INFO     [Seq]: sent #4, delay 1
     80.00ns INFO     [ResponseTest.env.driver]: answering #4
     85.00ns INFO     [Seq]: got #4, delay 1
     90.00ns INFO     [ResponseTest.env.driver]: answering #3
     95.00ns INFO     [Seq]: got #3, delay 4
    100.00ns INFO     [ResponseTest.env.driver]: answering #2
    105.00ns INFO     [Seq]: got #2, delay 7
    110.00ns INFO     [ResponseTest.env.driver]: answering #1
    115.00ns INFO     [Seq]: got #1, delay 10
    115.00ns INFO     ResponseTest PASSED
**********************************************************************
***********
** TEST                                          STATUS  SIM TIME (ns)
**
**********************************************************************
***********
** ResponseTest                                    PASS         115.00
**
**********************************************************************
***********
REGRESSION: PASS
```

Read the log in three passes. The `sent` lines run `#1, #2, #3, #4`, delays descending. The `answering` lines run `#4, #3, #2, #1` — the exact reverse, because the driver is working on all four at once and the short ones finish first. The `got` lines follow that same reversed order, and each one carries back its

own request's delay: `#4` with 1, `#1` with 10. The sequence never asked "what is next"; it asked four specific questions and got four specific answers.

Notice also that the driver accepts `#2` at 35 ns while `#1` still has seven ticks left on it. That is `try_next_item` and `item_done` doing their work: without them, `#1` would have to be finished and answered before `#2` could even be taken in, and there would be nothing left of this chapter.

Summary

Responses are the second half of the sequencer handshake, and identity is what keeps them sorted when several are in flight. The driver accepts a request with `try_next_item` — the non-blocking accept that exists so a driver can keep serving work it already holds — releases the sequence with `item_done` before the answer exists, and returns each answer later with `put_response` under the request's own `TxnId`. The sequence holds its tickets and claims each answer with `get_response` / `try_get_response(Some(ticket))`, polling so that it collects them in the order they were finished rather than the order it sent them.

Chapter 38 puts the response path to work on the real DUT — where each command cannot even be written until the previous answer is in hand, and one transaction is in flight at a time.

Chapter 38: Fibonacci Testbench: 7.2

Chapter 37 taught the response mechanism on a device that did nothing but wait. Testbench 7.2 puts it to work on the real DUT, computing the most traditional dependent stimulus there is: **the TinyALU produces the Fibonacci numbers**, and it cannot be given the next pair of operands until it has answered the last one.

```
0  1  1  2  3  5  8  13  21
```

Each command's operands are the answers to the two before it. There is no way to generate this stimulus in advance — no sequence of random values, no precomputed list that stays honest — it must be written one command at a time, with the DUT's answer in hand. That is the reason the sequence system exists, reduced to nine lines of `body`.

In the UVM... both source books compute Fibonacci the shared-handle way: the driver gets the sum from the BFM and writes it *into the sequence item*, and the sequence — still holding a handle to the same object — reads `cmd.result` when `finish_item` returns. The 7.2 variant sent a separate response instead: the driver called `rsp.set_id_info(req)` and `item_done(rsp)`, and the sequence awaited `get_response()`.

A driver that waits for the answer

```
// Chapter 38, Figure 1: The driver sends a command and returns its
result
#[derive(Component, Default)]
struct Driver {
    #[port(seq_item)]
    seq_item_port: SeqItemPort<AluCommand, AluResult>,
    #[port(publish)]
    result_ap: PublishPort<u64>,
}

impl Component for Driver {
    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let bfm: Rc<TinyAluBfm> = ConfigDb::get(Some(ctx), "",
"BFM")?;
        bfm.reset().await;
        loop {
            let item = self.seq_item_port.get_next_item().await;
            let cmd = item.payload();
            bfm.send_op(cmd.a, cmd.b, cmd.op).await;
            let result = bfm.get_result().await;
            self.result_ap.write(&result);
            self.seq_item_port
                .item_done(Some(AluResult { result: result as u16
}));
        }
    }
}
```

Chapter 36's driver fired and forgot. This one waits for *this* operation's answer before taking another item, and hands the answer back through `item_done(Some(...))`. The framework tags the response with the command's ticket automatically — nothing here performs `set_id_info`, the call a UVM driver could forget and whose absence was a run-time fatal. And note the analysis port: since this driver already holds every answer, it publishes results itself. The reason unfolds in figure 3.

The Fibonacci sequence

```
// Chapter 38, Figure 2: Nine numbers, eight of them from the DUT
#[derive(Default)]
struct FibonacciSeq;

impl Sequence for FibonacciSeq {
    type Req = AluCommand;
    type Rsp = AluResult;

    async fn body(&mut self, ctx: &mut SeqCtx<AluCommand,
AluResult>) -> Result<(), SeqError> {
        let mut prev: u8 = 0;
        let mut cur: u8 = 1;
        let mut fib = vec![prev as u16, cur as u16];

        for _ in 0..7 {
            let mut cmd = AluCommand { a: 0, b: 0, op: Ops::Add };
            ctx.start_item(&mut cmd).await?;
            cmd.a = prev;
            cmd.b = cur;
            let ticket = ctx.finish_item(cmd).await?;
            let sum = ctx.get_response(Some(ticket)).await.result;

            fib.push(sum);
            prev = cur;
            cur = sum as u8;
        }

        ctx.info(&format!("Fibonacci Sequence: {fib:?}"));
        Ok(())
    }
}
```

Read the middle three lines as one gesture: hand the command over, wait, take the answer. `cmd` moves at `finish_item` — the sequence has no further use for it, so nothing is cloned; a sequence that *did* want to keep the command it sent would write `finish_item(cmd.clone())`, and the compiler would insist if it forgot. The ticket comes back from `finish_item`, and `get_response(Some(ticket))` claims this command's answer.

Two things are worth saying about that `Some(ticket)`, since Chapter 37 is fresh. First, with one command in flight at a time — and Fibonacci *cannot* pipeline; the dependency forbids it — the ticket is never ambiguous, so passing it is documentation rather than necessity. It is good documentation: `get_response(Some(id))` says what you mean, and it keeps working if a later

testbench pipelines the same sequence. Asking for “whatever comes next” is right only for as long as there is only one thing coming. Second, for the UVM reader waiting for the shared-handle miracle — the driver writing `cmd.result` and the sequence reading it back out of the object it still holds — that move does not exist here, and nothing is missing. Writing into a shared handle is simply what returning a value looks like in a language where two names can point at one object. Rust has one owner, so the answer comes back as its *own value*, one line later either way.

A small pleasure, in passing: the sequence logs under its own name. pyuvm’s sequences cannot — `uvm_sequence` is not a `uvm_report_object`, so its Fibonacci reaches for `uvm_root().logger` — and the Primer’s uses a hand-typed string id. `ctx.info` in a sequence body is attributed like everything else.

The environment: no result monitor

```
// Chapter 38, Figure 3: No result monitor
#[derive(Component, Default)]
struct FibEnv {
    #[component]
    seqr: Sequencer<AluCommand, AluResult>,
    #[component]
    driver: RustdvComp,
    #[component]
    result_bus: AnalysisBus<u64>,
    #[component]
    watcher: RustdvComp,
}

impl Component for FibEnv {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        self.seqr = Sequencer::new();
        ConfigDb::set(None, "*", "SEQR", self.seqr.handle());
        self.driver = Driver::new_comp();
        self.result_bus = AnalysisBus::new();
        self.watcher = ResultWatcher::new_comp();
    }

    fn connect(&mut self, _ctx: &mut RustdvCtx) {
        self.seqr.seq_item_export().connect(&self.driver,
Driver::SEQ_ITEM_PORT);
        self.result_bus.pub_export().connect(&self.driver,
Driver::RESULT_AP);
        self.result_bus.sub_export().connect(&self.watcher,
ResultWatcher::INPUT);
    }

    fn start_of_simulation(&mut self, ctx: &mut RustdvCtx) {
        let bfm: Rc<TinyAluBfm> = ConfigDb::get(Some(ctx), "",
"BFM").expect("the test sets BFM");
        bfm.start_tasks();
    }
}
```

The `ResultMonitor` of testbenches 6.0 and 7.0 is gone, and the reason is worth a moment because it is an architecture decision you will face on real projects. This driver already awaits every answer — it is the component that *has* the result. A separate result monitor would be a second reader drawing from the same BFM queue, and the two would take turns stealing results from each other. So the driver publishes on its own analysis port: `result_bus.pub_export()` connects to the *driver*. Whoever holds the data publishes it; observation follows the data, not the org chart.

```
// Chapter 38, Figure 4: A subscriber checks the DUT actually added
#[derive(Component, Default)]
struct ResultWatcher {
    #[port(subscribe)]
    input: SubscribePort<u64>,
    seen: RustdvShared<SeenResults>,
}

impl Component for ResultWatcher {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        self.input.subscribe(self.seen.clone());
    }

    fn check(&mut self, ctx: &mut RustdvCtx, errors: &mut
CheckSink) {
        let seen = self.seen.get();
        let expected: Vec<u64> = vec![1, 2, 3, 5, 8, 13, 21];
        if seen.results == expected {
            ctx.info(&format!("adder produced {:?}",
seen.results));
        } else {
            errors.error(format!("expected {expected:?}, saw {:?}",
seen.results));
        }
    }
}
```

The sequence proves the numbers are Fibonacci; the watcher proves they came from the *adder*, rather than from the sequence's own arithmetic. A subscriber with seven expected values is a small scoreboard, and it closes the loop a self-checking testbench needs.

```
// Chapter 38, Figure 5: The test
#[rustdv::test]
#[derive(Component, Default)]
struct FibonacciTest {
    #[component]
    env: RustdvComp,
}

impl Component for FibonacciTest {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        let bfm = TinyAluBfm::new(&ctx.dut()).expect("TinyALU
signals");
        ConfigDb::set(None, "*", "BFM", Rc::new(bfm));
        self.env = FibEnv::new_comp();
    }

    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let _obj = ctx.raise_objection("computing Fibonacci");
        let seqr: Sequencer<AluCommand, AluResult> =
ConfigDb::get(Some(ctx), "", "SEQR")?;
        let mut seq = FibonacciSeq::default();
        seq.start(&seqr).await?;
        Ok(())
    }
}
```

No flush. Chapters 34 and 36 held their objections through twenty falling edges because their drivers fired and forgot; this one's `finish_item` does not return until the driver has the answer, so when the sequence ends, nothing is in the pipeline. The twenty-clock wait was a property of a driver that does not wait — not of sequences, and not of the DUT.

```
# Figure 6: The TinyALU computes Fibonacci

      0.00ns INFO     rustdv: found 1 test(s), RUSTDV_RANDOM_SEED=1
      0.00ns INFO     running FibonacciTest (1/1)  [ch38-fibonacci-
testbench-7.2/src/ch38_fibonacci_testbench_7_2.rs:228]
    170.00ns INFO     [FibonacciSeq]: Fibonacci Sequence: [0, 1, 1,
2, 3, 5, 8, 13, 21]
    170.00ns INFO     [FibonacciTest.env.watcher]: adder produced
[1, 2, 3, 5, 8, 13, 21]
    170.00ns INFO     FibonacciTest PASSED
**********************************************************************
***********
** TEST                                        STATUS  SIM TIME (ns)
**
**********************************************************************
***********
** FibonacciTest                                 PASS         170.00
**
**********************************************************************
***********
REGRESSION: PASS
```

Summary

Dependent stimulus is what sequences are for, and testbench 7.2 is the classic case: nine numbers, eight computed by the DUT, each command written only after the previous answer arrived. The driver awaits each result and returns it through `item_done(Some(...))`, ticketed automatically; the sequence claims it with `get_response(Some(ticket))` — documentation today, correctness the day the sequence is pipelined. The result monitor is gone because the driver holds the results, and observation follows the data. And the flush is gone because a driver that waits leaves nothing in flight — end-of-test bookkeeping got simpler by making the driver more honest about time.

One rung remains on the ladder both earlier books climbed: sequences that run other sequences, and the testbench that becomes a programming interface. Testbench 8.0 is next.

Chapter 39: Virtual Sequence Testbench: 8.0

The last three chapters built sequences that send items. This one builds sequences that send nothing at all. A **virtual sequence** is started without a sequencer; it sends no items of its own; it *starts other sequences*. That is the whole of the idea, and it is what lets a test be assembled from stimulus that already exists rather than written again. Testbench 8.0 is where both earlier books ended their climbs, and it ends this one's: after this chapter, the machinery is complete.

> **In the UVM…** we wrote a `TestAllSeq` extending `uvm_sequence` whose `body()` fetched the sequencer from the config database and ran `rand_seq.start(seqr)` then `max_seq.start(seqr)`; the test started the virtual sequence *without* a sequencer argument. A parallel variant forked the sub-sequences and joined them.

A program that runs programs

```
// Chapter 39, Figure 1: A virtual sequence starts other sequences
#[derive(Default)]
struct TestAllSeq;

impl Sequence for TestAllSeq {
    type Req = AluCommand;
    type Rsp = AluResult;

    async fn body(&mut self, ctx: &mut SeqCtx<AluCommand,
AluResult>) -> Result<(), SeqError> {
        let seqr: Sequencer<AluCommand, AluResult> =
ConfigDb::get(None, "", "SEQR")?;
        RandomSeq::default().start(&seqr).await?;
        MaxSeq::default().start(&seqr).await?;
        ctx.info("ran random, then max");
        Ok(())
    }
}
```

Three observations, in rising order of importance.

- The body finds its sequencer the same way a test does — in the ConfigDb, with the `None` context, because a sequence has no path to offset from. That was the reason Chapter 27 kept the null-context form.
- There is no `start_item` and no `finish_item`. Nothing here touches an item; `RandomSeq` and `MaxSeq` do their own item handling exactly as they did in Chapter 36, unchanged.
- It is the *same* `Sequence` trait. Nothing marks this sequence “virtual” except what it does — precisely as in SystemVerilog, where `runall_sequence extends uvm_sequence #(uvm_sequence_item)` and simply never sends one.

The test that starts it:

```rust
// Chapter 39, Figure 2: The test starts the virtual sequence — no
sequencer
#[rustdv::test]
#[derive(Component, Default)]
struct AluTest {
    #[component]
    env: RustdvComp,
}

impl Component for AluTest {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        let bfm = TinyAluBfm::new(&ctx.dut()).expect("TinyALU
signals");
        ConfigDb::set(None, "*", "BFM", Rc::new(bfm));
        self.env = AluEnv::new_comp();
    }

    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let _obj = ctx.raise_objection("running the virtual
sequence");
        create_seq::<TestAllSeq>().start_virtual().await?;
        Ok(())
    }
}
```

`start_virtual()` takes no sequencer, because a virtual sequence has none to take — everything it drives, it drives through sequencers it looked up itself. Why a second method rather than pyuvm's single `start` with an optional argument? Rust has no default arguments, so the choice was between `start(Some(&seqr))` at every ordinary call site — noise that says nothing — or `start(None)` at the virtual ones, where `None` fails to say "virtual." Two names, each meaning what it says.

One design question deserves an answer here, because a Rust-minded reader will already have asked it: *why not a separate `VirtualSequence` trait?* It would make calling `start_item` inside a virtual sequence a compile error instead of the run-time error pyuvm gives. It would also forbid a shape the UVM allows: *The UVM Primer*'s `parallel_sequence` is started *with* a sequencer and is still virtual in the sense that matters, and nothing stops a sequence from sending some items itself and delegating the rest. Two traits would buy a better error message at the cost of a capability, and the framework declines that trade, knowingly. Late binding keeps its options; the error, if you make it, arrives at run time with a name on it.

```
# Figure 3: Random, then max

      0.00ns INFO     running AluTest (1/3)  [ch39-virtual-
sequence-testbench-
8.0/src/ch39_virtual_sequence_testbench_8_0.rs:440]
    250.00ns INFO     [TestAllSeq]: ran random, then max
    250.00ns INFO     [AluTest.env.scoreboard]: PASSED: c1 Add 67 =
0128
    250.00ns INFO     [AluTest.env.scoreboard]: PASSED: 5e And 0b =
000a
    250.00ns INFO     [AluTest.env.scoreboard]: PASSED: b9 Xor 80 =
0039
    250.00ns INFO     [AluTest.env.scoreboard]: PASSED: a5 Mul 75 =
4b69
    250.00ns INFO     [AluTest.env.scoreboard]: PASSED: ff Add ff =
01fe
    250.00ns INFO     [AluTest.env.scoreboard]: PASSED: ff And ff =
00ff
    250.00ns INFO     [AluTest.env.scoreboard]: PASSED: ff Xor ff =
0000
    250.00ns INFO     [AluTest.env.scoreboard]: PASSED: ff Mul ff =
fe01
    250.00ns INFO     [AluTest.env.scoreboard]: Covered all
operations
    250.00ns INFO     AluTest PASSED
```

The same two sequences, at the same time

```
// Chapter 39, Figure 4: Running sub-sequences in parallel
#[derive(Default)]
struct TestAllParallelSeq;

impl Sequence for TestAllParallelSeq {
    type Req = AluCommand;
    type Rsp = AluResult;

    async fn body(&mut self, ctx: &mut SeqCtx<AluCommand,
AluResult>) -> Result<(), SeqError> {
        let seqr: Sequencer<AluCommand, AluResult> =
ConfigDb::get(None, "", "SEQR")?;
        let mut random = RandomSeq::default();
        let mut max = MaxSeq::default();

        let (a, b) = join2(random.start(&seqr),
max.start(&seqr)).await;
        a?;
        b?;
        ctx.info("ran random and max together");
        Ok(())
    }
}
```

`join2` is the `fork...join` you met in Chapter 16, doing here what `fork` / `join` does in every SystemVerilog virtual sequence: both sub-sequences run, and `body` continues when both are done. The sequencer arbitrates between the two live streams — FIFO order, one item each in turn — so the transcript alternates random operands with `0xff` operands, both patterns interleaved on one DUT.

One line of reasoning behind `join2` rather than `spawn`: a spawned task must own everything it touches (`'static` — the rule from Chapter 16), and a sub-sequence that borrows the parent sequence's state cannot promise that. Composing the two futures in place costs nothing and keeps that door open. The test differs from figure 2 by exactly one line — it starts `TestAllParallelSeq` instead — and is not worth a listing.

```
# Figure 5: The two sequences interleave at the sequencer

    250.00ns INFO     running ParallelTest (2/3)  [ch39-virtual-
sequence-testbench-
8.0/src/ch39_virtual_sequence_testbench_8_0.rs:461]
    500.00ns INFO     [TestAllParallelSeq]: ran random and max
together
    500.00ns INFO     [ParallelTest.env.scoreboard]: PASSED: c1 Add
67 = 0128
    500.00ns INFO     [ParallelTest.env.scoreboard]: PASSED: ff Add
ff = 01fe
    500.00ns INFO     [ParallelTest.env.scoreboard]: PASSED: 5e And
0b = 000a
    500.00ns INFO     [ParallelTest.env.scoreboard]: PASSED: ff And
ff = 00ff
    500.00ns INFO     [ParallelTest.env.scoreboard]: PASSED: b9 Xor
80 = 0039
    500.00ns INFO     [ParallelTest.env.scoreboard]: PASSED: ff Xor
ff = 0000
    500.00ns INFO     [ParallelTest.env.scoreboard]: PASSED: a5 Mul
75 = 4b69
    500.00ns INFO     [ParallelTest.env.scoreboard]: PASSED: ff Mul
ff = fe01
    500.00ns INFO     [ParallelTest.env.scoreboard]: Covered all
operations
    500.00ns INFO     ParallelTest PASSED
```

The testbench becomes a programming interface

The chapter's payoff is not running two canned sequences — it is what virtual sequences make possible for the *next* person on the team: an interface. First, one operation as a sequence:

```
// Chapter 39, Figure 6: One operation, as a sequence
struct OpSeq {
    a: u8,
    b: u8,
    op: Ops,
    result: Option<u16>,
}

impl Sequence for OpSeq {
    type Req = AluCommand;
    type Rsp = AluResult;

    async fn body(&mut self, ctx: &mut SeqCtx<AluCommand,
AluResult>) -> Result<(), SeqError> {
        let mut cmd = AluCommand { a: self.a, b: self.b, op:
self.op };
        ctx.start_item(&mut cmd).await?;
        let ticket = ctx.finish_item(cmd).await?;
        self.result =
Some(ctx.get_response(Some(ticket)).await.result);
        Ok(())
    }
}
```

`OpSeq` carries parameters, so it is constructed the ordinary way rather than through the factory — the factory's makers take no arguments, and both source books build their parameterized sequences by hand for the same reason. Its body is Chapter 38 in miniature: send one command, claim its response by ticket, store the answer.

```rust
// Chapter 39, Figure 7: The TinyALU programming interface
async fn do_op(
    seqr: &Sequencer<AluCommand, AluResult>,
    a: u8,
    b: u8,
    op: Ops,
) -> Result<u16, SeqError> {
    let mut seq = OpSeq { a, b, op, result: None };
    seq.start(seqr).await?;
    seq.result.ok_or_else(|| SeqError::from("the driver returned no
result"))
}

async fn do_add(seqr: &Sequencer<AluCommand, AluResult>, a: u8, b:
u8) -> Result<u16, SeqError> {
    do_op(seqr, a, b, Ops::Add).await
}
async fn do_and(seqr: &Sequencer<AluCommand, AluResult>, a: u8, b:
u8) -> Result<u16, SeqError> {
    do_op(seqr, a, b, Ops::And).await
}
async fn do_xor(seqr: &Sequencer<AluCommand, AluResult>, a: u8, b:
u8) -> Result<u16, SeqError> {
    do_op(seqr, a, b, Ops::Xor).await
}
async fn do_mul(seqr: &Sequencer<AluCommand, AluResult>, a: u8, b:
u8) -> Result<u16, SeqError> {
    do_op(seqr, a, b, Ops::Mul).await
}
```

This is the payoff. A test writer who has never opened the testbench gets four functions that take numbers and return numbers; sequencer, driver, handshake, and response envelope are all behind them. (In Python these read `seq.result` after `start` returns, because a coroutine cannot hand a value back through `start`; here the function returns what it computed, because that is what functions do.)

And with an interface in hand, a test is just a program:

```
// Chapter 39, Figure 8: Fibonacci, written as a program
#[derive(Default)]
struct FibonacciProgramSeq;

impl Sequence for FibonacciProgramSeq {
    type Req = AluCommand;
    type Rsp = AluResult;

    async fn body(&mut self, ctx: &mut SeqCtx<AluCommand,
AluResult>) -> Result<(), SeqError> {
        let seqr: Sequencer<AluCommand, AluResult> =
ConfigDb::get(None, "", "SEQR")?;
        let mut prev: u8 = 0;
        let mut cur: u8 = 1;
        let mut fib = vec![prev as u16, cur as u16];

        for _ in 0..7 {
            let sum = do_add(&seqr, prev, cur).await?;
            fib.push(sum);
            prev = cur;
            cur = sum as u8;
        }

        ctx.info(&format!("Fibonacci Sequence: {fib:?}"));
        Ok(())
    }
}
```

Compare this against Chapter 38's Fibonacci, where the handshake was visible at every step. The computation is identical; the sequence machinery has vanished into `do_add`. This is what a programming interface is for, and why a team that writes tests but not testbenches wants one.

Its test sets one extra ConfigDb value — `ConfigDb::set(None, "*", "CHECK_COVERAGE", false)` — because a program that only adds will never cover four operations, and the scoreboard reads that flag at check time. A test changing what the scoreboard demands, through the database, without touching it: the whole book's runtime-binding half, in one line.

```
# Figure 9: The TinyALU computes Fibonacci through the interface

    500.00ns INFO     running FibonacciProgramTest (3/3)  [ch39-
virtual-sequence-testbench-
8.0/src/ch39_virtual_sequence_testbench_8_0.rs:482]
    670.00ns INFO     [FibonacciProgramSeq]: Fibonacci Sequence:
[0, 1, 1, 2, 3, 5, 8, 13, 21]
    670.00ns INFO     [FibonacciProgramTest.env.scoreboard]:
PASSED: 00 Add 01 = 0001
    670.00ns INFO     [FibonacciProgramTest.env.scoreboard]:
PASSED: 01 Add 01 = 0002
    670.00ns INFO     [FibonacciProgramTest.env.scoreboard]:
PASSED: 01 Add 02 = 0003
    670.00ns INFO     [FibonacciProgramTest.env.scoreboard]:
PASSED: 02 Add 03 = 0005
    670.00ns INFO     [FibonacciProgramTest.env.scoreboard]:
PASSED: 03 Add 05 = 0008
    670.00ns INFO     [FibonacciProgramTest.env.scoreboard]:
PASSED: 05 Add 08 = 000d
    670.00ns INFO     [FibonacciProgramTest.env.scoreboard]: saw 1
of 4 ops (coverage not required)
    670.00ns INFO     FibonacciProgramTest PASSED
```

The environment underneath

The env this chapter runs on is Chapter 38's, with the driver that answers: it publishes each result on its own analysis port and returns it through `item_done(Some(...))`, and there is no separate `ResultMonitor` — the driver already awaits each answer, so it is the component that *has* it, and a second reader on the BFM's result queue would take turns stealing results from the first. The observation side, the scoreboard's two streams, and the `SEQR` handle in the ConfigDb are all exactly as you left them. Nothing in the environment knows that virtual sequences exist — which is the measure of the design: the top layer of the stimulus stack arrived, and no layer below it moved.

Summary

A virtual sequence is a sequence that starts sequences: same trait, no items of its own, started with `start_virtual()` because it has no sequencer to be

started on. Sequential composition is two `start` calls in a row; parallel composition is `join2` over two `start` futures, with the sequencer interleaving the streams. There is deliberately no `VirtualSequence` trait — a compile-time fence there would forbid the mixed shapes the UVM permits, and the framework takes the UVM's side of that trade with its eyes open. The chapter's real product is the interface pattern: an `OpSeq` with parameters, wrapped in `do_add`-style functions, until a test reads like arithmetic and the testbench underneath is invisible.

That is testbench 8.0, and with it every mechanism the UVM promised: phases, configuration, factory, TLM, analysis, transactions, sequences, and programs built from all of them. Chapter 40 returns to the shipped TinyALU testbench — the one the Interlude showed you before you could read it — and walks it end to end, with nothing left unexplained.

Chapter 40: The Complete TinyALU Testbench

The Interlude showed you this testbench before you could read it, and asked only for recognition. Thirty-nine chapters later, the deal completes: the same `tinyalu_tb` crate, walked with nothing left on faith. This is also the chapter to use as a template — the shipped testbench in the rustdv repository, the one its regression runs, organized the way a real project's would be.

Project layout

```
# Figure 1: The testbench crate

tinyalu_tb/src/
├── tinyalu_tb.rs    the crate root: BaseTest, RandomTest, MaxTest
├── alu_item.rs      transactions, Ops, and the predictor — plus
unit tests
├── alu_bfm.rs       the BFM: pins, protocol loops, queue-fed
methods
├── sequences.rs     BaseSeq, RandomSeq, MaxSeq over one shared
walk
├── components.rs    Driver, two monitors, Scoreboard, Coverage
└── env.rs           AluEnv: build, connect, and two ConfigDb knobs
```

One file per concern, and the crate root named after the crate — no file in this project is named `lib.rs`, so a stack trace or a log line always says *which* crate it came from. `alu_item.rs` ends with `#[cfg(test)]` unit tests: the predictor and the transaction derives are checked by `cargo test` on every build, no simulator anywhere — Chapter 14's capability, earning its keep in shipping code.

The tests

```
// Figure 2: BaseTest — build files the BFM; run starts whatever
the factory chose
#[derive(Component, Default)]
pub struct BaseTest {
    #[component]
    env: RustdvComp,
}

impl Component for BaseTest {
    fn build(&mut self, ctx: &mut RustdvCtx) {
        let bfm = TinyAluBfm::new(&ctx.dut()).expect("TinyALU
signals");
        ConfigDb::set(None, "*", "BFM", Rc::new(bfm));
        self.env = AluEnv::new_comp();
    }

    async fn run(&mut self, ctx: &mut RustdvCtx) -> Result<(),
TestError> {
        let _obj = ctx.raise_objection("stimulus");

        let seqr: Sequencer<alu_item::AluCommand,
alu_item::AluResult> =
            ConfigDb::get(Some(ctx), "", "SEQR")?;

        let mut seq = create_seq::<BaseSeq>();
        seq.start(&seqr).await?;

        let bfm: Rc<TinyAluBfm> = ConfigDb::get(Some(ctx), "",
"BFM")?;
        bfm.wait_idle().await;

        ctx.info("sequence complete");
        Ok(())
    }
}
```

Every line is a chapter. `build` constructs the BFM from the DUT handle and files it in the ConfigDb under `"*"` — the whole tree gets this one, and there is no singleton anywhere in the crate: the database asserts “one BFM under this name for this subtree,” which is a promise a two-interface testbench can keep, where a singleton’s “one BFM in the world” is not (Chapter 25). `run` finds the sequencer by name, builds its sequence *through the factory*, and starts it (Chapter 36).

No clock appears anywhere in it, and that is the point Chapter 19 made: `sim/hdl/tinyalu.sv` clocks itself, exactly as the chapters' copy of the design does, so this testbench only ever *waits* on edges. A BFM built that way ports to an emulation transactor unchanged; one that drives edges does not. The shipped testbench is not an exception to the discipline the book taught — it is the discipline, running.

One detail does differ from the chapters. The end of stimulus is `bfm.wait_idle().await`, not the twenty-clock flush of Chapters 34 and 36. Counting clocks worked, but it encoded a magic number — twenty, because the multiply is slowest — that would quietly go stale if the DUT grew a slower operation. `wait_idle` asks the *protocol* instead: it watches for the driver queue empty and the handshake quiet for two consecutive falling edges (two, because a command already popped but not yet driven must not fool it), then gives the monitors one more edge to flush. Same job, no magic number, and it moves with the DUT.

```
// Figure 3: Two tests, one testbench, no new components
#[rustdv::test(timeout_time = 500, timeout_unit = "us")]
#[derive(Component, Default)]
struct RandomTest {
    #[component]
    inner: RustdvComp,
}

impl Component for RandomTest {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        set_seq_override::<BaseSeq, RandomSeq>();
        self.inner = BaseTest::new_comp();
    }
}

#[rustdv::test(timeout_time = 500, timeout_unit = "us")]
#[derive(Component, Default)]
struct MaxTest {
    #[component]
    inner: RustdvComp,
}

impl Component for MaxTest {
    fn build(&mut self, _ctx: &mut RustdvCtx) {
        set_seq_override::<BaseSeq, MaxSeq>();
        self.inner = BaseTest::new_comp();
    }
}
```

Each registered test is one override plus `BaseTest` — Chapter 36's pattern, shipping. `RandomSeq` runs every operation five times with seeded random operands, so coverage is guaranteed by construction rather than hoped for; `MaxSeq` drives the `0xff op 0xff` corner once each — the corner random stimulus is unlikely to find on its own. Adding a third stimulus pattern to this testbench is a sequence and a six-line test; no component changes, which is the measure the whole book has been building toward. The `timeout_time` attributes are the runner's safety net: a hung handshake fails loudly at 500 microseconds instead of running forever.

The environment's two knobs

The Interlude showed `AluEnv` in full. The walk stops at its opening lines, because they are the ConfigDb doing structural work:

```
// Figure 4: Two choices a test can make from outside (env.rs,
build)
    fn build(&mut self, ctx: &mut RustdvCtx) {
        let activity: Active = ConfigDb::get(Some(ctx), "",
"IS_ACTIVE").unwrap_or(Active::Active);
        self.is_active = activity == Active::Active;
        self.with_coverage = ConfigDb::get(Some(ctx), "",
"WITH_COVERAGE").unwrap_or(true);

        self.seqr = Sequencer::new();
        ConfigDb::set(None, "*", "SEQR", self.seqr.handle());

        if self.is_active {
            self.driver = Driver::create_comp();
        }
        // ...
    }
```

`IS_ACTIVE` is the UVM's active/passive knob, typed: `Active` is an enum, so an illegal value cannot be filed, and `unwrap_or(Active::Active)` makes the ordinary case configure nothing. Look at what a passive env *is*: the driver slot is simply left empty — no driver constructed and told not to drive, no `None` checks downstream, just a component that does not exist and a `connect` that (three lines later) skips its wiring. This is why `build` had to be a phase: whether the driver exists is decided by configuration that must arrive *before* the children

do. `WITH_COVERAGE` works the same way for the coverage collector. And this is also the reason every component takes no constructor arguments — the factory's makers cannot supply any (Chapter 29), so everything a component needs arrives by name after it exists, which is exactly what makes the whole tree overridable.

The scoreboard's guards

The Interlude showed the scoreboard whole; the walk stops at the end of its `check`, on two guards that a lesser scoreboard omits:

```
// Figure 5: A scoreboard that cannot pass vacuously
(components.rs, check)
        if cmd_log.cmds.len() != result_log.results.len() {
            errors.error(format!(
                "scoreboard: saw {} commands and {} results",
                cmd_log.cmds.len(),
                result_log.results.len()
            ));
        }
        if self.compared == 0 {
            errors.error("scoreboard: nothing was
compared".to_string());
        }
```

The comparison loop zips commands against results, and a zip cannot complain about what never arrived — the shorter stream just ends the comparison, which is how a scoreboard passes while checking less than it saw (Chapter 34's warning). The first guard makes the count mismatch an error in its own right. The second refuses a clean pass with zero comparisons — the oldest trap in verification, a checker that never ran. Both guards exist because the failure they catch is silent by construction: the scoreboard owns its own storage (Chapter 32's rule — two `RustdvShared` logs behind two `SubscribePort`s), so a scoreboard that quietly stopped receiving would look identical to one that passed. The standing proof that the checking has teeth is a mutation run — corrupt the DUT's XOR into an OR and the scoreboard flags every affected transaction — verification of the verification, and these guards are what make that check stay meaningful.

The run

```
# Figure 6: The shipped testbench running

      0.00ns INFO     rustdv: found 2 test(s), RUSTDV_RANDOM_SEED=1
      0.00ns INFO     running RandomTest (1/2)
[rustdv/tinyalu_tb/src/tinyalu_tb.rs:77]
     70.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 296 }
     70.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 193, b: 103, op: Add }
     90.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 10 }
     90.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 94, b: 11, op: And }
    110.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 57 }
    110.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 185, b: 128, op: Xor }
    130.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 165, b: 117, op: Mul }
    160.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 19305 }
    180.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 168, b: 150, op: Add }
    180.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 318 }
    200.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 97, b: 254, op: And }
    200.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 96 }
    220.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 192, b: 138, op: Xor }
    220.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 74 }
    240.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 168, b: 59, op: Mul }
    270.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 9912 }
    290.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 340 }
    290.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 99, b: 241, op: Add }
    310.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 8 }
    310.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 238, b: 8, op: And }
    330.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 218 }
    330.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 70, b: 156, op: Xor }
    350.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 205, b: 172, op: Mul }
```

```
    380.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 35260 }
    400.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 159, b: 247, op: Add }
    400.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 406 }
    420.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 53, b: 171, op: And }
    420.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 33 }
    440.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 39, b: 138, op: Xor }
    440.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 173 }
    460.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 132, b: 186, op: Mul }
    490.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 24552 }
    510.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 137 }
    510.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 109, b: 28, op: Add }
    530.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 4 }
    530.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 23, b: 12, op: And }
    550.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 52 }
    550.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 245, b: 193, op: Xor }
    570.00ns INFO     [RandomTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 24, b: 60, op: Mul }
    600.00ns INFO     [RandomTest.inner.env.result_mon]:
result_monitor: AluResult { result: 1440 }
    630.00ns INFO     [RandomTest.inner]: sequence complete
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 193, b: 103, op: Add } out=AluResult
{ result: 296 } expected=AluResult { result: 296 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 94, b: 11, op: And } out=AluResult {
result: 10 } expected=AluResult { result: 10 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 185, b: 128, op: Xor } out=AluResult
{ result: 57 } expected=AluResult { result: 57 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 165, b: 117, op: Mul } out=AluResult
{ result: 19305 } expected=AluResult { result: 19305 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 168, b: 150, op: Add } out=AluResult
{ result: 318 } expected=AluResult { result: 318 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 97, b: 254, op: And } out=AluResult
{ result: 96 } expected=AluResult { result: 96 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 192, b: 138, op: Xor } out=AluResult
```

```
{ result: 74 } expected=AluResult { result: 74 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 168, b: 59, op: Mul } out=AluResult
{ result: 9912 } expected=AluResult { result: 9912 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 99, b: 241, op: Add } out=AluResult
{ result: 340 } expected=AluResult { result: 340 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 238, b: 8, op: And } out=AluResult {
result: 8 } expected=AluResult { result: 8 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 70, b: 156, op: Xor } out=AluResult
{ result: 218 } expected=AluResult { result: 218 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 205, b: 172, op: Mul } out=AluResult
{ result: 35260 } expected=AluResult { result: 35260 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 159, b: 247, op: Add } out=AluResult
{ result: 406 } expected=AluResult { result: 406 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 53, b: 171, op: And } out=AluResult
{ result: 33 } expected=AluResult { result: 33 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 39, b: 138, op: Xor } out=AluResult
{ result: 173 } expected=AluResult { result: 173 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 132, b: 186, op: Mul } out=AluResult
{ result: 24552 } expected=AluResult { result: 24552 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 109, b: 28, op: Add } out=AluResult
{ result: 137 } expected=AluResult { result: 137 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 23, b: 12, op: And } out=AluResult {
result: 4 } expected=AluResult { result: 4 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 245, b: 193, op: Xor } out=AluResult
{ result: 52 } expected=AluResult { result: 52 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: in=AluCommand { a: 24, b: 60, op: Mul } out=AluResult {
result: 1440 } expected=AluResult { result: 1440 } check=PASS
    630.00ns INFO     [RandomTest.inner.env.scoreboard]:
scoreboard: 20 compared, 0 mismatches
    630.00ns INFO     [RandomTest.inner.env.coverage]: coverage:
Add=5 And=5 Mul=5 Xor=5
    630.00ns INFO     RandomTest PASSED
    630.00ns INFO     running MaxTest (2/2)
[rustdv/tinyalu_tb/src/tinyalu_tb.rs:92]
    700.00ns INFO     [MaxTest.inner.env.result_mon]:
result_monitor: AluResult { result: 510 }
    700.00ns INFO     [MaxTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 255, b: 255, op: Add }
    720.00ns INFO     [MaxTest.inner.env.result_mon]:
result_monitor: AluResult { result: 255 }
    720.00ns INFO     [MaxTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 255, b: 255, op: And }
```

```
    740.00ns INFO     [MaxTest.inner.env.result_mon]:
result_monitor: AluResult { result: 0 }
    740.00ns INFO     [MaxTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 255, b: 255, op: Xor }
    760.00ns INFO     [MaxTest.inner.env.cmd_mon]: cmd_monitor:
AluCommand { a: 255, b: 255, op: Mul }
    790.00ns INFO     [MaxTest.inner.env.result_mon]:
result_monitor: AluResult { result: 65025 }
    820.00ns INFO     [MaxTest.inner]: sequence complete
    820.00ns INFO     [MaxTest.inner.env.scoreboard]: scoreboard:
in=AluCommand { a: 255, b: 255, op: Add } out=AluResult { result:
510 } expected=AluResult { result: 510 } check=PASS
    820.00ns INFO     [MaxTest.inner.env.scoreboard]: scoreboard:
in=AluCommand { a: 255, b: 255, op: And } out=AluResult { result:
255 } expected=AluResult { result: 255 } check=PASS
    820.00ns INFO     [MaxTest.inner.env.scoreboard]: scoreboard:
in=AluCommand { a: 255, b: 255, op: Xor } out=AluResult { result: 0
} expected=AluResult { result: 0 } check=PASS
    820.00ns INFO     [MaxTest.inner.env.scoreboard]: scoreboard:
in=AluCommand { a: 255, b: 255, op: Mul } out=AluResult { result:
65025 } expected=AluResult { result: 65025 } check=PASS
    820.00ns INFO     [MaxTest.inner.env.scoreboard]: scoreboard: 4
compared, 0 mismatches
    820.00ns INFO     [MaxTest.inner.env.coverage]: coverage: Add=1
And=1 Mul=1 Xor=1
    820.00ns INFO     MaxTest PASSED
**********************************************************************
***********
** TEST                                          STATUS  SIM TIME (ns)
**
**********************************************************************
***********
** RandomTest                                     PASS         630.00
**
** MaxTest                                        PASS         190.00
**
**********************************************************************
***********
REGRESSION: PASS
```

Twenty-four operations across two tests, each line stamped with the path of the component that wrote it, ending in the summary table. This is the same run the repository's regression asserts — the counts above are checked mechanically on every push, so the transcript you produce and the one in this book can only agree.

Inspection, waveforms, and RTL coverage

The Icarus run above is the four-state reference. From the repository root, `sim/run_rustdv.sh release verilator` runs the same Rust testbench with the shared Verilator host. Its default FAST mode exposes the DUT's top-level ports without waveform or coverage instrumentation. The launcher exports `RUSTDV_TOP=tinyalu`, selecting the DUT even when other root modules exist.

For internal visibility, set `RUSTDV_VERILATOR_MODE=inspect` and use `RUSTDV_VERILATOR_CONTROL_FILE` to name a `.vlt` file selecting signals with `public_flat_rw`. The TinyALU launcher supplies `sim/verilator-debug.vlt` by default. INSPECT adds those VPI-visible internals without recording waves. DEBUG uses the selected visibility and writes an FST from time zero; `RUSTDV_FST` chooses its destination. Both DEBUG and RECORD need the lz4 development headers and library used by Verilator's FST writer.

RECORD compiles waveform instrumentation but opens no file until the testbench requests a capture. Under `rustdv::sim::simulator_trace`, `start(path)`, `flush()`, `stop()`, and `status()` control that capture and report its state. Each operation runs inside `service_read_only(...)`: the supplied synchronous closure executes on the simulator thread after writes and combinational logic settle. Simulation time is held until it returns. A service that waits on another OS thread therefore needs a wall-clock deadline or cooperative cancellation; a simulation-time timeout cannot release it. `host_present()` and `host_format()` allow a service to choose its backend before entering ReadOnly. The API reports unavailable capabilities and invalid lifecycle use as errors. The supplied Verilator host records FST; the generic Rust API does not assume every simulator can do so.

RTL coverage answers a different question from the `Coverage` component in this testbench. That component counts observed operations: did the test exercise ADD, AND, XOR, and MUL? Verilator's COVERAGE build instruments RTL lines and expressions. Run `RUSTDV_VERILATOR_MODE=coverage RUSTDV_COVERAGE_FILE=/tmp/rustdv-coverage/tinyalu.dat sim/run_rustdv.sh release verilator`, then inspect the database with `verilator_coverage --report summary /tmp/rustdv-coverage/tinyalu.dat`. The host writes it after final blocks and end-of-simulation callbacks, retaining it even when a RustDV

check fails. It supplements the operation collector; setting `WITH_COVERAGE` controls only that functional collector.

Waveform and RTL-coverage builds are separate profiles. The scripts reject FST in COVERAGE mode and reject a coverage destination in other modes. These profiles use the repository's launchers and C++ simulator host in addition to the Rust crates. The repository's simulator guide documents their controls, and its targeted regressions exercise capture windows, settled inspection, and coverage output. The testbench above needs none of these optional services to run.

What to take with you

Use this crate as the template it is. Transactions are plain structs with derives and a hand-written `Display`, plus unit tests beside them (Chapter 35, Chapter 14). The BFM owns the pins, speaks the protocol on falling edges, and exposes queue-fed async methods — and hides one hand-written `Debug` impl so `ConfigDb::dump` names it as `TinyAluBfm` instead of dumping eight signal handles (a small kindness Chapter 28 makes you glad of). Sequences share one walk and vary one method; components take nothing at construction and ask the ConfigDb for what they need; the env reads its knobs before building, builds through the factory, and wires everything in `connect` with the one idiom; the tests are overrides on a shared base. Every piece was a chapter; together they are a working, checked, regression-guarded testbench — and now they are yours.

The climb the Interlude promised is done. What follows are the appendices: the maps back to the earlier books, the idiom translations from both source languages, and the full catalog of what rustdv provides.

Appendix A: Chapter Maps to the Earlier Books

Two books precede this one, and either prepares you for it: *The UVM Primer* (SystemVerilog) and *Python for RTL Verification* (Python/cocotb/pyuvm). This appendix maps every chapter of this book to its companion chapters in both, for readers who want to compare treatments — or to lend the right book to a colleague. A dash means the topic has no mirror in that book; Chapters 5 and 21 cover ground (ownership; macros) neither predecessor made visible.

Rust for RTL Verification	The UVM Primer	Python for RTL Verification
Ch. 1: Why Rust?	Ch. 1: Introduction	Why Python and why UVM?
Ch. 2: Rust Concepts	—	Python concepts
Ch. 3: Rust Basics	—	Python basics
Ch. 4: Conditions, Loops, and match	—	Conditions and loops / Ranges
Ch. 5: Ownership	*(no mirror — GC did this silently)*	*(no mirror)*
Ch. 6: Borrowing and References	*(no mirror)*	*(no mirror)*
Ch. 7: Structs, Enums, and Methods	Ch. 4: OOP; Ch. 7: Static Methods	Classes
Ch. 8: Collections	—	Python sequences / Lists / Strings / Dictionaries
Ch. 9: Result, Option, and the End of Exceptions	—	Exceptions
Ch. 10: Traits	Ch. 5: Classes and Extension; Ch. 6: Polymorphism	Inheritance / super() / protocols
Ch. 11: Generics	Ch. 8: Parameterized Class Definitions	(duck typing, throughout)
Ch. 12: Closures and Iterators	—	Generators / comprehensions

Rust for RTL Verification	The UVM Primer	Python for RTL Verification
Ch. 13: Smart Pointers	—	Protecting attributes
Ch. 14: Modules, Crates, and Cargo	—	Modules
Interlude: The Complete TinyALU Testbench	*(the destination, previewed)*	*(testbench 8.0, previewed)*
Ch. 15: async/await and the Executor	*(the simulator's scheduler, opened up)*	Coroutines
Ch. 16: Tasks, Channels, and Sim-Aware Queues	Ch. 17: Interthread Communication	cocotb Queue
Ch. 17: Simulating with rustdv-sim	—	Simulating with cocotb
Ch. 18: Basic Testbench: 1.0	Ch. 2: A Conventional Testbench	Basic testbench: 1.0
Ch. 19: TinyAluBfm	Ch. 3: Interfaces and BFMs	TinyAluBfm
Ch. 20: Struct-Based Testbench: 2.0	Ch. 10: An Object-Oriented Testbench	Class-based testbench: 2.0
Ch. 21: Macros	*(the `uvm_*_utils macros, demystified)*	(decorators; design patterns)
Ch. 22: Why UVM?	Ch. 1: Introduction	Why UVM?
Ch. 23: uvm_test Testbench: 3.0	Ch. 11: UVM Tests	uvm_test testbench: 3.0
Ch. 24: Components	Ch. 12: UVM Components	uvm_component
Ch. 25: uvm_env Testbench: 4.0	Ch. 13: UVM Environments	uvm_env testbench: 4.0
Ch. 26: Logging	Ch. 19: UVM Reporting	Logging
Ch. 27: Configuration	*(uvm_config_db, in passing)*	ConfigDB()
Ch. 28: Configuration Debugging	—	Debugging the ConfigDB()
Ch. 29: The Factory	Ch. 9: The Factory Pattern	The UVM factory
Ch. 30: Variation-Point Testbench: 5.0	—	UVM factory testbench: 5.0
Ch. 31: Component Communications	Ch. 14: A New Paradigm; Ch. 18: Put	Component communications

Rust for RTL Verification	The UVM Primer	Python for RTL Verification
	and Get Ports	
Ch. 32: Analysis Ports	Ch. 15: Talking to Multiple Objects	Analysis ports
Ch. 33: Components in Testbench 6.0	Ch. 16: Analysis Ports in a Testbench	Components in testbench 6.0
Ch. 34: Connections in Testbench 6.0	Ch. 18: Put and Get in Action; Ch. 22: UVM Agents	Connections in testbench 6.0
Ch. 35: Transactions	Ch. 20: Deep Operations; Ch. 21: UVM Transactions	uvm_object in Python
Ch. 36: Sequence Testbench: 7.0	Ch. 23: UVM Sequences	Sequence testbench: 7.0
Ch. 37: Out-of-Order Transactions: Testbench 7.1	Ch. 23: UVM Sequences	Fibonacci testbench: 7.1 / get_response() testbench: 7.2
Ch. 38: Fibonacci Testbench: 7.2	Ch. 23: UVM Sequences	Fibonacci testbench: 7.1
Ch. 39: Virtual Sequence Testbench: 8.0	Ch. 23: UVM Sequences	Virtual sequence testbench: 8.0
Ch. 40: The Complete TinyALU Testbench	*(no mirror)*	*(no mirror)*

This book has no closing what-comes-next chapter — the earlier books each wrote one, and their futures arrived on their own schedules. The book ends with the testbench.

Appendix B: Python → Rust Idiom Translations

For readers coming from cocotb and pyuvm (and *Python for RTL Verification*): the working translations this book used, gathered for reference. SystemVerilog readers want Appendix C, this table's twin. Legend: **[C]** cocotb, **[P]** pyuvm.

Language and runtime

Python	Rust	Chapter
`async def` coroutine, resumed via `send(None)`	`async fn` → `Future`, resumed via `poll()`	15
`@cocotb.test()`	`#[rustdv::test]`	15, 21
exceptions fail the test	`Result<(), TestError>`; panics = testbench bugs	9, 18
`cocotb.start_soon(coro)`	`spawn(future) -> TaskHandle<T>`	16
`await task`	`task.await` → `Result<T, TaskError>`	16
`task.kill()`	`handle.cancel()` — the future is dropped; cleanup in `Drop`	16
`Combine(...)` / `First(...)`	`join2` / `join!` / `first2` / `first!`	16
`try/except QueueFull`	`try_put` → `Result<(), T>` (rejected item handed back)	16, 31
decorator registration at import time	`linkme` gathers test registrations into a distributed slice	21
metaclass class registration	`#[derive(Component)]` supplies factory registration	21, 29
`getattr(obj, name)` dispatch	pass the function/closure itself	33
`logging` levels + handlers	`log::` levels, `set_level_for(prefix)`, `log_to_file`	26

cocotb layer

Python (cocotb)	Rust (rustdv-sim)	Chapter
`Timer(2, units="ns")`	`Timer::ns(2).await`	15
`RisingEdge(sig)` / `FallingEdge(sig)`	`sig.rising_edge().await` / `sig.falling_edge().await`	17
`ClockCycles(clk, n)`	`for _ in 0..n { clk.rising_edge().await; }`	17
`dut.sig` attribute magic	`dut.signal("sig")` returns `Result<LogicHandle, HandleError>`; `?` propagates failure	17
`sig.value = x` / `int(sig.value)`	`sig.set_u64(x)` / `sig.get_u64()?`	17
Python integers for wide buses	`get_u128()?` or arbitrary-width `get_bigint()?`; matching typed setters	17
four-state signal values	`get_logic()?` / `set_logic(&value)` with `LogicArray`	17
real and string signal values	`dut.real(name)?` / `dut.string(name)?`, then `get()` / `set()`	17
hierarchy and aggregate member lookup	`child(name)?` returns `SimHandle`; `as_*()?` checks the kind	17
`ReadWrite()` / `ReadOnly()`	`read_write().await` / `read_only().await`	17
`NextTimeStep()`	`next_time_step().await`	17
`Clock(dut.clk, 10, units="ns").start()`	`Clock::new(&clk, SimDuration::ns(10)).start()`	17
`cocotb.queue.Queue(maxsize=1)`	`Queue::new(Some(1))`; `Queue::unbounded()`	16
`Event` / `Lock`	`sim::Event` / `sim::Lock` (FIFO-fair, RAII guard)	16

The integer getter selects an unsigned width and rejects X/Z or an oversized signal. Use `BigUint` when the width exceeds the primitive types. Traversal and edge methods come from `HandleChildren` and `HandleEvent`, both imported

by the prelude. Real/string/aggregate access depends on simulator VPI support. The launcher selects the DUT root with `RUSTDV_TOP`.

For an application-defined inspection service, `service_read_only(closure)` runs a synchronous operation while holding settled simulation time. Runtime waveform capture uses `rustdv::sim::simulator_trace` and a trace-capable host; Chapter 40 describes the Verilator modes and their requirements.

pyuvm layer

Python (pyuvm)	Rust (rustdv)	Chapt
`@pyuvm.test()` on a class, `uvm_test_top`	`#[rustdv::test]` on a struct; the root is named after your test	23
`raise_objection()` / `drop_objection()`	`ctx.raise_objection(..)` → RAII `ObjectionGuard`; drop releases	23
`uvm_component(name, parent)` tree	children are struct fields; `#[derive(Component)]`; paths derived	24
the nine phases, pyuvm's traversal order	the nine phases, same order: `build`, `connect`, … `final_phase`	24
`self.logger`, `[uvm_test_top.comp]`	`ctx.info(..)`, same bracket format, path supplied by the walk	26
`ConfigDB().set/get`, wildcards, globals	`ConfigDb::set/get` — same paths, same globs, `Result` answers	25, 27
`except UVMConfigItemNotFound`	`match` on `ConfigError::NotFound { .. }`	28
metaclass registration + `create()`	`#[derive(Component)]` registers; `Foo::create_comp()`	21, 29
`set_type_override_by_type`	`Factory::set_type_override::<A, B>()` (also by name, by instance)	29, 30

Python (pyuvm)	Rust (rustdv)	Chapt
TLM-1 put/get/peek port classes	`PutPort` / `GetPort` / `PeekPort`, wired export-to-port through a `TlmFifo`	31
`UVMTLMConnectionError` (lazy, at first use)	elaboration sweep names every unwired port before run	31
`uvm_analysis_port.write()`	`PublishPort<T>::write(&T)` through an `AnalysisBus` hub	32
`uvm_subscriber` (one `write` per class)	a `Subscriber<T>` impl per stream — two streams, two impls	32, 34
`uvm_tlm_analysis_fifo`	absent — the subscriber owns its storage	32
`uvm_object` do_copy/do_compare/ `__str__`	`#[derive(Clone, PartialEq, Debug)]` + hand-written `Display`	35
`copy(other)` / `clone()`	`clone_from(&mut self, src)` / `clone()`	35
`uvm_sequence.body()`	`impl Sequence` — `type Req` / `type Rsp`, `async fn body(ctx)`	36
`start_item` / `finish_item`	`ctx.start_item(&mut req)` / `ctx.finish_item(req)` → ticket	36
`seq_item_port.get_next_item()`	`port.get_next_item().await` → `SeqItem<REQ>`	36
`item_done()` / `item_done(rsp)` + `set_id_info`	`item_done(None)` / `item_done(Some(rsp))` — auto-tagged	36, 38
`get_response()`	`get_response(Some(ticket))` / `try_get_response` — in order or by ticket	37, 38
(no pyuvm counterpart) `try_next_item`	`try_next_item()` → `Option` — the UVM's non-blocking accept, kept	37
`seq.start(seqr)` / `start(None)` for virtual	`seq.start(&seqr)` / `start_virtual()`	36, 39
`is_active` int from ConfigDB	`Active` enum from the ConfigDb; a passive env skips building the driver	40

Appendix C: SystemVerilog-UVM → rustdv Translations

For readers coming from SystemVerilog UVM (and *The UVM Primer*): where each piece of your working vocabulary went. Python readers want Appendix B, this table's twin.

Language level

SystemVerilog	Rust	Chapter
`byte`, `shortint`, `int`	`u8` / `i8`, `u16` / `i16`, `u32` / `i32` — no silent truncation	3
`logic` four-state values	`Logic` enum / `LogicArray` — no x in arithmetic	7, 17
Unsigned logic vectors read as integers	`get_u8()` through `get_u128()`, or `get_bigint()` for arbitrary widths; reads return `Result`	17
`real` / `string` variables	`RealHandle` (`f64`) / `StringHandle` (owned strings), with scheduled and immediate writes	17
Unpacked struct/union members	`AggregateHandle::child()` / `children()` through simulator VPI	17
A settled sampling point	`read_only().await`; writes are illegal until a later writable time step	17
A writable synchronization point	`read_write().await`; buffered writes apply before its waiters resume	17
`typedef enum` (an int in disguise)	`enum` — a real type; exhaustively matched	7
`case` + `default` (+ `unique` warnings)	`match` — missing cases are compile errors	4
`class ... extends`, `virtual`, `super.new()`	traits, default methods, composition + delegation	10
`pure virtual function` in a virtual class	a required trait method, checked at the `impl`	10

SystemVerilog	Rust	Chapter
parameterized class `#(type T = int)`	generics `<T: Bound>`, checked at definition	11
`class ... #(type REQ, type RSP = REQ)`	`SeqItemPort<REQ, RSP = REQ>` — same convention	11
`local` / `protected`	private-by-default, `pub` to export	14
`null` handle, `$cast`	`Option<T>`, exhaustive `match` — no null, no cast	9
status flags and sentinel returns	`Result<T, E>` + `?` — failure in the signature	9
`$sformatf`	`format!`	8
`fork` / `join_none` / `disable`	`spawn(future)` → `TaskHandle`; `handle.cancel()`	16
`forever`	`loop` (an expression — it can `break` with a value)	4
`mailbox #(T)`, `try_put` / `try_get`	`sim::Queue<T>` — same names, `Result` / `Option` answers	16
named `event`, `->done`, `@(done)`	`sim::Event` — `set()` / `wait().await`	16
`semaphore` (one key)	`sim::Lock` — FIFO-fair, RAII guard	16
`@(posedge clk)`, `#2ns`	`clk.rising_edge().await`, `Timer::ns(2).await`	15, 17
`` `define `` -style codegen (`` `uvm_*_utils ``)	attribute + derive macros — syntax trees, not text	21
package + `.f` file + vendor tarball	crate + `Cargo.toml` + crates.io	14
(no equivalent)	`cargo test` — unit tests with no simulator	14

These signal accesses use the simulator's VPI objects, rather than a compiled virtual interface. `RUSTDV_TOP` names the root and `HandleChildren` supplies name lookup. `SimHandle::as_*()` reports wrong-kind objects as `HandleError`. Integer access is unsigned and rejects X/Z or an oversized signal; use `LogicArray` to preserve four-state values. Packed aggregates exposed as logic use `LogicHandle`; unpacked member traversal requires simulator support.

Functional coverage in the book's collector is independent of Verilator RTL line/expression coverage. Chapter 40 describes COVERAGE mode, optional waveform capture, and `service_read_only()` for application-defined

inspection at a settled point. These services depend on the repository's simulator host/build configuration as well as the Rust API.

Methodology level

SystemVerilog UVM	rustdv	Chap
`class my_test extends uvm_test` + `run_test()`	`#[rustdv::test]` on a struct; the runner drives its phases	23
`phase.raise_objection(this)` / `drop_objection`	`ctx.raise_objection(..)` → RAII `ObjectionGuard` ; drop releases	23
`uvm_component(name, parent)` tree	children are struct fields; `#[derive(Component)]` ; paths derived by the walk	24
`build_phase` (top-down) / `connect_phase` (bottom-up)	`fn build(&mut self, ctx)` / `fn connect(&mut self, ctx)` — real phases, same directions	24
`run_phase` (objection-gated task)	`async fn run` — concurrent across the tree; ends when objections drain	24, 3
elaboration + post-run phases	same names; **top-down**, where SV runs them bottom-up	24
`` `uvm_info(id, msg, verbosity) ``	`ctx.info(..)` — same time/level/ `[path]` line format	26
`set_report_verbosity_level_hier()`	`ctx.set_logging_level_hier(..)`	26
`uvm_config_db#(T)::set/get` , wildcards	`ConfigDb::set(ctx, glob, key, v)` / `get` → `Result` — one key, no type in the address	25, 2
virtual interface via config database	`Rc<TinyAluBfm>` in the ConfigDb	25
a failed `get()` (silent `return 0`)	`ConfigError` naming which failure; `#[must_use]`	27, 2
`print_config()` / `+UVM_CONFIG_DB_TRACE`	`ConfigDb::print()` / `ConfigDb::set_tracing(true)`	28
`` `uvm_component_utils `` registration	`#[derive(Component)]` registers by name, universally	21, 2
`type_id::create("name", this)`	`Foo::create_comp()` — overridable (`new_comp() = new` , fixed)	29

SystemVerilog UVM	rustdv	Chap
`set_type_override_by_type` / `_by_name` / instance	`Factory::set_type_override::<A, B>()` / `_by_name` / `set_inst_override`	29, 3
`uvm_factory::get().print()`	`Factory::print()`	29
TLM-1 put/get/peek port + export + `connect()`	`PutPort` / `GetPort` / `PeekPort` + `fifo.put_export().connect(comp, PORT_NAME)`	31
`uvm_tlm_fifo` (with built-in taps)	`TlmFifo<T>` (with `put_ap()` / `get_ap()`)	31
`try_put()` returns a bit	`try_put(T)` → `Result<(), T>` — a refused item comes back	31
unconnected port found at first use	elaboration sweep names every unwired port before run	31
`uvm_analysis_port.write()`	`PublishPort<T>::write(&T)`, brokered by an `AnalysisBus` hub	32
`uvm_subscriber` (one `write` per class)	a `Subscriber<T>` impl per stream — two streams, two impls, no `imp_decl`	32, 3
`uvm_tlm_analysis_fifo` in scoreboards	absent — the subscriber owns its storage	32
`uvm_agent` + `is_active`	env reads `Active` from the ConfigDb; a passive env leaves the driver slot empty	40
`do_copy` / `do_compare` / `convert2string`	`#[derive(Clone, PartialEq, Debug)]` + hand-written `Display`	10, 3
`copy(other)` / `clone()`	`clone_from(&mut self, src)` / `clone()`	35
`uvm_field_*` macros (runtime field walking)	`derive` — the same generation, at compile time	21, 3
`uvm_sequence #(REQ, RSP)`, `body()`	`impl Sequence` — `type Req` / `type Rsp`, `async fn body(ctx)`	36
`start_item(req)` / `finish_item(req)`	`ctx.start_item(&mut req)` / `ctx.finish_item(req)` → ticket	36
`seq_item_port.get_next_item()` / `item_done()`	same names; `item_done(Some(rsp))` answers	36, 3
`try_next_item()` (absent from pyuvm)	`try_next_item()` → `Option<SeqItem<REQ>>`	37

SystemVerilog UVM	rustdv	Chap
`rsp.set_id_info(req)` + `get_response()`	auto-tagged; `get_response(Some(ticket))` / `try_get_response`	37, 3
`seq.start(seqr)` / virtual sequence with no sequencer	`seq.start(&seqr)` / `start_virtual()`	36, 3
sequencer grab/lock/priority arbitration	unported (FIFO arbitration only) — a recorded gap	36
`assert` (prints, simulation continues)	`assert!` (panic = test fails, on the spot)	9
`uvm_error` vs `uvm_fatal` (convention)	`CheckSink::error` = DUT check; `panic!` = testbench bug	9, 24

Appendix D: What rustdv Provides

Every Part II listing opens with `use rustdv::prelude::*` — the analog of `import uvm_pkg::*` and `from pyuvm import *`. This appendix is the complete reference for what that line brings into scope, plus the macros. The Chapter column points to where each name is taught; the Toolkit page (before Chapter 15) groups the same names by job. A dash means the name is provided for completeness but this book's examples never need it.

The prelude, alphabetically

Name	What it is	Chapter
`Active`	the active/passive agent knob, read from the ConfigDb (pyuvm's `is_active` int, as an enum)	40
`AggregateHandle`	typed member lookup and iteration for unpacked structs and unions exposed through VPI	17
`AnalysisBus`	the broadcast hub; it stores nothing	32
`BigUint`	arbitrary-width unsigned values, used by `get_bigint` and `set_bigint`	17
`build_all`	drive `build` across a component tree (the runner's job)	24
`channel`	make a `(Sender, Receiver)` queue pair with a capacity	—
`check_all`	drive `check` across a tree	24
`CheckSink`	the collector `check` phases write failures into	24
`Clock`	a software clock driver — taught once, then retired in favor of BFMs that wait on edges	17
`Component`	the lifecycle trait: `build`, `connect`, `run`, and the other phase methods	24

Name	What it is	Chapter
`ComponentNode`	the tree-traversal trait `#[derive(Component)]` implements	21, 24
`ConfigDb`	path-addressed runtime configuration; `get` returns a `Result` naming the cause	25, 27
`connect_all`	drive `connect` across a tree	24
`create_seq`	build a sequence through the sequence factory	36
`Either`	the answer from racing two differently-typed futures	—
`end_of_elaboration_all`	phase driver	24
`Event`	set once; everyone waiting wakes (SV: named `event`)	16
`extract_all`	phase driver	24
`Factory`	the component registry: build by type or name, override by type, name, or instance	29
`final_all`	phase driver	24
`first2`	race two futures; the first to finish wins (SV: `fork...join_any`)	16
`GetPort`	the consuming end of a TLM connection	31
`HandleChildren`	the lookup and traversal trait for hierarchy and aggregate handles	17
`HandleError`	what signal access returns instead of a crash	17, 19
`HandleEvent`	the trait supplying logic-edge and value-change triggers	17
`HierarchyHandle`	a handle to a scope in the design hierarchy	17
`join2`	run two futures together; wait for both (SV: `fork...join`)	16
`Lock`	mutual exclusion with an RAII guard (SV: a one-key `semaphore`)	16
`log`	the logging facade; policy is per hierarchy	15, 26
`Logic`, `LogicArray`	four-state values, scalar and vector	17

Name	What it is	Chapter
`LogicHandle`	a named DUT signal: read it, drive it; a typo'd name is an `Err`	17
`next_time_step`	trigger for the simulator's next time step	17
`NullTrigger`	the trigger that is ready the next time it is polled	15
`ObjectionGuard`	returned by `ctx.raise_objection`; the run phase ends when the last guard drops	23
`PeekPort`	the look-without-taking end of a TLM connection	31
`PortName`, `PortOwner`	how the elaboration check names an unconnected port	31
`print_hierarchy`	dump a component tree	—
`PublishPort`	the publishing end a component declares	32
`PutPort`	the producing end of a TLM connection	31
`Queue`	the sim-aware mailbox: bounded puts and empty gets block in simulated time (SV: `mailbox#(T)`)	16
`read_only`, `read_write`	settled sampling and writable synchronization triggers	17
`RealHandle`	a real variable read/written as `f64`	17
`Receiver`	the getting end `channel` returns	—
`report_all`	phase driver	24
`Rng`	the deterministic per-test random source behind `ctx.rng()`	20
`run_component_test`	run a component tree as a self-contained test	—
`run_extract_check_report`	drive the closing phases together	—
`RustdvComp`	a slot holding any factory-built component	29
`RustdvCtx`	the context: path, logging, rng, DUT handle, objection — the framework, handed as an argument	15
`RustdvSeq`	a slot holding any factory-built sequence — `RustdvComp`'s parallel	36

Name	What it is	Chapter
`RustdvShared`	a cloneable handle to one shared object — `Rc<RefCell>` wearing the framework's name	32
`Sender`	the putting end `channel` returns	—
`SeqCtx`	the context a sequence `body` receives	36
`SeqError`	what a sequence can fail with	36
`SeqItem`	the bounds a sequence-item type must meet	36
`SeqItemExport`, `SeqItemPort`	the driver's side of the sequencer handshake	36
`Sequence`	the trait with one method, `body` — a test program, not a component	36
`Sequencer`	grants sequences their turns; feeds the driver	36
`set_seq_override`	change which sequence `create_seq` builds	36
`service_read_only`	run a synchronous service closure at settled ReadOnly while simulation time is held	40
`sim_time_ns`	the current simulated time	—
`SimDuration`	an amount of simulated time	17
`spawn`, `spawn_named`	launch a concurrent task; the named form stamps its log lines	16
`start_all`	drive the run phase across a tree	24
`start_of_simulation_all`	phase driver	24
`StringHandle`	owned string reads and scheduled/immediate writes; embedded NUL bytes are rejected	17
`SubscribePort`	the subscribing end a component declares	32
`Subscriber`	the trait a subscriber implements once per stream	32
`TaskHandle`	what `spawn` returns: await it for the result, or `cancel()` it	16
`TestError`	the error a failing test returns; `Ok(())` is a pass	15

Name	What it is	Chapter
`Timer`	the simulated-time trigger: `Timer::ns(2).await`	15
`TlmFifo`	the FIFO two components share without learning each other's names	31
`TxnId`	the ticket `finish_item` returns; claims a response	37, 38
`with_timeout`	wrap an await with a deadline	—

The macros

Name	What it does	Chapter
`#[rustdv::test]`	registers a test with the runner (cocotb: `@cocotb.test()`)	15, 21
`#[derive(Component)]`	writes the component-tree plumbing (SV: the `uvm_component_utils` family)	21, 24
`vpi_bootstrap!()`	one line per testbench crate: exports the entry points the simulator loads	17
`first!`, `join!`	variadic race and join; call as `rustdv::first!` / `rustdv::join!` or import explicitly	16

On the surface, outside the prelude

A few names live on the crate but not in the prelude; reach them as `rustdv::Name`.

Name	What it is	Chapter
`ConfigError`	why a ConfigDb `get` failed, as a value	28
`ConnectError`	why a connection could not be made	31
`TlmFull`, `TlmEmpty`, `TlmError`	the channel layer's refusals: what `Sender::try_send` and `Receiver::try_recv` answer	—
`Maker`	the closure type the factory stores per registration	29

Name	What it is	Chapter
`ResponseQueue`	the store behind `get_response` — responses held for claiming, in order or by ticket (pyuvm's `ResponseQueue`)	—
`SimHandle`	the kind-tagged result of general child lookup; `as_*()` conversions return `Result`	17
`ValueError`	why a value read/conversion failed, including X/Z, width, and unavailable representations	17
`top_module`	root lookup selected by `RUSTDV_TOP`, falling back to the first root when unset	17
`TimeoutError`, `TaskError`, `Executor`, `TestRegistration`	infrastructure corners a testbench rarely touches	—

`HandleBase`, for `name()` and `full_name()`, lives at `rustdv::sim::handle::HandleBase`; import it explicitly. The low-level GPI layer has its own `rustdv::gpi::AnyHandle`, while simulator-layer child lookup returns `SimHandle`.

Optional simulator services

These names live under `rustdv::sim::simulator_trace`, outside the prelude. Chapter 40 explains their use with the repository's Verilator host.

Name	What it does
`host_present`, `host_format`	probe the optional host and its advertised waveform format before ReadOnly
`start`, `flush`, `stop`, `status`	control or query capture inside `service_read_only`
`TraceStatus`, `TraceState`	report availability, capture state, time window, and dump count
`TraceFormat`	identify FST or another advertised host format
`TraceError`	report an unavailable capability, invalid path, wrong state, or host failure

The shipped Verilator host uses FST. RECORD mode compiles capture support; FAST and INSPECT do not. RTL line/expression coverage uses the separate COVERAGE profile and `RUSTDV_COVERAGE_FILE`, independently of functional coverage collected by a Rust subscriber.

The whole of each layer is also re-exported for power users: `rustdv::sim`, `rustdv::runner`, `rustdv::gpi`.